HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

# CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Giáo trình học phần INT1306

Chương trình khung 2023 • 3 tín chỉ

Minh hoạ bằng ba ngôn ngữ

Julia • Python • C++

Mỗi thuật toán được trình bày theo trình tự: ý tưởng, giả mã, ba bản cài đặt, phân tích độ phức tạp và các lỗi thường gặp. Mỗi chương kết thúc bằng một ứng dụng trong trí tuệ nhân tạo.

Hà Nội

# Lời nói đầu

Giáo trình này được biên soạn cho học phần *Cấu trúc dữ liệu và Giải thuật*, mã số INT1306, thuộc khối kiến thức cơ sở ngành của chương trình đào tạo khung 2023. Nội dung và thứ tự các chương bám theo đề cương chi tiết học phần: bảy chương tương ứng với mười sáu tuần học, với phân bổ 30 tiết lý thuyết, 18 tiết bài tập và thảo luận, 12 tiết thực hành.

Tài liệu được viết với giả định người đọc **chưa có kinh nghiệm lập trình**. Phần I của chương 1 trình bày từ đầu những khái niệm cần thiết: biến, kiểu dữ liệu, câu lệnh rẽ nhánh, vòng lặp, mảng và hàm. Sinh viên đã học xong học phần tiên quyết có thể đọc lướt phần này, nhưng nên dừng lại ở các mục nêu khác biệt giữa ba ngôn ngữ được dùng, vì những khác biệt đó xuất hiện liên tục trong toàn bộ tài liệu.

**Về việc dùng ba ngôn ngữ.** Mỗi thuật toán trong giáo trình được cài đặt bằng Julia, Python và C++. Ba bản cài đặt luôn thực hiện cùng một thuật toán và luôn cho cùng kết quả, chỉ khác nhau về cú pháp. Lý do chọn ba ngôn ngữ như sau. C++ là ngôn ngữ phổ biến nhất trong các kỳ thi thực hành và trên các cổng luyện tập trực tuyến, đồng thời cho phép kiểm soát bộ nhớ ở mức thấp nên phù hợp để hiểu bản chất của con trỏ và cấp phát động. Python là ngôn ngữ của lĩnh vực trí tuệ nhân tạo, cú pháp ngắn gọn nên thích hợp để diễn đạt ý tưởng thuật toán. Julia có chỉ số mảng bắt đầu từ 1, trùng với cách đánh số trong giả mã và trong các công thức toán học, nhờ đó việc chuyển từ công thức trên giấy sang chương trình bớt được một nguồn sai sót phổ biến.

Sinh viên nên chọn một ngôn ngữ làm ngôn ngữ chính và đọc hai ngôn ngữ còn lại như bảng đối chiếu.

**Về phần ứng dụng.** Mỗi chương kết thúc bằng một mục trình bày một thuật toán trí tuệ nhân tạo sử dụng đúng cấu trúc dữ liệu hoặc kỹ thuật vừa học. Các mục này không phải phần đọc thêm cho vui: chúng cho thấy vì sao việc chọn cấu trúc dữ liệu quyết định một hệ thống có chạy được ở quy mô thực tế hay không. Bảy ứng dụng lần lượt là: thuật toán láng giềng gần nhất, thuật toán $k$ láng giềng gần nhất, chương trình giải Sudoku, thuật toán giải mã của mô hình ngôn ngữ, bộ nhớ phát lại trong học tăng cường, thuật toán tìm đường A sao, và cây quyết định cùng thuật toán chơi cờ minimax.

**Về mã nguồn.** Toàn bộ chương trình trong tài liệu đã được biên dịch và chạy thử. Các chương trình C++ được biên dịch bằng `g++` với chuẩn C++17, các chương trình Python chạy trên Python 3.11. Kết quả của những thuật toán có lời giải kiểm chứng được đều được đối chiếu với chương trình vét cạn trên dữ liệu ngẫu nhiên. Các số liệu thực nghiệm nêu trong tài liệu, chẳng hạn số nút phải duyệt của thuật toán quay lui hay số ô phải xét của thuật toán tìm đường, đều là số đo thực tế chứ không phải ước lượng.

Mọi góp ý về nội dung và về lỗi trong tài liệu xin gửi về bộ môn phụ trách học phần.

# Mục lục

**Lời nói đầu** **i**

**1 Thuật toán và độ phức tạp tính toán** **1**

1.1 Chuẩn bị về lập trình . . . 3
1.1.1 Ba ngôn ngữ dùng trong tài liệu . . . 3
1.1.2 Chuẩn bị môi trường làm việc . . . 3
1.1.3 Chương trình đầu tiên . . . 3
1.1.4 Biến và kiểu dữ liệu . . . 4
1.1.5 Nhập dữ liệu . . . 5
1.1.6 Các phép toán số học . . . 6
1.1.7 Câu lệnh rẽ nhánh . . . 6
1.1.8 Câu lệnh lặp . . . 7
1.1.9 Mảng . . . 9
1.1.10 Hàm . . . 10
1.1.11 Đọc thông báo lỗi . . . 11
1.1.12 Khung chương trình nộp bài . . . 12
1.2 Thuật toán . . . 13
1.2.1 Khái niệm thuật toán . . . 13
1.2.2 Các đặc trưng của thuật toán . . . 14
1.2.3 Các cách mô tả thuật toán . . . 14
1.2.4 Ví dụ: thuật toán Euclid . . . 15
1.3 Độ phức tạp tính toán . . . 16
1.3.1 Cách đánh giá thời gian chạy . . . 16
1.3.2 Ký hiệu tiệm cận . . . 17
1.3.3 Quy tắc tính độ phức tạp . . . 18
1.3.4 Độ phức tạp của thuật toán đệ quy . . . 19
1.3.5 Các lớp độ phức tạp và bảng tra . . . 19
1.3.6 Trường hợp tốt nhất, xấu nhất, trung bình . . . 20
1.3.7 Độ phức tạp không gian . . . 21
1.4 Các ví dụ áp dụng . . . 21
1.4.1 Kiểm tra số nguyên tố . . . 21
1.4.2 Đoạn con liên tiếp có tổng lớn nhất . . . 22
1.4.3 Dãy Fibonacci . . . 25
1.4.4 Đo thời gian chạy thực tế . . . 27
1.5 Ứng dụng: thuật toán láng giềng gần nhất . . . 27

1.5.1 Bài toán phân lớp . . . 27
1.5.2 Thuật toán láng giềng gần nhất . . . 28
1.5.3 Đánh giá độ phức tạp . . . 30
1.6 Bài tập chương 1 . . . 31
Bài tập chương 1 . . . 31

**2 Sắp xếp và Tìm kiếm** **33**
2.1 Bài toán sắp xếp . . . 33
2.1.1 Phát biểu bài toán . . . 33
2.1.2 Ba tiêu chí đánh giá . . . 33
2.1.3 Dãy dùng chung cho các ví dụ . . . 34
2.2 Các thuật toán sắp xếp đơn giản . . . 34
2.2.1 Sắp xếp kiểu đổi chỗ . . . 34
2.2.2 Sắp xếp kiểu lựa chọn . . . 35
2.2.3 Sắp xếp kiểu chèn trực tiếp . . . 37
2.2.4 Sắp xếp kiểu nổi bọt . . . 39
2.2.5 So sánh bốn thuật toán đơn giản . . . 40
2.3 Thuật toán sắp xếp nhanh . . . 41
2.3.1 Giới thiệu thuật toán . . . 41
2.3.2 Phép phân hoạch . . . 41
2.3.3 Toàn bộ quá trình sắp xếp . . . 42
2.3.4 Cài đặt thuật toán . . . 42
2.3.5 Độ phức tạp . . . 44
2.4 Thuật toán sắp xếp kiểu cơ số . . . 45
2.4.1 Giới thiệu thuật toán . . . 45
2.4.2 Mô tả thuật toán . . . 45
2.4.3 Cài đặt thuật toán . . . 46
2.4.4 Độ phức tạp và phạm vi áp dụng . . . 48
2.5 So sánh các thuật toán sắp xếp . . . 48
2.6 Các thuật toán tìm kiếm . . . 49
2.6.1 Tìm kiếm tuyến tính . . . 49
2.6.2 Tìm kiếm nhị phân . . . 50
2.6.3 Tìm kiếm nội suy . . . 52
2.7 Ứng dụng: thuật toán $k$ láng giềng gần nhất . . . 54
2.7.1 Từ một láng giềng đến $k$ láng giềng . . . 54
2.7.2 Vì sao phải chuẩn hoá đặc trưng . . . 55
2.7.3 Cài đặt . . . 56
2.7.4 Chọn $K$ mục có điểm cao nhất . . . 58
2.8 Bài tập chương 2 . . . 60

**3 Bài toán Duyệt** **63**
3.1 Bài toán liệt kê . . . 63
3.1.1 Ba cấu hình cơ bản . . . 63
3.1.2 Thứ tự từ điển . . . 64
3.2 Thuật toán sinh kế tiếp . . . 64
3.2.1 Nguyên lý . . . 64
3.2.2 Sinh xâu nhị phân . . . 64
3.2.3 Sinh tổ hợp chập $k$ của $n$ . . . 66
3.2.4 Sinh hoán vị . . . 67
3.3 Thuật toán đệ quy . . . 69
3.3.1 Khái niệm . . . 69
3.3.2 Bài toán Tháp Hà Nội . . . 69
3.3.3 Liệt kê bằng đệ quy . . . 71
3.3.4 Đệ quy và ngăn xếp . . . 72
3.4 Thuật toán quay lui . . . 72
3.4.1 Sơ đồ chung . . . 72
3.4.2 Cây không gian trạng thái . . . 73
3.4.3 Bài toán $n$ quân hậu . . . 73
3.4.4 Hiệu quả của việc cắt nhánh . . . 75
3.5 Thuật toán nhánh và cận . . . 76
3.5.1 Ý tưởng . . . 76
3.5.2 Bài toán người du lịch . . . 76
3.5.3 Đo hiệu quả của cận . . . 78
3.6 Ứng dụng: bài toán thoả ràng buộc và chương trình giải Sudoku . . . 79
3.6.1 Bài toán thoả ràng buộc . . . 79
3.6.2 Sudoku là một bài toán thoả ràng buộc . . . 80
3.6.3 Cách thứ nhất: quay lui theo thứ tự quét . . . 80
3.6.4 Cách thứ hai: heuristic chọn biến ràng buộc nhất . . . 82
3.6.5 Đo hiệu quả của heuristic . . . 84
3.7 Bài tập chương 3 . . . 85

**4 Các mô hình thuật toán** **88**
4.1 Mô hình chia để trị . . . 88
4.1.1 Sơ đồ chung . . . 88
4.1.2 Sắp xếp trộn . . . 89
4.1.3 Tìm đồng thời giá trị nhỏ nhất và lớn nhất . . . 91
4.1.4 Luỹ thừa nhanh . . . 92
4.2 Mô hình tham lam . . . 93
4.2.1 Sơ đồ chung . . . 93

4.2.2 Bài toán đổi tiền . . . 93
4.2.3 Bài toán chọn hoạt động . . . 94
4.3 Mô hình quy hoạch động . . . 96
4.3.1 Hai điều kiện áp dụng . . . 96
4.3.2 Quy hoạch động trên mảng một chiều . . . 97
4.3.3 Quy hoạch động trên ma trận . . . 100
4.4 Ứng dụng: thuật toán giải mã của mô hình ngôn ngữ . . . 103
4.4.1 Mô hình ngôn ngữ hai từ . . . 104
4.4.2 Giải mã tham lam . . . 104
4.4.3 Giải mã theo chùm . . . 105
4.4.4 Thuật toán Viterbi . . . 107
4.5 Bài tập chương 4 . . . 111

**5 Danh sách liên kết, Ngăn xếp, Hàng đợi** **114**
5.1 Danh sách và hai cách cài đặt . . . 114
5.1.1 So sánh hai cách . . . 114
5.2 Danh sách liên kết đơn . . . 115
5.2.1 Cấu trúc nút . . . 115
5.2.2 Duyệt danh sách . . . 116
5.2.3 Thêm phần tử . . . 117
5.2.4 Xoá phần tử . . . 119
5.2.5 Đảo ngược danh sách . . . 121
5.2.6 Danh sách liên kết kép . . . 122
5.3 Ngăn xếp . . . 124
5.3.1 Định nghĩa . . . 124
5.3.2 Biểu diễn bằng mảng . . . 124
5.3.3 Ứng dụng 1: kiểm tra dấu ngoặc . . . 125
5.3.4 Ứng dụng 2: biểu thức hậu tố . . . 127
5.3.5 Ứng dụng 3: khử đệ quy . . . 129
5.4 Hàng đợi . . . 130
5.4.1 Định nghĩa . . . 130
5.4.2 Cách cài đặt sai và cách cài đặt đúng . . . 130
5.5 Ứng dụng: bộ nhớ phát lại trong học tăng cường . . . 132
5.5.1 Bài toán . . . 132
5.5.2 Yêu cầu đối với cấu trúc dữ liệu . . . 133
5.5.3 Cài đặt . . . 133
5.6 Bài tập chương 5 . . . 136

**6 Đồ thị và Đồ thị có trọng số** **139**

6.1 Khái niệm và định nghĩa . . . 139
6.2 Biểu diễn đồ thị . . . 140
6.2.1 Ma trận kề . . . 140
6.2.2 Danh sách kề . . . 140
6.2.3 Chọn cách biểu diễn nào . . . 141
6.3 Các thuật toán tìm kiếm trên đồ thị . . . 142
6.3.1 Tìm kiếm theo chiều sâu . . . 142
6.3.2 Tìm kiếm theo chiều rộng . . . 143
6.4 Đồ thị có trọng số và bài toán đường đi ngắn nhất . . . 146
6.4.1 Biểu diễn đồ thị có trọng số . . . 146
6.4.2 Vì sao tìm kiếm theo chiều rộng không đủ . . . 146
6.4.3 Thuật toán Dijkstra . . . 147
6.5 Bài toán cây bao trùm nhỏ nhất . . . 150
6.5.1 Thuật toán Prim . . . 150
6.5.2 Thuật toán Kruskal . . . 151
6.6 Ứng dụng: thuật toán tìm đường A sao . . . 153
6.6.1 Bài toán . . . 153
6.6.2 Ý tưởng của A sao . . . 153
6.6.3 Cài đặt . . . 154
6.6.4 Đo hiệu quả . . . 156
6.7 Bài tập chương 6 . . . 158

**7 Cây nhị phân** **161**
7.1 Định nghĩa và phân loại . . . 161
7.1.1 Định nghĩa . . . 161
7.1.2 Phân loại . . . 162
7.2 Biểu diễn cây nhị phân . . . 162
7.2.1 Biểu diễn bằng danh sách liên kết . . . 162
7.2.2 Biểu diễn bằng mảng . . . 163
7.3 Thao tác trên cây nhị phân tổng quát . . . 163
7.3.1 Ba cách duyệt . . . 163
7.3.2 Duyệt theo mức . . . 165
7.3.3 Các thao tác đệ quy khác . . . 165
7.4 Cây nhị phân tìm kiếm . . . 166
7.4.1 Tìm kiếm . . . 166
7.4.2 Chèn . . . 167
7.4.3 Xoá . . . 167
7.4.4 Vấn đề chiều cao . . . 169
7.5 Cây nhị phân tìm kiếm cân bằng . . . 170

7.5.1 Phép quay . . . 170
7.6 Cây nhị phân gần đầy và đống . . . 174
7.6.1 Hai thao tác cơ bản . . . 174
7.7 Ứng dụng: cây quyết định và thuật toán chơi cờ . . . 177
7.7.1 Cây quyết định . . . 177
7.7.2 Thuật toán minimax . . . 180
7.7.3 Cắt tỉa alpha-beta . . . 180
7.7.4 Đo hiệu quả cắt tỉa . . . 183
7.8 Bài tập chương 7 . . . 184

**Tài liệu tham khảo** **187**

# Danh sách hình vẽ


1.1 Biến là một ô nhớ có tên. Lệnh gán ghi giá trị vào ô đó, ghi đè giá trị cũ nếu có. . . 4

1.2 Cùng một mảng năm phần tử, hai cách đánh chỉ số. Nhầm lẫn giữa hai cách này sinh ra lỗi lệch một đơn vị, loại lỗi khiến sinh viên mất điểm nhiều nhất trong bài thi thực hành. . . . 9

1.3 Sơ đồ khối thuật toán Euclid tìm ước chung lớn nhất, kèm chú giải các ký hiệu chuẩn. . . . 14

1.4 Tốc độ tăng của các lớp độ phức tạp, trục tung theo thang lô-ga-rit. Đường $2^n$ vượt khỏi khung hình khi $n$ mới chỉ khoảng 12, còn $\log_2 n$ gần như nằm ngang. . . . 18

1.5 Vòng lặp tam giác. Đây là dạng của cả ba thuật toán sắp xếp cơ bản trong chương 2. 19

1.6 Đoạn con liên tiếp có tổng lớn nhất của dãy $-2, 3, -1, 4, -5, 2$. . . . 22

1.7 Số phép toán của ba lời giải cho bài toán đoạn con có tổng lớn nhất, với $n = 10^5$. . . 25

1.8 Cây các lời gọi khi tính `fib(5)` bằng đệ quy thuần. . . . 25

1.9 Bộ dữ liệu hoa diên vĩ rút gọn. Mẫu mới được gán nhãn của điểm gần nó nhất. . . 28

2.1 Một bước của sắp xếp chèn trực tiếp: chèn giá trị 2 vào đoạn đã sắp xếp $(1, 3, 5, 8, 9)$. 37

2.2 Một lần phân hoạch dãy $(6, 1, 7, 5, 3, 8, 2, 4)$ với chốt lấy ở giữa đoạn. . . . 42

2.3 Cây các lời gọi đệ quy của sắp xếp nhanh trên dãy $(6, 1, 7, 5, 3, 8, 2, 4)$. Ô viền mờ là đoạn có một phần tử, dừng ngay. . . . 43

2.4 Lượt phân phối theo hàng đơn vị của sắp xếp cơ số. Các số vào cùng thùng giữ nguyên thứ tự tương đối. . . . 46

2.5 Tìm kiếm nhị phân giá trị 23. Mỗi bước loại bỏ khoảng một nửa số ứng viên còn lại. 50

2.6 Cùng một tập dữ liệu trước và sau khi chuẩn hoá min-max. Hình sao là mẫu cần phân loại; hình tròn là nhãn "chưa đạt", hình vuông là nhãn "đạt". Ba đoạn nét đứt nối mẫu mới với ba láng giềng gần nhất sau chuẩn hoá, cả ba đều mang nhãn "đạt". . . . 56

3.1 Sinh xâu nhị phân kế tiếp: từ 0111 sang 1000. . . . 64

3.2 Sinh hoán vị kế tiếp: từ $(1, 3, 5, 4, 2)$ sang $(1, 4, 2, 3, 5)$. . . . 67

3.3 Phân rã đệ quy của bài toán Tháp Hà Nội. . . . 70

3.4 Đánh số hai loại đường chéo trên bàn cờ $4 \times 4$ và một trong hai lời giải của bài toán bốn quân hậu. . . . 73

3.5 Nguyên tắc cắt của nhánh và cận. Cây con dưới nút bị cắt không bao giờ được sinh ra. . . . 76

3.6 Đề Sudoku dùng trong mục này, có 30 số cho trước và 51 ô trống. Hai ô trống được chú thích có số ứng viên rất khác nhau: một ô chỉ còn một ứng viên, ô kia còn sáu. Heuristic MRV luôn chọn ô ít ứng viên nhất, tức ô tô cam, thay vì ô trống đầu tiên gặp khi quét từ trái sang phải. . . . 80

4.1 Cây đệ quy của một thuật toán chia để trị với $a = 2$, $b = 2$, $d = 1$. Mỗi mức tốn $O(n)$, có $\log_2 n$ mức. . . . 89
4.2 Một bước của phép trộn hai dãy đã sắp xếp. . . . 89
4.3 Mười một hoạt động sắp theo thời điểm kết thúc tăng dần. Bốn hoạt động tô đậm là lời giải của thuật toán tham lam. . . . 95
4.4 Ma trận chi phí $C$ và bảng $f$ tương ứng. Các ô tô đậm tạo thành đường đi có tổng chi phí nhỏ nhất, bằng 9. . . . 102
4.5 Cây các câu có thể sinh ra. Tham lam đi theo nhánh đậm màu xanh đậm và bỏ lỡ nhánh tốt nhất vì bước đầu tiên của nhánh đó có xác suất thấp hơn. . . . 104
4.6 Lưới Viterbi cho câu "tôi ăn cơm". Mỗi ô giữ xác suất lớn nhất của một đường đi tới ô đó; đường đậm là đường được truy vết. . . . 108

5.1 Hai cách cài đặt danh sách. Số nhỏ màu xám là địa chỉ ô nhớ. . . . 114
5.2 Chèn nút mới sau nút $p$. Đổi thứ tự hai phép gán sẽ làm mất phần đuôi của danh sách. . . . 118
5.3 Xoá nút $p$ khỏi danh sách liên kết đơn: nối nút đứng trước thẳng vào nút đứng sau. 119
5.4 Danh sách liên kết kép: mỗi nút có hai liên kết. . . . 122
5.5 Ba trạng thái của ngăn xếp. Mọi thao tác chỉ xảy ra ở đỉnh. . . . 124
5.6 Hàng đợi vòng sức chứa 5 sau khi thêm $1..5$, lấy ra một phần tử, rồi thêm giá trị 6. Giá trị mới quay về ô 0. . . . 130
5.7 Bộ nhớ phát lại sức chứa 5 sau khi thêm lần lượt các giá trị $1..8$. Ba giá trị cũ nhất đã bị ghi đè. . . . 133

6.1 Đồ thị vô hướng dùng làm ví dụ trong chương: 8 đỉnh, 10 cạnh. . . . 140
6.2 Danh sách kề của đồ thị ở hình 6.1. . . . 141
6.3 Tìm kiếm theo chiều rộng chia đồ thị thành các mức theo khoảng cách tới đỉnh xuất phát. . . . 143
6.4 Đồ thị có trọng số dùng trong mục này. Số trên mỗi cạnh là trọng số. . . . 147
6.5 Cây bao trùm nhỏ nhất của đồ thị ở hình 6.4. . . . 153
6.6 Dijkstra và A sao trên cùng bản đồ. Cả hai cho đường dài 18 bước, nhưng A sao chỉ xét 75 ô so với 114 ô. . . . 157

7.1 Cây nhị phân dùng làm ví dụ trong chương. . . . 161
7.2 Cây gần đầy và mảng biểu diễn nó. Không có ô trống nào, và quan hệ cha con tính được bằng phép số học trên chỉ số. . . . 163
7.3 Ba bước xoá một nút có hai con khỏi cây nhị phân tìm kiếm. . . . 168
7.4 Phép quay phải. Chỉ ba liên kết thay đổi, nên phép quay tốn $O(1)$. . . . 170
7.5 Thêm giá trị 20 vào đống cực đại. Phần tử mới đi lên dọc theo đường từ lá về gốc. . 174
7.6 Cây quyết định học được từ tập dữ liệu ở bảng 7.6. . . . 178
7.7 Một lần cắt tỉa alpha-beta. Hai nút đánh dấu không bao giờ được sinh ra. . . . 181

# Danh sách bảng


1.1 Vai trò của ba ngôn ngữ trong giáo trình. . . . 3
1.2 Các kiểu dữ liệu cơ bản. . . . 5
1.3 Các phép toán số học và khác biệt giữa ba ngôn ngữ. . . . 6
1.4 Bảng đối chiếu các thao tác trên mảng. . . . 10
1.5 Các lỗi hay gặp trong tuần đầu và thông báo tương ứng. . . . 12
1.6 Sáu đặc trưng của thuật toán. . . . 14
1.7 Lần vết thuật toán Euclid với $a = 48$, $b = 18$. . . . 16
1.8 Thời gian chạy ước tính với $10^8$ phép toán mỗi giây. . . . 17
1.9 Các lớp độ phức tạp thường gặp. . . . 20
1.10 Bảng tra giới hạn dữ liệu. Dùng bảng này *trước khi* viết dòng mã đầu tiên. . . . 20
1.11 Ý nghĩa thực tế của độ phức tạp $O(n \cdot d)$. . . . 30

2.1 Lần vết thuật toán sắp xếp kiểu đổi chỗ trên dãy $(8, 3, 5, 1, 9, 2)$. . . . 35
2.2 Lần vết thuật toán sắp xếp kiểu lựa chọn trên dãy $(8, 3, 5, 1, 9, 2)$. . . . 36
2.3 Lần vết thuật toán sắp xếp chèn trực tiếp trên dãy $(8, 3, 5, 1, 9, 2)$. . . . 38
2.4 Lần vết thuật toán sắp xếp nổi bọt trên dãy $(8, 3, 5, 1, 9, 2)$. . . . 39
2.5 Bốn thuật toán sắp xếp đơn giản trên dãy $(8, 3, 5, 1, 9, 2)$ và trong trường hợp tổng quát. . . . 41
2.6 Lần vết phép phân hoạch đoạn $[0..7]$ của dãy $(6, 1, 7, 5, 3, 8, 2, 4)$, chốt bằng 5. . . . 42
2.7 Toàn bộ các lần phân hoạch khi sắp xếp nhanh dãy $(6, 1, 7, 5, 3, 8, 2, 4)$. . . . 42
2.8 Thời gian sắp xếp đo được, đơn vị mi-li-giây. . . . 48
2.9 Lần vết tìm kiếm nhị phân trên dãy $(2, 5, 8, 12, 16, 23, 38, 56, 72, 91)$. . . . 51
2.10 So sánh ba thuật toán tìm kiếm. . . . 54
2.11 Tập huấn luyện của ví dụ phân loại. . . . 55
2.12 Khoảng cách từ mẫu $(14, 800)$ tới tám mẫu huấn luyện, trước và sau khi chuẩn hoá. Ba mẫu gần nhất trong mỗi cột được in đậm. . . . 55

3.1 Ba loại cấu hình cơ bản và số lượng của chúng. . . . 63
3.2 Số cấu hình và thời gian duyệt hết, với tốc độ $10^8$ cấu hình mỗi giây. . . . 63
3.3 Giới hạn độ sâu đệ quy mặc định. . . . 72
3.4 Bài toán $n$ quân hậu: số lời giải và số nút phải duyệt. . . . 75
3.5 Ma trận chi phí của ví dụ 8 thành phố. . . . 79
3.6 Số nút phải duyệt của bài toán người du lịch với 8 thành phố. . . . 79
3.7 Số lần gọi hàm đệ quy khi giải cùng một đề Sudoku 51 ô trống. . . . 84

4.1 Ba mô hình thuật toán và điều kiện áp dụng. . . . 88

4.2 Mô hình ngôn ngữ dùng trong ví dụ. Mỗi hàng có tổng bằng 1. . . . . 104
4.3 Toàn bộ sáu câu độ dài ba và điểm của chúng. . . . . 104
4.4 Ba thuật toán giải mã trong mục này. . . . . 110

5.1 Độ phức tạp các thao tác trên hai cách cài đặt danh sách. . . . . 115
5.2 Thời gian chèn $n$ phần tử vào *đầu* danh sách, đơn vị mi-li-giây. . . . . 115
5.3 So sánh danh sách liên kết đơn và kép. . . . . 124
5.4 Vì sao mảng vòng là lựa chọn duy nhất thoả cả ba yêu cầu. . . . . 135

6.1 Các loại đồ thị. . . . . 139
6.2 Ma trận kề của đồ thị ở hình 6.1. . . . . 140
6.3 So sánh hai cách biểu diễn đồ thị. . . . . 141
6.4 So sánh hai thuật toán duyệt. . . . . 146
6.5 So sánh Prim và Kruskal. . . . . 153
6.6 Số ô được lấy ra khỏi hàng đợi trên bản đồ $9 \times 20$. . . . . 156

7.1 Các loại cây nhị phân. . . . . 162
7.2 Ba cách duyệt cây nhị phân và kết quả trên cây ở hình 7.1. . . . . 164
7.3 Chiều cao đo được khi chèn 100 giá trị khác nhau. . . . . 169
7.4 Bốn tình huống mất cân bằng và cách xử lý. . . . . 170
7.5 So sánh ba cấu trúc lưu tập giá trị có thứ tự. . . . . 173
7.6 Tập dữ liệu huấn luyện: có đi chơi hay không. . . . . 177
7.7 Số nút phải sinh ra khi giải cờ ca-rô $3 \times 3$ từ bàn cờ trống. . . . . 183

CHƯƠNG 1

# Thuật toán và độ phức tạp tính toán

> **Nội dung chương.** Chương này có hai nhiệm vụ. Nhiệm vụ thứ nhất là trang bị đủ kiến thức lập trình để người chưa từng viết chương trình nào vẫn theo được các chương sau. Nhiệm vụ thứ hai là nội dung chính của tuần học đầu tiên theo đề cương: khái niệm thuật toán, các cách mô tả thuật toán, và phương pháp đánh giá độ phức tạp tính toán. Phần cuối chương xây dựng một chương trình phân lớp trong trí tuệ nhân tạo chỉ dùng vòng lặp và phép so sánh.

Sau khi học xong chương này, sinh viên cần làm được ba việc. Thứ nhất, viết và chạy được chương trình đơn giản bằng ít nhất một trong ba ngôn ngữ Julia, Python và C++. Thứ hai, nhìn một đoạn chương trình và xác định được độ phức tạp của nó theo ký hiệu $O$ lớn. Thứ ba, khi nhìn giới hạn dữ liệu trong đề bài, ước lượng được thuật toán nào chạy kịp và thuật toán nào không.

## 1.1 Chuẩn bị về lập trình

Mục này dành cho sinh viên chưa có kinh nghiệm lập trình. Sinh viên đã học xong học phần Tin học cơ sở 2 nên đọc lướt, chỉ dừng lại ở mục 1.1.6 và mục 1.1.9 vì hai mục đó nêu những khác biệt giữa ba ngôn ngữ mà tài liệu sẽ dùng liên tục.

### 1.1.1 Ba ngôn ngữ dùng trong tài liệu

Đề cương học phần không ràng buộc ngôn ngữ cài đặt. Bảng 1.1 tóm tắt lý do chọn ba ngôn ngữ và vai trò của từng ngôn ngữ trong tài liệu.

**Bảng 1.1.** Vai trò của ba ngôn ngữ trong giáo trình.

| Ngôn ngữ | Điểm mạnh | Dùng trong tài liệu để |
|---|---|---|
| C++ | Chạy nhanh, thư viện chuẩn đầy đủ, kiểm soát bộ nhớ ở mức thấp | Chuẩn bị cho kỳ thi thực hành; hiểu con trỏ và cấp phát động ở chương 5 và 7 |
| Python | Cú pháp ngắn gọn, là ngôn ngữ chuẩn của lĩnh vực trí tuệ nhân tạo | Diễn đạt ý tưởng thuật toán; toàn bộ phần ứng dụng cuối mỗi chương |
| Julia | Chỉ số mảng bắt đầu từ 1, trùng với giả mã; tốc độ gần C | Đối chiếu trực tiếp giữa công thức toán học và chương trình |

Ba ngôn ngữ giống nhau đến mức đáng kể ở phần cơ bản. Vòng lặp, rẽ nhánh và hàm gần như trùng nhau về ý nghĩa, chỉ khác cách viết. Học kỹ một ngôn ngữ thì hai ngôn ngữ còn lại chỉ mất vài giờ để làm quen.

### 1.1.2 Chuẩn bị môi trường làm việc

Sinh viên có thể cài đặt trên máy cá nhân, hoặc dùng công cụ trực tuyến nếu máy yếu. Với C++, tài liệu dùng chuẩn C++17 và trình biên dịch `g++`. Lệnh biên dịch và chạy như sau:

```
g++ -std=c++17 -O2 -o chuongtrinh chuongtrinh.cpp
./chuongtrinh
```

Tuỳ chọn `-O2` bật chế độ tối ưu của trình biên dịch. Khi đo thời gian chạy để so sánh các thuật toán, nhất định phải bật tuỳ chọn này; nếu không, kết quả đo chậm hơn thực tế nhiều lần và dẫn tới kết luận sai.

### 1.1.3 Chương trình đầu tiên

JULIA

```
println("Chuong trinh dau tien")
```

PYTHON

```
print("Chuong trinh dau tien")
```

C++

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Chuong trinh dau tien" << endl;
    return 0;
}
```

Bản Julia và Python chỉ có một dòng. Bản C++ dài hơn vì ngôn ngữ này đòi hỏi khai báo tường minh. Từng dòng có ý nghĩa như sau.

Dòng `#include <iostream>` yêu cầu trình biên dịch nạp thư viện nhập xuất chuẩn. Thiếu dòng này thì `cout` chưa được định nghĩa và chương trình không biên dịch được.

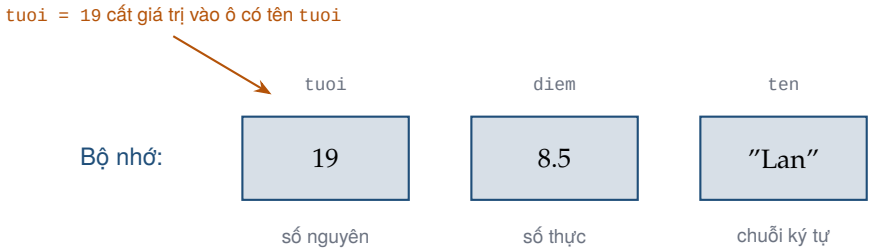
Dòng `using namespace std;` cho phép viết `cout` thay vì `std::cout`. Trong các dự án lớn người ta tránh dòng này để không gây xung đột tên, nhưng với bài tập và bài thi thì nó rút ngắn mã nguồn đáng kể.

Hàm `main` là nơi chương trình bắt đầu chạy. Cặp ngoặc nhọn bao lấy thân hàm. Toán tử `<<` đưa dữ liệu ra màn hình và có thể nối liên tiếp nhiều lần. Lệnh `return 0;` báo cho hệ điều hành biết chương trình kết thúc bình thường.

Mỗi câu lệnh trong C++ phải kết thúc bằng dấu chấm phẩy. Thiếu dấu chấm phẩy là lỗi biên dịch phổ biến nhất của người mới học.

### 1.1.4 Biến và kiểu dữ liệu

Biến là một vùng nhớ có tên, dùng để lưu một giá trị. Lệnh gán có dạng `tên_biến = giá_trị`, nghĩa là lấy giá trị bên phải cất vào vùng nhớ bên trái. Hình 1.1 minh hoạ ý nghĩa đó.




**Hình 1.1.** Biến là một ô nhớ có tên. Lệnh gán ghi giá trị vào ô đó, ghi đè giá trị cũ nếu có.

Dấu bằng ở đây không mang nghĩa so sánh như trong toán học. Vì thế lệnh `x = x + 1` là hợp lệ: lấy giá trị hiện có của `x`, cộng thêm 1, rồi cất trở lại vào chính `x`.

JULIA

```julia
tuoi = 19
diem = 8.5
ten = "Lan"
dat = true

println(ten, " ", tuoi, " tuoi, diem ", diem)
```

PYTHON

```python
tuoi = 19
diem = 8.5
ten = "Lan"
dat = True

print(ten, tuoi, "tuoi, diem", diem)
```

C++

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    int tuoi = 19;
    double diem = 8.5;
    string ten = "Lan";
    bool dat = true;

    cout << ten << " " << tuoi << " tuoi, diem " << diem << endl;
    return 0;
}
```

Khác biệt cơ bản giữa ba ngôn ngữ nằm ở việc khai báo kiểu. Trong C++, mỗi biến phải được khai báo kiểu trước khi dùng và kiểu đó không đổi trong suốt vòng đời của biến. Trong Julia và Python, kiểu được xác định tự động từ giá trị gán vào.

**Bảng 1.2.** Các kiểu dữ liệu cơ bản.

| Ý nghĩa | Julia | Python | C++ |
|---|---|---|---|
| Số nguyên | `Int` (64 bit) | `int` (không giới hạn) | `int` (32 bit), `long long` (64 bit) |
| Số thực | `Float64` | `float` | `double` |
| Chuỗi | `String` | `str` | `string` |
| Ký tự đơn | `Char`, viết `'a'` | không phân biệt với chuỗi | `char`, viết `'a'` |
| Logic | `true` / `false` | `True` / `False` | `true` / `false` |

**Tràn số.** Kiểu `int` trong C++ chiếm 32 bit, chứa được các số từ khoảng $-2{,}1 \times 10^9$ đến $2{,}1 \times 10^9$. Khi kết quả vượt quá khoảng này, giá trị bị quay vòng và trở thành số âm; chương trình vẫn chạy nhưng cho kết quả sai và rất khó phát hiện. Khi có nguy cơ vượt ngưỡng, phải dùng `long long`. Julia cũng có giới hạn tương tự với `Int` 64 bit. Python không gặp vấn đề này vì số nguyên của Python không bị giới hạn độ lớn.

### 1.1.5 Nhập dữ liệu

Các bài tập trên cổng luyện tập trực tuyến đều lấy dữ liệu từ luồng vào chuẩn.

JULIA

```julia
n = parse(Int, readline())                # đọc một số nguyên trên một dòng
a = parse.(Int, split(readline()))        # đọc nhiều số trên cùng một dòng
```

PYTHON

```python
n = int(input())                          # đọc một số nguyên
a = list(map(int, input().split()))       # đọc nhiều số trên cùng một dòng
```

C++

```cpp
int n;
cin >> n;                                 // đọc một số nguyên

vector<int> a(n);
for (int i = 0; i < n; i++) cin >> a[i];  // đọc n số
```

Trong C++, toán tử >> tự bỏ qua khoảng trắng và dấu xuống dòng, nên không cần phân biệt dữ liệu nằm trên một dòng hay nhiều dòng. Trong Julia và Python, hàm đọc lấy trọn một dòng nên phải tự tách bằng hàm `split`.

### 1.1.6 Các phép toán số học

Mục này ngắn nhưng quan trọng, vì phép chia hoạt động khác nhau ở ba ngôn ngữ và đây là nguồn lỗi thường xuyên.

**Bảng 1.3.** Các phép toán số học và khác biệt giữa ba ngôn ngữ.

| Phép toán | Julia | Python | C++ |
|---|---|---|---|
| Cộng, trừ, nhân | `+ - *` | `+ - *` | `+ - *` |
| Chia lấy phần nguyên | `7 ÷ 2` hoặc `div(7,2)` | `7 // 2` | `7 / 2` khi cả hai là số nguyên |
| Chia lấy kết quả thực | `7 / 2` | `7 / 2` | `7.0 / 2` |
| Lấy phần dư | `7 % 2` | `7 % 2` | `7 % 2` |
| Luỹ thừa | `2^10` | `2 ** 10` | `pow(2, 10)` |

Ba dòng đầu cần đọc kỹ. Trong C++, biểu thức `7 / 2` cho kết quả `3`, vì cả hai toán hạng đều là số nguyên nên trình biên dịch dùng phép chia nguyên. Muốn có `3.5`, ít nhất một toán hạng phải là số thực. Ngược lại, trong Julia và Python thì `7 / 2` luôn cho `3.5`, và muốn có `3` thì phải dùng `÷` hoặc `//`.

Lỗi hay gặp là viết `(a + b) / 2` để tính trung bình cộng hai số nguyên trong C++ rồi nhận kết quả bị cắt phần thập phân; hoặc viết `n / 2` trong Julia để chia đôi chỉ số mảng rồi nhận về số thực và gặp lỗi khi dùng làm chỉ số.

Ký hiệu `÷` trong Julia được gõ bằng cách gõ `\div` rồi nhấn phím Tab. Nếu bàn phím không tiện, có thể dùng `div(a, b)`.

### 1.1.7 Câu lệnh rẽ nhánh

JULIA

```julia
diem = 8.5

if diem >= 8.0
    println("Gioi")
elseif diem >= 6.5
    println("Kha")
else
    println("Trung binh")
end
```

PYTHON

```python
diem = 8.5

if diem >= 8.0:
    print("Gioi")
elif diem >= 6.5:
    print("Kha")
else:
    print("Trung binh")
```

C++

```cpp
double diem = 8.5;

if (diem >= 8.0) {
    cout << "Gioi" << endl;
} else if (diem >= 6.5) {
    cout << "Kha" << endl;
} else {
    cout << "Trung binh" << endl;
}
```

Ba ngôn ngữ đánh dấu phạm vi khối lệnh theo ba cách khác nhau. Julia dùng từ khoá `end` để đóng khối. Python dùng độ thụt đầu dòng, quy ước chuẩn là bốn dấu cách; thụt sai là chương trình sai. C++ dùng cặp ngoặc nhọn và bắt buộc đặt điều kiện trong ngoặc tròn.

Các phép so sánh giống nhau ở cả ba ngôn ngữ: `==` bằng, `!=` khác, cùng với `<`, `>`, `<=`, `>=`. Cần phân biệt `=` là phép gán còn `==` là phép so sánh. Trong C++, viết nhầm `if (x = 5)` không gây lỗi biên dịch mà lặng lẽ gán 5 cho `x` rồi coi điều kiện là đúng, nên đây là lỗi khó phát hiện.

### 1.1.8 Câu lệnh lặp

Ví dụ dưới đây tính tổng các số từ 1 đến $n$.

JULIA

```julia
function tong_1_den_n(n)
    tong = 0
    for i in 1:n              # i lần lượt nhận 1, 2, ..., n
        tong = tong + i
    end
    return tong
end

println(tong_1_den_n(10))     # 55
```

PYTHON

```python
def tong_1_den_n(n):
    tong = 0
    for i in range(1, n + 1):     # i nhận 1, 2, ..., n; số n+1 không lấy
        tong = tong + i
    return tong

print(tong_1_den_n(10))           # 55
```

C++

```cpp
long long tong_1_den_n(int n) {
    long long tong = 0;
    for (int i = 1; i <= n; i++) {    // khởi tạo; điều kiện; bước nhảy
        tong = tong + i;
    }
    return tong;
}
```

Cách viết cận của vòng lặp là chỗ dễ nhầm khi dùng nhiều ngôn ngữ. Julia viết `1:n` và lấy cả giá trị `n`. Python viết `range(1, n+1)` và không lấy giá trị cuối, nên muốn chạy đến `n` phải ghi `n+1`. C++ có ba thành phần trong ngoặc tròn: khởi tạo biến đếm, điều kiện để tiếp tục, và bước nhảy sau mỗi lần lặp.

Khi số lần lặp chưa biết trước, ta dùng vòng lặp `while`. Ví dụ sau đếm xem phải chia đôi một số bao nhiêu lần thì về 1.

JULIA

```julia
function dem_chia_doi(n)
    so_lan = 0
    while n > 1
        n = n ÷ 2
        so_lan = so_lan + 1
    end
    return so_lan
end

println(dem_chia_doi(100))    # 6
```

PYTHON

```python
def dem_chia_doi(n):
    so_lan = 0
    while n > 1:
        n = n // 2
        so_lan = so_lan + 1
    return so_lan

print(dem_chia_doi(100))      # 6
```

C++

```cpp
int dem_chia_doi(int n) {
    int so_lan = 0;
    while (n > 1) {
```

```
        n = n / 2;
        so_lan = so_lan + 1;
    }
    return so_lan;
}
```

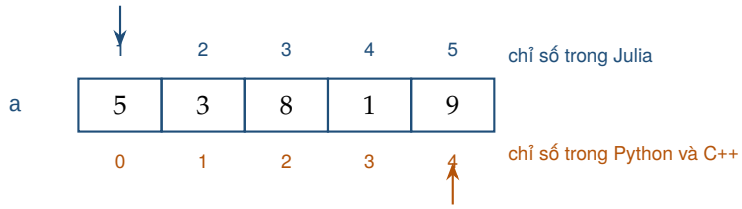
Kết quả bằng 6 vì dãy $100, 50, 25, 12, 6, 3, 1$ có sáu bước chia. Đây là một quan sát sẽ được dùng lại nhiều lần trong chương: khi biến đếm bị chia đôi mỗi vòng lặp thì số vòng lặp xấp xỉ $\log_2 n$.

Hai câu lệnh điều khiển vòng lặp có ở cả ba ngôn ngữ: `break` thoát khỏi vòng lặp ngay lập tức, `continue` bỏ qua phần còn lại của lần lặp hiện tại và sang lần tiếp theo.

Vòng lặp `while` mà điều kiện không bao giờ sai sẽ chạy mãi mãi. Đây là lỗi thường gặp nhất của người mới học và cũng dễ nhận ra: chương trình không dừng và không in gì thêm. Nhấn tổ hợp phím Ctrl + C để dừng.

### 1.1.9 Mảng

Mảng là dãy các ô nhớ liền nhau, được đánh số thứ tự, dùng chung một tên. Muốn lưu điểm của 50 sinh viên, ta không đặt 50 tên biến mà dùng một mảng 50 phần tử. Hình 1.2 minh hoạ điểm khác biệt quan trọng nhất giữa ba ngôn ngữ.




**Hình 1.2.** Cùng một mảng năm phần tử, hai cách đánh chỉ số. Nhầm lẫn giữa hai cách này sinh ra lỗi lệch một đơn vị, loại lỗi khiến sinh viên mất điểm nhiều nhất trong bài thi thực hành.

**JULIA**

```
a = [5, 3, 8, 1, 9]

println(a[1])          # 5      phần tử đầu tiên có chỉ số 1
println(a[5])          # 9      phần tử cuối
println(a[end])        # 9      cách viết khác cho phần tử cuối
println(length(a))     # 5      số phần tử

a[2] = 100             # sửa phần tử thứ hai
push!(a, 7)            # thêm 7 vào cuối mảng
b = zeros(Int, 10)     # mảng 10 số nguyên, tất cả bằng 0
```

**PYTHON**

```
a = [5, 3, 8, 1, 9]

print(a[0])            # 5      phần tử đầu tiên có chỉ số 0
print(a[4])            # 9
print(a[-1])           # 9      chỉ số âm đếm từ cuối
print(len(a))          # 5

a[1] = 100             # sửa phần tử thứ hai
a.append(7)            # thêm 7 vào cuối
b = [0] * 10           # mảng 10 số 0
```

C++

```cpp
#include <vector>
using namespace std;

vector<int> a = {5, 3, 8, 1, 9};

cout << a[0] << endl;        // 5    chỉ số bắt đầu từ 0
cout << a[4] << endl;        // 9
cout << a.back() << endl;    // 9    phần tử cuối
cout << a.size() << endl;    // 5

a[1] = 100;                  // sửa phần tử thứ hai
a.push_back(7);              // thêm 7 vào cuối
vector<int> b(10, 0);        // mảng 10 số 0
```

**Bảng 1.4.** Bảng đối chiếu các thao tác trên mảng.

| Việc cần làm | Julia | Python | C++ |
|---|---|---|---|
| Phần tử đầu | `a[1]` | `a[0]` | `a[0]` |
| Phần tử cuối | `a[end]` | `a[-1]` | `a.back()` |
| Số phần tử | `length(a)` | `len(a)` | `a.size()` |
| Duyệt theo chỉ số | `for i in 1:n` | `for i in range(n)` | `for (int i=0; i<n; i++)` |
| Duyệt theo giá trị | `for x in a` | `for x in a` | `for (int x : a)` |
| Duyệt ngược | `for i in n:-1:1` | `for i in range(n-1,-1,-1)` | `for (int i=n-1; i>=0; i--)` |

**Truy cập ngoài phạm vi mảng.** Trong C++, truy cập phần tử ngoài phạm vi mảng không gây lỗi ngay mà đọc vào một vùng nhớ bất kỳ. Chương trình có thể chạy được nhưng cho kết quả sai, hoặc dừng đột ngột ở một chỗ hoàn toàn khác với chỗ sai thật. Julia và Python báo lỗi ngay tại dòng sai. Đây là lý do khi mới học, việc gỡ lỗi bằng Python hoặc Julia thường nhanh hơn.

### 1.1.10 Hàm

Hàm là đoạn chương trình có tên, nhận vào một số giá trị và trả về kết quả. Viết hàm giúp tránh lặp lại cùng một đoạn mã và giúp chia bài toán lớn thành các phần nhỏ dễ kiểm tra.

JULIA

```julia
function dien_tich_hcn(dai, rong)
    return dai * rong
end

println(dien_tich_hcn(5, 3))        # 15
```

PYTHON

```python
def dien_tich_hcn(dai, rong):
    return dai * rong

print(dien_tich_hcn(5, 3))          # 15
```

C++

```cpp
double dien_tich_hcn(double dai, double rong) {
    return dai * rong;
}
```

Trong C++, hàm phải khai báo kiểu của giá trị trả về và kiểu của từng tham số. Hàm không trả về gì thì kiểu trả về là `void`.

Một khác biệt quan trọng của C++ là cách truyền tham số. Mặc định, tham số được truyền theo giá trị, nghĩa là hàm nhận một bản sao và mọi thay đổi bên trong hàm không ảnh hưởng tới biến gốc. Muốn hàm sửa được biến gốc, phải thêm dấu `&` để truyền theo tham chiếu.

C++

```cpp
void tang_gap_doi(int x)      { x = x * 2; }   // không có tác dụng ra ngoài
void tang_gap_doi_ref(int &x) { x = x * 2; }   // sửa được biến gốc

int x = 10;
tang_gap_doi(x);       // x vẫn bằng 10
tang_gap_doi_ref(x);   // x bằng 20
```

Python không có khái niệm tham chiếu tường minh, nhưng có một khác biệt cùng tác dụng: số nguyên và chuỗi không sửa được tại chỗ, còn danh sách thì sửa được. Vì vậy gán lại tham số bên trong hàm không có tác dụng ra ngoài, nhưng sửa phần tử của danh sách thì có.

PYTHON

```python
def gan_lai(a):
    a = [9, 9, 9]        # chỉ đổi tên a bên trong hàm

def sua_phan_tu(a):
    a[0] = 9             # sửa đúng danh sách gốc

x = [1, 2, 3]
gan_lai(x)
print(x)                 # [1, 2, 3]
sua_phan_tu(x)
print(x)                 # [9, 2, 3]
```

Julia hoạt động giống Python ở điểm này, và có thêm một quy ước đặt tên: hàm nào sửa tham số đầu vào thì tên kết thúc bằng dấu chấm than. Chẳng hạn `sort(a)` trả về mảng mới và giữ nguyên `a`, còn `sort!(a)` sắp xếp trực tiếp trên `a`. Quên dấu chấm than là lỗi khiến chương trình chạy nhưng không có tác dụng gì.

Ngoài ra, trong Julia còn một lý do kỹ thuật để đặt mọi thứ vào trong hàm: mã lệnh nằm trong hàm được biên dịch tối ưu và chạy nhanh hơn nhiều lần so với mã lệnh viết ở phạm vi toàn cục.

### 1.1.11 Đọc thông báo lỗi

Chương trình báo lỗi là chuyện bình thường. Điều cần rèn là đọc được thông báo lỗi để biết sai ở đâu.

Với lỗi biên dịch của C++, cần đọc thông báo **đầu tiên** chứ không phải thông báo cuối. Một lỗi cú pháp thường kéo theo hàng chục thông báo phụ; sửa lỗi đầu tiên xong thì phần lớn các thông báo còn lại tự biến mất.

**Bảng 1.5.** Các lỗi hay gặp trong tuần đầu và thông báo tương ứng.

| Tình huống | Julia | Python | C++ |
|---|---|---|---|
| Dùng biến chưa khai báo | `UndefVarError` | `NameError` | `error: not declared` |
| Truy cập ngoài mảng | `BoundsError` | `IndexError` | không báo lỗi, chạy sai |
| Sai kiểu dữ liệu | `MethodError` | `TypeError` | `error: invalid operands` |
| Thiếu dấu đóng khối | `syntax: incomplete` | `IndentationError` | `error: expected '}'` |
| Chia cho 0 (số nguyên) | `DivideError` | `ZeroDivisionError` | chương trình dừng đột ngột |

### 1.1.12 Khung chương trình nộp bài

Các bài tập của học phần thường có dạng: dòng đầu ghi số bộ dữ liệu kiểm thử, mỗi bộ gồm một vài dòng dữ liệu.

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

void solve() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    // xử lý
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int t;
    cin >> t;
    while (t--) solve();
    return 0;
}
```

Hai dòng `ios_base::sync_with_stdio(false)` và `cin.tie(NULL)` tắt cơ chế đồng bộ giữa luồng vào ra của C++ và của C. Với dữ liệu lớn, hai dòng này làm tốc độ đọc nhanh lên nhiều lần và thường là ranh giới giữa bài nộp đúng hạn và bài nộp quá thời gian. Ngoài ra nên dùng `"\n"` thay cho `endl`, vì `endl` buộc đẩy dữ liệu ra thiết bị sau mỗi lần gọi.

Tệp tiêu đề `<bits/stdc++.h>` nạp gần như toàn bộ thư viện chuẩn cùng lúc. Nó chỉ có trên trình biên dịch `g++` và không thuộc chuẩn C++, nhưng rất tiện khi thi vì khỏi phải nhớ tên từng tệp tiêu đề.

Khung tương ứng trong hai ngôn ngữ còn lại như sau.

JULIA

```julia
function solve()
    n = parse(Int, readline())
    a = parse.(Int, split(readline()))
    # xử lý
```

```
end

function main()
    t = parse(Int, readline())
    for _ in 1:t
        solve()
    end
end

main()
```

PYTHON

```python
import sys
input = sys.stdin.readline          # đọc nhanh hơn hàm input mặc định

def solve():
    n = int(input())
    a = list(map(int, input().split()))
    # xử lý

def main():
    t = int(input())
    for _ in range(t):
        solve()

main()
```

Trong Julia, `parse.(Int, ...)` có dấu chấm trước ngoặc: đó là cú pháp áp dụng hàm lên từng phần tử của mảng, tương ứng với `map` của Python.

Trong Python, việc gán lại tên `input` thành `sys.stdin.readline` là mẹo quen thuộc khi dữ liệu vào lớn, vì hàm `input` mặc định chậm hơn đáng kể. Cần lưu ý hàm này giữ lại ký tự xuống dòng ở cuối, nên khi đọc chuỗi phải gọi thêm `.strip()`.

## 1.2 Thuật toán

### 1.2.1 Khái niệm thuật toán

**Định nghĩa 1.1.** (*Thuật toán*) Thuật toán là một dãy hữu hạn các thao tác được sắp thứ tự, khi thực hiện trên dữ liệu vào hợp lệ thì dừng lại sau hữu hạn bước và cho ra kết quả mong muốn.

Định nghĩa này có ba ý cần chú ý. Thứ nhất, thuật toán là một dãy thao tác **hữu hạn** và phải **dừng**; một quy trình chạy mãi không dừng thì không phải thuật toán. Thứ hai, các thao tác phải được **sắp thứ tự**, nghĩa là thứ tự thực hiện có ý nghĩa và không được tuỳ tiện đảo. Thứ ba, thuật toán phải cho **kết quả mong muốn** với mọi dữ liệu vào hợp lệ, chứ không phải chỉ đúng với một vài trường hợp.

Cần phân biệt thuật toán với chương trình. Thuật toán là phương pháp ở mức trừu tượng, độc lập với việc ai thực hiện và thực hiện bằng công cụ gì. Chương trình là cách hiện thực hoá thuật toán đó trên một cấu trúc dữ liệu cụ thể, bằng một ngôn ngữ cụ thể. Cùng một thuật toán tìm kiếm, cài trên mảng và cài trên danh sách liên kết cho hai chương trình khác nhau, và độ phức tạp cũng khác nhau. Quan hệ này được Niklaus Wirth diễn đạt trong công thức quen thuộc: cấu trúc dữ liệu cộng với thuật toán bằng chương trình.

### 1.2.2 Các đặc trưng của thuật toán

Một dãy hướng dẫn chỉ được gọi là thuật toán khi thoả mãn đủ sáu tính chất trong bảng 1.6.

**Bảng 1.6.** Sáu đặc trưng của thuật toán.

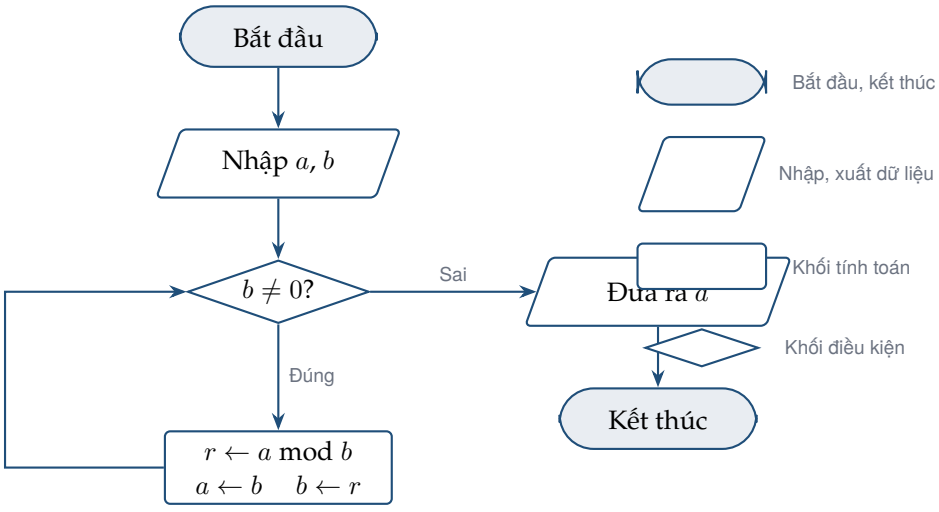
| Đặc trưng | Nội dung | Ví dụ vi phạm |
|---|---|---|
| Có đầu vào | Nhận 0 hoặc nhiều dữ liệu vào, mỗi dữ liệu thuộc tập xác định rõ | Yêu cầu "sắp xếp dãy" mà không nói dãy gồm kiểu dữ liệu gì |
| Có đầu ra | Cho ít nhất một kết quả, quan hệ rõ ràng với dữ liệu vào | Thủ tục chỉ in ra màn hình, không xác định kết quả là gì |
| Tính xác định | Mỗi bước rõ ràng, chỉ hiểu được theo một nghĩa | "Chọn một phần tử tương đối lớn" |
| Tính hữu hạn | Dừng sau hữu hạn bước với mọi dữ liệu hợp lệ | `while n != 1: n = n - 2` với $n$ lẻ |
| Tính hiệu quả | Mỗi bước đủ cơ bản để làm được trong thời gian hữu hạn | "Kiểm tra tất cả các số nguyên tố" |
| Tính phổ dụng | Giải được cả một lớp bài toán, không chỉ một bộ dữ liệu | Chương trình in sẵn đáp án cho đúng một bộ dữ liệu kiểm thử |

### 1.2.3 Các cách mô tả thuật toán

Có bốn cách mô tả thuật toán, xếp theo mức độ hình thức tăng dần.

**Liệt kê từng bước** bằng ngôn ngữ tự nhiên, có đánh số thứ tự. Cách này dễ đọc với mọi người nhưng dài dòng và dễ gây hiểu nhầm khi thuật toán phức tạp.

**Sơ đồ khối** dùng các hình vẽ nối bằng mũi tên. Hình 1.3 minh hoạ thuật toán Euclid bằng sơ đồ khối, kèm chú giải ý nghĩa từng loại hình. Sơ đồ khối cho thấy luồng điều khiển rất trực quan nhưng trở nên rối rắm khi thuật toán có đệ quy hoặc nhiều vòng lặp lồng nhau.




**Hình 1.3.** Sơ đồ khối thuật toán Euclid tìm ước chung lớn nhất, kèm chú giải các ký hiệu chuẩn.

**Giả mã** là cách viết trung gian giữa ngôn ngữ tự nhiên và ngôn ngữ lập trình. Nó đủ chính xác để không gây hiểu nhầm nhưng bỏ qua các chi tiết kỹ thuật như khai báo kiểu hay đọc dữ liệu. Đây là cách được dùng trong hầu hết giáo trình và là cách nên dùng khi làm bài thi tự luận, vì sinh viên không bị trừ điểm do sai cú pháp của một ngôn ngữ cụ thể.

GIẢ MÃ

```
TÌM-MAX(a, n)
1  max ← a[1]
2  for i ← 2 to n
3      if a[i] > max
4          max ← a[i]
5  return max
```

Ký hiệu mũi tên đọc là "gán". Các dòng được đánh số để tiện trích dẫn khi phân tích.

**Ngôn ngữ lập trình** là cách mô tả chính xác tuyệt đối và chạy được, nhưng lẫn nhiều chi tiết kỹ thuật vào ý tưởng thuật toán.

### 1.2.4 Ví dụ: thuật toán Euclid

Xét bài toán tìm ước chung lớn nhất của hai số nguyên dương. Cách học ở phổ thông là phân tích cả hai số ra thừa số nguyên tố rồi lấy tích các thừa số chung. Cách đó đúng nhưng chậm, vì việc phân tích ra thừa số nguyên tố là bài toán khó.

Thuật toán Euclid dựa trên một nhận xét đơn giản.

**Định lý 1.1.** *Nếu* $b \neq 0$ *thì* $\gcd(a, b) = \gcd(b,\ a \bmod b)$. *Ngoài ra* $\gcd(a, 0) = a$.

*Chứng minh.* Đặt $r = a \bmod b$, khi đó $a = qb + r$ với $q$ là thương của phép chia. Nếu $d$ là ước chung của $a$ và $b$ thì $d$ chia hết $r = a - qb$, nên $d$ là ước chung của $b$ và $r$. Ngược lại, nếu $d$ là ước chung của $b$ và $r$ thì $d$ chia hết $a = qb + r$, nên $d$ là ước chung của $a$ và $b$. Vậy tập ước chung của cặp $(a, b)$ trùng với tập ước chung của cặp $(b, r)$, do đó hai cặp có cùng ước chung lớn nhất. ■

Từ nhận xét đó ta có thuật toán: thay cặp $(a, b)$ bằng cặp $(b, a \bmod b)$ cho tới khi số thứ hai bằng 0, khi đó số thứ nhất là kết quả cần tìm.

GIẢ MÃ

```
EUCLID(a, b)
1  while b ≠ 0
2      r ← a mod b
3      a ← b
4      b ← r
5  return a
```

JULIA

```
function ucln(a, b)
    while b != 0
        a, b = b, a % b      # gán đồng thời: vế phải tính hết rồi mới gán
    end
    return a
end

println(ucln(48, 18))        # 6
```

PYTHON

```
def ucln(a, b):
    while b != 0:
        a, b = b, a % b
```

```
    return a

print(ucln(48, 18))          # 6
```

C++

```cpp
int ucln(int a, int b) {
    while (b != 0) {
        int r = a % b;       // C++ không có gán đồng thời, phải dùng biến tạm
        a = b;
        b = r;
    }
    return a;
}
```

Julia và Python cho phép gán đồng thời, nghĩa là vế phải được tính hết trước rồi mới gán sang vế trái. C++ không có cú pháp này nên phải dùng biến tạm. Nếu viết `a = b; b = a % b;` mà không có biến tạm thì kết quả sai, vì đến dòng thứ hai giá trị của a đã bị thay đổi.

**Ví dụ 1.1.** *Lần vết thuật toán Euclid*. Bảng 1.7 theo dõi từng vòng lặp với $a = 48$ và $b = 18$.

**Bảng 1.7.** Lần vết thuật toán Euclid với $a = 48$, $b = 18$.

| Trước vòng lặp thứ | $a$ | $b$ | $a \bmod b$ |
|---|---|---|---|
| 1 | 48 | 18 | 12 |
| 2 | 18 | 12 | 6 |
| 3 | 12 | 6 | 0 |
| Sau vòng 3 | 6 | 0 | dừng, trả về 6 |

Ta kiểm tra thuật toán này thoả mãn đủ sáu đặc trưng. Đầu vào là hai số nguyên, đầu ra là ước chung lớn nhất của chúng. Mỗi bước là phép chia lấy dư, vừa xác định vừa cơ bản. Tính hữu hạn được bảo đảm vì số dư luôn nhỏ hơn số chia, nên giá trị của $b$ giảm ngặt sau mỗi vòng lặp; một dãy số tự nhiên giảm ngặt không thể vô hạn nên nó buộc phải chạm 0. Thuật toán đúng với mọi cặp số nguyên dương nên có tính phổ dụng.

## 1.3 Độ phức tạp tính toán

### 1.3.1 Cách đánh giá thời gian chạy

Câu hỏi thực tế nhất khi giải một bài toán là: thuật toán mình định viết có chạy kịp trong giới hạn thời gian không. Trả lời được câu hỏi này trước khi viết chương trình là kỹ năng quan trọng nhất của học phần.

Cách đơn giản nhất là bấm giờ chạy thử, nhưng cách đó có ba nhược điểm khiến nó không dùng được để so sánh thuật toán. Kết quả phụ thuộc cấu hình máy, nên hai người đo được hai con số khác nhau. Kết quả phụ thuộc ngôn ngữ và trình biên dịch, nên cùng một thuật toán viết bằng C++ và Python cho thời gian chênh nhau hàng chục lần. Và kết quả phụ thuộc bộ dữ liệu đem đo, nên một thuật toán có thể nhanh trên bộ này và chậm trên bộ khác.

Cách đánh giá đúng là đếm số phép toán mà thuật toán phải thực hiện, tính theo kích thước dữ liệu vào. Ta không đếm mọi phép toán mà chỉ đếm **phép toán tích cực**, tức phép toán có số lần

thực hiện ít nhất cũng tỉ lệ thuận với tổng số phép toán của thuật toán. Với thuật toán sắp xếp, phép toán tích cực thường là phép so sánh hai phần tử; với thuật toán tìm kiếm là phép so sánh khoá.

Ký hiệu $T(n)$ là số phép toán tích cực khi kích thước dữ liệu vào là $n$. Xét ba đoạn chương trình sau.

C++

```cpp
// Đoạn 1
int x = a[0];                          // T(n) = 1

// Đoạn 2
long long tong = 0;
for (int i = 0; i < n; i++)
    tong += a[i];                      // T(n) = n

// Đoạn 3
int dem = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        dem++;                         // T(n) = n * n
```

Đoạn 1 luôn thực hiện đúng một phép gán, không phụ thuộc $n$. Đoạn 2 có vòng lặp chạy $n$ lần. Đoạn 3 có hai vòng lặp lồng nhau, vòng ngoài chạy $n$ lần và mỗi lần vòng trong lại chạy $n$ lần.

Bảng 1.8 cho thấy khác biệt giữa $T(n) = n$ và $T(n) = n^2$ với giả định máy thực hiện được $10^8$ phép toán mỗi giây.

**Bảng 1.8.** Thời gian chạy ước tính với $10^8$ phép toán mỗi giây.

| $n$ | $T(n) = n$ | $T(n) = n^2$ |
|---|---|---|
| $10^3$ | tức thì | tức thì |
| $10^5$ | tức thì | khoảng 100 giây |
| $10^6$ | 0,01 giây | khoảng 3 giờ |
| $10^7$ | 0,1 giây | khoảng 11 ngày |

### 1.3.2 Ký hiệu tiệm cận

Khi $n$ lớn, các chi tiết nhỏ trong công thức $T(n)$ không còn quan trọng. Xét $T(n) = 3n^2 + 5n + 100$ với $n = 10^6$: số hạng $3n^2$ bằng ba nghìn tỉ, còn $5n + 100$ chỉ khoảng năm triệu, chiếm chưa tới một phần nửa triệu của tổng.

**Định nghĩa 1.2.** (*Ký hiệu $O$ lớn*) Ta viết $T(n) = O(f(n))$ nếu tồn tại hằng số $c > 0$ và số $n_0$ sao cho $T(n) \le c \cdot f(n)$ với mọi $n \ge n_0$.

**Ví dụ 1.2.** *Chứng minh một chặn trên.* Chứng minh $3n^2 + 5n + 100 = O(n^2)$.

Với mọi $n \ge 10$ ta có $5n \le n^2$ (vì $5 \le n$) và $100 \le n^2$ (vì $n^2 \ge 100$). Do đó với mọi $n \ge 10$:

$$3n^2 + 5n + 100 \le 3n^2 + n^2 + n^2 = 5n^2.$$

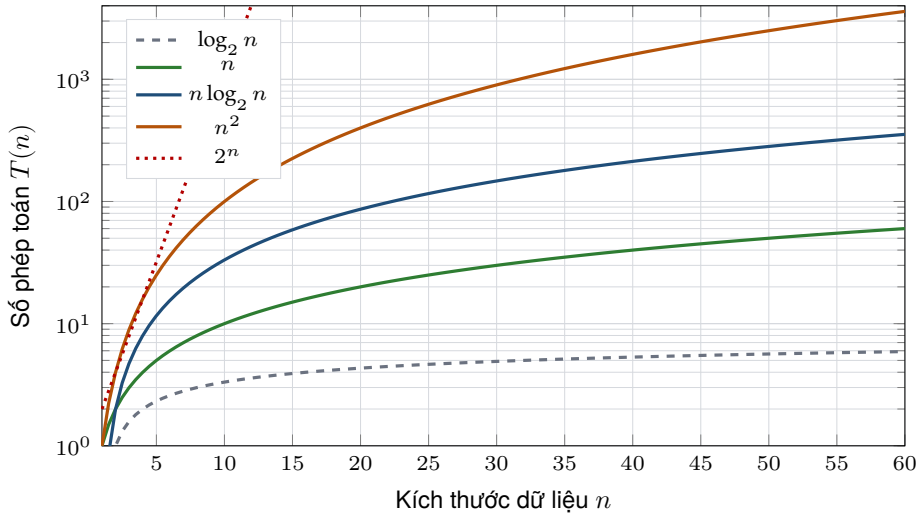
Chọn $c = 5$ và $n_0 = 10$ thì định nghĩa được thoả mãn.

**Định nghĩa 1.3.** (*Ký hiệu $\Omega$ và $\Theta$*) Ta viết $T(n) = \Omega(f(n))$ nếu tồn tại $c > 0$ và $n_0$ sao cho $T(n) \ge$

$c \cdot f(n)$ với mọi $n \geq n_0$; đây là một chặn dưới. Ta viết $T(n) = \Theta(f(n))$ khi đồng thời $T(n) = O(f(n))$ và $T(n) = \Omega(f(n))$; đây là một chặn khít.

Với $T(n) = 3n^2 + 5n + 100$, ta đã có $T(n) = O(n^2)$; mặt khác hiển nhiên $T(n) \geq 3n^2$ nên $T(n) = \Omega(n^2)$. Kết hợp lại, $T(n) = \Theta(n^2)$.

Trong thực hành, người ta dùng $O$ nhiều hơn cả và áp dụng hai quy tắc rút gọn: bỏ mọi hằng số nhân, và chỉ giữ số hạng tăng nhanh nhất. Hình 1.4 cho thấy vì sao hai quy tắc đó hợp lý: khi $n$ đủ lớn, đường cong của lớp cao hơn vượt hẳn lên và mọi khác biệt về hằng số trở nên không đáng kể.




**Hình 1.4.** Tốc độ tăng của các lớp độ phức tạp, trục tung theo thang lô-ga-rit. Đường $2^n$ vượt khỏi khung hình khi $n$ mới chỉ khoảng 12, còn $\log_2 n$ gần như nằm ngang.

### 1.3.3 Quy tắc tính độ phức tạp

**Quy tắc hằng số.** Một khối lệnh không chứa vòng lặp và không gọi hàm có độ phức tạp $O(1)$, bất kể nó gồm một hay năm mươi câu lệnh, vì số lệnh không phụ thuộc $n$.

**Quy tắc cộng.** Hai khối lệnh thực hiện tuần tự có độ phức tạp $O(f)$ và $O(g)$ thì tổng thời gian là $O(f + g)$, và vì ta chỉ giữ số hạng lớn nhất nên kết quả bằng $O(\max(f, g))$.

**Quy tắc nhân.** Vòng lặp chạy $O(f)$ lần, mỗi lần thân vòng tốn $O(g)$, thì tổng là $O(f \cdot g)$.

**Vòng lặp có cận phụ thuộc.** Khi cận của vòng trong phụ thuộc biến của vòng ngoài, phải tính tổng thay vì nhân trực tiếp. Xét đoạn mã sau.

JULIA

```julia
for i in 1:n
    for j in i+1:n        # vòng trong chạy n - i lần
        # O(1)
    end
end
```

Tổng số lần thực hiện thân vòng là

$$\sum_{i=1}^{n} (n - i) = (n-1) + (n-2) + \cdots + 1 + 0 = \frac{n(n-1)}{2}.$$

Theo quy tắc bỏ hằng số nhân, độ phức tạp vẫn là $O(n^2)$. Hình 1.5 minh hoạ vì sao dạng vòng

lặp này được gọi là vòng lặp tam giác, và vì sao nó vẫn thuộc lớp $O(n^2)$ dù chỉ chạy khoảng một nửa số ô của hình vuông.




**Hình 1.5.** Vòng lặp tam giác. Đây là dạng của cả ba thuật toán sắp xếp cơ bản trong chương 2.

**Vòng lặp nhân hoặc chia biến đếm.** Khi biến đếm được nhân đôi hoặc chia đôi sau mỗi vòng, số vòng lặp là $\log_2 n$.

C++

```cpp
int i = 1;
while (i <= n) {
    i = i * 2;        // i nhận 1, 2, 4, 8, ..., nên chạy log2(n) vòng
}
```

### 1.3.4 Độ phức tạp của thuật toán đệ quy

Với thuật toán đệ quy, ta lập hệ thức truy hồi cho $T(n)$ rồi giải hệ thức đó.

**Ví dụ 1.3.** *Tìm kiếm nhị phân.* Mỗi lần gọi loại bỏ một nửa dãy, cho $T(n) = T(n/2) + O(1)$. Khai triển liên tiếp: $T(n) = T(n/4) + 2 = T(n/8) + 3 = \cdots = T(1) + \log_2 n$. Vậy $T(n) = O(\log n)$.

**Ví dụ 1.4.** *Sắp xếp trộn.* Thuật toán chia dãy làm đôi, sắp xếp từng nửa rồi trộn lại; bước trộn tốn thời gian tuyến tính, cho $T(n) = 2T(n/2) + O(n)$. Cây các lời gọi đệ quy có $\log_2 n$ mức; ở mức thứ $k$ có $2^k$ bài toán con, mỗi bài kích thước $n/2^k$, nên tổng chi phí mỗi mức là $2^k \cdot O(n/2^k) = O(n)$. Nhân với số mức, ta được $T(n) = O(n \log n)$.

**Định lý 1.2.** (*Định lý thợ, dạng đơn giản*) *Cho $T(n) = a\,T(n/b) + O(n^d)$ với $a \geq 1$, $b > 1$, $d \geq 0$. Khi đó $T(n) = O(n^d)$ nếu $d > \log_b a$; $T(n) = O(n^d \log n)$ nếu $d = \log_b a$; và $T(n) = O(n^{\log_b a})$ nếu $d < \log_b a$.*

Áp dụng cho sắp xếp trộn: $a = 2, b = 2, d = 1$; ta có $\log_2 2 = 1 = d$ nên rơi vào trường hợp thứ hai, cho $T(n) = O(n \log n)$. Áp dụng cho tìm kiếm nhị phân: $a = 1, b = 2, d = 0$; ta có $\log_2 1 = 0 = d$ nên $T(n) = O(\log n)$.

### 1.3.5 Các lớp độ phức tạp và bảng tra

Một cách cảm nhận trực quan là xem điều gì xảy ra khi kích thước dữ liệu tăng gấp đôi. Với $O(\log n)$, thời gian tăng thêm một lượng nhỏ cố định. Với $O(n)$, thời gian tăng gấp đôi. Với $O(n^2)$, thời gian tăng gấp bốn. Với $O(2^n)$, thời gian bình phương lên.

**Bảng 1.9.** Các lớp độ phức tạp thường gặp.

| Ký hiệu | Tên gọi | Thuật toán tiêu biểu |
|---|---|---|
| $O(1)$ | Hằng số | Truy cập phần tử mảng, thao tác trên ngăn xếp |
| $O(\log n)$ | Lô-ga-rit | Tìm kiếm nhị phân, thao tác trên heap |
| $O(n)$ | Tuyến tính | Duyệt mảng, tìm kiếm tuần tự |
| $O(n \log n)$ | Tuyến tính lô-ga-rit | Sắp xếp nhanh, sắp xếp trộn, sắp xếp vun đống |
| $O(n^2)$ | Bình phương | Ba thuật toán sắp xếp cơ bản |
| $O(n^3)$ | Lập phương | Floyd–Warshall, nhân ma trận trực tiếp |
| $O(2^n)$ | Hàm mũ | Duyệt mọi tập con, quay lui không cắt nhánh |
| $O(n!)$ | Giai thừa | Duyệt mọi hoán vị |

Bảng 1.10 là công cụ dùng hằng ngày khi làm bài tập: nhìn giới hạn $n$ trong đề, xác định ngay lớp thuật toán cần nhắm tới. Bảng lập theo giả định máy chấm thực hiện khoảng $10^8$ phép toán mỗi giây.

**Bảng 1.10.** Bảng tra giới hạn dữ liệu. Dùng bảng này *trước khi* viết dòng mã đầu tiên.

| Giới hạn của $n$ | Độ phức tạp tối đa | Hướng tiếp cận |
|---|---|---|
| $n \le 11$ | $O(n!)$ | Duyệt mọi hoán vị |
| $n \le 25$ | $O(2^n \cdot n)$ | Duyệt mọi tập con, quay lui |
| $n \le 100$ | $O(n^4)$ | Quy hoạch động nhiều chiều |
| $n \le 400$ | $O(n^3)$ | Floyd–Warshall, quy hoạch động trên ma trận |
| $n \le 5\,000$ | $O(n^2)$ | Quy hoạch động hai vòng lặp, sắp xếp cơ bản |
| $n \le 10^5$ | $O(n \log n)$ | Sắp xếp, tìm kiếm nhị phân, Dijkstra có heap |
| $n \le 10^7$ | $O(n)$ | Duyệt một lượt, hai con trỏ, sàng nguyên tố |
| $n > 10^8$ | $O(\log n)$, $O(1)$ | Công thức toán học, luỹ thừa nhanh |

**Cách dùng bảng tra.** Nếu đề cho $n \le 10^5$ mà lời giải dự định có hai vòng lặp lồng nhau thì bài chắc chắn quá thời gian. Hãy dừng lại nghĩ cách khác ngay lúc đó, thay vì viết xong rồi mới phát hiện. Thói quen tra bảng trước khi viết mã tiết kiệm rất nhiều thời gian trong phòng thi.

### 1.3.6 Trường hợp tốt nhất, xấu nhất, trung bình

Với cùng một kích thước $n$, thời gian chạy còn phụ thuộc dữ liệu cụ thể. Xét bài toán tìm giá trị $x$ trong mảng $n$ phần tử bằng cách duyệt tuần tự từ đầu.

Trong trường hợp tốt nhất, $x$ nằm ngay ở vị trí đầu tiên, thuật toán dừng sau một phép so sánh, độ phức tạp $O(1)$. Trong trường hợp xấu nhất, $x$ nằm ở vị trí cuối hoặc không có trong mảng, thuật toán thực hiện $n$ phép so sánh, độ phức tạp $O(n)$. Trong trường hợp trung bình, giả sử $x$

có mặt và vị trí phân bố đều, số phép so sánh kỳ vọng là

$$\frac{1+2+\cdots+n}{n} = \frac{n+1}{2},$$

vẫn là $O(n)$.

Khi nói "độ phức tạp của thuật toán" mà không nói gì thêm, người ta ngầm hiểu là trường hợp xấu nhất. Đây là con số duy nhất cho phép khẳng định chương trình không quá thời gian với mọi bộ dữ liệu.

### 1.3.7 Độ phức tạp không gian

Ngoài thời gian, ta còn quan tâm tới lượng bộ nhớ mà thuật toán cần. Độ phức tạp không gian đo bộ nhớ **phụ**, không tính phần bộ nhớ chứa dữ liệu vào.

Sắp xếp chèn dùng $O(1)$ bộ nhớ phụ vì nó sắp xếp ngay trên mảng gốc. Sắp xếp trộn cần một mảng phụ cùng kích thước nên tốn $O(n)$.

Một điểm hay bị bỏ sót là chi phí bộ nhớ của đệ quy. Mỗi lời gọi hàm chiếm một khung trên ngăn xếp lời gọi, chứa tham số và biến cục bộ của lời gọi đó. Đệ quy sâu $n$ mức tốn $O(n)$ bộ nhớ ngăn xếp, và khi độ sâu vượt quá giới hạn, thường vào khoảng $10^5$ mức, chương trình dừng đột ngột do tràn ngăn xếp.

## 1.4 Các ví dụ áp dụng

### 1.4.1 Kiểm tra số nguyên tố

Cách trực tiếp là thử mọi ước từ 2 đến $n-1$, cho độ phức tạp $O(n)$. Nhưng ta có nhận xét: nếu $n = p \cdot q$ với $p \le q$ thì $p \le \sqrt{n}$. Do đó nếu $n$ có ước thực sự thì nó phải có một ước không vượt quá $\sqrt{n}$, và chỉ cần thử tới đó.

JULIA

```julia
function la_nguyen_to(n)
    n < 2 && return false
    n % 2 == 0 && return n == 2
    d = 3
    while d * d <= n            # tương đương d <= sqrt(n) nhưng tránh số thực
        n % d == 0 && return false
        d = d + 2               # chỉ thử các ước lẻ
    end
    return true
end
```

PYTHON

```python
def la_nguyen_to(n):
    if n < 2:
        return False
    if n % 2 == 0:
        return n == 2
    d = 3
    while d * d <= n:
        if n % d == 0:
            return False
        d = d + 2
    return True
```

C++

```cpp
bool la_nguyen_to(long long n) {
    if (n < 2) return false;
    if (n % 2 == 0) return n == 2;
    for (long long d = 3; d * d <= n; d += 2) {
        if (n % d == 0) return false;
    }
    return true;
}
```

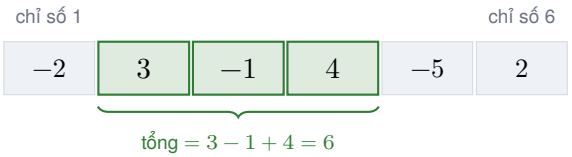
Vòng lặp chạy khoảng $\sqrt{n}/2$ lần nên độ phức tạp là $O(\sqrt{n})$. Với $n = 10^{12}$, cách đúng cần khoảng $10^6$ phép toán và cho kết quả tức thì, còn cách thử mọi ước cần $10^{12}$ phép toán, tức khoảng ba giờ.

Chú ý cách viết điều kiện `d * d <= n` thay vì `d <= sqrt(n)`. Cách thứ hai gọi hàm căn bậc hai trên số thực ở mỗi vòng lặp, vừa chậm hơn vừa có nguy cơ sai do sai số làm tròn.

Trong Julia, cú pháp `điều_kiện && return giá_trị` là cách viết tắt của `if điều_kiện; return giá_trị; end`. Cách viết này rất phổ biến trong mã nguồn Julia và được dùng nhiều trong tài liệu.

### 1.4.2 Đoạn con liên tiếp có tổng lớn nhất

Cho dãy $n$ số nguyên, trong đó có thể có số âm. Tìm đoạn gồm các phần tử liên tiếp có tổng lớn nhất. Chẳng hạn với dãy $-2, 3, -1, 4, -5, 2$ thì đoạn tốt nhất là $3, -1, 4$ với tổng bằng 6, như minh hoạ ở hình 1.6.




**Hình 1.6.** Đoạn con liên tiếp có tổng lớn nhất của dãy $-2, 3, -1, 4, -5, 2$.

Ví dụ này đáng nghiên cứu kỹ vì cùng một bài toán có ba lời giải với ba độ phức tạp khác nhau, và mỗi lần cải tiến đều xuất phát từ một nhận xét cụ thể.

**Cách thứ nhất.** Duyệt mọi cặp chỉ số đầu và cuối, với mỗi đoạn thì cộng lại từ đầu để lấy tổng. Ba vòng lặp lồng nhau cho độ phức tạp $O(n^3)$.

JULIA

```julia
function cach1(a)
    n = length(a)
    best = a[1]
    for i in 1:n
        for j in i:n
            s = 0
            for k in i:j          # vòng lặp thứ ba
                s = s + a[k]
            end
            if s > best
                best = s
            end
        end
    end
```

```
    return best
end
```

PYTHON

```python
def cach1(a):
    n = len(a)
    best = a[0]
    for i in range(n):
        for j in range(i, n):
            s = 0
            for k in range(i, j + 1):    # vòng lặp thứ ba
                s = s + a[k]
            if s > best:
                best = s
    return best
```

C++

```cpp
long long cach1(const vector<int> &a) {
    int n = a.size();
    long long best = a[0];
    for (int i = 0; i < n; i++)
        for (int j = i; j < n; j++) {
            long long s = 0;
            for (int k = i; k <= j; k++) s += a[k];    // vòng lặp thứ ba
            if (s > best) best = s;
        }
    return best;
}
```

**Cách thứ hai.** Nhận xét rằng khi mở rộng đoạn thêm một phần tử ở bên phải, tổng mới chỉ bằng tổng cũ cộng thêm phần tử đó. Không cần cộng lại từ đầu, nhờ vậy bỏ được vòng lặp trong cùng và độ phức tạp giảm còn $O(n^2)$.

JULIA

```julia
function cach2(a)
    n = length(a)
    best = a[1]
    for i in 1:n
        s = 0
        for j in i:n
            s = s + a[j]                # cộng dồn thay vì tính lại
            if s > best
                best = s
            end
        end
    end
    return best
end
```

PYTHON

```python
def cach2(a):
    n = len(a)
```

```python
    best = a[0]
    for i in range(n):
        s = 0
        for j in range(i, n):
            s = s + a[j]            # cộng dồn thay vì tính lại
            if s > best:
                best = s
    return best
```

C++

```cpp
long long cach2(const vector<int> &a) {
    int n = a.size();
    long long best = a[0];
    for (int i = 0; i < n; i++) {
        long long s = 0;
        for (int j = i; j < n; j++) {
            s += a[j];
            if (s > best) best = s;
        }
    }
    return best;
}
```

**Cách thứ ba, thuật toán Kadane.** Gọi cur là tổng lớn nhất của đoạn kết thúc **đúng tại** vị trí đang xét. Đoạn đó chỉ có hai khả năng: hoặc chỉ gồm một mình phần tử hiện tại, hoặc là đoạn tốt nhất kết thúc tại vị trí trước đó nối thêm phần tử hiện tại. Ta lấy phương án lớn hơn.

JULIA

```julia
function kadane(a)
    best = a[1]
    cur = a[1]
    for i in 2:length(a)
        cur = max(a[i], cur + a[i])     # bắt đầu lại, hay nối tiếp?
        best = max(best, cur)
    end
    return best
end
```

PYTHON

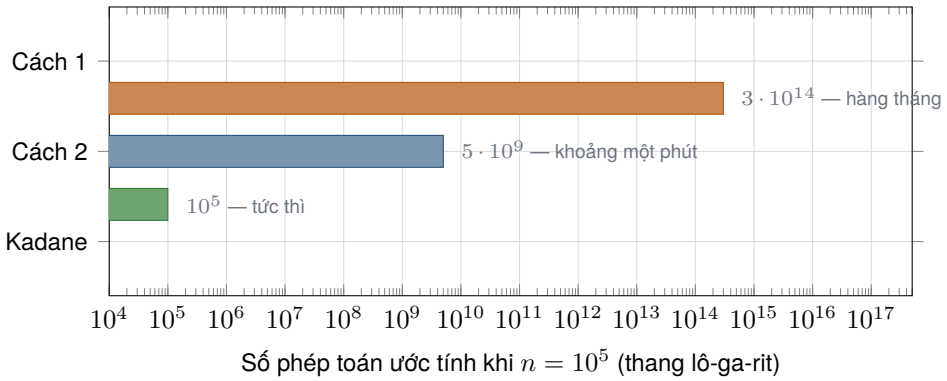
```python
def kadane(a):
    best = a[0]
    cur = a[0]
    for i in range(1, len(a)):
        cur = max(a[i], cur + a[i])
        best = max(best, cur)
    return best
```

C++

```cpp
long long kadane(const vector<int> &a) {
    long long best = a[0], cur = a[0];
    for (int i = 1; i < (int)a.size(); i++) {
        cur = max((long long)a[i], cur + a[i]);
```

```
            best = max(best, cur);
        }
        return best;
    }
```

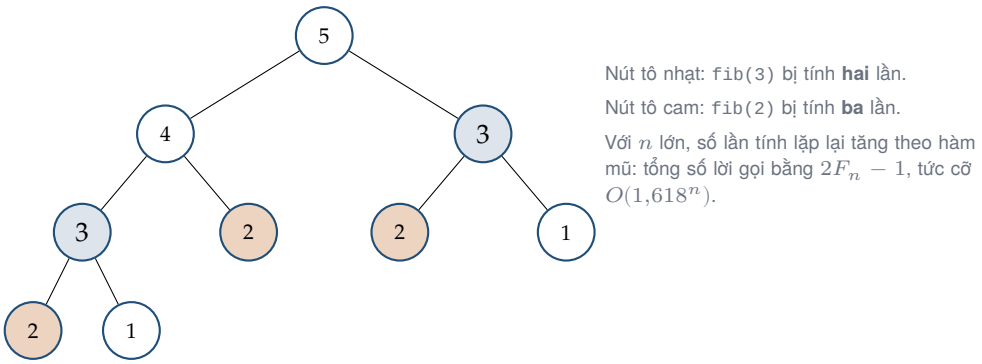
Hình 1.7 so sánh số phép toán của ba cách với $n = 10^5$. Thang đo là lô-ga-rit, nghĩa là mỗi vạch cách nhau một bậc gấp mười lần.




**Hình 1.7.** Số phép toán của ba lời giải cho bài toán đoạn con có tổng lớn nhất, với $n = 10^5$.

### 1.4.3 Dãy Fibonacci

Dãy Fibonacci được định nghĩa bằng $F_1 = F_2 = 1$ và $F_n = F_{n-1} + F_{n-2}$. Định nghĩa này dịch thẳng sang chương trình đệ quy, nhưng chương trình thu được chạy rất chậm. Hình 1.8 chỉ ra nguyên nhân: các nhánh đệ quy tính lại cùng một giá trị nhiều lần.




**Hình 1.8.** Cây các lời gọi khi tính `fib(5)` bằng đệ quy thuần.

Có hai cách khắc phục. Cách thứ nhất là ghi nhớ kết quả đã tính để lần sau lấy ra dùng lại.

JULIA

```
bo_nho = Dict{Int, Int}()

function fib_memo(n)
    n <= 2 && return 1
    haskey(bo_nho, n) && return bo_nho[n]      # đã tính rồi
    bo_nho[n] = fib_memo(n-1) + fib_memo(n-2)
end
```

PYTHON

```python
bo_nho = {}

def fib_memo(n):
    if n <= 2:
        return 1
    if n in bo_nho:
        return bo_nho[n]                        # đã tính rồi
    bo_nho[n] = fib_memo(n-1) + fib_memo(n-2)
    return bo_nho[n]
```

C++

```cpp
map<int, long long> bo_nho;

long long fib_memo(int n) {
    if (n <= 2) return 1;
    if (bo_nho.count(n)) return bo_nho[n];          // đã tính rồi
    return bo_nho[n] = fib_memo(n-1) + fib_memo(n-2);
}
```

Cách thứ hai là bỏ đệ quy, tính lần lượt từ nhỏ đến lớn và chỉ giữ hai giá trị gần nhất.

JULIA

```julia
function fib_lap(n)
    n <= 2 && return 1
    a, b = 1, 1
    for _ in 3:n
        a, b = b, a + b
    end
    return b
end
```

PYTHON

```python
def fib_lap(n):
    if n <= 2:
        return 1
    a, b = 1, 1
    for _ in range(3, n + 1):
        a, b = b, a + b
    return b
```

C++

```cpp
long long fib_lap(int n) {
    if (n <= 2) return 1;
    long long a = 1, b = 1;
    for (int i = 3; i <= n; i++) {
        long long c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

Cả hai cách đều cho độ phức tạp $O(n)$, riêng cách thứ hai chỉ tốn $O(1)$ bộ nhớ. Với $n = 50$, chương trình cho kết quả $F_{50} = 12\,586\,269\,025$ ngay lập tức, trong khi bản đệ quy thuần cần khoảng $2{,}5 \times 10^{10}$ lời gọi.

Ý tưởng ghi nhớ kết quả các bài toán con ở cách thứ nhất là hạt nhân của quy hoạch động, sẽ được trình bày đầy đủ ở chương 4.

### 1.4.4 Đo thời gian chạy thực tế

Sau khi phân tích lý thuyết, sinh viên nên tự kiểm chứng bằng cách đo thời gian thực tế.

JULIA

```julia
@time tong_1_den_n(100_000_000)
```

PYTHON

```python
import time

t0 = time.time()
tong_1_den_n(100000000)
print("Thoi gian:", time.time() - t0, "giay")
```

C++

```cpp
#include <chrono>

auto t0 = chrono::high_resolution_clock::now();
tong_1_den_n(100000000);
auto t1 = chrono::high_resolution_clock::now();
cout << "Thoi gian: " << chrono::duration<double>(t1 - t0).count() << " giay";
```

**Kiểm chứng bảng lý thuyết.** Cài cả ba cách giải bài toán đoạn con tổng lớn nhất, chạy với $n$ bằng 200, 400, 800 rồi lập bảng thời gian. Với cách 1, thời gian phải tăng khoảng tám lần mỗi khi $n$ gấp đôi; với cách 2 là bốn lần; với Kadane là hai lần. Nếu số liệu đo được khớp với dự đoán thì sinh viên có thể tin vào phân tích của mình.
Khi đo bằng C++ nhớ biên dịch với tuỳ chọn `-O2`. Khi đo bằng Julia, cần chạy hàm một lần trước rồi mới đo lần thứ hai, vì lần chạy đầu tiên bao gồm cả thời gian biên dịch.

## 1.5 Ứng dụng: thuật toán láng giềng gần nhất

### 1.5.1 Bài toán phân lớp

Trong cách lập trình thông thường, người lập trình viết ra các quy tắc và máy áp dụng chúng lên dữ liệu. Trong học máy, một nhánh của trí tuệ nhân tạo, trình tự bị đảo ngược: người ta đưa cho máy các ví dụ đã có sẵn đáp án, máy tự rút ra quy tắc, rồi áp dụng quy tắc đó cho các trường hợp mới.

Bài toán **phân lớp** là dạng bài toán học máy phổ biến nhất. Ta có một tập dữ liệu gồm các mẫu, mỗi mẫu được mô tả bằng một số đặc trưng và đi kèm một nhãn cho biết mẫu đó thuộc lớp nào. Nhiệm vụ là dự đoán nhãn cho một mẫu mới chưa biết.

Để minh hoạ, ta dùng bộ dữ liệu hoa diên vĩ rút gọn, do nhà thống kê Ronald Fisher công bố

năm 1936 và từ đó trở thành ví dụ chuẩn trong các giáo trình học máy. Mỗi bông hoa được mô tả bằng hai đặc trưng là chiều dài và chiều rộng cánh hoa, và thuộc một trong ba loài.

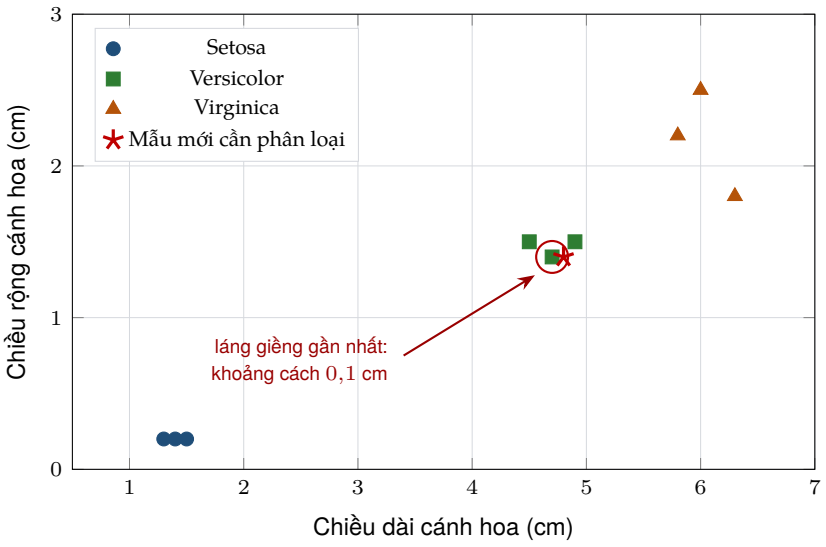
### 1.5.2 Thuật toán láng giềng gần nhất

Thuật toán phân lớp đơn giản nhất dựa trên một giả thiết trực quan: hai mẫu có đặc trưng gần giống nhau thì nhiều khả năng cùng thuộc một lớp. Từ đó suy ra cách làm: để dự đoán nhãn cho mẫu mới, hãy tìm trong tập dữ liệu mẫu nào giống nó nhất rồi trả về nhãn của mẫu đó.

Để máy so sánh được mức độ giống nhau, ta phải quy nó thành con số. Coi mỗi bông hoa là một điểm trên mặt phẳng với toạ độ là cặp chiều dài và chiều rộng; hai bông càng giống nhau thì hai điểm càng gần nhau. Khoảng cách giữa hai điểm $(x_1, y_1)$ và $(x_2, y_2)$ được tính theo công thức Pythagore:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

Hình 1.9 biểu diễn toàn bộ tập dữ liệu trên mặt phẳng cùng một mẫu mới cần phân loại.




**Hình 1.9.** Bộ dữ liệu hoa diên vĩ rút gọn. Mẫu mới được gán nhãn của điểm gần nó nhất.

GIẢ MÃ

```
LÁNG-GIỀNG-GẦN-NHẤT(mẫu_mới, dữ_liệu)
1  kc_nhỏ_nhất ← +∞
2  kết_quả ← rỗng
3  for mỗi mẫu trong dữ_liệu
4      d ← khoảng cách từ mẫu_mới tới mẫu đó
5      if d < kc_nhỏ_nhất
6          kc_nhỏ_nhất ← d
7          kết_quả ← nhãn của mẫu đó
8  return kết_quả
```

So sánh giả mã này với thuật toán TÌM-MAX ở mục 1.2, ta thấy chúng có cùng cấu trúc. Khác biệt là ở đây ta tìm giá trị nhỏ nhất thay vì lớn nhất, và trả về nhãn đi kèm giá trị đó chứ không trả về chính giá trị.

Việc khởi tạo biến khoảng cách nhỏ nhất bằng một giá trị rất lớn là cần thiết. Nếu khởi tạo bằng 0 thì điều kiện ở dòng 5 không bao giờ đúng, vì mọi khoảng cách đều không âm, và hàm sẽ trả về kết quả rỗng.

PYTHON

```python
du_lieu = [
    (1.4, 0.2, "Setosa"),
    (1.3, 0.2, "Setosa"),
    (1.5, 0.2, "Setosa"),
    (4.7, 1.4, "Versicolor"),
    (4.5, 1.5, "Versicolor"),
    (4.9, 1.5, "Versicolor"),
    (6.0, 2.5, "Virginica"),
    (5.8, 2.2, "Virginica"),
    (6.3, 1.8, "Virginica"),
]

def khoang_cach(x1, y1, x2, y2):
    return ((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5

def phan_loai(dai, rong, du_lieu):
    kc_nho_nhat = float("inf")
    ket_qua = None
    for (d, r, loai) in du_lieu:
        kc = khoang_cach(dai, rong, d, r)
        if kc < kc_nho_nhat:
            kc_nho_nhat = kc
            ket_qua = loai
    return ket_qua

print(phan_loai(4.8, 1.4, du_lieu))    # Versicolor
print(phan_loai(1.6, 0.3, du_lieu))    # Setosa
print(phan_loai(6.1, 2.3, du_lieu))    # Virginica
```

JULIA

```julia
khoang_cach(x1, y1, x2, y2) = sqrt((x1 - x2)^2 + (y1 - y2)^2)

function phan_loai(dai, rong, du_lieu)
    kc_nho_nhat = Inf
    ket_qua = ""
    for (d, r, loai) in du_lieu
        kc = khoang_cach(dai, rong, d, r)
        if kc < kc_nho_nhat
            kc_nho_nhat = kc
            ket_qua = loai
        end
    end
    return ket_qua
end
```

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Hoa {
    double dai, rong;
    string loai;
};

double khoang_cach(double x1, double y1, double x2, double y2) {
```

```
    return sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2));
}

string phan_loai(double dai, double rong, const vector<Hoa> &du_lieu) {
    double kc_nho_nhat = 1e18;
    string ket_qua = "";
    for (int i = 0; i < (int)du_lieu.size(); i++) {
        double kc = khoang_cach(dai, rong, du_lieu[i].dai, du_lieu[i].rong);
        if (kc < kc_nho_nhat) {
            kc_nho_nhat = kc;
            ket_qua = du_lieu[i].loai;
        }
    }
    return ket_qua;
}
```

Bản C++ dùng `struct` để gom ba thông tin của một bông hoa thành một kiểu dữ liệu. Cách này rõ ràng hơn so với việc dùng ba mảng song song, và sẽ được dùng lại nhiều lần ở các chương sau.

Chương trình vừa viết không hề "hiểu" hoa là gì. Nó chỉ tính khoảng cách và chọn giá trị nhỏ nhất. Phần lớn các thuật toán học máy, khi bóc tách tới tầng dưới cùng, cũng chỉ gồm những phép tính đơn giản như vậy lặp lại rất nhiều lần.

### 1.5.3 Đánh giá độ phức tạp

Đặt $n$ là số mẫu trong tập dữ liệu và $d$ là số đặc trưng mô tả mỗi mẫu. Thuật toán duyệt qua cả $n$ mẫu; với mỗi mẫu, việc tính khoảng cách cần $O(d)$ phép toán. Vậy chi phí để phân lớp một mẫu mới là $T(n, d) = O(n \cdot d)$.

**Bảng 1.11.** Ý nghĩa thực tế của độ phức tạp $O(n \cdot d)$.

| **Bài toán** | $n$ | $d$ | **Số phép toán mỗi lần dự đoán** |
|---|---|---|---|
| Bộ hoa diên vĩ đầy đủ | 150 | 4 | 600, tức thì |
| Nhận dạng chữ số viết tay | 60 000 | 784 | $4{,}7 \times 10^7$, dưới một giây |
| Tìm ảnh tương tự trong kho ảnh | $10^7$ | 512 | $5 \times 10^9$, khoảng một phút cho một ảnh |

Dòng cuối cho thấy giới hạn của thuật toán. Không ai chấp nhận chờ một phút để tìm một tấm ảnh. Điều đáng chú ý là vấn đề nằm ở thuật toán chứ không nằm ở tốc độ máy: dù máy nhanh gấp mười lần thì vẫn mất sáu giây cho một truy vấn.

Ý tưởng tìm mẫu giống nhất thì đúng và dễ hiểu. Nhưng để nó chạy được trên mười triệu ảnh thì cần đến các cấu trúc dữ liệu và giải thuật của các chương sau: sắp xếp và tìm kiếm nhị phân ở chương 2, cây tìm kiếm và cây phân hoạch không gian ở chương 7.

Thuật toán này còn một điểm yếu nữa: nó chỉ dựa vào đúng một mẫu gần nhất, nên nếu mẫu đó bị ghi nhãn sai thì kết quả dự đoán cũng sai theo. Cách khắc phục là xét $k$ mẫu gần nhất rồi lấy nhãn chiếm đa số. Muốn tìm $k$ mẫu gần nhất thì cần sắp xếp, và đó là nội dung của chương 2.

## 1.6 Bài tập chương 1

### Câu hỏi ôn tập

1. Nêu định nghĩa thuật toán và sáu đặc trưng của thuật toán. Với mỗi đặc trưng, cho một ví dụ vi phạm.
2. Phân biệt thuật toán và chương trình. Cho một ví dụ về cùng một thuật toán cài đặt trên hai cấu trúc dữ liệu khác nhau và cho hai độ phức tạp khác nhau.
3. Vì sao không dùng thời gian chạy thực tế để so sánh hai thuật toán?
4. Phát biểu định nghĩa hình thức của ba ký hiệu $O$, $\Omega$ và $\Theta$.
5. Trong C++, biểu thức `7 / 2` cho kết quả bao nhiêu? Còn trong Julia và Python? Giải thích sự khác biệt.
6. Mảng năm phần tử được đánh chỉ số như thế nào trong mỗi ngôn ngữ?
7. Khi nói "độ phức tạp của thuật toán" mà không nói gì thêm, người ta hiểu là trường hợp nào? Vì sao?
8. Vì sao trong thuật toán láng giềng gần nhất phải khởi tạo biến khoảng cách nhỏ nhất bằng một giá trị rất lớn?

### Bài tập tính toán

9. Chứng minh $T(n) = 5n^3 + 100n^2 + 2000 = \Theta(n^3)$ bằng cách chỉ ra các hằng số $c_1$, $c_2$, $n_0$ cụ thể.
10. Xác định độ phức tạp của các đoạn chương trình sau và giải thích:

    (a) hai vòng lặp `for i in 1:n` viết liên tiếp nhau;

    (b) `for i in 1:n; for j in 1:i; for k in 1:j; end; end; end;`

    (c) `i = n; while i > 1; i = i ÷ 3; end;`

    (d) `for i in 1:n; j = 1; while j <= n; j = j * 2; end; end.`
11. Giải các hệ thức truy hồi sau bằng định lý thợ: (a) $T(n) = 4T(n/2) + n$; (b) $T(n) = 2T(n/2) + n^2$; (c) $T(n) = 3T(n/3) + n$; (d) $T(n) = 9T(n/3) + n^2$.
12. Cho $n \le 2 \times 10^5$ và giới hạn thời gian một giây. Với mỗi độ phức tạp sau, cho biết có khả thi không và giải thích: $O(n^2)$, $O(n\sqrt{n})$, $O(n \log^2 n)$, $O(2^n)$.

### Bài tập lập trình mức cơ bản

13. Viết chương trình in ra các số từ 1 đến 100, mỗi số một dòng. Sau đó sửa để chỉ in các số chia hết cho 3 hoặc cho 5. Viết bằng cả ba ngôn ngữ.
14. Viết hàm `dem_am(a)` đếm số phần tử âm trong mảng và hàm `tim_min(a)` trả về giá trị nhỏ nhất. Viết bằng cả ba ngôn ngữ, chú ý sự khác biệt về chỉ số.
15. Viết hàm đảo ngược một mảng tại chỗ, nghĩa là không dùng mảng phụ.
16. Cài đặt thuật toán Euclid bằng cả ba ngôn ngữ. Bổ sung biến đếm số vòng lặp và chạy thử với các cặp số Fibonacci liên tiếp, chẳng hạn 89 và 55, hoặc 144 và 89. Sinh viên có nhận xét gì về số vòng lặp?

### Bài tập lập trình mức vận dụng

17. Cài đặt cả ba cách giải bài toán đoạn con có tổng lớn nhất. Chạy với $n$ bằng 200, 400, 800 và lập bảng thời gian. Đối chiếu với dự đoán lý thuyết.

18. Viết hàm kiểm tra số nguyên tố theo hai cách: thử mọi ước tới $n-1$, và thử tới $\sqrt{n}$. Đo thời gian với $n = 10^9 + 7$ và so sánh.

19. Cài đặt cả ba cách tính số Fibonacci. Với cách đệ quy thuần, thêm biến đếm số lời gọi hàm và lập bảng số lời gọi khi $n$ bằng 10, 20, 30. Số lời gọi tăng theo quy luật nào?

20. Viết chương trình đếm số phép so sánh thực tế của tìm kiếm tuần tự trên mảng ngẫu nhiên $n = 1000$ phần tử, chạy 10 000 lần, rồi so giá trị trung bình thu được với dự đoán lý thuyết $(n+1)/2$.

21. Cho dãy $n$ số nguyên và số nguyên $X$. Viết ba chương trình đếm số cặp chỉ số $(i, j)$ với $i < j$ và $a_i + a_j = X$, có độ phức tạp lần lượt là $O(n^2)$, $O(n \log n)$ và $O(n)$ trung bình.

### Bài tập phần trí tuệ nhân tạo

22. Sửa hàm `phan_loai` để trả về đồng thời tên loài và khoảng cách tới láng giềng gần nhất. Thông tin khoảng cách có ích ở chỗ nào?

23. Bổ sung đặc trưng thứ ba cho mỗi bông hoa. Sửa hàm khoảng cách để làm việc với ba chiều, sau đó viết lại hàm khoảng cách bằng vòng lặp để nó làm việc được với số chiều tuỳ ý.

24. Thêm biến đếm số phép toán vào hàm `phan_loai` và in ra tổng số phép toán đã thực hiện. Kiểm chứng rằng con số này tỉ lệ thuận với $n \times d$.

25. Tìm một cặp giá trị chiều dài và chiều rộng mà thuật toán láng giềng gần nhất cho kết quả không hợp lý theo cảm nhận của sinh viên. Giải thích nguyên nhân và đề xuất hướng khắc phục.

26. Sinh ngẫu nhiên một tập dữ liệu gồm $n = 100\,000$ mẫu với $d = 100$ đặc trưng, rồi đo thời gian phân lớp một mẫu mới bằng cả ba ngôn ngữ. Lập bảng so sánh và giải thích chênh lệch.

CHƯƠNG 2

# Sắp xếp và Tìm kiếm

Chương này trình bày hai nhóm thuật toán có mặt trong hầu hết mọi chương trình thực tế: sắp xếp một dãy phần tử theo thứ tự và tìm một phần tử trong dãy. Thứ tự các mục bám theo đề cương học phần: bốn thuật toán sắp xếp đơn giản, thuật toán sắp xếp nhanh, thuật toán sắp xếp kiểu cơ số, rồi ba thuật toán tìm kiếm.[1]

Hai nhóm thuật toán này gắn với nhau chặt chẽ. Tìm kiếm nhị phân, thuật toán nhanh nhất trong chương, chỉ chạy được trên dãy đã sắp xếp. Vì vậy trong nhiều bài toán, chi phí sắp xếp một lần được bù lại bởi hàng nghìn lần tìm kiếm sau đó. Mục cuối chương áp dụng cả hai vào thuật toán $k$ láng giềng gần nhất và vào bài toán gợi ý $K$ mục có điểm cao nhất.

## 2.1 Bài toán sắp xếp

### 2.1.1 Phát biểu bài toán

**Định nghĩa 2.1.** (*Bài toán sắp xếp*) Cho dãy $n$ phần tử $a_1, a_2, \ldots, a_n$ và một quan hệ thứ tự $\leq$ trên tập giá trị của các phần tử. Hãy sắp xếp lại dãy thành $a_{\sigma(1)}, a_{\sigma(2)}, \ldots, a_{\sigma(n)}$ với $\sigma$ là một hoán vị của $\{1, 2, \ldots, n\}$, sao cho

$$a_{\sigma(1)} \leq a_{\sigma(2)} \leq \ldots \leq a_{\sigma(n)}.$$

Định nghĩa trên nói rằng kết quả phải là một *hoán vị* của dãy ban đầu, tức là không được thêm, bớt hay thay đổi giá trị của phần tử nào. Đây là điều kiện thường bị bỏ quên khi kiểm thử: một chương trình trả về dãy tăng dần nhưng làm mất một phần tử vẫn là chương trình sai.

Trong toàn chương, ta quy ước sắp xếp theo thứ tự **không giảm**. Muốn sắp xếp giảm dần chỉ cần đảo chiều mọi phép so sánh, đổi < thành > và ngược lại.

### 2.1.2 Ba tiêu chí đánh giá

Các thuật toán sắp xếp được so sánh theo ba tiêu chí.

1. **Số phép so sánh.** Mỗi lần chương trình hỏi "phần tử này có nhỏ hơn phần tử kia không" là một phép so sánh. Đây là phép toán tích cực của mọi thuật toán sắp xếp dựa trên so sánh.
2. **Số phép đổi chỗ hoặc di chuyển.** Khi phần tử là số nguyên, chi phí đổi chỗ không đáng kể. Khi phần tử là bản ghi lớn, chẳng hạn một hồ sơ sinh viên gồm nhiều trường, chi phí này vượt xa chi phí so sánh và trở thành yếu tố quyết định.
3. **Bộ nhớ phụ.** Thuật toán chỉ dùng thêm một vài biến được gọi là sắp xếp **tại chỗ** (*in-place*). Thuật toán cần thêm một mảng kích thước $n$ thì tốn $O(n)$ bộ nhớ phụ.

Một tính chất thứ tư cũng quan trọng trong thực tế.

**Định nghĩa 2.2.** (*Tính ổn định*) Thuật toán sắp xếp được gọi là **ổn định** (*stable*) nếu hai phần tử có khoá bằng nhau vẫn giữ nguyên thứ tự tương đối như trong dãy ban đầu.

[1] Đề cương học phần đánh số các mục của chương này là 2.1, 2.2, 2.4, 2.5, 2.6; mục 2.3 không xuất hiện. Tài liệu giữ nguyên thứ tự nội dung nhưng đánh số lại liên tục để tiện trích dẫn.

**Ví dụ 2.1.** *Vì sao tính ổn định có ích.* Cho danh sách sinh viên đã sắp theo tên. Sắp lại danh sách đó theo lớp bằng một thuật toán ổn định thì trong mỗi lớp, sinh viên vẫn giữ thứ tự theo tên. Nếu dùng thuật toán không ổn định, thứ tự theo tên bên trong từng lớp bị xáo trộn và ta phải sắp xếp theo cặp khoá (lớp, tên) ngay từ đầu.

> **Chú ý.** Trong bốn thuật toán đơn giản của mục 2.2, hai thuật toán chèn trực tiếp và nổi bọt là ổn định; hai thuật toán đổi chỗ và lựa chọn thì không, vì chúng đổi chỗ hai phần tử ở xa nhau và có thể nhảy qua một phần tử có khoá bằng.

### 2.1.3 Dãy dùng chung cho các ví dụ

Để tiện đối chiếu, cả bốn thuật toán đơn giản trong chương này đều được lần vết trên cùng một dãy sáu phần tử:

$$a = (8,\ 3,\ 5,\ 1,\ 9,\ 2).$$

Kết quả đúng trong mọi trường hợp là $(1, 2, 3, 5, 8, 9)$. Các bảng lần vết trong chương được sinh trực tiếp từ chương trình đã cài đặt, không phải viết tay, nên số phép so sánh và số phép đổi chỗ nêu trong bảng là số đo thật.

## 2.2 Các thuật toán sắp xếp đơn giản

Bốn thuật toán trong mục này đều có độ phức tạp $O(n^2)$ và đều có dạng hai vòng lặp lồng nhau. Chúng khác nhau ở chỗ vòng lặp trong làm gì.

### 2.2.1 Sắp xếp kiểu đổi chỗ

**Ý tưởng.** Xét lần lượt từng vị trí $i$ từ đầu dãy. Với vị trí $i$ đang xét, so sánh $a_i$ với mọi phần tử đứng sau nó; hễ gặp phần tử nhỏ hơn thì đổi chỗ ngay. Sau khi vòng trong kết thúc, $a_i$ chắc chắn là phần tử nhỏ nhất trong đoạn từ $i$ đến hết dãy.

GIẢ MÃ

```
SAP-XEP-DOI-CHO(a, n)
1  for i ← 1 to n-1
2      for j ← i+1 to n
3          if a[j] < a[i]
4              đổi chỗ a[i] và a[j]
```

Bảng 2.1 cho trạng thái của dãy sau mỗi lần vòng ngoài kết thúc. Phần in đậm là đoạn đã đúng vị trí cuối cùng.

Tổng cộng thuật toán thực hiện 15 phép so sánh và 9 phép đổi chỗ.

JULIA

```julia
function sap_xep_doi_cho!(a)
    n = length(a)
    for i in 1:n-1
        for j in i+1:n
            if a[j] < a[i]
                a[i], a[j] = a[j], a[i]
            end
        end
```

**Bảng 2.1.** Lần vết thuật toán sắp xếp kiểu đổi chỗ trên dãy $(8, 3, 5, 1, 9, 2)$.

| $i$ | Dãy sau khi vòng trong kết thúc | Số phép đổi chỗ ở bước này |
|---|---|---|
| — | 8 3 5 1 9 2 | — |
| 1 | **1** 8 5 3 9 2 | 2 |
| 2 | **1 2** 8 5 9 3 | 3 |
| 3 | **1 2 3** 8 9 5 | 2 |
| 4 | **1 2 3 5** 9 8 | 1 |
| 5 | **1 2 3 5 8** 9 | 1 |

```
    end
    return a
end
```

PYTHON

```python
def sap_xep_doi_cho(a):
    n = len(a)
    for i in range(n - 1):
        for j in range(i + 1, n):
            if a[j] < a[i]:
                a[i], a[j] = a[j], a[i]
    return a
```

C++

```cpp
void sap_xep_doi_cho(vector<int> &a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[i])
                swap(a[i], a[j]);
}
```

**Phân tích.** Vòng ngoài chạy $n-1$ lần, vòng trong ứng với $i$ chạy $n-i$ lần. Tổng số phép so sánh là

$$\sum_{i=1}^{n-1}(n-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} = O(n^2),$$

không phụ thuộc dữ liệu vào. Số phép đổi chỗ nằm giữa 0 và $n(n-1)/2$. Thuật toán chạy tại chỗ, dùng thêm đúng một biến tạm khi đổi chỗ.

**Nhận xét 2.1.** Đây là thuật toán tốn nhiều phép đổi chỗ nhất trong bốn thuật toán đơn giản, vì nó đổi chỗ ngay mỗi khi gặp phần tử nhỏ hơn thay vì chỉ ghi nhớ vị trí. Thuật toán ở mục sau sửa đúng nhược điểm này.

### 2.2.2 Sắp xếp kiểu lựa chọn

**Ý tưởng.** Vẫn xét từng vị trí $i$, nhưng vòng trong chỉ *ghi nhớ* vị trí của phần tử nhỏ nhất trong đoạn còn lại chứ không đổi chỗ. Hết vòng trong mới thực hiện đúng một phép đổi chỗ.

GIẢ MÃ

```
SAP-XEP-LUA-CHON(a, n)
1  for i ← 1 to n-1
2      m ← i                        // vị trí phần tử nhỏ nhất tạm thời
3      for j ← i+1 to n
4          if a[j] < a[m]
5              m ← j
6      if m ≠ i
7          đổi chỗ a[i] và a[m]
```

**Bảng 2.2.** Lần vết thuật toán sắp xếp kiểu lựa chọn trên dãy $(8, 3, 5, 1, 9, 2)$.

| $i$ | Vị trí nhỏ nhất | Dãy sau khi đổi chỗ | Số phép đổi chỗ |
|---|---|---|---|
| — | — | 8 3 5 1 9 2 | — |
| 1 | 4 | **1** 3 5 8 9 2 | 1 |
| 2 | 6 | **1 2** 5 8 9 3 | 1 |
| 3 | 6 | **1 2 3** 8 9 5 | 1 |
| 4 | 6 | **1 2 3 5** 9 8 | 1 |
| 5 | 6 | **1 2 3 5 8** 9 | 1 |

Số phép so sánh vẫn là 15, nhưng số phép đổi chỗ giảm từ 9 xuống 5.

JULIA

```julia
function sap_xep_lua_chon!(a)
    n = length(a)
    for i in 1:n-1
        m = i
        for j in i+1:n
            if a[j] < a[m]
                m = j
            end
        end
        if m != i
            a[i], a[m] = a[m], a[i]
        end
    end
    return a
end
```

PYTHON

```python
def sap_xep_lua_chon(a):
    n = len(a)
    for i in range(n - 1):
        m = i
        for j in range(i + 1, n):
            if a[j] < a[m]:
                m = j
        if m != i:
            a[i], a[m] = a[m], a[i]
    return a
```

C++

```cpp
void sap_xep_lua_chon(vector<int> &a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        int m = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[m]) m = j;
        if (m != i) swap(a[i], a[m]);
    }
}
```

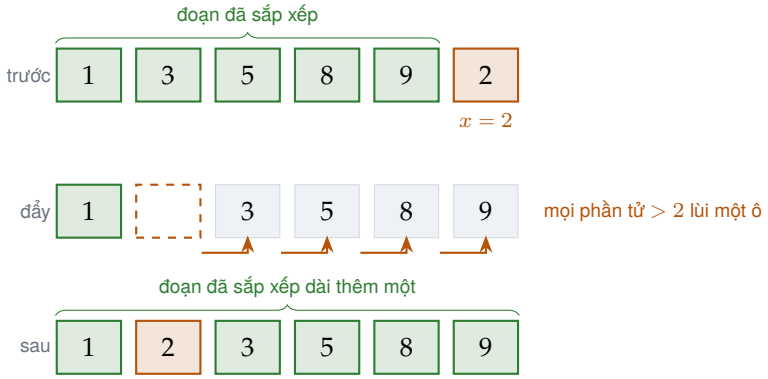
**Phân tích.** Số phép so sánh vẫn là $n(n-1)/2 = O(n^2)$ trong mọi trường hợp. Số phép đổi chỗ nhiều nhất là $n-1$, tức $O(n)$.

**Nhận xét 2.2.** Vì số phép đổi chỗ chỉ là $O(n)$, sắp xếp kiểu lựa chọn là lựa chọn hợp lý khi mỗi phần tử là một bản ghi lớn và chi phí di chuyển dữ liệu cao hơn hẳn chi phí so sánh. Ngoài trường hợp này, các thuật toán $O(n \log n)$ ở các mục sau luôn tốt hơn.

### 2.2.3 Sắp xếp kiểu chèn trực tiếp

**Ý tưởng.** Giữ đoạn đầu dãy luôn ở trạng thái đã sắp xếp. Lấy phần tử tiếp theo, đẩy lùi mọi phần tử lớn hơn nó về sau một vị trí, rồi đặt nó vào chỗ trống vừa tạo ra. Cách làm này giống hệt cách người chơi bài xếp lại quân bài mới rút vào đúng chỗ trong tay bài đã sắp.

Hình 2.1 minh hoạ một bước chèn.




**Hình 2.1.** Một bước của sắp xếp chèn trực tiếp: chèn giá trị 2 vào đoạn đã sắp xếp $(1, 3, 5, 8, 9)$.

GIẢ MÃ

```
SAP-XEP-CHEN(a, n)
1  for i ← 2 to n
2      x ← a[i]                    // phần tử cần chèn
3      j ← i-1
4      while j ≥ 1 and a[j] > x
5          a[j+1] ← a[j]           // đẩy lùi một vị trí
6          j ← j-1
7      a[j+1] ← x                  // đặt x vào chỗ trống
```

Thuật toán thực hiện 12 phép so sánh và 9 phép di chuyển, ít phép so sánh hơn hai thuật toán trước. Nguyên nhân là ở bước $i = 5$, phần tử 9 đã lớn hơn mọi phần tử đứng trước nên vòng `while` dừng ngay sau một phép so sánh.

**Bảng 2.3.** Lần vết thuật toán sắp xếp chèn trực tiếp trên dãy $(8, 3, 5, 1, 9, 2)$.

| $i$ | $x = a_i$ | Dãy sau khi chèn $x$ | Số phép di chuyển |
|---|---|---|---|
| — | — | 8 3 5 1 9 2 | — |
| 2 | 3 | **3 8** 5 1 9 2 | 1 |
| 3 | 5 | **3 5 8** 1 9 2 | 1 |
| 4 | 1 | **1 3 5 8** 9 2 | 3 |
| 5 | 9 | **1 3 5 8 9** 2 | 0 |
| 6 | 2 | **1 2 3 5 8 9** | 4 |

JULIA

```julia
function sap_xep_chen!(a)
    n = length(a)
    for i in 2:n
        x = a[i]
        j = i - 1
        while j >= 1 && a[j] > x
            a[j+1] = a[j]
            j -= 1
        end
        a[j+1] = x
    end
    return a
end
```

PYTHON

```python
def sap_xep_chen(a):
    n = len(a)
    for i in range(1, n):
        x = a[i]
        j = i - 1
        while j >= 0 and a[j] > x:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = x
    return a
```

C++

```cpp
void sap_xep_chen(vector<int> &a) {
    int n = a.size();
    for (int i = 1; i < n; i++) {
        int x = a[i], j = i - 1;
        while (j >= 0 && a[j] > x) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = x;
    }
}
```

**Thứ tự hai điều kiện.** Điều kiện `j >= 0 && a[j] > x` không được đổi chỗ. Nếu viết `a[j] > x && j >= 0` thì khi $j$ bằng $-1$, chương trình đọc `a[-1]` trước khi kiểm tra $j$, gây lỗi truy cập ngoài mảng trong C++ và đọc nhầm phần tử cuối mảng trong Python. Cả ba ngôn ngữ đều tính `&&` theo kiểu *short-circuit*: nếu vế trái sai thì vế phải không được tính.

**Phân tích.** Trường hợp xấu nhất là dãy giảm dần: mỗi phần tử phải lùi về tận đầu dãy, cho $1 + 2 + \cdots + (n-1) = n(n-1)/2 = O(n^2)$ phép so sánh. Trường hợp tốt nhất là dãy đã sắp xếp: vòng `while` dừng ngay lập tức ở mọi bước, cho đúng $n - 1$ phép so sánh và $O(n)$ thời gian.

**Nhận xét 2.3.** Tính chất "nhanh trên dữ liệu gần như đã sắp xếp" khiến sắp xếp chèn được dùng làm bước cuối trong các thư viện sắp xếp chuẩn: thuật toán $O(n \log n)$ chạy trước để chia dãy thành các đoạn ngắn gần đúng thứ tự, sau đó sắp xếp chèn hoàn tất. Số đo ở mục 2.5 cho thấy trên dãy gần như đã sắp xếp với $n = 100\,000$, sắp xếp chèn chỉ mất 25,8 ms so với 1036,7 ms trên dãy ngẫu nhiên, tức nhanh hơn khoảng 40 lần.

### 2.2.4 Sắp xếp kiểu nổi bọt

**Ý tưởng.** Duyệt dãy từ cuối về đầu, so sánh từng cặp phần tử kề nhau và đổi chỗ nếu chúng sai thứ tự. Sau lượt duyệt thứ nhất, phần tử nhỏ nhất "nổi" lên vị trí đầu dãy như bọt khí nổi lên mặt nước. Lặp lại với phần còn lại.

GIẢ MÃ

```
SAP-XEP-NOI-BOT(a, n)
1  for i ← 1 to n-1
2      đổi ← false
3      for j ← n downto i+1
4          if a[j] < a[j-1]
5              đổi chỗ a[j] và a[j-1]
6              đổi ← true
7      if đổi = false                // dãy đã có thứ tự
8          dừng
```

Biến `đổi` ở dòng 2 là một cải tiến quan trọng: nếu suốt một lượt duyệt không có phép đổi chỗ nào thì dãy đã có thứ tự và thuật toán dừng sớm. Nhờ vậy sắp xếp nổi bọt đạt $O(n)$ trên dãy đã sắp xếp, giống sắp xếp chèn.

**Bảng 2.4.** Lần vết thuật toán sắp xếp nổi bọt trên dãy $(8, 3, 5, 1, 9, 2)$.

| Lượt | Dãy sau lượt duyệt | Có đổi chỗ |
|---|---|---|
| — | 8 3 5 1 9 2 | — |
| 1 | **1** 8 3 5 2 9 | có |
| 2 | **1 2** 8 3 5 9 | có |
| 3 | **1 2 3** 8 5 9 | có |
| 4 | **1 2 3 5** 8 9 | có |
| 5 | **1 2 3 5 8 9** | không, dừng |

Thuật toán thực hiện 15 phép so sánh và 9 phép đổi chỗ. Lượt thứ năm không có phép đổi chỗ nào nên vòng ngoài dừng ở đó thay vì chạy tiếp.

JULIA

```julia
function sap_xep_noi_bot!(a)
    n = length(a)
    for i in 1:n-1
        doi = false
        for j in n:-1:i+1
            if a[j] < a[j-1]
                a[j], a[j-1] = a[j-1], a[j]
                doi = true
            end
        end
        doi || break            # không đổi chỗ lần nào thì dừng
    end
    return a
end
```

PYTHON

```python
def sap_xep_noi_bot(a):
    n = len(a)
    for i in range(n - 1):
        doi = False
        for j in range(n - 1, i, -1):
            if a[j] < a[j - 1]:
                a[j], a[j - 1] = a[j - 1], a[j]
                doi = True
        if not doi:
            break               # không đổi chỗ lần nào thì dừng
    return a
```

C++

```cpp
void sap_xep_noi_bot(vector<int> &a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        bool doi = false;
        for (int j = n - 1; j > i; j--)
            if (a[j] < a[j - 1]) {
                swap(a[j], a[j - 1]);
                doi = true;
            }
        if (!doi) break;        // không đổi chỗ lần nào thì dừng
    }
}
```

**Vòng lặp lùi trong ba ngôn ngữ.** Julia viết `n:-1:i+1`, trong đó số $-1$ ở giữa là bước nhảy. Python viết `range(n-1, i, -1)`, chạy từ $n-1$ xuống $i+1$ và *không* lấy giá trị $i$. C++ viết vòng `for` với `j--`. Viết nhầm cận dưới ở đây là lỗi thường gặp: nếu để vòng trong chạy tới $j = i$ thì chương trình đọc `a[i-1]`, tức đọc vào đoạn đã sắp xếp xong.

### 2.2.5 So sánh bốn thuật toán đơn giản

Ba cột đầu là số đo trên dãy sáu phần tử nói trên; hai cột tiếp là độ phức tạp tiệm cận. Cần lưu ý rằng số liệu trên một dãy sáu phần tử không đủ để kết luận thuật toán nào nhanh hơn trong thực tế; phần đo thời gian ở mục 2.5 mới cho câu trả lời có ý nghĩa.

**Bảng 2.5.** Bốn thuật toán sắp xếp đơn giản trên dãy $(8, 3, 5, 1, 9, 2)$ và trong trường hợp tổng quát.

| Thuật toán | So sánh | Đổi chỗ | Tốt nhất | Xấu nhất | Ổn định |
|---|---|---|---|---|---|
| Đổi chỗ | 15 | 9 | $O(n^2)$ | $O(n^2)$ | không |
| Lựa chọn | 15 | 5 | $O(n^2)$ | $O(n^2)$ | không |
| Chèn | 12 | 9 | $O(n)$ | $O(n^2)$ | có |
| Nổi bọt | 15 | 9 | $O(n)$ | $O(n^2)$ | có |

**Bài 2.1.** Cài đặt cả bốn thuật toán trong một chương trình. Thêm hai biến đếm toàn cục `so_sanh` và `doi_cho`, tăng chúng ở đúng chỗ, rồi chạy thử với ba loại dữ liệu vào: dãy ngẫu nhiên, dãy đã sắp xếp tăng dần, dãy sắp xếp giảm dần, mỗi loại với $n = 1000$. Lập bảng kết quả và giải thích vì sao sắp xếp chèn và sắp xếp nổi bọt cho số liệu rất khác nhau giữa ba loại dữ liệu trong khi hai thuật toán còn lại thì không.

## 2.3 Thuật toán sắp xếp nhanh

### 2.3.1 Giới thiệu thuật toán

Sắp xếp nhanh (*QuickSort*) do C. A. R. Hoare đề xuất năm 1960, là thuật toán sắp xếp được dùng nhiều nhất trong thực tế. Nó theo mô hình chia để trị, sẽ được trình bày tổng quát ở chương 4.

**Ý tưởng.** Chọn một phần tử của dãy làm **chốt** (*pivot*). Sắp xếp lại dãy sao cho mọi phần tử nhỏ hơn chốt nằm về bên trái, mọi phần tử lớn hơn chốt nằm về bên phải. Bước này gọi là **phân hoạch**. Sau khi phân hoạch, bài toán ban đầu tách thành hai bài toán con độc lập, mỗi bài toán con lại được giải bằng chính thuật toán này.

Điểm mấu chốt là sau phân hoạch, *không cần* so sánh phần tử ở nửa trái với phần tử ở nửa phải nữa. Chính việc loại bỏ hẳn các phép so sánh này đưa độ phức tạp từ $O(n^2)$ xuống $O(n \log n)$.

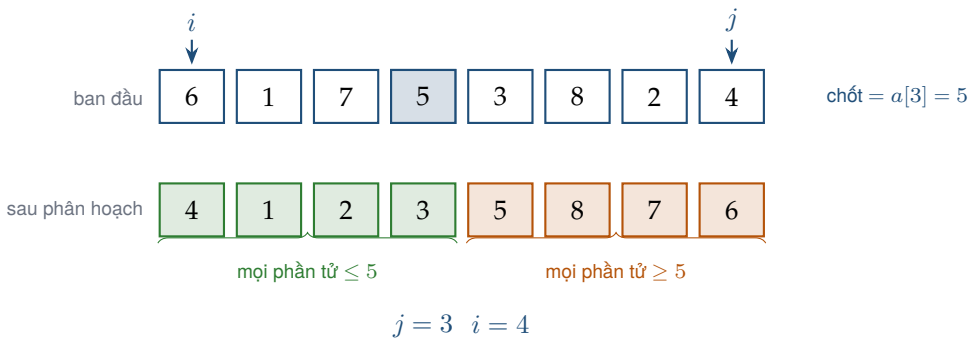
### 2.3.2 Phép phân hoạch

Có nhiều cách phân hoạch. Tài liệu này dùng sơ đồ hai con trỏ chạy ngược chiều của Hoare, với chốt lấy ở giữa đoạn.

1. Đặt `chot` $= a[(l + r)/2]$, đặt $i = l$ và $j = r$.
2. Tăng $i$ chừng nào $a[i] <$ `chot`; giảm $j$ chừng nào $a[j] >$ `chot`.
3. Nếu $i \le j$: đổi chỗ $a[i]$ và $a[j]$, rồi tăng $i$, giảm $j$.
4. Lặp lại bước 2 và 3 cho tới khi $i > j$.

Khi vòng lặp dừng, đoạn $[l..j]$ chỉ chứa các phần tử không lớn hơn chốt và đoạn $[i..r]$ chỉ chứa các phần tử không nhỏ hơn chốt. Hai đoạn này có thể không phủ kín $[l..r]$: các vị trí nằm giữa $j$ và $i$ đã đúng chỗ.

Bảng 2.6 ghi lại toàn bộ lần phân hoạch trong hình 2.2. Chỉ số trong bảng đánh từ 0 để trùng với bản cài đặt Python và C++.




**Hình 2.2.** Một lần phân hoạch dãy $(6, 1, 7, 5, 3, 8, 2, 4)$ với chốt lấy ở giữa đoạn.

**Bảng 2.6.** Lần vết phép phân hoạch đoạn $[0..7]$ của dãy $(6, 1, 7, 5, 3, 8, 2, 4)$, chốt bằng 5.

| Thao tác | Dãy sau thao tác |
|---|---|
| bắt đầu, $i = 0, j = 7$ | 6 1 7 5 3 8 2 4 |
| đổi chỗ $a[0] \leftrightarrow a[7]$ | **4** 1 7 5 3 8 2 **6** |
| đổi chỗ $a[2] \leftrightarrow a[6]$ | 4 1 **2** 5 3 8 **7** 6 |
| đổi chỗ $a[3] \leftrightarrow a[4]$ | 4 1 2 **3** **5** 8 7 6 |
| dừng với $i = 4, j = 3$ | 4 1 2 3 \| 5 8 7 6 |

### 2.3.3 Toàn bộ quá trình sắp xếp

Sau lần phân hoạch đầu tiên, thuật toán gọi đệ quy trên $[0..3]$ rồi trên $[4..7]$. Bảng 2.7 ghi lại tất cả các lần phân hoạch cho tới khi dãy được sắp xếp xong.

**Bảng 2.7.** Toàn bộ các lần phân hoạch khi sắp xếp nhanh dãy $(6, 1, 7, 5, 3, 8, 2, 4)$.

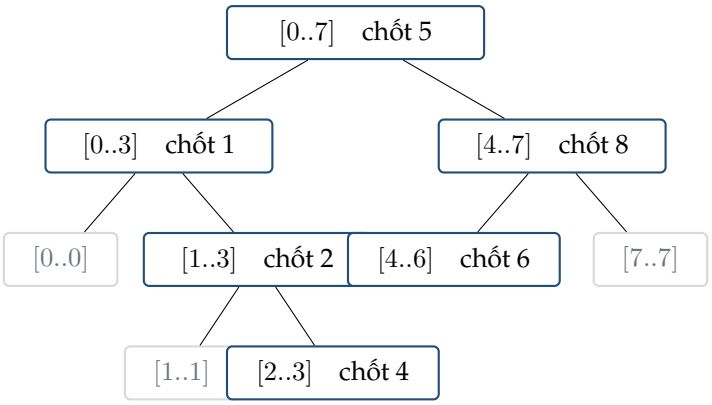
| Mức đệ quy | Đoạn | Chốt | Dãy sau khi phân hoạch xong |
|---|---|---|---|
| 0 | $[0..7]$ | 5 | 4 1 2 3 5 8 7 6 |
| 1 | $[0..3]$ | 1 | 1 4 2 3 5 8 7 6 |
| 2 | $[1..3]$ | 2 | 1 2 4 3 5 8 7 6 |
| 3 | $[2..3]$ | 4 | 1 2 3 4 5 8 7 6 |
| 1 | $[4..7]$ | 8 | 1 2 3 4 5 6 7 8 |
| 2 | $[4..6]$ | 6 | 1 2 3 4 5 6 7 8 |

Hình 2.3 biểu diễn cùng quá trình đó dưới dạng cây các lời gọi đệ quy.

### 2.3.4 Cài đặt thuật toán

GIẢ MÃ

```
SAP-XEP-NHANH(a, l, r)
1  if l ≥ r
2      return                        // đoạn rỗng hoặc một phần tử
3  chot ← a[(l+r) div 2]
4  i ← l;  j ← r
5  while i ≤ j
6      while a[i] < chot:  i ← i+1
7      while a[j] > chot:  j ← j-1
8      if i ≤ j
```




**Hình 2.3.** Cây các lời gọi đệ quy của sắp xếp nhanh trên dãy $(6, 1, 7, 5, 3, 8, 2, 4)$. Ô viền mờ là đoạn có một phần tử, dừng ngay.

```
9          if i < j: đổi chỗ a[i] và a[j]
10         i ← i+1;  j ← j-1
11 SAP-XEP-NHANH(a, l, j)
12 SAP-XEP-NHANH(a, i, r)
```

**JULIA**

```julia
function sap_xep_nhanh!(a, l = 1, r = length(a))
    l >= r && return a
    chot = a[(l + r) ÷ 2]
    i, j = l, r
    while i <= j
        while a[i] < chot;  i += 1;  end
        while a[j] > chot;  j -= 1;  end
        if i <= j
            if i < j
                a[i], a[j] = a[j], a[i]
            end
            i += 1
            j -= 1
        end
    end
    sap_xep_nhanh!(a, l, j)
    sap_xep_nhanh!(a, i, r)
    return a
end
```

**PYTHON**

```python
def sap_xep_nhanh(a, l=None, r=None):
    if l is None:
        l, r = 0, len(a) - 1
    if l >= r:
        return a
    chot = a[(l + r) // 2]
    i, j = l, r
    while i <= j:
        while a[i] < chot:
            i += 1
        while a[j] > chot:
            j -= 1
```

```
        if i <= j:
            if i < j:
                a[i], a[j] = a[j], a[i]
            i += 1
            j -= 1
    sap_xep_nhanh(a, l, j)
    sap_xep_nhanh(a, i, r)
    return a
```

C++

```cpp
void sap_xep_nhanh(vector<int> &a, int l, int r) {
    if (l >= r) return;
    int chot = a[(l + r) / 2];
    int i = l, j = r;
    while (i <= j) {
        while (a[i] < chot) i++;
        while (a[j] > chot) j--;
        if (i <= j) {
            if (i < j) swap(a[i], a[j]);
            i++;
            j--;
        }
    }
    sap_xep_nhanh(a, l, j);
    sap_xep_nhanh(a, i, r);
}
// gọi: sap_xep_nhanh(a, 0, (int)a.size() - 1);
```

**Ba lỗi hay gặp khi cài đặt sắp xếp nhanh.**

1. Viết `a[i] <= chot` thay cho `a[i] < chot`. Khi mọi phần tử bằng nhau, con trỏ $i$ chạy thẳng qua hết đoạn, hai lời gọi đệ quy không thu nhỏ được đoạn và chương trình lặp vô hạn.
2. Lấy chốt bằng *giá trị* `a[(l+r)/2]` chứ không phải bằng *chỉ số*. Nếu lưu chỉ số rồi trong quá trình đổi chỗ phần tử ở chỉ số đó bị chuyển đi nơi khác, giá trị chốt thay đổi giữa chừng và thuật toán sai.
3. Gọi đệ quy nhầm thành `(a, l, i)` và `(a, j, r)`. Hai đoạn này chồng lên nhau, sinh vòng lặp vô hạn.

### 2.3.5 Độ phức tạp

**Định lý 2.1.** (*Độ phức tạp trung bình của sắp xếp nhanh*) *Nếu mỗi lần phân hoạch chia đoạn thành hai phần có kích thước xấp xỉ bằng nhau thì sắp xếp nhanh có độ phức tạp* $O(n \log n)$.

*Chứng minh.* Gọi $T(n)$ là số phép so sánh khi sắp xếp $n$ phần tử. Phép phân hoạch duyệt đoạn đúng một lượt nên tốn $O(n)$. Nếu hai nửa có kích thước $n/2$ thì

$$T(n) = 2\,T(n/2) + O(n).$$

Khai triển liên tiếp: $T(n) = 2(2T(n/4) + n/2) + n = 4T(n/4) + 2n$, và sau $k$ lần là $T(n) = 2^k\,T(n/2^k) + kn$. Quá trình dừng khi $n/2^k = 1$, tức $k = \log_2 n$, cho $T(n) = n \cdot T(1) + n \log_2 n = O(n \log n)$. ∎

**Nhận xét 2.4.** [Trường hợp xấu nhất] Nếu mỗi lần phân hoạch chốt lại là phần tử nhỏ nhất hoặc lớn nhất của đoạn thì một nửa rỗng và nửa kia chỉ nhỏ đi một phần tử, cho

$$T(n) = T(n-1) + O(n) = O(n^2).$$

Với cách lấy chốt ở giữa đoạn, trường hợp này hầu như không xảy ra trên dữ liệu thực. Nhưng nếu lấy chốt là `a[l]`, dãy đã sắp xếp sẵn lại rơi đúng vào trường hợp xấu nhất, và đó là dạng dữ liệu rất hay gặp. Đây là lý do không nên lấy chốt ở đầu đoạn.

Ngoài thời gian, sắp xếp nhanh còn tốn bộ nhớ cho ngăn xếp lời gọi đệ quy, sâu $O(\log n)$ trong trường hợp trung bình và $O(n)$ trong trường hợp xấu nhất. Với $n$ lớn và dữ liệu bất lợi, chương trình có thể tràn ngăn xếp trước khi chạy hết thời gian cho phép.

> **Bài 2.2.** Cài đặt hai bản sắp xếp nhanh, một bản lấy chốt bằng `a[l]` và một bản lấy chốt ở giữa đoạn. Chạy cả hai trên ba loại dữ liệu với $n = 100\,000$: dãy ngẫu nhiên, dãy tăng dần, dãy giảm dần. Ghi lại thời gian chạy. Bản lấy chốt đầu đoạn sẽ tràn ngăn xếp hoặc chạy rất lâu trên hai loại dữ liệu sau; hãy giải thích bằng cây đệ quy tương ứng.

## 2.4 Thuật toán sắp xếp kiểu cơ số

### 2.4.1 Giới thiệu thuật toán

Mọi thuật toán sắp xếp dựa trên so sánh đều cần ít nhất $\Omega(n \log n)$ phép so sánh; đây là một kết quả lý thuyết đã được chứng minh. Muốn nhanh hơn, thuật toán phải khai thác cấu trúc của dữ liệu thay vì chỉ so sánh.

Sắp xếp kiểu cơ số (*Radix Sort*) làm đúng điều đó: nó không so sánh hai phần tử với nhau lần nào. Thay vào đó, nó phân phối các số vào thùng theo từng chữ số, bắt đầu từ hàng đơn vị.

### 2.4.2 Mô tả thuật toán

Với dãy số nguyên không âm có nhiều nhất $d$ chữ số, thuật toán thực hiện $d$ lượt. Ở lượt thứ $k$:

1. Chuẩn bị 10 thùng, đánh số từ 0 đến 9.
2. Duyệt dãy theo thứ tự hiện tại, bỏ mỗi số vào thùng ứng với chữ số hàng thứ $k$ của nó.
3. Nối các thùng lại theo thứ tự 0, 1, ..., 9 để được dãy mới.

Điều kiện để thuật toán đúng là bước phân phối phải *ổn định*: các số vào cùng một thùng phải giữ nguyên thứ tự tương đối. Nhờ tính ổn định, kết quả sắp xếp theo các hàng thấp hơn không bị phá vỡ khi sắp xếp theo hàng cao hơn.

**Ví dụ 2.2.** *Lần vết sắp xếp cơ số.* Sắp xếp dãy $(329, 457, 657, 839, 436, 720, 355)$.

**Lượt 1 — hàng đơn vị.**

| | |
|---|---|
| Thùng 0 | 720 |
| Thùng 5 | 355 |
| Thùng 6 | 436 |
| Thùng 7 | 457, 657 |
| Thùng 9 | 329, 839 |
| Kết quả | 720, 355, 436, 457, 657, 329, 839 |

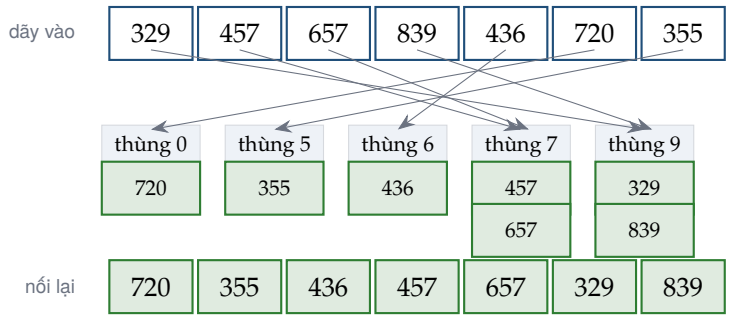
**Lượt 2 — hàng chục.**

| | |
|---|---|
| Thùng 2 | 720, 329 |
| Thùng 3 | 436, 839 |
| Thùng 5 | 355, 457, 657 |
| Kết quả | 720, 329, 436, 839, 355, 457, 657 |

**Lượt 3 — hàng trăm.**

| | |
|---|---|
| Thùng 3 | 329, 355 |
| Thùng 4 | 436, 457 |
| Thùng 6 | 657 |
| Thùng 7 | 720 |
| Thùng 8 | 839 |
| Kết quả | 329, 355, 436, 457, 657, 720, 839 |

Sau ba lượt, dãy đã được sắp xếp.




**Hình 2.4.** Lượt phân phối theo hàng đơn vị của sắp xếp cơ số. Các số vào cùng thùng giữ nguyên thứ tự tương đối.

### 2.4.3 Cài đặt thuật toán

Bản cài đặt dưới đây không dùng danh sách thùng thật mà dùng *sắp xếp đếm phân phối*: đếm số lượng phần tử ứng với mỗi chữ số, cộng dồn để biết vị trí bắt đầu của từng nhóm, rồi ghi thẳng vào mảng kết quả. Cách này nhanh hơn và không cần cấp phát động.

JULIA

```julia
function sap_xep_co_so!(a)
    isempty(a) && return a
    mx = maximum(a)
    tmp = similar(a)
    exp = 1
    while mx ÷ exp > 0
        dem = zeros(Int, 10)
        for x in a
            dem[(x ÷ exp) % 10 + 1] += 1        # +1 vì Julia đánh số từ 1
        end
        for d in 2:10
            dem[d] += dem[d-1]                  # cộng dồn
        end
        for i in length(a):-1:1                 # duyệt lùi để giữ tính ổn định
            d = (a[i] ÷ exp) % 10 + 1
            tmp[dem[d]] = a[i]
```

```
            dem[d] -= 1
        end
        a .= tmp
        exp *= 10
    end
    return a
end
```

PYTHON

```python
def sap_xep_co_so(a):
    if not a:
        return a
    mx = max(a)
    exp = 1
    while mx // exp > 0:
        dem = [0] * 10
        for x in a:
            dem[(x // exp) % 10] += 1
        for d in range(1, 10):
            dem[d] += dem[d - 1]                # cộng dồn
        tmp = [0] * len(a)
        for i in range(len(a) - 1, -1, -1):     # duyệt lùi để giữ tính ổn định
            d = (a[i] // exp) % 10
            dem[d] -= 1
            tmp[dem[d]] = a[i]
        a[:] = tmp
        exp *= 10
    return a
```

C++

```cpp
void sap_xep_co_so(vector<int> &a) {
    if (a.empty()) return;
    int mx = *max_element(a.begin(), a.end());
    vector<int> tmp(a.size());
    for (long long exp = 1; mx / exp > 0; exp *= 10) {
        int dem[10] = {0};
        for (int x : a) dem[(x / exp) % 10]++;
        for (int d = 1; d < 10; d++) dem[d] += dem[d - 1];   // cộng dồn
        for (int i = a.size() - 1; i >= 0; i--) {            // duyệt lùi
            int d = (a[i] / exp) % 10;
            tmp[--dem[d]] = a[i];
        }
        a = tmp;
    }
}
```

**Vì sao phải duyệt lùi.** Sau khi cộng dồn, `dem[d]` bằng số phần tử có chữ số nhỏ hơn hoặc bằng $d$, tức bằng vị trí *ngay sau* nhóm $d$ trong mảng kết quả. Duyệt mảng từ cuối về đầu và mỗi lần giảm `dem[d]` đi một, các phần tử cùng nhóm được ghi vào từ phải sang trái, do đó phần tử đứng trước trong mảng gốc cũng đứng trước trong mảng kết quả. Nếu duyệt xuôi, thứ tự trong mỗi nhóm bị đảo ngược, thuật toán mất tính ổn định và cho kết quả sai ngay từ lượt thứ hai.

### 2.4.4 Độ phức tạp và phạm vi áp dụng

Mỗi lượt duyệt mảng đúng ba lần, tốn $O(n + b)$ với $b = 10$ là số thùng. Có $d$ lượt nên tổng chi phí là $O(d\,(n + b))$. Khi $d$ và $b$ là hằng số, độ phức tạp trở thành $O(n)$, nhanh hơn mọi thuật toán dựa trên so sánh.

Đổi lại, thuật toán có ba hạn chế.

- Chỉ áp dụng trực tiếp cho số nguyên không âm hoặc dữ liệu tách được thành các chữ số. Với số âm phải tách riêng phần âm, sắp xếp theo trị tuyệt đối rồi đảo ngược.
- Cần thêm mảng phụ kích thước $n$, tức $O(n)$ bộ nhớ phụ.
- Nếu các số rất lớn thì $d$ lớn theo, và hằng số $d$ có thể lấn át lợi thế so với $O(n \log n)$.

## 2.5 So sánh các thuật toán sắp xếp

Bảng 2.8 ghi thời gian chạy đo được của các thuật toán trong chương. Chương trình viết bằng C++, biên dịch bằng `g++` với tuỳ chọn `-O2`, dữ liệu là số nguyên ngẫu nhiên trong khoảng $[0,\ 10^6)$. Mỗi ô là thời gian của một lần chạy; dấu gạch ngang nghĩa là phép đo bị bỏ qua vì quá lâu.

**Bảng 2.8.** Thời gian sắp xếp đo được, đơn vị mi-li-giây.

| Thuật toán | $n = 10^4$ | $n = 10^5$ | $n = 10^6$ |
|---|---|---|---|
| Nổi bọt | 200,4 | 27 848,3 | — |
| Lựa chọn | 123,3 | 12 323,8 | — |
| Chèn trực tiếp | 11,1 | 1 036,7 | — |
| Sắp xếp nhanh | 0,6 | 7,3 | 86,4 |
| Sắp xếp cơ số | 0,4 | 4,3 | 42,7 |
| `std::sort` của thư viện | 0,5 | 6,0 | 71,9 |

Ba nhận xét rút ra từ bảng số liệu.

1. Khi $n$ tăng gấp mười, thời gian của các thuật toán $O(n^2)$ tăng khoảng một trăm lần: nổi bọt đi từ 200 ms lên gần 28 giây. Thời gian của các thuật toán $O(n \log n)$ chỉ tăng khoảng mười hai lần.
2. Ở $n = 10^5$, sắp xếp nhanh chạy nhanh hơn sắp xếp nổi bọt khoảng 3800 lần. Khoảng cách này không thể bù bằng máy tính mạnh hơn.
3. Trong ba thuật toán $O(n^2)$, sắp xếp chèn nhanh hơn hẳn hai thuật toán kia dù cùng lớp độ phức tạp, vì vòng trong của nó chỉ gồm một phép so sánh và một phép gán, còn vòng trong của hai thuật toán kia có thêm phép đổi chỗ hoặc phép cập nhật chỉ số.

Trên dãy gần như đã sắp xếp với $n = 100\,000$ (dãy tăng dần rồi hoán vị ngẫu nhiên 1000 cặp), sắp xếp chèn chỉ mất 25,8 ms thay vì 1036,7 ms.

> **Khi nào nên tự viết thuật toán sắp xếp.** Trong bài toán thực tế, nên dùng hàm sắp xếp của thư viện chuẩn: `sort!` của Julia, `sorted/list.sort` của Python, `std::sort` của C++. Chúng đã được tối ưu kỹ và tránh được trường hợp xấu nhất. Việc tự cài đặt trong chương này nhằm hai mục đích: hiểu cơ chế bên trong, và chuẩn bị cho các bài thi yêu cầu cài đặt thủ công.

**Bài 2.3.** Đo lại bảng 2.8 trên máy của mình. Kết quả tuyệt đối sẽ khác vì phần cứng khác, nhưng tỉ lệ giữa các dòng phải tương tự. Sau đó chạy thêm với dữ liệu là dãy đã sắp xếp sẵn và ghi nhận thuật toán nào nhanh lên, thuật toán nào chậm đi.

## 2.6 Các thuật toán tìm kiếm

**Định nghĩa 2.3.** (*Bài toán tìm kiếm*) Cho dãy $a_1, a_2, \ldots, a_n$ và một giá trị $x$. Hãy tìm chỉ số $i$ sao cho $a_i = x$, hoặc kết luận rằng $x$ không có trong dãy.

Dãy dùng cho các ví dụ trong mục này là

$$a = (2,\ 5,\ 8,\ 12,\ 16,\ 23,\ 38,\ 56,\ 72,\ 91),$$

đã sắp xếp tăng dần, gồm 10 phần tử. Chỉ số được đánh từ 0 để trùng với bản cài đặt Python và C++.

### 2.6.1 Tìm kiếm tuyến tính

**Ý tưởng.** Duyệt lần lượt từ đầu dãy, so sánh từng phần tử với $x$, gặp thì trả về chỉ số, hết dãy mà không gặp thì trả về giá trị báo không tìm thấy.

GIẢ MÃ

```
TIM-TUYEN-TINH(a, n, x)
1  for i ← 1 to n
2      if a[i] = x
3          return i
4  return KHONG-TIM-THAY
```

JULIA

```julia
function tim_tuyen_tinh(a, x)
    for i in 1:length(a)
        if a[i] == x
            return i
        end
    end
    return 0                # 0 là chỉ số không hợp lệ trong Julia
end
```

PYTHON

```python
def tim_tuyen_tinh(a, x):
    for i in range(len(a)):
        if a[i] == x:
            return i
    return -1               # -1 là quy ước báo không tìm thấy
```

C++

```cpp
int tim_tuyen_tinh(const vector<int> &a, int x) {
    for (int i = 0; i < (int)a.size(); i++)
        if (a[i] == x) return i;
```

```
    return -1;
}
```

**Giá trị báo không tìm thấy.** Julia đánh chỉ số từ 1 nên giá trị 0 là chỉ số không hợp lệ và dùng được làm tín hiệu báo lỗi; Python và C++ đánh chỉ số từ 0 nên phải dùng $-1$. Trong Python, chỉ số $-1$ lại là một chỉ số *hợp lệ*, trỏ vào phần tử cuối mảng. Vì vậy phải kiểm tra `if vt != -1` chứ không được viết `if a[vt] == x` sau khi gọi hàm.

**Phân tích.** Trường hợp tốt nhất là $x$ nằm ở đầu dãy, tốn một phép so sánh. Trường hợp xấu nhất là $x$ ở cuối dãy hoặc không có trong dãy, tốn $n$ phép so sánh. Trung bình, nếu $x$ chắc chắn có trong dãy và mọi vị trí đều đồng khả năng thì số phép so sánh là

$$\frac{1 + 2 + \cdots + n}{n} = \frac{n+1}{2} = O(n).$$

Ưu điểm của tìm kiếm tuyến tính là không đòi hỏi gì ở dữ liệu: dãy không cần sắp xếp, không cần truy cập ngẫu nhiên. Đây là thuật toán tìm kiếm duy nhất dùng được trên danh sách liên kết ở chương 5.

### 2.6.2 Tìm kiếm nhị phân

**Điều kiện áp dụng.** Dãy phải đã được sắp xếp.

**Ý tưởng.** So sánh $x$ với phần tử ở giữa dãy. Nếu bằng thì xong. Nếu $x$ lớn hơn, mọi phần tử từ giữa trở về trước đều nhỏ hơn $x$ nên có thể loại bỏ toàn bộ nửa trái; ngược lại loại bỏ nửa phải. Mỗi bước loại đi một nửa số ứng viên.




**Hình 2.5.** Tìm kiếm nhị phân giá trị 23. Mỗi bước loại bỏ khoảng một nửa số ứng viên còn lại.

GIẢ MÃ

```
TIM-NHI-PHAN(a, n, x)
1  l ← 1;  r ← n
2  while l ≤ r
3      giua ← l + (r-l) div 2
4      if a[giua] = x
5          return giua
6      if a[giua] < x
7          l ← giua + 1              // x nằm ở nửa phải
8      else
9          r ← giua - 1              // x nằm ở nửa trái
10 return KHONG-TIM-THAY
```

Bảng 2.9 lần vết hai lần tìm kiếm trên dãy 10 phần tử.

**Bảng 2.9.** Lần vết tìm kiếm nhị phân trên dãy $(2, 5, 8, 12, 16, 23, 38, 56, 72, 91)$.

| Bước | $l$ | $r$ | giua | Kết luận |
|---|---|---|---|---|
| *Tìm $x = 23$* | | | | |
| 1 | 0 | 9 | 4 | $a[4] = 16 < 23$, đặt $l = 5$ |
| 2 | 5 | 9 | 7 | $a[7] = 56 > 23$, đặt $r = 6$ |
| 3 | 5 | 6 | 5 | $a[5] = 23$, tìm thấy |
| *Tìm $x = 7$ (không có trong dãy)* | | | | |
| 1 | 0 | 9 | 4 | $a[4] = 16 > 7$, đặt $r = 3$ |
| 2 | 0 | 3 | 1 | $a[1] = 5 < 7$, đặt $l = 2$ |
| 3 | 2 | 3 | 2 | $a[2] = 8 > 7$, đặt $r = 1$ |
| — | 2 | 1 | — | $l > r$, dừng, không tìm thấy |

**JULIA**

```julia
function tim_nhi_phan(a, x)
    l, r = 1, length(a)
    while l <= r
        giua = l + (r - l) ÷ 2
        if a[giua] == x
            return giua
        elseif a[giua] < x
            l = giua + 1
        else
            r = giua - 1
        end
    end
    return 0
end
```

**PYTHON**

```python
def tim_nhi_phan(a, x):
    l, r = 0, len(a) - 1
    while l <= r:
        giua = l + (r - l) // 2
        if a[giua] == x:
            return giua
        elif a[giua] < x:
            l = giua + 1
        else:
            r = giua - 1
    return -1
```

**C++**

```cpp
int tim_nhi_phan(const vector<int> &a, int x) {
    int l = 0, r = (int)a.size() - 1;
    while (l <= r) {
        int giua = l + (r - l) / 2;
        if (a[giua] == x) return giua;
        if (a[giua] < x) l = giua + 1;
        else             r = giua - 1;
    }
```

```
    return -1;
}
```

**Vì sao viết `l + (r-l)/2` chứ không phải `(l+r)/2`.** Hai biểu thức bằng nhau về mặt toán học. Nhưng trong C++, nếu $l$ và $r$ đều gần giá trị lớn nhất của kiểu `int` thì $l+r$ vượt quá phạm vi biểu diễn và cho kết quả âm, làm chương trình truy cập ngoài mảng. Biểu thức $l+(r-l)/2$ không bao giờ vượt quá $r$ nên luôn an toàn. Đây là lỗi từng tồn tại nhiều năm trong thư viện chuẩn của một số ngôn ngữ. Trong Python và Julia, số nguyên tự động mở rộng nên lỗi này không xảy ra, nhưng nên giữ cùng một thói quen viết ở cả ba ngôn ngữ.

**Định lý 2.2.** *(Độ phức tạp của tìm kiếm nhị phân) Tìm kiếm nhị phân trên dãy $n$ phần tử thực hiện nhiều nhất $\lfloor \log_2 n \rfloor + 1$ phép so sánh, tức có độ phức tạp $O(\log n)$.*

*Chứng minh.* Gọi $m_k$ là số phần tử còn trong đoạn tìm kiếm sau $k$ bước. Ban đầu $m_0 = n$. Mỗi bước loại bỏ phần tử ở giữa cùng với một nửa đoạn, nên $m_{k+1} \le \lfloor m_k/2 \rfloor$. Suy ra $m_k \le n/2^k$. Vòng lặp dừng khi đoạn rỗng, tức khi $n/2^k < 1$, xảy ra với $k > \log_2 n$. Vậy số bước nhiều nhất là $\lfloor \log_2 n \rfloor + 1$. ∎

Sự khác biệt giữa $O(n)$ và $O(\log n)$ được minh hoạ bằng số đo sau. Trên dãy $n = 1\,000\,000$ phần tử, thực hiện 1000 truy vấn ngẫu nhiên: tìm kiếm tuyến tính thực hiện tổng cộng 740 686 263 phép so sánh, tìm kiếm nhị phân thực hiện 19 418 phép, ít hơn khoảng 38 000 lần.

**Hàm thư viện.** Trong bài thi và trong công việc, nên dùng hàm có sẵn thay vì tự viết: `searchsortedfirst` của Julia, `bisect.bisect_left` của Python, `std::lower_bound` của C++. Cả ba trả về vị trí đầu tiên có giá trị không nhỏ hơn $x$, tiện hơn hẳn khi cần đếm số phần tử nhỏ hơn $x$ hoặc khi dãy có phần tử trùng nhau.

C++

```
auto it = lower_bound(a.begin(), a.end(), x);
if (it != a.end() && *it == x)
    cout << "tim thay tai vi tri " << (it - a.begin()) << "\n";
else
    cout << "khong tim thay\n";
```

### 2.6.3 Tìm kiếm nội suy

**Điều kiện áp dụng.** Dãy đã sắp xếp và các giá trị phân bố tương đối đều.

**Ý tưởng.** Tìm kiếm nhị phân luôn chọn điểm giữa, không quan tâm giá trị cần tìm là bao nhiêu. Người tra từ điển thì không làm vậy: tìm từ bắt đầu bằng chữ "b" thì mở gần đầu quyển, tìm từ bắt đầu bằng chữ "x" thì mở gần cuối. Tìm kiếm nội suy đưa cách nghĩ đó vào công thức: thay vì lấy điểm giữa, ước lượng vị trí của $x$ bằng phép nội suy tuyến tính giữa hai đầu đoạn:

$$\texttt{giua} = l + \left\lfloor \frac{(x - a_l)\,(r - l)}{a_r - a_l} \right\rfloor .$$

JULIA

```julia
function tim_noi_suy(a, x)
    l, r = 1, length(a)
    while l <= r && a[l] <= x <= a[r]
        if a[l] == a[r]
            return a[l] == x ? l : 0
        end
        giua = l + ((x - a[l]) * (r - l)) ÷ (a[r] - a[l])
        if a[giua] == x
            return giua
        elseif a[giua] < x
            l = giua + 1
        else
            r = giua - 1
        end
    end
    return 0
end
```

PYTHON

```python
def tim_noi_suy(a, x):
    l, r = 0, len(a) - 1
    while l <= r and a[l] <= x <= a[r]:
        if a[l] == a[r]:
            return l if a[l] == x else -1
        giua = l + (x - a[l]) * (r - l) // (a[r] - a[l])
        if a[giua] == x:
            return giua
        elif a[giua] < x:
            l = giua + 1
        else:
            r = giua - 1
    return -1
```

C++

```cpp
int tim_noi_suy(const vector<int> &a, int x) {
    int l = 0, r = (int)a.size() - 1;
    while (l <= r && x >= a[l] && x <= a[r]) {
        if (a[l] == a[r]) return a[l] == x ? l : -1;
        int giua = l + (int)((long long)(x - a[l]) * (r - l) / (a[r] - a[l]));
        if (a[giua] == x) return giua;
        if (a[giua] < x) l = giua + 1;
        else             r = giua - 1;
    }
    return -1;
}
```

**Ba chi tiết bắt buộc.** Điều kiện vòng lặp phải có thêm `a[l] <= x <= a[r]`. Không có điều kiện này, khi $x$ nằm ngoài khoảng giá trị của đoạn, công thức nội suy cho chỉ số nằm ngoài mảng. Trường hợp `a[l] == a[r]` phải xử lý riêng vì mẫu số bằng 0. Trong C++, tích $(x - a_l)(r - l)$ phải ép sang `long long` vì với mảng lớn và giá trị lớn, tích này tràn kiểu `int`.

**Ví dụ 2.3.** *So sánh số bước.* Trên dãy đều $(10, 20, 30, \ldots, 200)$ gồm 20 phần tử:

| Giá trị cần tìm | Nội suy | Nhị phân |
|---|---|---|
| $x = 70$ | 1 bước | 3 bước |
| $x = 190$ | 1 bước | 4 bước |
| $x = 10$ | 1 bước | 4 bước |

Với dữ liệu phân bố hoàn toàn đều, công thức nội suy trỏ đúng ngay từ lần đầu.

**Phân tích.** Trên dữ liệu phân bố đều, tìm kiếm nội suy có độ phức tạp trung bình $O(\log \log n)$, nhanh hơn hẳn tìm kiếm nhị phân. Nhưng trên dữ liệu phân bố lệch, chẳng hạn dãy gồm các số $1, 2, \ldots, 99$ rồi một số $10^9$ ở cuối, công thức nội suy liên tục trỏ về gần đầu đoạn và độ phức tạp xấu nhất trở lại $O(n)$. Tìm kiếm nhị phân giữ $O(\log n)$ trong mọi trường hợp, nên trong bài thi có ràng buộc thời gian, tìm kiếm nhị phân vẫn là lựa chọn an toàn hơn.

**Bảng 2.10.** So sánh ba thuật toán tìm kiếm.

| Thuật toán | Yêu cầu dữ liệu | Trung bình | Xấu nhất | Ghi chú |
|---|---|---|---|---|
| Tuyến tính | không | $O(n)$ | $O(n)$ | dùng được trên mọi cấu trúc |
| Nhị phân | đã sắp xếp | $O(\log n)$ | $O(\log n)$ | lựa chọn mặc định |
| Nội suy | đã sắp xếp, phân bố đều | $O(\log \log n)$ | $O(n)$ | nhanh nhất khi dữ liệu đều |

**Bài 2.4.** Sinh một mảng $n = 10^6$ số nguyên đã sắp xếp theo hai cách: phân bố đều (`a[i] = i * 10`) và phân bố lệch (`a[i] = i` với $i < n-1$, riêng phần tử cuối bằng $10^9$). Với mỗi mảng, chạy 1000 truy vấn ngẫu nhiên bằng cả ba thuật toán, đếm tổng số phép so sánh. Đối chiếu kết quả với bảng 2.10 và giải thích vì sao tìm kiếm nội suy chậm hẳn đi trên mảng thứ hai.

## 2.7 Ứng dụng: thuật toán $k$ láng giềng gần nhất

Mục 1.5 của chương 1 đã trình bày thuật toán láng giềng gần nhất: gán cho mẫu mới nhãn của mẫu huấn luyện gần nó nhất. Mục này mở rộng thuật toán đó theo hai hướng, và cả hai hướng đều dùng đến các thuật toán vừa học trong chương.

### 2.7.1 Từ một láng giềng đến $k$ láng giềng

Thuật toán một láng giềng có một điểm yếu rõ rệt: nó phụ thuộc hoàn toàn vào một mẫu duy nhất. Nếu mẫu đó bị ghi sai nhãn, mọi mẫu mới rơi gần nó đều bị phân loại sai.

Thuật toán $k$ láng giềng gần nhất khắc phục bằng cách lấy $k$ mẫu gần nhất rồi cho chúng bỏ phiếu. Nhãn nào xuất hiện nhiều nhất trong $k$ mẫu đó được gán cho mẫu mới. Một mẫu bị ghi sai nhãn khi đó chỉ là một phiếu trong $k$ phiếu.

**Định nghĩa 2.4.** (*Thuật toán $k$ láng giềng gần nhất*) Cho tập huấn luyện $D = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ với $x_i$ là véc-tơ đặc trưng và $y_i$ là nhãn, cho mẫu mới $x$ và số nguyên dương $k$. Sắp xếp $D$ theo khoảng cách $d(x, x_i)$ tăng dần, lấy $k$ mẫu đầu tiên, và trả về nhãn xuất hiện nhiều nhất trong $k$ nhãn tương ứng.

Định nghĩa nói "sắp xếp $D$ theo khoảng cách": đây chính là chỗ thuật toán sắp xếp của chương

này được dùng. Giá trị $k$ thường lấy lẻ để tránh hoà phiếu khi có hai lớp.

### 2.7.2 Vì sao phải chuẩn hoá đặc trưng

Khoảng cách Euclid cộng bình phương chênh lệch của mọi đặc trưng. Nếu các đặc trưng có đơn vị đo khác nhau, đặc trưng nào có giá trị lớn hơn sẽ áp đảo tổng, và các đặc trưng còn lại gần như không có tác dụng.

Xét bài toán dự đoán một sinh viên có đạt học phần hay không, dựa trên hai đặc trưng: số bài tập đã nộp (từ 0 đến 20) và tổng thời gian lập trình tính bằng phút (từ 0 đến 3000). Tập huấn luyện gồm tám mẫu trong bảng 2.11.

**Bảng 2.11.** Tập huấn luyện của ví dụ phân loại.

| Mẫu | Số bài nộp | Số phút | Nhãn |
|---|---|---|---|
| 1 | 2 | 300 | chưa đạt |
| 2 | 3 | 450 | chưa đạt |
| 3 | 4 | 600 | chưa đạt |
| 4 | 15 | 1800 | đạt |
| 5 | 16 | 2000 | đạt |
| 6 | 18 | 2400 | đạt |
| 7 | 5 | 2200 | chưa đạt |
| 8 | 17 | 700 | đạt |

Mẫu cần phân loại là $x = (14,\ 800)$: một sinh viên nộp 14 bài, dùng 800 phút. Bảng 2.12 so sánh kết quả khi tính khoảng cách trên dữ liệu thô và trên dữ liệu đã chuẩn hoá.

**Bảng 2.12.** Khoảng cách từ mẫu $(14, 800)$ tới tám mẫu huấn luyện, trước và sau khi chuẩn hoá. Ba mẫu gần nhất trong mỗi cột được in đậm.

| Mẫu | Nhãn | Khoảng cách thô | | Sau chuẩn hoá | |
|---|---|---|---|---|---|
| | | giá trị | hạng | giá trị | hạng |
| 8 | đạt | **100,045** | **1** | **0,1935** | **1** |
| 3 | chưa đạt | **200,250** | **2** | 0,6322 | 4 |
| 2 | chưa đạt | **350,173** | **3** | 0,7074 | 5 |
| 1 | chưa đạt | 500,144 | 4 | 0,7869 | 6 |
| 4 | đạt | 1000,000 | 5 | **0,4803** | **2** |
| 5 | đạt | 1200,002 | 6 | **0,5849** | **3** |
| 7 | chưa đạt | 1400,029 | 7 | 0,8723 | 8 |
| 6 | đạt | 1600,005 | 8 | 0,8019 | 7 |
| Kết luận với $k = 3$ | | **chưa đạt** | | **đạt** | |

Hai cột cho hai kết luận trái ngược nhau. Nguyên nhân nằm ở chênh lệch thang đo. Số phút chênh nhau hàng trăm, số bài nộp chênh nhau vài đơn vị; bình phương của hai chênh lệch đó khác nhau khoảng bốn bậc, nên khoảng cách thô gần như chỉ phản ánh chênh lệch về số phút. Ba mẫu "gần nhất" theo khoảng cách thô thực chất chỉ là ba mẫu có số phút gần 800 nhất, bất kể số bài nộp là 4 hay 17.

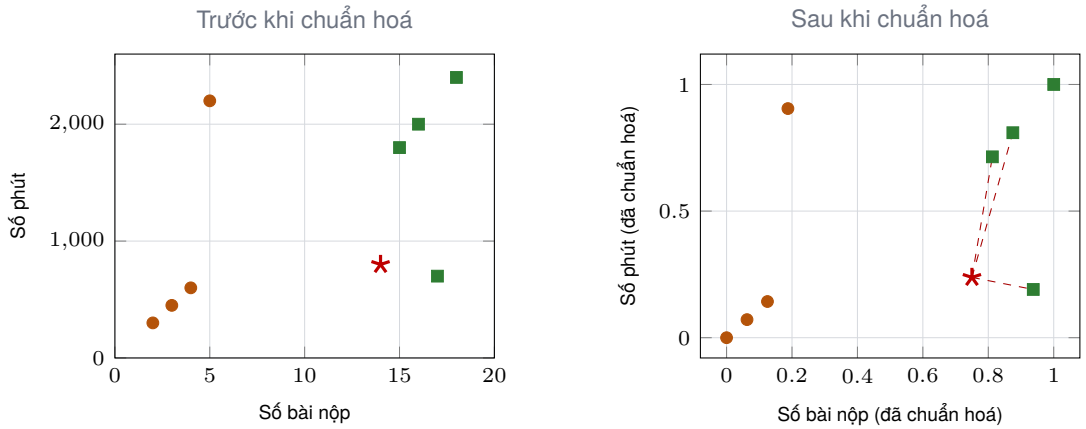
Phép chuẩn hoá thường dùng nhất là **chuẩn hoá min-max**, đưa mọi đặc trưng về đoạn $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Với tập huấn luyện trên, số bài nộp nằm trong $[2, 18]$ và số phút nằm trong $[300, 2400]$, nên mẫu mới $(14, 800)$ trở thành

$$\left(\frac{14-2}{18-2}, \frac{800-300}{2400-300}\right) = (0{,}7500,\ 0{,}2381).$$

Sau chuẩn hoá, hai đặc trưng đóng góp ngang nhau, và ba mẫu gần nhất là các mẫu 8, 4, 5 — đều mang nhãn "đạt". Kết luận này phù hợp với trực giác: sinh viên nộp 14 bài chỉ trong 800 phút là người làm việc hiệu quả.




**Hình 2.6.** Cùng một tập dữ liệu trước và sau khi chuẩn hoá min-max. Hình sao là mẫu cần phân loại; hình tròn là nhãn "chưa đạt", hình vuông là nhãn "đạt". Ba đoạn nét đứt nối mẫu mới với ba láng giềng gần nhất sau chuẩn hoá, cả ba đều mang nhãn "đạt".

> **Sai lầm thường gặp khi chuẩn hoá.** Giá trị $x_{\min}$ và $x_{\max}$ phải lấy từ *tập huấn luyện* và được ghi lại để dùng cho mọi mẫu mới về sau. Nếu mỗi lần có mẫu mới lại tính lại $x_{\min}, x_{\max}$ trên tập gồm cả mẫu mới, kết quả phân loại của các mẫu cũ sẽ thay đổi theo thời gian mà không có lý do nào chính đáng.

### 2.7.3 Cài đặt

GIẢ MÃ

```
K-LANG-GIENG(D, x, k)
1  for i ← 1 to n
2      d[i] ← khoảng cách từ x tới D[i].đặc_trưng
3  sắp xếp các chỉ số 1..n theo d tăng dần
4  đếm số phiếu của từng nhãn trong k chỉ số đầu
5  return nhãn có nhiều phiếu nhất
```

JULIA

```julia
function chuan_hoa(X)
    n, m = size(X)
    cmin = [minimum(X[:, j]) for j in 1:m]
```

```
    cmax = [maximum(X[:, j]) for j in 1:m]
    Z = similar(X, Float64)
    for j in 1:m
        khoang = cmax[j] - cmin[j]
        for i in 1:n
            Z[i, j] = khoang == 0 ? 0.0 : (X[i, j] - cmin[j]) / khoang
        end
    end
    return Z, cmin, cmax
end

khoang_cach(u, v) = sqrt(sum((u .- v) .^ 2))

function k_lang_gieng(Z, y, z_moi, k)
    n = size(Z, 1)
    d = [khoang_cach(Z[i, :], z_moi) for i in 1:n]
    thu_tu = sortperm(d)                    # chỉ số theo d tăng dần
    phieu = Dict{String, Int}()
    for i in thu_tu[1:k]
        phieu[y[i]] = get(phieu, y[i], 0) + 1
    end
    return argmax(phieu)                    # nhãn có nhiều phiếu nhất
end
```

**PYTHON**

```
import math
from collections import Counter

def chuan_hoa(X):
    m = len(X[0])
    cmin = [min(hang[j] for hang in X) for j in range(m)]
    cmax = [max(hang[j] for hang in X) for j in range(m)]
    Z = []
    for hang in X:
        z = []
        for j in range(m):
            khoang = cmax[j] - cmin[j]
            z.append(0.0 if khoang == 0 else (hang[j] - cmin[j]) / khoang)
        Z.append(z)
    return Z, cmin, cmax

def khoang_cach(u, v):
    return math.sqrt(sum((p - q) ** 2 for p, q in zip(u, v)))

def k_lang_gieng(Z, y, z_moi, k):
    d = [(khoang_cach(Z[i], z_moi), y[i]) for i in range(len(Z))]
    d.sort(key=lambda t: t[0])              # sắp xếp theo khoảng cách
    phieu = Counter(nhan for _, nhan in d[:k])
    return phieu.most_common(1)[0][0]
```

**C++**

```
struct Mau { vector<double> x; string y; };

void chuan_hoa(vector<Mau> &D, vector<double> &cmin, vector<double> &cmax) {
    int m = D[0].x.size();
    cmin.assign(m, 1e18);
```

```
    cmax.assign(m, -1e18);
    for (auto &mau : D)
        for (int j = 0; j < m; j++) {
            cmin[j] = min(cmin[j], mau.x[j]);
            cmax[j] = max(cmax[j], mau.x[j]);
        }
    for (auto &mau : D)
        for (int j = 0; j < m; j++) {
            double khoang = cmax[j] - cmin[j];
            mau.x[j] = (khoang == 0) ? 0.0 : (mau.x[j] - cmin[j]) / khoang;
        }
}

double khoang_cach(const vector<double> &u, const vector<double> &v) {
    double s = 0;
    for (size_t j = 0; j < u.size(); j++) s += (u[j]-v[j]) * (u[j]-v[j]);
    return sqrt(s);
}

string k_lang_gieng(const vector<Mau> &D, const vector<double> &z, int k) {
    vector<pair<double, string>> d;
    for (auto &mau : D) d.push_back({khoang_cach(mau.x, z), mau.y});
    sort(d.begin(), d.end(),
         [](const auto &p, const auto &q) { return p.first < q.first; });
    map<string, int> phieu;
    for (int i = 0; i < k; i++) phieu[d[i].second]++;
    string tot = ""; int nhieu = -1;
    for (auto &[nhan, so] : phieu)
        if (so > nhieu) { nhieu = so; tot = nhan; }
    return tot;
}
```

**Sắp xếp chỉ số hay sắp xếp cặp.** Bản Julia dùng `sortperm`, trả về *hoán vị chỉ số* chứ không sắp xếp mảng gốc; nhờ vậy quan hệ giữa khoảng cách và nhãn không bị mất. Bản Python và C++ dùng cách khác: ghép khoảng cách và nhãn thành một cặp rồi sắp xếp cặp đó. Cả hai cách đều đúng, nhưng không được sắp xếp riêng mảng khoảng cách rồi vẫn dùng mảng nhãn cũ — đó là lỗi làm hỏng toàn bộ kết quả mà chương trình vẫn chạy bình thường.
Một chi tiết nhỏ nữa: khi hai nhãn được số phiếu bằng nhau, ba bản cài đặt trên phân xử khác nhau, vì cách duyệt từ điển của ba ngôn ngữ khác nhau. Muốn ba bản cho kết quả trùng nhau trong mọi trường hợp, phải quy định rõ luật phân xử, chẳng hạn chọn nhãn có tổng khoảng cách nhỏ hơn, hoặc chọn nhãn đứng trước theo thứ tự từ điển.

### 2.7.4 Chọn $K$ mục có điểm cao nhất

Một hệ gợi ý tính điểm phù hợp cho từng mục trong kho rồi trả về $K$ mục có điểm cao nhất. Đây cũng là bước cuối của $k$ láng giềng: lấy $k$ khoảng cách nhỏ nhất trong $n$ khoảng cách.

Có hai cách làm.

1. Sắp xếp toàn bộ $n$ phần tử rồi lấy $K$ phần tử đầu, tốn $O(n \log n)$.
2. Chạy $K$ lượt của thuật toán sắp xếp lựa chọn, mỗi lượt đưa phần tử lớn nhất còn lại lên đầu, tốn $O(nK)$.

Cách nào tốt hơn phụ thuộc vào quan hệ giữa $K$ và $\log_2 n$. Với $n = 100\,000$ và $K = 5$, số đo thực

tế là 499 985 phép so sánh cho cách thứ hai, so với $n \log_2 n \approx 1\,660\,964$ cho cách thứ nhất: cách thứ hai tiết kiệm hơn ba lần. Khi $K$ lớn hơn $\log_2 n \approx 17$, quan hệ đảo lại.

**PYTHON**

```python
def top_k_bang_lua_chon(diem, K):
    a = diem[:]                       # sao chép để không phá dữ liệu gốc
    for i in range(K):
        m = i
        for j in range(i + 1, len(a)):
            if a[j] > a[m]:
                m = j
        a[i], a[m] = a[m], a[i]
    return a[:K]

def top_k_bang_sap_xep(diem, K):
    return sorted(diem, reverse=True)[:K]
```

**JULIA**

```julia
function top_k_bang_lua_chon(diem, K)
    a = copy(diem)
    for i in 1:K
        m = i
        for j in i+1:length(a)
            if a[j] > a[m]
                m = j
            end
        end
        a[i], a[m] = a[m], a[i]
    end
    return a[1:K]
end

top_k_bang_sap_xep(diem, K) = sort(diem, rev = true)[1:K]
```

**C++**

```cpp
vector<int> top_k_bang_lua_chon(vector<int> a, int K) {
    for (int i = 0; i < K; i++) {
        int m = i;
        for (int j = i + 1; j < (int)a.size(); j++)
            if (a[j] > a[m]) m = j;
        swap(a[i], a[m]);
    }
    return vector<int>(a.begin(), a.begin() + K);
}

vector<int> top_k_bang_sap_xep(vector<int> a, int K) {
    // chỉ cần K phần tử đầu đúng thứ tự, không cần sắp xếp cả mảng
    partial_sort(a.begin(), a.begin() + K, a.end(), greater<int>());
    return vector<int>(a.begin(), a.begin() + K);
}
```

Hàm `partial_sort` của thư viện chuẩn C++ giải đúng bài toán này với độ phức tạp $O(n \log K)$, tốt hơn cả hai cách nêu trên. Nó dùng cấu trúc dữ liệu đống, sẽ được trình bày ở mục 7.6 của chương 7.

**Bài 2.5.** Cài đặt đầy đủ chương trình phân loại $k$ láng giềng gần nhất theo các bước sau.

1. Đọc tập huấn luyện từ tệp văn bản, mỗi dòng gồm các đặc trưng và nhãn.
2. Tính $x_{\min}, x_{\max}$ của từng cột và chuẩn hoá tập huấn luyện.
3. Nhận mẫu mới từ bàn phím, chuẩn hoá bằng đúng $x_{\min}, x_{\max}$ đã lưu ở bước 2.
4. Tính khoảng cách tới mọi mẫu, sắp xếp, lấy $k$ mẫu đầu, đếm phiếu.
5. In ra $k$ láng giềng kèm khoảng cách, rồi in nhãn dự đoán.

Chạy chương trình với dữ liệu ở bảng 2.11 và mẫu $(14, 800)$, với $k = 1$, $k = 3$, $k = 5$. So sánh kết quả khi bỏ bước 2 và khi giữ bước 2. Kết quả phải trùng với bảng 2.12.

**Tóm tắt.** Bài toán sắp xếp và bài toán tìm kiếm gắn với nhau: tìm kiếm nhị phân đòi hỏi dãy đã sắp xếp, và bù lại đưa chi phí mỗi truy vấn từ $O(n)$ xuống $O(\log n)$. Bốn thuật toán sắp xếp đơn giản đều là $O(n^2)$ nhưng khác nhau về số phép đổi chỗ, về tính ổn định và về hành vi trên dữ liệu gần như đã sắp xếp. Sắp xếp nhanh đạt $O(n \log n)$ trung bình nhờ loại bỏ các phép so sánh giữa hai nửa sau khi phân hoạch. Sắp xếp cơ số đạt $O(n)$ nhờ không so sánh mà phân phối theo chữ số, đổi lại chỉ áp dụng cho dữ liệu tách được thành chữ số. Trong thuật toán $k$ láng giềng gần nhất, bước sắp xếp theo khoảng cách quyết định chi phí, và bước chuẩn hoá đặc trưng quyết định tính đúng đắn.

## 2.8 Bài tập chương 2

### Câu hỏi ôn tập

1. Phát biểu bài toán sắp xếp. Vì sao kết quả bắt buộc phải là một hoán vị của dãy ban đầu?
2. Thế nào là thuật toán sắp xếp ổn định? Nêu một tình huống thực tế trong đó tính ổn định là bắt buộc.
3. Vì sao sắp xếp kiểu lựa chọn thực hiện ít phép đổi chỗ hơn sắp xếp kiểu đổi chỗ, trong khi số phép so sánh của hai thuật toán bằng nhau?
4. Sắp xếp chèn trực tiếp có độ phức tạp tốt nhất là $O(n)$. Dạng dữ liệu vào nào cho trường hợp tốt nhất đó?
5. Vai trò của biến `đổi` trong sắp xếp nổi bọt là gì? Bỏ biến này đi thì độ phức tạp thay đổi thế nào?
6. Mô tả phép phân hoạch của sắp xếp nhanh. Sau khi phân hoạch, phát biểu nào đúng về hai đoạn con?
7. Vì sao không nên chọn chốt là phần tử đầu đoạn?
8. Vì sao bước phân phối của sắp xếp cơ số bắt buộc phải ổn định?
9. Tìm kiếm nhị phân đòi hỏi điều kiện gì ở dữ liệu vào? Điều gì xảy ra nếu áp dụng nó cho dãy chưa sắp xếp?
10. Vì sao tìm kiếm nội suy có thể chậm hơn tìm kiếm nhị phân dù độ phức tạp trung bình của nó tốt hơn?

### Bài tập tính toán

11. Lần vết bốn thuật toán sắp xếp đơn giản trên dãy $(7, 2, 9, 4, 1, 6)$. Lập bảng theo mẫu của bảng 2.1, đếm số phép so sánh và số phép đổi chỗ của từng thuật toán.

12. Lần vết sắp xếp nhanh trên dãy $(5, 3, 8, 4, 2, 7, 1, 6)$ với chốt lấy ở giữa đoạn. Vẽ cây các lời gọi đệ quy.

13. Lần vết sắp xếp cơ số trên dãy $(170, 45, 75, 90, 802, 24, 2, 66)$. Ghi nội dung mười thùng sau mỗi lượt.

14. Cho dãy đã sắp xếp $(3, 9, 14, 20, 27, 35, 41, 58, 66, 79, 88, 95)$. Lần vết tìm kiếm nhị phân cho $x = 41$ và cho $x = 30$. Đếm số phép so sánh trong mỗi trường hợp.

15. Với dãy ở bài 14, lần vết tìm kiếm nội suy cho $x = 88$ và so sánh số bước với tìm kiếm nhị phân.

16. Chứng minh rằng sắp xếp chèn trực tiếp thực hiện đúng $n - 1$ phép so sánh khi dãy vào đã sắp xếp tăng dần.

17. Một dãy $n$ phần tử được gọi là *gần như đã sắp xếp* nếu mỗi phần tử cách vị trí đúng của nó không quá $k$ vị trí. Chứng minh sắp xếp chèn trực tiếp chạy trong $O(nk)$ trên dãy như vậy.

18. Đếm số phép so sánh của sắp xếp nhanh trên dãy đã sắp xếp tăng dần $(1, 2, \ldots, n)$ khi chốt lấy ở giữa đoạn. Kết quả thuộc lớp độ phức tạp nào?

### Bài tập lập trình mức cơ bản

19. Cài đặt cả bốn thuật toán sắp xếp đơn giản bằng ba ngôn ngữ. Viết một hàm kiểm thử sinh 1000 mảng ngẫu nhiên, so sánh kết quả của từng thuật toán với hàm sắp xếp của thư viện.

20. Sửa cả bốn thuật toán để sắp xếp giảm dần, chỉ bằng cách đổi chiều phép so sánh.

21. Viết hàm kiểm tra một mảng đã sắp xếp tăng dần hay chưa, độ phức tạp $O(n)$.

22. Cho một mảng và một số $x$, đếm số phần tử nhỏ hơn $x$ bằng tìm kiếm nhị phân, độ phức tạp $O(\log n)$ sau khi đã sắp xếp.

23. Cho hai mảng đã sắp xếp, trộn chúng thành một mảng đã sắp xếp trong $O(n + m)$, không dùng hàm sắp xếp.

24. Cho một mảng $n$ số nguyên, tìm phần tử xuất hiện nhiều lần nhất. Trình bày hai lời giải: một dùng hai vòng lặp lồng nhau $O(n^2)$, một dùng sắp xếp $O(n \log n)$.

### Bài tập lập trình mức vận dụng

25. Cho mảng $n$ số nguyên và số $S$. Tìm hai phần tử có tổng bằng $S$. Trình bày lời giải $O(n \log n)$ dùng sắp xếp kết hợp tìm kiếm nhị phân, và lời giải $O(n)$ dùng hai con trỏ sau khi sắp xếp.

26. Cho mảng $n$ số nguyên, tìm phần tử lớn thứ $K$. Trình bày ba lời giải với độ phức tạp $O(nK)$, $O(n \log n)$ và $O(n)$ trung bình (dựa trên phép phân hoạch của sắp xếp nhanh).

27. Cho danh sách sinh viên gồm mã, tên và điểm. Sắp xếp theo điểm giảm dần; nếu điểm bằng nhau thì theo tên tăng dần. Cài đặt bằng cách so sánh cặp khoá, không gọi hàm sắp xếp hai lần.

28. Cho mảng đã sắp xếp có thể có phần tử trùng nhau và một giá trị $x$. Viết hai hàm: tìm vị trí xuất hiện đầu tiên và vị trí xuất hiện cuối cùng của $x$, mỗi hàm $O(\log n)$. Từ đó đếm số lần xuất hiện của $x$.

29. Cho hàm $f$ đơn điệu tăng trên tập số nguyên và giá trị $y$. Tìm số nguyên $t$ nhỏ nhất thoả $f(t) \geq y$ bằng tìm kiếm nhị phân trên miền giá trị. Áp dụng để tìm căn bậc hai nguyên của một số nguyên dương $N$ trong $O(\log N)$.

30. Cài đặt sắp xếp cơ số cho mảng có cả số âm lẫn số dương.

### Bài tập phần trí tuệ nhân tạo

31. Cài đặt chương trình ở bài thực hành 2.5 và kiểm tra kết quả trùng với bảng 2.12.

32. Thêm vào chương trình bài 31 tuỳ chọn chuẩn hoá theo trung bình và độ lệch chuẩn $x' = (x - \mu)/\sigma$ thay cho min-max. So sánh kết quả phân loại của hai cách trên cùng tập dữ liệu.

33. Sửa chương trình để trọng số phiếu của mỗi láng giềng tỉ lệ nghịch với khoảng cách, tức phiếu của láng giềng thứ $i$ có trọng số $1/(d_i + \epsilon)$. So sánh với cách bỏ phiếu đều nhau khi $k = 7$.

34. Viết chương trình đo thời gian phân loại một mẫu mới khi tập huấn luyện có $n = 10^3, 10^4, 10^5$ mẫu và số đặc trưng $m = 10$. Xác nhận thời gian tăng tuyến tính theo $n$, đúng với độ phức tạp $O(nm + n \log n)$.

35. Cho tập huấn luyện $n = 10^6$ mẫu và $10^4$ mẫu cần phân loại. Ước lượng tổng thời gian nếu mỗi mẫu mới phải sắp xếp lại toàn bộ khoảng cách, rồi ước lượng lại nếu chỉ dùng $K$ lượt lựa chọn với $K = 5$. Kiểm chứng bằng chương trình đo thực tế.

CHƯƠNG 3

# Bài toán Duyệt

Hai chương trước giải các bài toán có lời giải trực tiếp: muốn sắp xếp thì chạy thuật toán sắp xếp, muốn tìm kiếm thì chạy thuật toán tìm kiếm. Chương này xử lý một lớp bài toán khác, trong đó lời giải không tính ra được mà phải *tìm* trong một tập rất lớn các khả năng: xếp tám quân hậu lên bàn cờ sao cho không quân nào ăn được quân nào, điền số vào lưới Sudoku, chọn tuyến đường ngắn nhất đi qua tất cả các thành phố.

Với những bài toán như vậy, ta phải sinh ra các khả năng rồi kiểm tra. Vấn đề là số khả năng thường tăng theo hàm mũ hoặc giai thừa, nên việc sinh phải được tổ chức cẩn thận: sinh đủ, không sinh trùng, và quan trọng nhất là biết dừng sớm ở những nhánh chắc chắn không dẫn tới lời giải. Bốn kỹ thuật của chương này — sinh kế tiếp, đệ quy, quay lui, nhánh và cận — xếp theo đúng thứ tự tăng dần của khả năng cắt bỏ đó.

## 3.1 Bài toán liệt kê

### 3.1.1 Ba cấu hình cơ bản

Phần lớn bài toán duyệt quy về việc liệt kê một trong ba loại cấu hình sau.

**Bảng 3.1.** Ba loại cấu hình cơ bản và số lượng của chúng.

| Cấu hình | Mô tả | Số lượng | Ví dụ với $n = 4$ |
|---|---|---|---|
| Xâu nhị phân độ dài $n$ | dãy $n$ ký tự, mỗi ký tự là 0 hoặc 1 | $2^n$ | 16 xâu |
| Tổ hợp chập $k$ của $n$ | tập con $k$ phần tử, không kể thứ tự | $\binom{n}{k}$ | $\binom{4}{2} = 6$ |
| Hoán vị của $n$ phần tử | sắp xếp lại $n$ phần tử, có kể thứ tự | $n!$ | 24 |

Ba con số ở cột thứ ba cho thấy ngay giới hạn của phương pháp duyệt. Bảng 3.2 lấy giả định máy tính xử lý được $10^8$ cấu hình mỗi giây.

**Bảng 3.2.** Số cấu hình và thời gian duyệt hết, với tốc độ $10^8$ cấu hình mỗi giây.

| $n$ | $2^n$ | thời gian | $n!$ | thời gian |
|---|---|---|---|---|
| 10 | 1 024 | tức thì | 3 628 800 | 0,04 giây |
| 20 | 1 048 576 | 0,01 giây | $2{,}4 \cdot 10^{18}$ | 770 năm |
| 30 | $1{,}1 \cdot 10^{9}$ | 11 giây | $2{,}7 \cdot 10^{32}$ | không khả thi |
| 50 | $1{,}1 \cdot 10^{15}$ | 130 ngày | $3{,}0 \cdot 10^{64}$ | không khả thi |

Kết luận rút ra: duyệt toàn bộ chỉ dùng được khi $n$ nhỏ, cỡ $n \le 20$ với xâu nhị phân và $n \le 11$ với hoán vị. Với $n$ lớn hơn, bắt buộc phải cắt bỏ bớt không gian tìm kiếm, và đó là nội dung của các mục 3.4 và 3.5.

### 3.1.2 Thứ tự từ điển

Để việc liệt kê có thể kiểm tra được, ta cần một thứ tự xác định giữa các cấu hình.

**Định nghĩa 3.1.** (*Thứ tự từ điển*) Cho hai dãy $x = (x_1, \ldots, x_n)$ và $y = (y_1, \ldots, y_n)$. Ta nói $x$ đứng trước $y$ theo **thứ tự từ điển**, viết $x < y$, nếu tồn tại chỉ số $i$ sao cho $x_j = y_j$ với mọi $j < i$ và $x_i < y_i$.

Đây chính là cách sắp xếp các từ trong từ điển: so sánh từng ký tự từ trái sang phải, gặp ký tự khác nhau đầu tiên thì kết luận. Với xâu nhị phân độ dài 4, thứ tự từ điển cho dãy 0000, 0001, 0010, 0011, ..., 1111, tức đúng thứ tự đếm từ 0 đến 15 trong hệ nhị phân.

## 3.2 Thuật toán sinh kế tiếp

### 3.2.1 Nguyên lý

Phương pháp sinh kế tiếp dựa trên nhận xét: nếu biết cấu hình đầu tiên, biết cấu hình cuối cùng, và biết cách đi từ một cấu hình sang cấu hình liền sau nó theo thứ tự từ điển, thì ta liệt kê được toàn bộ mà không cần đệ quy và không cần bộ nhớ phụ.

GIẢ MÃ

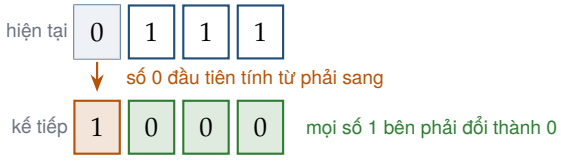
```
1  LIET-KE-BANG-SINH()
2  1  x ← cấu hình đầu tiên
3  2  repeat
4  3      xử lý x
5  4      x ← cấu hình kế tiếp của x
6  5  until không còn cấu hình kế tiếp
```

Toàn bộ khó khăn nằm ở dòng 4. Với mỗi loại cấu hình, ta phải trả lời hai câu hỏi: khi nào một cấu hình là cuối cùng, và nếu chưa phải cuối cùng thì cấu hình liền sau nó là gì.

### 3.2.2 Sinh xâu nhị phân

**Cấu hình đầu tiên** là $00\ldots0$, **cấu hình cuối cùng** là $11\ldots1$.

**Quy tắc sinh.** Việc chuyển sang xâu kế tiếp giống hệt phép cộng thêm 1 trong hệ nhị phân: tìm từ phải sang trái chữ số 0 đầu tiên, đổi nó thành 1, và đổi tất cả các chữ số 1 ở bên phải nó thành 0.




**Hình 3.1.** Sinh xâu nhị phân kế tiếp: từ 0111 sang 1000.

GIẢ MÃ

```
1  SINH-XAU-NHI-PHAN-KE-TIEP(x, n)
2  1  i ← n
3  2  while i ≥ 1 and x[i] = 1
4  3      x[i] ← 0
5  4      i ← i-1
6  5  if i < 1
7  6      return HET                  // đang ở xâu 11...1
8  7  x[i] ← 1
```

JULIA

```julia
function liet_ke_xau_nhi_phan(n)
    x = zeros(Int, n)
    println(join(x))
    while true
        i = n
        while i >= 1 && x[i] == 1
            x[i] = 0
            i -= 1
        end
        i < 1 && break              # đã là xâu 11...1
        x[i] = 1
        println(join(x))
    end
end
```

PYTHON

```python
def liet_ke_xau_nhi_phan(n):
    x = [0] * n
    print("".join(map(str, x)))
    while True:
        i = n - 1
        while i >= 0 and x[i] == 1:
            x[i] = 0
            i -= 1
        if i < 0:
            break                   # đã là xâu 11...1
        x[i] = 1
        print("".join(map(str, x)))
```

C++

```cpp
void liet_ke_xau_nhi_phan(int n) {
    vector<int> x(n, 0);
    for (int v : x) cout << v;  cout << "\n";
    while (true) {
        int i = n - 1;
        while (i >= 0 && x[i] == 1) { x[i] = 0; i--; }
        if (i < 0) break;           // đã là xâu 11...1
        x[i] = 1;
        for (int v : x) cout << v;  cout << "\n";
    }
}
```

**Ví dụ 3.1.** *Kết quả với* $n = 4$. Chương trình in ra 16 xâu, đúng bằng $2^4$:

```
0000  0001  0010  0011  0100  0101  0110  0111
1000  1001  1010  1011  1100  1101  1110  1111
```

**Ứng dụng: liệt kê tập con.** Mỗi xâu nhị phân độ dài $n$ tương ứng với một tập con của tập $n$ phần tử: $x_i = 1$ nghĩa là phần tử thứ $i$ có mặt trong tập con. Vì vậy bài toán liệt kê tập con và bài toán liệt kê xâu nhị phân là một. Trong C++, cách viết ngắn gọn thường dùng là duyệt số nguyên từ 0 đến $2^n - 1$ rồi kiểm tra từng bit bằng phép `(m >> i) & 1`.

### 3.2.3 Sinh tổ hợp chập $k$ của $n$

Tổ hợp được biểu diễn bằng dãy tăng nghiêm ngặt $1 \le x_1 < x_2 < \cdots < x_k \le n$.

**Cấu hình đầu tiên** là $(1, 2, \dots, k)$, **cấu hình cuối cùng** là $(n-k+1, \dots, n)$.

**Quy tắc sinh.** Vị trí $x_i$ đã đạt giá trị lớn nhất có thể của nó khi $x_i = n - k + i$. Tìm từ phải sang trái vị trí $i$ đầu tiên chưa đạt giá trị lớn nhất, tăng $x_i$ lên 1, rồi đặt các vị trí phía sau về giá trị nhỏ nhất còn hợp lệ: $x_j = x_{j-1} + 1$ với mọi $j > i$.

**GIẢ MÃ**

```
SINH-TO-HOP-KE-TIEP(x, n, k)
1  i ← k
2  while i ≥ 1 and x[i] = n-k+i
3      i ← i-1
4  if i < 1
5      return HET
6  x[i] ← x[i] + 1
7  for j ← i+1 to k
8      x[j] ← x[j-1] + 1
```

**JULIA**

```julia
function liet_ke_to_hop(n, k)
    x = collect(1:k)
    println(x)
    while true
        i = k
        while i >= 1 && x[i] == n - k + i
            i -= 1
        end
        i < 1 && break
        x[i] += 1
        for j in i+1:k
            x[j] = x[j-1] + 1
        end
        println(x)
    end
end
```

**PYTHON**

```python
def liet_ke_to_hop(n, k):
    x = list(range(1, k + 1))
    print(x)
    while True:
        i = k - 1
        while i >= 0 and x[i] == n - k + i + 1:
            i -= 1
        if i < 0:
            break
        x[i] += 1
        for j in range(i + 1, k):
            x[j] = x[j - 1] + 1
        print(x)
```

C++

```cpp
void liet_ke_to_hop(int n, int k) {
    vector<int> x(k);
    for (int i = 0; i < k; i++) x[i] = i + 1;
    auto in = [&]() { for (int v : x) cout << v << " "; cout << "\n"; };
    in();
    while (true) {
        int i = k - 1;
        while (i >= 0 && x[i] == n - k + i + 1) i--;
        if (i < 0) break;
        x[i]++;
        for (int j = i + 1; j < k; j++) x[j] = x[j - 1] + 1;
        in();
    }
}
```

**Chú ý về chỉ số.** Điều kiện "đã đạt giá trị lớn nhất" là $x_i = n - k + i$ khi đánh chỉ số từ 1 (Julia), nhưng là $x_i = n - k + i + 1$ khi đánh chỉ số từ 0 (Python, C++), vì vị trí thứ $i$ theo cách đánh từ 0 tương ứng với vị trí thứ $i+1$ theo cách đánh từ 1. Đây là chỗ sai nhiều nhất khi chuyển chương trình giữa hai ngôn ngữ.

**Ví dụ 3.2.** *Kết quả với* $n = 5$, $k = 3$. Chương trình in ra $\binom{5}{3} = 10$ tổ hợp theo đúng thứ tự từ điển:

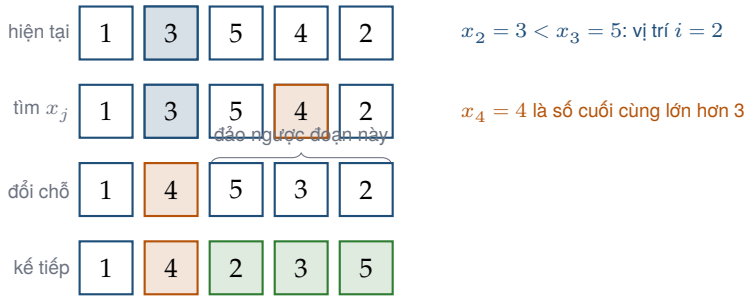
```
(1,2,3) (1,2,4) (1,2,5) (1,3,4) (1,3,5) (1,4,5) (2,3,4) (2,3,5) (2,4,5) (3,4,5)
```

### 3.2.4 Sinh hoán vị

**Cấu hình đầu tiên** là $(1, 2, \dots, n)$, **cấu hình cuối cùng** là $(n, n-1, \dots, 1)$.

**Quy tắc sinh.** Tìm từ phải sang trái vị trí $i$ đầu tiên thoả $x_i < x_{i+1}$; nếu không có thì dãy đang giảm dần và đây là hoán vị cuối cùng. Nếu có, tìm từ phải sang trái phần tử $x_j$ đầu tiên lớn hơn $x_i$, đổi chỗ $x_i$ với $x_j$, rồi đảo ngược toàn bộ đoạn từ $i+1$ đến $n$.

Ba bước này đảm bảo hoán vị thu được là hoán vị nhỏ nhất trong số các hoán vị lớn hơn hoán vị hiện tại.




**Hình 3.2.** Sinh hoán vị kế tiếp: từ $(1, 3, 5, 4, 2)$ sang $(1, 4, 2, 3, 5)$.

GIẢ MÃ

```
SINH-HOAN-VI-KE-TIEP(x, n)
1  i ← n-1
2  while i ≥ 1 and x[i] > x[i+1]
3       i ← i-1
4  if i < 1
```

```
5      return HET                  // dãy đang giảm dần
6  j ← n
7  while x[j] < x[i]
8      j ← j-1
9  đổi chỗ x[i] và x[j]
10 đảo ngược đoạn x[i+1..n]
```

**JULIA**

```julia
function liet_ke_hoan_vi(n)
    x = collect(1:n)
    println(x)
    while true
        i = n - 1
        while i >= 1 && x[i] > x[i+1]
            i -= 1
        end
        i < 1 && break
        j = n
        while x[j] < x[i]
            j -= 1
        end
        x[i], x[j] = x[j], x[i]
        reverse!(x, i+1, n)
        println(x)
    end
end
```

**PYTHON**

```python
def liet_ke_hoan_vi(n):
    x = list(range(1, n + 1))
    print(x)
    while True:
        i = n - 2
        while i >= 0 and x[i] > x[i + 1]:
            i -= 1
        if i < 0:
            break
        j = n - 1
        while x[j] < x[i]:
            j -= 1
        x[i], x[j] = x[j], x[i]
        x[i + 1:] = reversed(x[i + 1:])
        print(x)
```

**C++**

```cpp
void liet_ke_hoan_vi(int n) {
    vector<int> x(n);
    for (int i = 0; i < n; i++) x[i] = i + 1;
    auto in = [&]() { for (int v : x) cout << v << " "; cout << "\n"; };
    in();
    while (true) {
        int i = n - 2;
        while (i >= 0 && x[i] > x[i + 1]) i--;
        if (i < 0) break;
```

```cpp
        int j = n - 1;
        while (x[j] < x[i]) j--;
        swap(x[i], x[j]);
        reverse(x.begin() + i + 1, x.end());
        in();
    }
}
// Thư viện chuẩn C++ có sẵn: while (next_permutation(x.begin(), x.end())) ...
```

**Bài 3.1.** Chạy cả ba chương trình sinh với $n = 4$ và đối chiếu số cấu hình in ra với công thức: phải được đúng 16, $\binom{4}{2} = 6$ và 24. Sau đó thay lệnh in bằng một biến đếm và chạy với $n = 20$, $\binom{20}{10}$, $n = 8$. Kết quả phải là 1 048 576, 184 756 và 40 320.

## 3.3 Thuật toán đệ quy

### 3.3.1 Khái niệm

**Định nghĩa 3.2.** (*Thuật toán đệ quy*) Một thuật toán được gọi là **đệ quy** nếu trong quá trình thực hiện, nó gọi lại chính nó trên dữ liệu có kích thước nhỏ hơn.

Mọi thuật toán đệ quy đúng đắn đều gồm hai thành phần bắt buộc.

1. **Trường hợp cơ sở.** Trường hợp nhỏ nhất, giải trực tiếp được, không gọi đệ quy nữa.
2. **Bước đệ quy.** Cách đưa bài toán về một hoặc nhiều bài toán cùng dạng nhưng nhỏ hơn, kèm cách ghép lời giải của chúng thành lời giải của bài toán ban đầu.

Thiếu thành phần thứ nhất, chương trình gọi vô hạn và dừng với lỗi tràn ngăn xếp. Thiếu tính "nhỏ hơn" ở thành phần thứ hai cũng cho hậu quả tương tự dù trường hợp cơ sở có tồn tại.

### 3.3.2 Bài toán Tháp Hà Nội

**Đề bài.** Có ba cọc $A$, $B$, $C$ và $n$ đĩa kích thước khác nhau, ban đầu xếp chồng trên cọc $A$ theo thứ tự đĩa nhỏ nằm trên. Cần chuyển toàn bộ sang cọc $C$, mỗi lần chỉ chuyển một đĩa từ đỉnh cọc này sang đỉnh cọc khác, và không bao giờ được đặt đĩa lớn lên trên đĩa nhỏ.

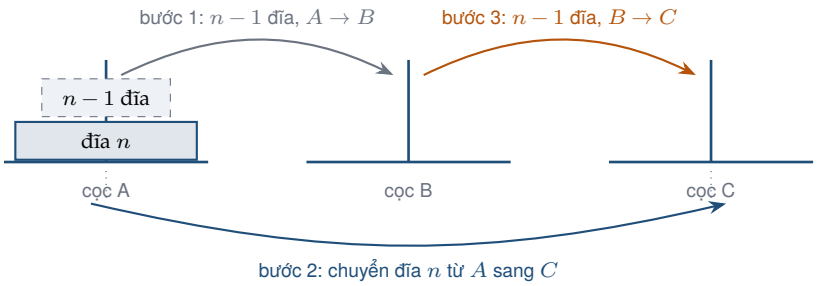
**Lời giải đệ quy.** Muốn chuyển $n$ đĩa từ $A$ sang $C$, chỉ có một cách duy nhất về mặt logic:

1. chuyển $n - 1$ đĩa trên cùng từ $A$ sang $B$, dùng $C$ làm cọc trung gian;
2. chuyển đĩa lớn nhất từ $A$ sang $C$;
3. chuyển $n - 1$ đĩa từ $B$ sang $C$, dùng $A$ làm cọc trung gian.

Trường hợp cơ sở là $n = 0$: không có đĩa nào, không làm gì cả.

GIẢ MÃ

```
HA-NOI(n, A, B, C)                  // chuyển n đĩa từ A sang C, B trung gian
1  if n = 0
2      return
3  HA-NOI(n-1, A, C, B)
4  in ra "chuyển đĩa n từ A sang C"
5  HA-NOI(n-1, B, A, C)
```




**Hình 3.3.** Phân rã đệ quy của bài toán Tháp Hà Nội.

**JULIA**

```julia
function ha_noi(n, a = 'A', b = 'B', c = 'C')
    n == 0 && return
    ha_noi(n-1, a, c, b)
    println("chuyển đĩa $n từ $a sang $c")
    ha_noi(n-1, b, a, c)
end
```

**PYTHON**

```python
def ha_noi(n, a='A', b='B', c='C'):
    if n == 0:
        return
    ha_noi(n - 1, a, c, b)
    print(f"chuyển đĩa {n} từ {a} sang {c}")
    ha_noi(n - 1, b, a, c)
```

**C++**

```cpp
void ha_noi(int n, char a = 'A', char b = 'B', char c = 'C') {
    if (n == 0) return;
    ha_noi(n - 1, a, c, b);
    cout << "chuyen dia " << n << " tu " << a << " sang " << c << "\n";
    ha_noi(n - 1, b, a, c);
}
```

**Thứ tự tham số.** Ba lời gọi khác nhau ở thứ tự ba tham số cọc, và đây là chỗ dễ nhầm nhất. Quy ước dùng ở đây là `ha_noi(n, nguồn, trung_gian, đích)`. Ở dòng thứ nhất, đích tạm thời là $B$ nên $C$ đóng vai trò trung gian; ở dòng thứ ba, nguồn là $B$ nên $A$ đóng vai trò trung gian. Viết nhầm thứ tự thì chương trình vẫn chạy và vẫn in ra $2^n - 1$ dòng, nhưng dãy nước đi không hợp lệ.

**Ví dụ 3.3.** *Kết quả với* $n = 3$. Chương trình in ra bảy dòng, đúng bằng $2^3 - 1$:

```
1: A→C  2: A→B  1: C→B  3: A→C  1: B→A  2: B→C  1: A→C
```

**Định lý 3.1.** *(Số bước của Tháp Hà Nội) Thuật toán trên thực hiện đúng* $2^n - 1$ *lần chuyển đĩa, và đây là số bước ít nhất có thể.*

*Chứng minh.* Gọi $S(n)$ là số lần chuyển đĩa. Từ thuật toán ta có $S(0) = 0$ và $S(n) = 2\,S(n-1) + 1$. Chứng minh $S(n) = 2^n - 1$ bằng quy nạp: đúng với $n = 0$ vì $2^0 - 1 = 0$; giả sử đúng với $n - 1$, khi đó $S(n) = 2(2^{n-1} - 1) + 1 = 2^n - 1$.

Về tính tối ưu: để chuyển được đĩa lớn nhất khỏi cọc $A$, toàn bộ $n-1$ đĩa phía trên bắt buộc phải nằm gọn trên một cọc khác, và cọc đó không thể là cọc đích vì đĩa lớn nhất phải xuống dưới cùng. Vậy mọi lời giải đều phải chứa một lần chuyển $n-1$ đĩa đi và một lần chuyển $n-1$ đĩa về, cho $S(n) \geq 2S(n-1)+1$. ■

Số đo thực tế xác nhận công thức: với $n = 10$ chương trình thực hiện 1023 lần chuyển, với $n = 20$ là 1 048 575 lần. Độ phức tạp $O(2^n)$ ở đây không phải do thuật toán kém mà do bản chất bài toán: kết quả cần in ra đã dài $2^n - 1$ dòng.

### 3.3.3 Liệt kê bằng đệ quy

Ba bài toán liệt kê ở mục 3.2 đều viết lại được bằng đệ quy, và bản đệ quy thường ngắn hơn, dễ kiểm tra tính đúng đắn hơn.

JULIA

```julia
function liet_ke_nhi_phan_dq(n)
    x = zeros(Int, n)
    function thu(i)
        if i > n
            println(join(x))
            return
        end
        for v in 0:1
            x[i] = v
            thu(i + 1)
        end
    end
    thu(1)
end
```

PYTHON

```python
def liet_ke_nhi_phan_dq(n):
    x = [0] * n
    def thu(i):
        if i == n:
            print("".join(map(str, x)))
            return
        for v in (0, 1):
            x[i] = v
            thu(i + 1)
    thu(0)
```

C++

```cpp
void thu(int i, int n, vector<int> &x) {
    if (i == n) {
        for (int v : x) cout << v;  cout << "\n";
        return;
    }
    for (int v = 0; v <= 1; v++) {
        x[i] = v;
        thu(i + 1, n, x);
    }
}
```

Bản đệ quy dài hơn bản sinh kế tiếp về số dòng nhưng đơn giản hơn về mặt ý tưởng: không cần biết cấu hình nào là cuối cùng, không cần công thức tính cấu hình kế tiếp. Quan trọng hơn, nó cho phép *dừng sớm*: nếu tại vị trí $i$ ta biết chắc mọi cách điền tiếp đều không dẫn tới lời giải thì chỉ việc không gọi đệ quy. Đó chính là nội dung của mục sau.

### 3.3.4 Đệ quy và ngăn xếp

Mỗi lời gọi hàm chiếm một khung trên **ngăn xếp lời gọi**, chứa các tham số, các biến địa phương và địa chỉ trở về. Khung này chỉ được giải phóng khi lời gọi kết thúc. Do đó bộ nhớ mà một thuật toán đệ quy chiếm tỉ lệ với *độ sâu* của cây đệ quy, chứ không phải với tổng số lời gọi.

**Bảng 3.3.** Giới hạn độ sâu đệ quy mặc định.

| Ngôn ngữ | Giới hạn mặc định | Cách nới rộng |
|---|---|---|
| Python | 1000 lời gọi lồng nhau | `sys.setrecursionlimit(10**6)` |
| C++ | khoảng $10^5$–$10^6$, tuỳ kích thước khung | tăng kích thước ngăn xếp khi biên dịch, hoặc khử đệ quy |
| Julia | tương tự C++ | khử đệ quy hoặc dùng vòng lặp |

> **Chú ý.** Python báo lỗi rõ ràng `RecursionError` khi vượt giới hạn. C++ thì không: chương trình bị hệ điều hành kết thúc đột ngột, thường báo `Segmentation fault`, không kèm thông tin gì về nguyên nhân. Khi một chương trình C++ có đệ quy sâu dừng bất thường mà không rõ lý do, tràn ngăn xếp là nghi vấn đầu tiên cần kiểm tra.

## 3.4 Thuật toán quay lui

### 3.4.1 Sơ đồ chung

Quay lui (*backtracking*) là kỹ thuật xây dựng lời giải theo từng thành phần. Tại mỗi bước, ta thử lần lượt các giá trị hợp lệ cho thành phần đang xét; với mỗi giá trị, ta đi tiếp sang thành phần sau; khi quay về, ta trả lại trạng thái như trước khi thử để chuẩn bị thử giá trị khác.

GIẢ MÃ

```
1  QUAY-LUI(i)
2  1  if i > n
3  2      ghi nhận một lời giải
4  3      return
5  4  for mỗi giá trị v thuộc tập ứng viên của vị trí i
6  5      if v chấp nhận được tại vị trí i
7  6          x[i] ← v
8  7          đánh dấu v đã dùng
9  8          QUAY-LUI(i+1)
10 9          bỏ đánh dấu v          // trả lại trạng thái, đây là bước "quay lui"
```

Dòng 9 là chỗ cho thuật toán cái tên của nó. Quên dòng này là lỗi phổ biến nhất khi cài đặt quay lui: chương trình chạy, không báo lỗi, nhưng bỏ sót phần lớn lời giải vì trạng thái bị "bẩn" sau mỗi nhánh.

### 3.4.2 Cây không gian trạng thái

Quá trình quay lui tương ứng với việc duyệt theo chiều sâu một cây, gọi là **cây không gian trạng thái**. Gốc là cấu hình rỗng; nút ở mức $i$ ứng với một cách điền $i$ thành phần đầu tiên; lá ở mức $n$ ứng với một cấu hình đầy đủ.

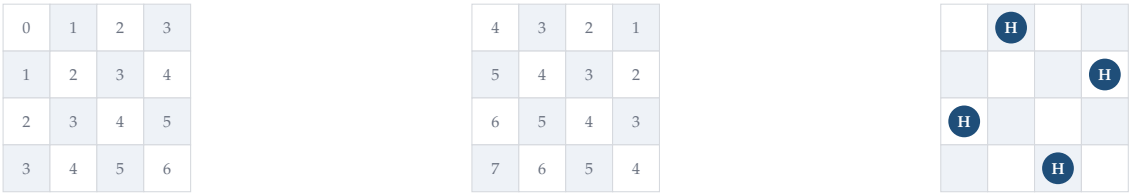
Điều kiện ở dòng 5 cho phép **cắt nhánh**: nếu $v$ không chấp nhận được, toàn bộ cây con bên dưới không được sinh ra. Với các bài toán có ràng buộc chặt, phần cây bị cắt lớn hơn phần được duyệt rất nhiều lần.

### 3.4.3 Bài toán $n$ quân hậu

**Đề bài.** Đặt $n$ quân hậu lên bàn cờ $n \times n$ sao cho không quân nào ăn được quân nào. Quân hậu ăn theo hàng, theo cột và theo hai đường chéo.

**Nhận xét thu gọn không gian.** Mỗi hàng có đúng một quân hậu, nên lời giải biểu diễn được bằng dãy $x_1, \dots, x_n$ trong đó $x_i$ là cột của quân hậu ở hàng $i$. Vì mỗi cột cũng có đúng một quân, dãy này là một hoán vị. Nhận xét đó đã giảm không gian tìm kiếm từ $\binom{n^2}{n}$ xuống $n!$: với $n = 8$, từ khoảng $4{,}4 \cdot 10^9$ xuống 40 320.

**Điều kiện đường chéo.** Hai quân hậu ở $(i, x_i)$ và $(j, x_j)$ cùng nằm trên một đường chéo khi và chỉ khi $|i - j| = |x_i - x_j|$. Kiểm tra điều kiện này bằng vòng lặp tốn $O(n)$ cho mỗi lần đặt quân. Có cách nhanh hơn: nhận xét rằng mọi ô trên cùng một đường chéo xuôi có $i + j$ bằng nhau, và mọi ô trên cùng một đường chéo ngược có $i - j$ bằng nhau. Vì vậy chỉ cần ba mảng đánh dấu và mỗi lần kiểm tra tốn $O(1)$.




**Hình 3.4.** Đánh số hai loại đường chéo trên bàn cờ $4 \times 4$ và một trong hai lời giải của bài toán bốn quân hậu.

**GIẢ MÃ**

```
1  DAT-HAU(i)
2  1  if i > n
3  2      ghi nhận lời giải
4  3      return
5  4  for j ← 1 to n
6  5      if cot[j] = false and cheo1[i+j] = false and cheo2[i-j+n] = false
7  6          x[i] ← j
8  7          cot[j] ← cheo1[i+j] ← cheo2[i-j+n] ← true
9  8          DAT-HAU(i+1)
10 9          cot[j] ← cheo1[i+j] ← cheo2[i-j+n] ← false
```

**JULIA**

```julia
function n_quan_hau(n)
    x = zeros(Int, n)
    cot   = falses(n)
    cheo1 = falses(2n)          # theo i + j
    cheo2 = falses(2n)          # theo i - j + n
```

```
    so_loi_giai = 0
    function dat(i)
        if i > n
            so_loi_giai += 1
            return
        end
        for j in 1:n
            if !cot[j] && !cheo1[i+j] && !cheo2[i-j+n]
                x[i] = j
                cot[j] = cheo1[i+j] = cheo2[i-j+n] = true
                dat(i + 1)
                cot[j] = cheo1[i+j] = cheo2[i-j+n] = false
            end
        end
    end
    dat(1)
    return so_loi_giai
end
```

PYTHON

```
def n_quan_hau(n):
    x = [0] * n
    cot   = [False] * n
    cheo1 = [False] * (2 * n)       # theo i + j
    cheo2 = [False] * (2 * n)       # theo i - j + n
    so_loi_giai = 0

    def dat(i):
        nonlocal so_loi_giai
        if i == n:
            so_loi_giai += 1
            return
        for j in range(n):
            if not cot[j] and not cheo1[i + j] and not cheo2[i - j + n]:
                x[i] = j
                cot[j] = cheo1[i + j] = cheo2[i - j + n] = True
                dat(i + 1)
                cot[j] = cheo1[i + j] = cheo2[i - j + n] = False

    dat(0)
    return so_loi_giai
```

C++

```
int N, so_loi_giai;
vector<int> x;
vector<bool> cot, cheo1, cheo2;

void dat(int i) {
    if (i == N) { so_loi_giai++; return; }
    for (int j = 0; j < N; j++)
        if (!cot[j] && !cheo1[i + j] && !cheo2[i - j + N]) {
            x[i] = j;
            cot[j] = cheo1[i + j] = cheo2[i - j + N] = true;
            dat(i + 1);
            cot[j] = cheo1[i + j] = cheo2[i - j + N] = false;
        }
```

```
}

int n_quan_hau(int n) {
    N = n; so_loi_giai = 0;
    x.assign(n, 0);
    cot.assign(n, false);
    cheo1.assign(2 * n, false);
    cheo2.assign(2 * n, false);
    dat(0);
    return so_loi_giai;
}
```

**Vì sao cộng thêm $n$ vào chỉ số đường chéo ngược.** Hiệu $i - j$ nhận giá trị từ $-(n-1)$ đến $n-1$, tức có thể âm, mà chỉ số mảng thì không được âm trong C++ và Julia. Cộng thêm $n$ đưa khoảng giá trị về $[1, 2n-1]$, luôn hợp lệ. Trong Python, chỉ số âm là hợp lệ nhưng mang nghĩa "đếm từ cuối mảng", nên nếu quên cộng $n$ thì chương trình chạy bình thường mà cho kết quả sai — loại lỗi khó phát hiện nhất.

### 3.4.4 Hiệu quả của việc cắt nhánh

Bảng 3.4 là số đo thực tế: số nút của cây không gian trạng thái mà chương trình thực sự duyệt, so với số hoán vị mà phương pháp vét cạn phải sinh ra.

**Bảng 3.4.** Bài toán $n$ quân hậu: số lời giải và số nút phải duyệt.

| $n$ | Số lời giải | Số nút quay lui | $n!$ | Tỉ lệ tiết kiệm |
|---|---|---|---|---|
| 4 | 2 | 17 | 24 | 1,4 lần |
| 5 | 10 | 54 | 120 | 2,2 lần |
| 6 | 4 | 153 | 720 | 4,7 lần |
| 7 | 40 | 552 | 5 040 | 9,1 lần |
| 8 | 92 | 2 057 | 40 320 | 19,6 lần |
| 9 | 352 | 8 394 | 362 880 | 43,2 lần |
| 10 | 724 | 35 539 | 3 628 800 | 102,1 lần |

Cột cuối cho thấy đặc điểm quan trọng nhất của quay lui: lợi ích của việc cắt nhánh *tăng theo* kích thước bài toán. Ở $n = 4$ quay lui gần như không tiết kiệm gì, nhưng ở $n = 10$ nó chỉ duyệt một phần trăm số cấu hình. Với $n = 8$, cây quay lui có 2057 nút trong khi vét cạn phải xét 40 320 hoán vị.

**Bài 3.2.** Cài đặt bài toán $n$ quân hậu theo hai cách: cách thứ nhất sinh mọi hoán vị rồi kiểm tra, cách thứ hai dùng quay lui với ba mảng đánh dấu. Thêm biến đếm số nút và đo thời gian chạy cho $n$ từ 4 đến 11. Lập bảng đối chiếu với bảng 3.4. Cách thứ nhất sẽ không chạy nổi với $n = 12$; hãy giải thích bằng số liệu cụ thể.

## 3.5 Thuật toán nhánh và cận

### 3.5.1 Ý tưởng

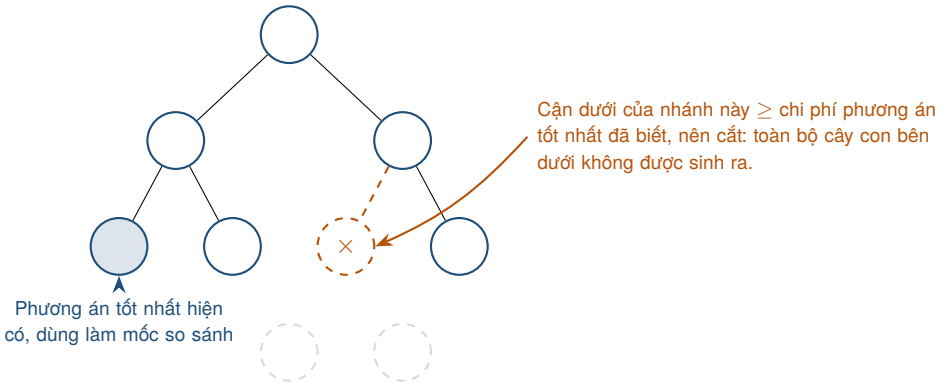
Quay lui cắt nhánh dựa trên *tính hợp lệ*: nếu giá trị đang thử vi phạm ràng buộc thì bỏ nhánh. Cách cắt này chỉ áp dụng được cho bài toán liệt kê hoặc bài toán tìm một lời giải thoả ràng buộc.

Với bài toán *tối ưu* — tìm phương án có chi phí nhỏ nhất hoặc giá trị lớn nhất — mọi phương án đều hợp lệ, nên không cắt được theo cách trên. Nhánh và cận bổ sung một tiêu chí cắt thứ hai, dựa trên *giá trị*: nếu chứng minh được rằng mọi phương án trong một nhánh đều tệ hơn phương án tốt nhất đã biết, thì cắt cả nhánh đó.

**Định nghĩa 3.3.** (*Cận dưới*) Với bài toán cực tiểu, một **cận dưới** của nhánh là một giá trị chắc chắn không lớn hơn chi phí của mọi phương án đầy đủ nằm trong nhánh đó.

Quy tắc cắt: nếu cận dưới của nhánh $\geq$ chi phí phương án tốt nhất đã tìm được, cắt nhánh. Điều kiện đúng đắn là cận dưới phải *thật sự* là cận dưới; nếu ước lượng quá cao, thuật toán có thể cắt nhầm nhánh chứa lời giải tối ưu và cho kết quả sai.

Cận càng chặt thì càng cắt được nhiều, nhưng tính cận cũng tốn thời gian. Việc chọn cận là sự đánh đổi giữa hai chi phí đó.




**Hình 3.5.** Nguyên tắc cắt của nhánh và cận. Cây con dưới nút bị cắt không bao giờ được sinh ra.

### 3.5.2 Bài toán người du lịch

**Đề bài.** Cho $n$ thành phố và ma trận chi phí $C$, trong đó $C[i][j]$ là chi phí đi từ thành phố $i$ đến thành phố $j$. Tìm hành trình xuất phát từ thành phố 0, đi qua mỗi thành phố còn lại đúng một lần, rồi quay về thành phố 0, sao cho tổng chi phí nhỏ nhất.

Số hành trình cần xét là $(n-1)!$. Với $n = 8$ là 5040, còn với $n = 15$ đã là hơn 87 tỉ.

**Cận dưới đơn giản.** Gọi $c_{\min}$ là chi phí của cạnh rẻ nhất trong toàn bộ ma trận. Nếu hành trình hiện tại đã đi qua $k$ thành phố với chi phí $f$, thì còn phải đi thêm $n-k+1$ cạnh nữa (kể cả cạnh quay về), nên tổng chi phí cuối cùng không nhỏ hơn

$$f + (n - k + 1)\, c_{\min}.$$

Nếu giá trị này đã lớn hơn hoặc bằng chi phí hành trình tốt nhất đã tìm được, ta cắt nhánh.

GIẢ MÃ

```
NGUOI-DU-LICH(k, f)                 // đã đi k thành phố, chi phí f
1  if k = n
2      tong ← f + C[x[k]][0]        // cạnh quay về
```

```
3      if tong < ky_luc
4          ky_luc ← tong;  luu lại hành trình
5      return
6  for j ← 1 to n-1
7      if j chưa thăm
8          f_moi ← f + C[x[k]][j]
9          if f_moi + (n-k)*c_min < ky_luc      // ← cận dưới
10             đánh dấu j đã thăm;  x[k+1] ← j
11             NGUOI-DU-LICH(k+1, f_moi)
12             bỏ đánh dấu j
13
```

**JULIA**

```julia
function nguoi_du_lich(C)
    n = size(C, 1)
    c_min = minimum(C[i, j] for i in 1:n, j in 1:n if i != j)
    ky_luc = typemax(Int)
    tot_nhat = Int[]
    x = [1]                          # bắt đầu ở thành phố 1
    tham = falses(n);  tham[1] = true
    so_nut = 0
    function thu(k, f)
        so_nut += 1
        if k == n
            tong = f + C[x[end], 1]
            if tong < ky_luc
                ky_luc = tong
                tot_nhat = copy(x)
            end
            return
        end
        for j in 2:n
            if !tham[j]
                f_moi = f + C[x[end], j]
                if f_moi + (n - k) * c_min < ky_luc
                    tham[j] = true;  push!(x, j)
                    thu(k + 1, f_moi)
                    pop!(x);  tham[j] = false
                end
            end
        end
    end
    thu(1, 0)
    return ky_luc, tot_nhat, so_nut
end
```

**PYTHON**

```python
def nguoi_du_lich(C):
    n = len(C)
    c_min = min(C[i][j] for i in range(n) for j in range(n) if i != j)
    ky_luc = float("inf")
    tot_nhat = []
    x = [0]                          # bắt đầu ở thành phố 0
    tham = [False] * n;  tham[0] = True
    so_nut = 0

    def thu(k, f):
```

```
        nonlocal ky_luc, tot_nhat, so_nut
        so_nut += 1
        if k == n:
            tong = f + C[x[-1]][0]
            if tong < ky_luc:
                ky_luc = tong
                tot_nhat = x[:]
            return
        for j in range(1, n):
            if not tham[j]:
                f_moi = f + C[x[-1]][j]
                if f_moi + (n - k) * c_min < ky_luc:
                    tham[j] = True;  x.append(j)
                    thu(k + 1, f_moi)
                    x.pop();  tham[j] = False

    thu(1, 0)
    return ky_luc, tot_nhat, so_nut
```

C++

```
int n, c_min, ky_luc, so_nut;
vector<vector<int>> C;
vector<int> x, tot_nhat;
vector<bool> tham;

void thu(int k, int f) {
    so_nut++;
    if (k == n) {
        int tong = f + C[x.back()][0];
        if (tong < ky_luc) { ky_luc = tong; tot_nhat = x; }
        return;
    }
    for (int j = 1; j < n; j++)
        if (!tham[j]) {
            int f_moi = f + C[x.back()][j];
            if (f_moi + (n - k) * c_min < ky_luc) {   // cận dưới
                tham[j] = true;  x.push_back(j);
                thu(k + 1, f_moi);
                x.pop_back();    tham[j] = false;
            }
        }
}
```

### 3.5.3 Đo hiệu quả của cận

Bảng 3.5 là ma trận chi phí dùng để đo, gồm 8 thành phố, đối xứng, cạnh rẻ nhất bằng 11.

Chạy chương trình hai lần, một lần bỏ dòng kiểm tra cận và một lần giữ nguyên, cho kết quả sau.

Hành trình tối ưu là $0 \to 1 \to 2 \to 6 \to 7 \to 5 \to 3 \to 4 \to 0$ với tổng chi phí 119, và cả hai phương pháp đều tìm ra đúng hành trình này. Khác biệt nằm ở số nút: 13 700 so với 499, tức nhánh và cận chỉ duyệt khoảng 3,6 phần trăm số nút, nhanh hơn 27,5 lần.

> **Thứ tự duyệt ảnh hưởng tới hiệu quả cắt.** Nhánh và cận chỉ cắt được khi đã có một phương án tốt để làm mốc. Vì vậy, tìm nhanh một phương án tốt ngay từ đầu là điều kiện để cắt

**Bảng 3.5.** Ma trận chi phí của ví dụ 8 thành phố.

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 12 | 29 | 22 | 13 | 24 | 31 | 18 |
| 1 | 12 | 0 | 19 | 30 | 25 | 27 | 17 | 21 |
| 2 | 29 | 19 | 0 | 15 | 28 | 33 | 20 | 26 |
| 3 | 22 | 30 | 15 | 0 | 16 | 11 | 34 | 23 |
| 4 | 13 | 25 | 28 | 16 | 0 | 14 | 32 | 19 |
| 5 | 24 | 27 | 33 | 11 | 14 | 0 | 21 | 15 |
| 6 | 31 | 17 | 20 | 34 | 32 | 21 | 0 | 13 |
| 7 | 18 | 21 | 26 | 23 | 19 | 15 | 13 | 0 |

**Bảng 3.6.** Số nút phải duyệt của bài toán người du lịch với 8 thành phố.

| Phương pháp | Số nút duyệt | Hành trình tìm được |
|---|---|---|
| Quay lui thuần, không dùng cận | 13 700 | độ dài 119 |
| Nhánh và cận | 499 | độ dài 119 |

được nhiều. Trong bài toán người du lịch, một cải tiến đơn giản là ở mỗi bước ưu tiên thử thành phố gần nhất trước; trong bài toán cái túi, sắp xếp các đồ vật theo tỉ lệ giá trị trên trọng lượng giảm dần trước khi duyệt.

**Bài 3.3.** Cài đặt bài toán cái túi bằng nhánh và cận. Đề bài: cho $n$ đồ vật, đồ vật thứ $i$ có trọng lượng $w_i$ và giá trị $v_i$, cho sức chứa $W$; chọn tập đồ vật có tổng trọng lượng không vượt $W$ và tổng giá trị lớn nhất.
Dùng cận trên kiểu tham lam phân số: sắp xếp đồ vật theo $v_i/w_i$ giảm dần, rồi từ vị trí đang xét lấy tham lam cho tới khi đầy túi, cho phép lấy một phần của đồ vật cuối. Giá trị thu được là cận trên vì lời giải nguyên không thể tốt hơn lời giải phân số.
Kiểm thử với $W = 26$ và năm đồ vật $(w, v)$ lần lượt là $(12, 24)$, $(7, 13)$, $(11, 23)$, $(8, 15)$, $(9, 16)$. Kết quả đúng là giá trị 51, chọn ba đồ vật $(11, 23)$, $(8, 15)$, $(7, 13)$. Đếm số nút với cận và không có cận; kết quả phải là 12 nút so với 44 nút.

## 3.6 Ứng dụng: bài toán thoả ràng buộc và chương trình giải Sudoku

### 3.6.1 Bài toán thoả ràng buộc

Nhiều bài toán trong trí tuệ nhân tạo có chung một dạng: xếp lịch thi, phân công ca trực, tô màu bản đồ, bố trí linh kiện trên bảng mạch. Chúng được mô hình hoá thống nhất như sau.

**Định nghĩa 3.4.** (*Bài toán thoả ràng buộc*) Một **bài toán thoả ràng buộc** (*Constraint Satisfaction Problem*, viết tắt CSP) gồm ba thành phần: một tập *biến* $X = \{X_1, \dots, X_m\}$; với mỗi biến $X_i$ một *miền giá trị* $D_i$; và một tập *ràng buộc* quy định những tổ hợp giá trị nào được phép. Lời giải là một phép gán giá trị cho mọi biến sao cho không ràng buộc nào bị vi phạm.

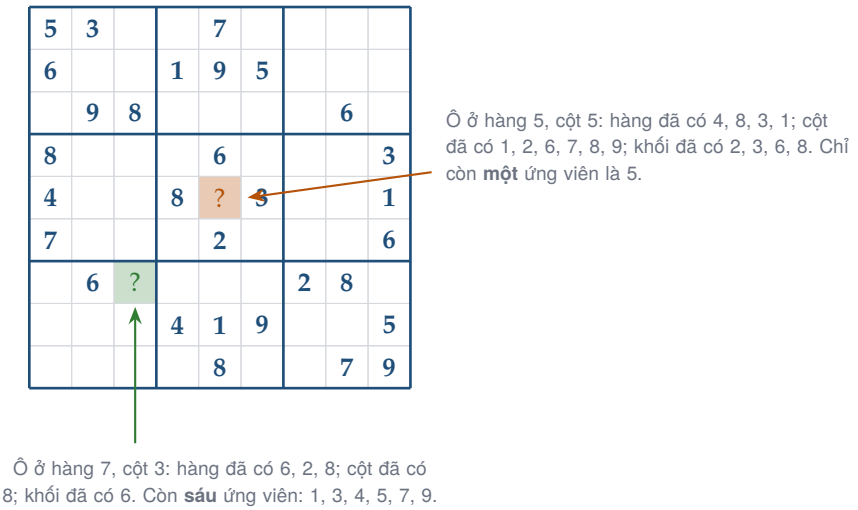
Thuật toán cơ bản để giải CSP chính là quay lui của mục 3.4: chọn một biến chưa gán, thử từng giá trị trong miền của nó, kiểm tra ràng buộc, đi tiếp. Vì vậy toàn bộ kỹ thuật của chương này áp dụng trực tiếp.

### 3.6.2 Sudoku là một bài toán thoả ràng buộc

Lưới Sudoku $9 \times 9$ mô hình hoá thành CSP như sau:

- **Biến:** mỗi ô trống là một biến, tổng cộng nhiều nhất 81 biến.
- **Miền giá trị:** $\{1, 2, \dots, 9\}$.
- **Ràng buộc:** trong mỗi hàng, mỗi cột và mỗi khối $3 \times 3$, các giá trị phải khác nhau đôi một. Tổng cộng 27 ràng buộc.

Đề Sudoku dùng trong mục này có 51 ô trống, nên không gian tìm kiếm thô là $9^{51} \approx 4{,}6 \cdot 10^{48}$ cách điền. Một máy tính xét được $10^9$ cách mỗi giây vẫn cần hơn $10^{32}$ năm để duyệt hết. Quay lui giải xong trong vài phần nghìn giây, vì các ràng buộc cắt bỏ gần như toàn bộ không gian đó.




**Hình 3.6.** Đề Sudoku dùng trong mục này, có 30 số cho trước và 51 ô trống. Hai ô trống được chú thích có số ứng viên rất khác nhau: một ô chỉ còn một ứng viên, ô kia còn sáu. Heuristic MRV luôn chọn ô ít ứng viên nhất, tức ô tô cam, thay vì ô trống đầu tiên gặp khi quét từ trái sang phải.

### 3.6.3 Cách thứ nhất: quay lui theo thứ tự quét

Cách đơn giản nhất là quét lưới từ trái sang phải, từ trên xuống dưới, gặp ô trống đầu tiên thì thử các giá trị từ 1 đến 9.

GIẢ MÃ

```
GIAI-SUDOKU()
1  tìm ô trống đầu tiên (r, c) theo thứ tự quét
2  if không còn ô trống
3      return ĐÚNG                      // đã điền xong
4  for v ← 1 to 9
5      if HOP-LE(r, c, v)
6          b[r][c] ← v
7          if GIAI-SUDOKU() = ĐÚNG
8              return ĐÚNG
9          b[r][c] ← 0                   // quay lui
10 return SAI
```

**PYTHON**

```python
def hop_le(b, r, c, v):
    for i in range(9):
        if b[r][i] == v or b[i][c] == v:
            return False
    br, bc = 3 * (r // 3), 3 * (c // 3)
    for i in range(br, br + 3):
        for j in range(bc, bc + 3):
            if b[i][j] == v:
                return False
    return True

def giai_sudoku(b):
    for r in range(9):
        for c in range(9):
            if b[r][c] == 0:
                for v in range(1, 10):
                    if hop_le(b, r, c, v):
                        b[r][c] = v
                        if giai_sudoku(b):
                            return True
                        b[r][c] = 0     # quay lui
                return False            # ô này không điền được giá trị nào
    return True                         # không còn ô trống
```

**JULIA**

```julia
function hop_le(b, r, c, v)
    for i in 1:9
        (b[r, i] == v || b[i, c] == v) && return false
    end
    br, bc = 3 * ((r-1) ÷ 3), 3 * ((c-1) ÷ 3)
    for i in br+1:br+3, j in bc+1:bc+3
        b[i, j] == v && return false
    end
    return true
end

function giai_sudoku!(b)
    for r in 1:9, c in 1:9
        if b[r, c] == 0
            for v in 1:9
                if hop_le(b, r, c, v)
                    b[r, c] = v
                    giai_sudoku!(b) && return true
                    b[r, c] = 0             # quay lui
                end
            end
            return false
        end
    end
    return true
end
```

**C++**

```cpp
int b[9][9];

```

```cpp
bool hop_le(int r, int c, int v) {
    for (int i = 0; i < 9; i++)
        if (b[r][i] == v || b[i][c] == v) return false;
    int br = 3 * (r / 3), bc = 3 * (c / 3);
    for (int i = br; i < br + 3; i++)
        for (int j = bc; j < bc + 3; j++)
            if (b[i][j] == v) return false;
    return true;
}

bool giai_sudoku() {
    for (int r = 0; r < 9; r++)
        for (int c = 0; c < 9; c++)
            if (b[r][c] == 0) {
                for (int v = 1; v <= 9; v++)
                    if (hop_le(r, c, v)) {
                        b[r][c] = v;
                        if (giai_sudoku()) return true;
                        b[r][c] = 0;        // quay lui
                    }
                return false;
            }
    return true;
}
```

### 3.6.4 Cách thứ hai: heuristic chọn biến ràng buộc nhất

Thứ tự quét ở cách thứ nhất là tuỳ tiện: không có lý do nào để ô trên cùng bên trái được xét trước. Một cách chọn tốt hơn dựa trên nhận xét sau: nếu một ô chỉ còn hai ứng viên thì thử sai nhiều nhất một lần, còn nếu một ô còn bảy ứng viên thì có thể thử sai tới sáu lần, mỗi lần kéo theo cả một cây con.

**Định nghĩa 3.5.** (*Heuristic MRV*) **Minimum Remaining Values** (viết tắt MRV), hay heuristic "biến ràng buộc nhất", là quy tắc luôn chọn biến chưa gán có số ứng viên hợp lệ ít nhất.

MRV còn cho thêm một lợi ích: nếu có ô nào không còn ứng viên nào, MRV phát hiện ngay và cắt nhánh lập tức, thay vì phải đi sâu thêm nhiều mức mới nhận ra mâu thuẫn.

GIẢ MÃ

```
GIAI-SUDOKU-MRV()
1  tot ← rỗng;  it_nhat ← 10
2  for mỗi ô trống (r, c)
3      uv ← tập giá trị hợp lệ tại (r, c)
4      if |uv| < it_nhat
5          it_nhat ← |uv|;  tot ← (r, c, uv)
6          if it_nhat = 0
7              return SAI                // mâu thuẫn, cắt ngay
8  if tot = rỗng
9      return ĐÚNG                       // không còn ô trống
10 for v thuộc uv của tot
11     b[r][c] ← v
12     if GIAI-SUDOKU-MRV() = ĐÚNG
13         return ĐÚNG
14     b[r][c] ← 0
15 return SAI
```

PYTHON

```python
def ung_vien(b, r, c):
    return [v for v in range(1, 10) if hop_le(b, r, c, v)]

def giai_sudoku_mrv(b):
    tot = None
    it_nhat = 10
    for r in range(9):
        for c in range(9):
            if b[r][c] == 0:
                uv = ung_vien(b, r, c)
                if len(uv) < it_nhat:
                    it_nhat = len(uv)
                    tot = (r, c, uv)
                    if it_nhat == 0:
                        return False        # ô này không điền được gì
    if tot is None:
        return True                         # không còn ô trống
    r, c, uv = tot
    for v in uv:
        b[r][c] = v
        if giai_sudoku_mrv(b):
            return True
        b[r][c] = 0
    return False
```

JULIA

```julia
ung_vien(b, r, c) = [v for v in 1:9 if hop_le(b, r, c, v)]

function giai_sudoku_mrv!(b)
    tot = nothing
    it_nhat = 10
    for r in 1:9, c in 1:9
        if b[r, c] == 0
            uv = ung_vien(b, r, c)
            if length(uv) < it_nhat
                it_nhat = length(uv)
                tot = (r, c, uv)
                it_nhat == 0 && return false
            end
        end
    end
    tot === nothing && return true
    r, c, uv = tot
    for v in uv
        b[r, c] = v
        giai_sudoku_mrv!(b) && return true
        b[r, c] = 0
    end
    return false
end
```

C++

```cpp
bool giai_sudoku_mrv() {
    int br = -1, bc = -1, it_nhat = 10;
    vector<int> tot_uv;
```

```
    for (int r = 0; r < 9; r++)
        for (int c = 0; c < 9; c++)
            if (b[r][c] == 0) {
                vector<int> uv;
                for (int v = 1; v <= 9; v++)
                    if (hop_le(r, c, v)) uv.push_back(v);
                if ((int)uv.size() < it_nhat) {
                    it_nhat = uv.size();
                    br = r; bc = c; tot_uv = uv;
                    if (it_nhat == 0) return false;
                }
            }
    if (br == -1) return true;              // không còn ô trống
    for (int v : tot_uv) {
        b[br][bc] = v;
        if (giai_sudoku_mrv()) return true;
        b[br][bc] = 0;
    }
    return false;
}
```

### 3.6.5 Đo hiệu quả của heuristic

Cả hai chương trình chạy trên đúng đề Sudoku ở hình 3.6 và cho cùng một lời giải. Khác biệt nằm ở số lần gọi hàm đệ quy.

**Bảng 3.7.** Số lần gọi hàm đệ quy khi giải cùng một đề Sudoku 51 ô trống.

| Cách chọn ô để điền | Số lần gọi hàm |
|---|---|
| Ô trống đầu tiên theo thứ tự quét | 4 209 |
| Ô có ít ứng viên nhất (heuristic MRV) | 52 |

Con số 52 gần bằng số ô trống là 51, nghĩa là với đề này, MRV gần như không thử sai lần nào: mỗi lời gọi điền đúng một ô và đi tiếp. Tỉ lệ giảm là hơn 80 lần, chỉ nhờ đổi thứ tự xét chứ không đổi thuật toán.

**Nhận xét 3.1.** Đây là một hiện tượng chung của các bài toán thoả ràng buộc: thuật toán nền vẫn là quay lui, nhưng thứ tự chọn biến quyết định chi phí thực tế. Ngoài MRV còn hai heuristic hay dùng khác. **Degree heuristic** chọn biến liên quan tới nhiều ràng buộc nhất, dùng để phá thế hoà khi nhiều biến cùng có số ứng viên ít nhất. **Least constraining value** chọn thứ tự thử giá trị sao cho giá trị nào để lại nhiều lựa chọn nhất cho các biến còn lại thì thử trước.

**Bài 3.4.** Cài đặt cả hai chương trình giải Sudoku theo các bước sau.

1. Đọc đề từ tệp văn bản gồm 9 dòng, mỗi dòng 9 chữ số, dùng số 0 cho ô trống.
2. Viết hàm `hop_le` và kiểm thử riêng nó: với đề ở hình 3.6, ô ở hàng 5 cột 5 phải có đúng một ứng viên là 5, còn ô ở hàng 7 cột 3 phải có sáu ứng viên là 1, 3, 4, 5, 7, 9.
3. Cài đặt bản quét tuần tự, thêm biến đếm số lần gọi hàm.
4. Cài đặt bản MRV, dùng lại nguyên hàm `hop_le`.
5. Chạy cả hai trên cùng đề, in ra lời giải và số lần gọi hàm. Kết quả phải trùng bảng 3.7:

4209 và 52.

6. Kiểm tra hai lời giải trùng nhau từng ô.

Sau đó thử với một đề khó hơn, chẳng hạn đề chỉ có 17 số cho trước, và so sánh lại. Chênh lệch giữa hai cách sẽ còn lớn hơn nhiều.

**Tóm tắt.** Bài toán duyệt xuất hiện khi lời giải phải tìm trong một tập cấu hình có kích thước tăng theo hàm mũ hoặc giai thừa. Sinh kế tiếp liệt kê đầy đủ theo thứ tự từ điển, không dùng đệ quy, nhưng không cắt bỏ được gì. Đệ quy diễn đạt bài toán gọn hơn và mở đường cho việc dừng sớm. Quay lui cắt nhánh theo tính hợp lệ; với bài toán $n$ quân hậu, mức tiết kiệm tăng từ 1,4 lần ở $n = 4$ lên 102 lần ở $n = 10$. Nhánh và cận cắt thêm theo giá trị, đưa bài toán người du lịch 8 thành phố từ 13 700 nút xuống 499 nút. Cuối cùng, thứ tự chọn biến cũng là một công cụ độc lập: heuristic MRV giảm số lời gọi khi giải Sudoku từ 4209 xuống 52 mà không thay đổi thuật toán nền.

## 3.7 Bài tập chương 3

### Câu hỏi ôn tập

1. Nêu ba loại cấu hình cơ bản của bài toán liệt kê và công thức đếm của mỗi loại.
2. Phương pháp sinh kế tiếp cần biết những gì để liệt kê đầy đủ một loại cấu hình?
3. Phát biểu quy tắc sinh xâu nhị phân kế tiếp và giải thích vì sao nó tương đương với phép cộng thêm 1 trong hệ nhị phân.
4. Hai thành phần bắt buộc của một thuật toán đệ quy là gì? Điều gì xảy ra nếu thiếu một trong hai?
5. Vì sao bộ nhớ mà đệ quy chiếm tỉ lệ với độ sâu của cây đệ quy chứ không phải với tổng số lời gọi?
6. Dòng lệnh nào trong sơ đồ quay lui cho kỹ thuật này cái tên của nó? Hậu quả của việc quên dòng đó là gì?
7. Trong bài toán $n$ quân hậu, vì sao biểu diễn lời giải bằng một hoán vị lại thu gọn được không gian tìm kiếm?
8. Nêu điều kiện để hai quân hậu cùng nằm trên một đường chéo. Vì sao dùng $i + j$ và $i - j$ lại kiểm tra được điều kiện đó trong $O(1)$?
9. Phân biệt tiêu chí cắt nhánh của quay lui và của nhánh và cận.
10. Điều gì xảy ra nếu hàm tính cận dưới cho giá trị lớn hơn chi phí thật của phương án tối ưu trong nhánh?
11. Phát biểu bài toán thoả ràng buộc. Mô hình hoá Sudoku theo ba thành phần của định nghĩa.
12. Heuristic MRV là gì? Vì sao nó giảm được số lời gọi mà không cần đổi thuật toán?

### Bài tập tính toán

13. Liệt kê bằng tay toàn bộ xâu nhị phân độ dài 4 theo thứ tự từ điển, chỉ rõ vị trí $i$ được tìm thấy ở mỗi bước sinh.

14. Cho tổ hợp $(2, 4, 6)$ với $n = 7$, $k = 3$. Tính ba tổ hợp kế tiếp.

15. Cho hoán vị $(2, 5, 4, 3, 1)$. Tính ba hoán vị kế tiếp, ghi rõ chỉ số $i$, chỉ số $j$ và đoạn bị đảo ngược ở mỗi bước.

16. Vẽ toàn bộ cây không gian trạng thái của bài toán bốn quân hậu, đánh dấu các nhánh bị cắt. Đếm số nút và đối chiếu với con số 17 trong bảng 3.4.

17. Tính số bước của Tháp Hà Nội với $n = 15$ và ước lượng thời gian in ra màn hình nếu mỗi dòng mất 1 mi-li-giây.

18. Với ma trận chi phí ở bảng 3.5, tính cận dưới của nhánh ứng với hành trình đã đi $0 \to 1 \to 2$.

19. Cho đề Sudoku ở hình 3.6. Liệt kê ứng viên của các ô $(0, 2)$, $(1, 1)$ và $(2, 0)$, rồi cho biết MRV sẽ chọn ô nào trong ba ô đó.

### Bài tập lập trình mức cơ bản

20. Cài đặt cả ba thuật toán sinh kế tiếp bằng ba ngôn ngữ, kiểm tra số cấu hình sinh ra khớp với công thức đếm.

21. Viết chương trình liệt kê mọi tập con của tập $\{1, \dots, n\}$ có tổng bằng một số $S$ cho trước.

22. Viết chương trình liệt kê mọi chỉnh hợp chập $k$ của $n$ phần tử, cả bằng sinh kế tiếp lẫn bằng đệ quy.

23. Viết hàm đệ quy tính $a^n$ với $O(\log n)$ phép nhân, dựa trên nhận xét $a^n = (a^{n/2})^2$ khi $n$ chẵn.

24. Cài đặt Tháp Hà Nội, in ra dãy nước đi và kiểm tra số dòng bằng $2^n - 1$.

25. Viết chương trình liệt kê mọi cách chia số nguyên dương $n$ thành tổng các số nguyên dương không tăng dần.

26. Cài đặt bài toán $n$ quân hậu, in ra bàn cờ của lời giải đầu tiên với $n = 8$.

### Bài tập lập trình mức vận dụng

27. Bài toán mã đi tuần: tìm một đường đi của quân mã qua tất cả các ô của bàn cờ $n \times n$, mỗi ô đúng một lần. Cài đặt bằng quay lui, sau đó thêm heuristic Warnsdorff (luôn đi tới ô có ít nước đi tiếp theo nhất) và so sánh số nút duyệt với $n = 8$.

28. Bài toán tô màu đồ thị: cho đồ thị $n$ đỉnh và số màu $k$, tìm cách tô sao cho hai đỉnh kề nhau khác màu. Cài đặt bằng quay lui và áp dụng MRV.

29. Bài toán xếp lịch thi: cho danh sách môn thi và danh sách sinh viên đăng ký, xếp các môn vào số ca thi ít nhất sao cho không sinh viên nào có hai môn cùng ca. Mô hình hoá thành bài toán tô màu đồ thị và giải lại bằng chương trình ở bài 28.

30. Cài đặt bài toán cái túi bằng nhánh và cận theo hướng dẫn ở bài thực hành 3.3, kiểm chứng kết quả 51 và số nút 12 so với 44.

31. Cải tiến chương trình người du lịch: ở mỗi bước, sắp xếp các thành phố chưa thăm theo khoảng cách tăng dần trước khi thử. Đo lại số nút trên ma trận ở bảng 3.5 và so sánh với 499.

32. Viết chương trình sinh đề Sudoku: bắt đầu từ một lời giải đầy đủ, lần lượt xoá bớt các ô sao cho đề vẫn có nghiệm duy nhất. Kiểm tra tính duy nhất bằng cách đếm số lời giải thay vì dừng ở lời giải đầu tiên.

### Bài tập phần trí tuệ nhân tạo

33. Cài đặt cả hai chương trình giải Sudoku theo bài thực hành 3.4 và xác nhận kết quả 4209 so với 52.

34. Bổ sung heuristic *least constraining value*: trong hàm MRV, sắp xếp các ứng viên của ô được chọn theo số ứng viên mà chúng loại bỏ khỏi các ô liên quan, ít nhất trước. Đo lại số lời gọi.

35. Cài đặt kỹ thuật *forward checking*: sau khi gán một giá trị, loại ngay giá trị đó khỏi miền của các ô cùng hàng, cùng cột, cùng khối; nếu miền nào rỗng thì quay lui lập tức. So sánh số lời gọi với ba phiên bản đã có.

36. Viết chương trình đo số lời gọi trung bình trên 100 đề Sudoku ngẫu nhiên, với cả bốn phiên bản: quét tuần tự, MRV, MRV kết hợp *least constraining value*, và MRV kết hợp *forward checking*. Lập bảng kết quả và rút ra kết luận về mức đóng góp của từng kỹ thuật.

37. Mô hình hoá bài toán $n$ quân hậu thành bài toán thoả ràng buộc, rồi giải bằng chính chương trình CSP tổng quát viết cho Sudoku. So sánh số nút với chương trình chuyên dụng ở mục 3.4.

CHƯƠNG 4

# Các mô hình thuật toán

Ba chương trước trình bày các thuật toán cụ thể. Chương này lùi lại một bước và trình bày ba *mô hình* thiết kế thuật toán: chia để trị, tham lam và quy hoạch động. Mô hình không phải là thuật toán, mà là khuôn mẫu để thiết kế thuật toán cho một bài toán mới. Nhiều thuật toán đã gặp thực ra là trường hợp riêng của các mô hình này: sắp xếp nhanh và tìm kiếm nhị phân theo mô hình chia để trị, cách ghi nhớ kết quả khi tính dãy Fibonacci ở mục 1.4 là quy hoạch động.

Điểm chung của cả ba mô hình là chia bài toán thành các bài toán con. Chúng khác nhau ở quan hệ giữa các bài toán con đó.

**Bảng 4.1.** Ba mô hình thuật toán và điều kiện áp dụng.

| Mô hình | Quan hệ giữa các bài toán con | Điều kiện áp dụng |
|---|---|---|
| Chia để trị | các bài toán con rời nhau, không chồng lấn | bài toán chia được thành phần độc lập, ghép lời giải được |
| Tham lam | mỗi bước chọn một khả năng, không quay lại | lựa chọn tốt nhất tại chỗ dẫn tới lời giải tối ưu toàn cục |
| Quy hoạch động | các bài toán con chồng lấn, được tính một lần và dùng lại | bài toán có cấu trúc con tối ưu và các bài toán con lặp lại |

## 4.1 Mô hình chia để trị

### 4.1.1 Sơ đồ chung

Chia để trị (*divide and conquer*) gồm ba bước.

1. **Chia.** Tách bài toán kích thước $n$ thành $a$ bài toán con cùng dạng, mỗi bài toán con kích thước khoảng $n/b$.
2. **Trị.** Giải từng bài toán con, thường bằng đệ quy. Khi kích thước đủ nhỏ thì giải trực tiếp.
3. **Kết hợp.** Ghép lời giải của các bài toán con thành lời giải của bài toán ban đầu.

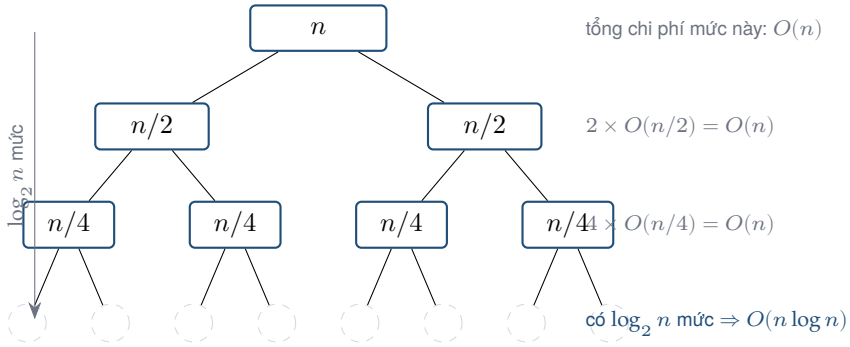
GIẢ MÃ

```
CHIA-DE-TRI(P)
1  if kích thước của P đủ nhỏ
2      return giải trực tiếp P
3  chia P thành P₁, P₂, ..., Pₐ
4  for i ← 1 to a
5      kq[i] ← CHIA-DE-TRI(Pᵢ)
6  return kết hợp kq[1..a]
```

Nếu bước chia và bước kết hợp tốn $O(n^d)$ thì thời gian chạy thoả hệ thức truy hồi

$$T(n) = a\,T(n/b) + O(n^d),$$

và định lý thợ ở mục 1.3.4 cho ngay nghiệm.

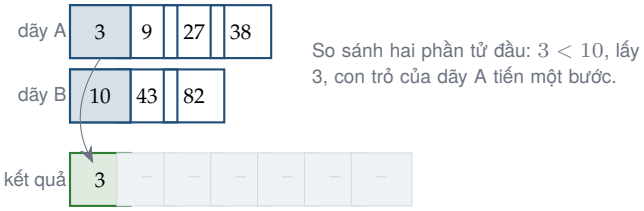



**Hình 4.1.** Cây đệ quy của một thuật toán chia để trị với $a = 2, b = 2, d = 1$. Mỗi mức tốn $O(n)$, có $\log_2 n$ mức.

### 4.1.2 Sắp xếp trộn

**Ý tưởng.** Chia dãy thành hai nửa, sắp xếp mỗi nửa bằng đệ quy, rồi trộn hai nửa đã sắp xếp thành một dãy có thứ tự.

Bước trộn là phần đáng chú ý: trộn hai dãy *đã có thứ tự* chỉ tốn $O(n)$, vì mỗi lần chỉ cần so sánh hai phần tử đầu của hai dãy và lấy phần tử nhỏ hơn.




**Hình 4.2.** Một bước của phép trộn hai dãy đã sắp xếp.

**GIẢ MÃ**

```
SAP-XEP-TRON(a, l, r)
1  if l ≥ r
2      return
3  m ← (l+r) div 2
4  SAP-XEP-TRON(a, l, m)
5  SAP-XEP-TRON(a, m+1, r)
6  TRON(a, l, m, r)

TRON(a, l, m, r)
1  i ← l;  j ← m+1;  t ← mảng tạm rỗng
2  while i ≤ m and j ≤ r
3      if a[i] ≤ a[j]:  thêm a[i] vào t;  i ← i+1
4      else:            thêm a[j] vào t;  j ← j+1
5  thêm phần còn lại của hai đoạn vào t
6  chép t trở lại a[l..r]
```

**JULIA**

```
function tron!(a, l, m, r)
    t = Int[]
    i, j = l, m + 1
    while i <= m && j <= r
        if a[i] <= a[j]
```

```
            push!(t, a[i]); i += 1
        else
            push!(t, a[j]); j += 1
        end
    end
    while i <= m;  push!(t, a[i]); i += 1;  end
    while j <= r;  push!(t, a[j]); j += 1;  end
    for k in 1:length(t)
        a[l + k - 1] = t[k]
    end
end

function sap_xep_tron!(a, l = 1, r = length(a))
    l >= r && return a
    m = (l + r) ÷ 2
    sap_xep_tron!(a, l, m)
    sap_xep_tron!(a, m + 1, r)
    tron!(a, l, m, r)
    return a
end
```

**PYTHON**

```
def tron(a, l, m, r):
    t = []
    i, j = l, m + 1
    while i <= m and j <= r:
        if a[i] <= a[j]:
            t.append(a[i]); i += 1
        else:
            t.append(a[j]); j += 1
    t.extend(a[i:m + 1])
    t.extend(a[j:r + 1])
    a[l:r + 1] = t

def sap_xep_tron(a, l=0, r=None):
    if r is None:
        r = len(a) - 1
    if l >= r:
        return a
    m = (l + r) // 2
    sap_xep_tron(a, l, m)
    sap_xep_tron(a, m + 1, r)
    tron(a, l, m, r)
    return a
```

**C++**

```
void tron(vector<int> &a, int l, int m, int r) {
    vector<int> t;
    t.reserve(r - l + 1);
    int i = l, j = m + 1;
    while (i <= m && j <= r)
        t.push_back(a[i] <= a[j] ? a[i++] : a[j++]);
    while (i <= m) t.push_back(a[i++]);
    while (j <= r) t.push_back(a[j++]);
    for (int k = 0; k < (int)t.size(); k++) a[l + k] = t[k];
}
```

```cpp

void sap_xep_tron(vector<int> &a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    sap_xep_tron(a, l, m);
    sap_xep_tron(a, m + 1, r);
    tron(a, l, m, r);
}
```

**Dấu <= trong phép trộn.** Viết `a[i] <= a[j]` chứ không phải `a[i] < a[j]`. Khi hai phần tử bằng nhau, dấu <= lấy phần tử của nửa trái trước, giữ nguyên thứ tự ban đầu, nên sắp xếp trộn là thuật toán ổn định. Đổi thành < thì mất tính ổn định mà kết quả sắp xếp vẫn đúng, nên lỗi này rất khó phát hiện bằng kiểm thử thông thường.

**Định lý 4.1.** *(Độ phức tạp của sắp xếp trộn) Sắp xếp trộn có độ phức tạp $O(n \log n)$ trong mọi trường hợp, và cần $O(n)$ bộ nhớ phụ.*

*Chứng minh.* Bước chia tốn $O(1)$, hai lời gọi đệ quy trên hai nửa, bước trộn duyệt đúng một lượt qua $n$ phần tử nên tốn $O(n)$. Vậy $T(n) = 2\,T(n/2) + O(n)$, tức $a = 2, b = 2, d = 1$. Vì $d = \log_b a = 1$, định lý thợ cho $T(n) = O(n \log n)$. Kết quả này không phụ thuộc dữ liệu vào vì bước chia luôn chia đôi, khác với sắp xếp nhanh mà độ sâu phụ thuộc cách chọn chốt. ∎

**Ví dụ 4.1.** *Số phép so sánh đo được.* Chạy chương trình có gắn biến đếm trên dãy ngẫu nhiên:

| $n$ | Số phép so sánh đo được | $n \log_2 n$ |
|---:|---:|---:|
| 8 | 17 | 24 |
| 1 000 | 8 687 | 9 966 |
| 100 000 | 1 536 325 | 1 660 964 |

Số đo luôn nhỏ hơn $n \log_2 n$ một chút vì khi một nửa hết trước, phần còn lại của nửa kia được chép thẳng, không tốn phép so sánh nào.

### 4.1.3 Tìm đồng thời giá trị nhỏ nhất và lớn nhất

**Bài toán.** Cho mảng $n$ phần tử, tìm cả giá trị nhỏ nhất lẫn giá trị lớn nhất.

Cách trực tiếp duyệt một lượt và so sánh mỗi phần tử với cả hai biến, tốn $2n - 2$ phép so sánh. Chia để trị làm tốt hơn: chia mảng làm đôi, tìm cặp (nhỏ nhất, lớn nhất) của mỗi nửa, rồi kết hợp bằng hai phép so sánh.

PYTHON

```python
def min_max(a, l, r):
    if l == r:                      # một phần tử
        return a[l], a[l]
    if r == l + 1:                  # hai phần tử, một phép so sánh
        return (a[l], a[r]) if a[l] < a[r] else (a[r], a[l])
    m = (l + r) // 2
    nho1, lon1 = min_max(a, l, m)
    nho2, lon2 = min_max(a, m + 1, r)
    return min(nho1, nho2), max(lon1, lon2)
```

JULIA

```julia
function min_max(a, l, r)
    l == r && return (a[l], a[l])
    r == l + 1 && return a[l] < a[r] ? (a[l], a[r]) : (a[r], a[l])
    m = (l + r) ÷ 2
    nho1, lon1 = min_max(a, l, m)
    nho2, lon2 = min_max(a, m + 1, r)
    return (min(nho1, nho2), max(lon1, lon2))
end
```

C++

```cpp
pair<int,int> min_max(const vector<int> &a, int l, int r) {
    if (l == r) return {a[l], a[l]};
    if (r == l + 1) return {min(a[l], a[r]), max(a[l], a[r])};
    int m = (l + r) / 2;
    auto t = min_max(a, l, m);
    auto p = min_max(a, m + 1, r);
    return {min(t.first, p.first), max(t.second, p.second)};
}
```

**Phân tích.** Gọi $T(n)$ là số phép so sánh. Ta có $T(2) = 1$ và $T(n) = 2\,T(n/2) + 2$. Giải hệ thức này được $T(n) = \frac{3n}{2} - 2$, ít hơn $2n - 2$ khoảng một phần tư. Với $n = 10^6$, tiết kiệm khoảng nửa triệu phép so sánh.

### 4.1.4 Luỹ thừa nhanh

**Bài toán.** Tính $a^n \bmod M$ với $n$ rất lớn.

Nhận xét: $a^n = (a^{n/2})^2$ khi $n$ chẵn, và $a^n = a \cdot (a^{(n-1)/2})^2$ khi $n$ lẻ. Mỗi bước giảm $n$ đi một nửa, cho $O(\log n)$ phép nhân thay vì $n - 1$ phép.

JULIA

```julia
function luy_thua(a, n, M)
    n == 0 && return 1 % M
    nua = luy_thua(a, n ÷ 2, M)
    kq = nua * nua % M
    isodd(n) && (kq = kq * a % M)
    return kq
end
```

PYTHON

```python
def luy_thua(a, n, M):
    if n == 0:
        return 1 % M
    nua = luy_thua(a, n // 2, M)
    kq = nua * nua % M
    if n % 2 == 1:
        kq = kq * a % M
    return kq
```

C++

```cpp
long long luy_thua(long long a, long long n, long long M) {
    if (n == 0) return 1 % M;
    long long nua = luy_thua(a, n / 2, M);
    long long kq = nua * nua % M;
    if (n & 1) kq = kq * a % M;
    return kq;
}
```

**Chú ý.** Phải gọi đệ quy *một lần* rồi bình phương kết quả. Nếu viết `luy_thua(a, n/2, M) * luy_thua(a, n/2, M)`, chương trình gọi hai lần với cùng tham số, số lời gọi trở lại $O(n)$ và mất hết lợi ích. Trong C++, tích của hai số cỡ $10^9$ vượt phạm vi `int` nên mọi biến trung gian phải khai báo `long long`.

**Ví dụ 4.2.** $3^{45} \bmod (10^9+7) = 644\,897\,553$. Thuật toán thực hiện 6 lời gọi đệ quy thay vì 44 phép nhân.

**Bài 4.1.** Cài đặt sắp xếp trộn bằng ba ngôn ngữ, thêm biến đếm phép so sánh, chạy trên dãy ngẫu nhiên với $n = 8, 1000, 100\,000$ và đối chiếu với bảng số liệu ở ví dụ trên. Sau đó chạy trên dãy đã sắp xếp và dãy sắp xếp ngược; số phép so sánh gần như không đổi, khác hẳn sắp xếp nhanh. Hãy giải thích.

## 4.2 Mô hình tham lam

### 4.2.1 Sơ đồ chung

Thuật toán tham lam (*greedy*) xây dựng lời giải theo từng bước; ở mỗi bước nó chọn khả năng có vẻ tốt nhất tại thời điểm đó và *không bao giờ xét lại* lựa chọn đã làm.

GIẢ MÃ

```
THAM-LAM(tập ứng viên C)
1  S ← rỗng
2  while C ≠ rỗng and S chưa là lời giải đầy đủ
3      x ← phần tử tốt nhất của C theo hàm chọn
4      loại x khỏi C
5      if them x vao S van chap nhan duoc
6          S ← S + {x}
7  return S
```

Ưu điểm của tham lam là đơn giản và nhanh, thường chỉ $O(n \log n)$ do bước sắp xếp ban đầu. Nhược điểm là nó *không phải lúc nào cũng cho lời giải tối ưu*. Vì vậy với mỗi bài toán, phải chứng minh được tính tối ưu trước khi dùng.

### 4.2.2 Bài toán đổi tiền

**Bài toán.** Cho các mệnh giá và một số tiền $S$. Trả đủ $S$ bằng ít tờ tiền nhất.

**Thuật toán tham lam.** Luôn lấy tờ có mệnh giá lớn nhất còn dùng được.

PYTHON

```python
def doi_tien(menh_gia, S):
    menh_gia = sorted(menh_gia, reverse=True)
    kq = {}
    con = S
    for m in menh_gia:
        if con >= m:
            kq[m] = con // m
            con %= m
    return kq, con                  # con > 0 nghĩa là không trả đủ được
```

JULIA

```julia
function doi_tien(menh_gia, S)
    mg = sort(menh_gia, rev = true)
    kq = Dict{Int,Int}()
    con = S
    for m in mg
        if con >= m
            kq[m] = con ÷ m
            con %= m
        end
    end
    return kq, con
end
```

C++

```cpp
vector<pair<int,int>> doi_tien(vector<int> menh_gia, int S) {
    sort(menh_gia.rbegin(), menh_gia.rend());
    vector<pair<int,int>> kq;
    int con = S;
    for (int m : menh_gia)
        if (con >= m) { kq.push_back({m, con / m}); con %= m; }
    return kq;                      // nếu con > 0 thì không trả đủ được
}
```

**Ví dụ 4.3.** *Tiền Việt Nam*. Với các mệnh giá 500 000, 200 000, 100 000, 50 000, 20 000, 10 000, 5 000, 2 000, 1 000 và $S = 386\,000$, thuật toán tham lam trả 7 tờ: một tờ 200 000, một tờ 100 000, một tờ 50 000, một tờ 20 000, một tờ 10 000, một tờ 5 000 và một tờ 1 000. Chương trình quy hoạch động ở mục 4.3 cũng cho 7 tờ, nên trong trường hợp này tham lam là tối ưu.

**Ví dụ 4.4.** *Khi tham lam sai*. Xét bộ mệnh giá $\{1, 3, 4\}$ và $S = 6$. Tham lam lấy tờ 4, còn lại 2, phải lấy thêm hai tờ 1, tổng cộng **ba** tờ. Lời giải tối ưu là hai tờ 3, tổng cộng **hai** tờ.

Hai ví dụ trên cho thấy tính đúng đắn của thuật toán tham lam *phụ thuộc vào dữ liệu*, không phải vào thuật toán. Bộ mệnh giá của hầu hết các đồng tiền thực tế có tính chất bảo đảm tham lam tối ưu; bộ mệnh giá tuỳ ý thì không.

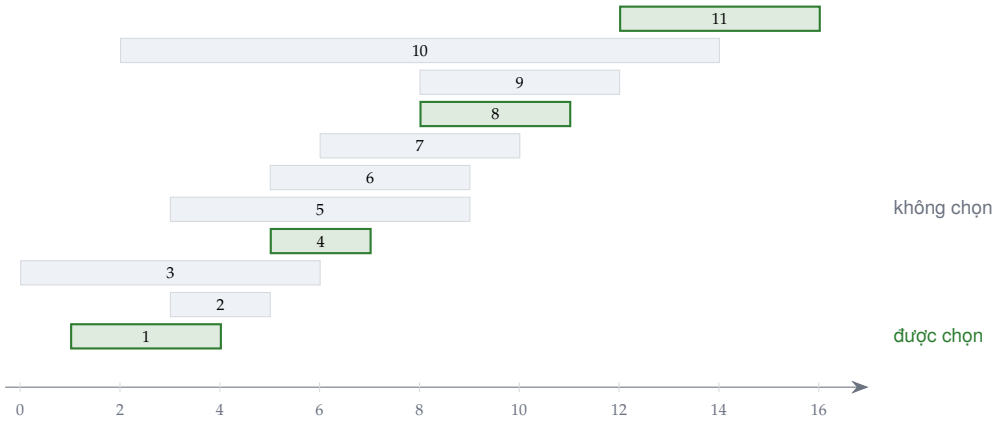
### 4.2.3 Bài toán chọn hoạt động

**Bài toán.** Cho $n$ hoạt động, hoạt động thứ $i$ bắt đầu lúc $s_i$ và kết thúc lúc $f_i$. Chỉ có một phòng, hai hoạt động không được chồng thời gian. Chọn nhiều hoạt động nhất có thể.

**Thuật toán tham lam.** Sắp xếp theo thời điểm kết thúc tăng dần, rồi duyệt và chọn mọi hoạt động bắt đầu sau khi hoạt động vừa chọn kết thúc.

**Định lý 4.2.** (*Tính tối ưu của tiêu chí kết thúc sớm nhất*) *Chiến lược chọn hoạt động kết thúc sớm nhất trong số các hoạt động còn dùng được luôn cho số hoạt động nhiều nhất.*

*Chứng minh.* Gọi $a_1$ là hoạt động kết thúc sớm nhất và $O$ là một lời giải tối ưu bất kỳ. Nếu $a_1 \in O$ thì không phải làm gì. Nếu không, gọi $b$ là hoạt động kết thúc sớm nhất trong $O$. Vì $a_1$ kết thúc sớm nhất trong tất cả nên $f_{a_1} \leq f_b$. Thay $b$ bằng $a_1$ trong $O$: tập thu được vẫn hợp lệ, vì mọi hoạt động khác trong $O$ đều bắt đầu sau $f_b \geq f_{a_1}$, và có cùng số phần tử nên vẫn tối ưu. Lặp lại lập luận này cho các bài toán con còn lại, ta được một lời giải tối ưu chứa toàn bộ các hoạt động do thuật toán tham lam chọn. ∎




**Hình 4.3.** Mười một hoạt động sắp theo thời điểm kết thúc tăng dần. Bốn hoạt động tô đậm là lời giải của thuật toán tham lam.

**PYTHON**

```python
def chon_hoat_dong(hd):
    hd = sorted(hd, key=lambda t: t[1])     # sắp theo thời điểm kết thúc
    kq = []
    het = float("-inf")
    for s, f in hd:
        if s >= het:
            kq.append((s, f))
            het = f
    return kq
```

**JULIA**

```julia
function chon_hoat_dong(hd)
    hd = sort(hd, by = t -> t[2])           # sắp theo thời điểm kết thúc
    kq = Tuple{Int,Int}[]
    het = typemin(Int)
    for (s, f) in hd
        if s >= het
            push!(kq, (s, f))
            het = f
        end
    end
    return kq
end
```

C++

```cpp
vector<pair<int,int>> chon_hoat_dong(vector<pair<int,int>> hd) {
    sort(hd.begin(), hd.end(),
         [](auto &a, auto &b) { return a.second < b.second; });
    vector<pair<int,int>> kq;
    int het = INT_MIN;
    for (auto &[s, f] : hd)
        if (s >= het) { kq.push_back({s, f}); het = f; }
    return kq;
}
```

**Ví dụ 4.5.** Với mười một hoạt động $(1,4)$, $(3,5)$, $(0,6)$, $(5,7)$, $(3,9)$, $(5,9)$, $(6,10)$, $(8,11)$, $(8,12)$, $(2,14)$, $(12,16)$, thuật toán chọn bốn hoạt động $(1,4)$, $(5,7)$, $(8,11)$, $(12,16)$. Chương trình vét cạn trên mọi tập con xác nhận bốn là số nhiều nhất có thể.

**Các tiêu chí tham lam sai.** Ba tiêu chí sau nghe hợp lý nhưng đều cho kết quả không tối ưu: chọn hoạt động bắt đầu sớm nhất (hoạt động $(0,6)$ chiếm chỗ của hai hoạt động ngắn), chọn hoạt động ngắn nhất, chọn hoạt động chồng lấn với ít hoạt động khác nhất. Đây là lý do bắt buộc phải chứng minh trước khi tin vào một tiêu chí tham lam.

**Bài 4.2.** Cài đặt bài toán chọn hoạt động bằng ba ngôn ngữ. Viết thêm một chương trình vét cạn duyệt mọi tập con để kiểm chứng kết quả với $n \le 20$. Sau đó cài đặt ba tiêu chí tham lam sai nêu trong khối chú ý và tìm bộ dữ liệu cụ thể chứng minh chúng sai.

## 4.3 Mô hình quy hoạch động

### 4.3.1 Hai điều kiện áp dụng

Quy hoạch động (*dynamic programming*) áp dụng được khi bài toán có hai tính chất sau.

1. **Cấu trúc con tối ưu.** Lời giải tối ưu của bài toán chứa trong nó lời giải tối ưu của các bài toán con.
2. **Các bài toán con chồng lấn.** Cùng một bài toán con xuất hiện nhiều lần trong quá trình giải.

Tính chất thứ nhất cũng có ở chia để trị. Tính chất thứ hai mới là điều làm nên quy hoạch động: vì bài toán con lặp lại, ta tính nó *một lần* và ghi kết quả vào bảng để dùng lại, thay vì tính lại mỗi lần gặp.

**Ví dụ 4.6.** *Dãy Fibonacci nhìn lại.* Mục 1.4 đã dẫn ra ví dụ này. Số liệu đo được: tính $F_{20}$ bằng đệ quy thuần cần 13 529 lời gọi, tính $F_{30}$ cần 1 664 079 lời gọi. Bản quy hoạch động chỉ cần đúng $n$ bước vì mỗi giá trị $F_i$ được tính một lần.

Có hai cách cài đặt quy hoạch động.

- **Từ trên xuống** (*top-down*): giữ nguyên hàm đệ quy, thêm một bảng ghi nhớ; trước khi tính thì tra bảng, tính xong thì ghi vào bảng. Cách này còn gọi là **ghi nhớ** (*memoization*).
- **Từ dưới lên** (*bottom-up*): xác định thứ tự tính sao cho khi tính đến một ô thì mọi ô nó phụ thuộc đã có sẵn, rồi điền bảng bằng vòng lặp. Cách này nhanh hơn vì không tốn chi phí gọi hàm và không có nguy cơ tràn ngăn xếp.

Quy trình thiết kế một lời giải quy hoạch động gồm bốn bước, và nên làm đúng theo thứ tự này.

1. **Định nghĩa trạng thái.** Đặt tên và nêu rõ ý nghĩa của $f[\dots]$. Đây là bước quan trọng nhất; định nghĩa sai thì ba bước sau đều vô ích.
2. **Viết công thức truy hồi.** $f$ tại một trạng thái tính từ những trạng thái nào.
3. **Xác định trường hợp cơ sở** và thứ tự tính.
4. **Truy vết** để khôi phục lời giải, nếu đề bài yêu cầu chứ không chỉ hỏi giá trị tối ưu.

### 4.3.2 Quy hoạch động trên mảng một chiều

#### 4.3.2.1 Dãy con tăng dài nhất

**Bài toán.** Cho dãy $a_1, \dots, a_n$. Tìm dãy con (không nhất thiết liên tiếp) dài nhất có các phần tử tăng dần.

**Bước 1 — trạng thái.** Đặt $f[i]$ là độ dài dãy con tăng dài nhất *kết thúc đúng tại* phần tử $a_i$. Chữ "đúng tại" là mấu chốt: nếu định nghĩa $f[i]$ là "độ dài dãy con tăng dài nhất trong $i$ phần tử đầu" thì không viết được công thức truy hồi, vì ta không biết dãy đó kết thúc ở đâu để nối thêm.

**Bước 2 — công thức truy hồi.**

$$f[i] = 1 + \max\{\, f[j] \;:\; j < i,\ a_j < a_i \,\},$$

với quy ước $\max \emptyset = 0$.

**Bước 3 — cơ sở.** $f[i] = 1$ với mọi $i$, ứng với dãy con chỉ gồm chính $a_i$. Tính $f$ theo $i$ tăng dần.

Đáp số là $\max_i f[i]$, không phải $f[n]$.

**Ví dụ 4.7.** *Lần vết.* Với dãy $a = (3, 10, 2, 1, 20, 4, 6, 21, 8)$:

| $i$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| $a_i$ | 3 | 10 | 2 | 1 | 20 | 4 | 6 | 21 | 8 |
| $f[i]$ | 1 | 2 | 1 | 1 | 3 | 2 | 3 | **4** | 4 |

Đáp số là 4. Một dãy con đạt độ dài đó là $(3, 10, 20, 21)$. Chương trình vét cạn duyệt mọi tập con xác nhận 4 là độ dài lớn nhất.

PYTHON

```python
def day_con_tang(a):
    n = len(a)
    f  = [1] * n
    tu = [-1] * n                  # tu[i] = chỉ số đứng trước i trong dãy con
    for i in range(n):
        for j in range(i):
            if a[j] < a[i] and f[j] + 1 > f[i]:
                f[i] = f[j] + 1
                tu[i] = j
    k = max(range(n), key=lambda i: f[i])
    day = []
    while k != -1:                 # truy vết ngược
        day.append(a[k])
        k = tu[k]
    return f[max(range(n), key=lambda i: f[i])], list(reversed(day))
```

JULIA

```julia
function day_con_tang(a)
    n = length(a)
    f  = ones(Int, n)
    tu = fill(-1, n)
    for i in 1:n, j in 1:i-1
        if a[j] < a[i] && f[j] + 1 > f[i]
            f[i] = f[j] + 1
            tu[i] = j
        end
    end
    k = argmax(f)
    do_dai = f[k]
    day = Int[]
    while k != -1
        push!(day, a[k])
        k = tu[k]
    end
    return do_dai, reverse(day)
end
```

C++

```cpp
pair<int, vector<int>> day_con_tang(const vector<int> &a) {
    int n = a.size();
    vector<int> f(n, 1), tu(n, -1);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (a[j] < a[i] && f[j] + 1 > f[i]) {
                f[i] = f[j] + 1;
                tu[i] = j;
            }
    int k = max_element(f.begin(), f.end()) - f.begin();
    int do_dai = f[k];
    vector<int> day;
    for (int t = k; t != -1; t = tu[t]) day.push_back(a[t]);
    reverse(day.begin(), day.end());
    return {do_dai, day};
}
```

**Phân tích.** Hai vòng lặp lồng nhau cho $O(n^2)$ thời gian và $O(n)$ bộ nhớ. Có lời giải $O(n \log n)$ dùng tìm kiếm nhị phân trên một mảng phụ, xem bài tập cuối chương.

#### 4.3.2.2 Bài toán cái túi

**Bài toán.** Cho $n$ đồ vật, đồ vật thứ $i$ có trọng lượng $w_i$ và giá trị $v_i$, và một cái túi chứa được tối đa trọng lượng $W$. Mỗi đồ vật hoặc lấy trọn hoặc không lấy. Chọn tập đồ vật có tổng giá trị lớn nhất.

**Bước 1 — trạng thái.** $f[i][j]$ là giá trị lớn nhất đạt được khi chỉ được dùng $i$ đồ vật đầu tiên và sức chứa còn lại là $j$.

**Bước 2 — truy hồi.** Với đồ vật thứ $i$ có hai khả năng:

$$f[i][j] = \begin{cases} f[i-1][j] & \text{nếu } w_i > j \text{ (không lấy được)}, \\ \max\big(f[i-1][j],\ f[i-1][j-w_i] + v_i\big) & \text{nếu } w_i \le j. \end{cases}$$

**Bước 3 — cơ sở.** $f[0][j] = 0$ với mọi $j$. Tính theo $i$ tăng dần.

**Ví dụ 4.8.** *Bảng đầy đủ.* Với $W = 10$ và bốn đồ vật $(w, v)$ lần lượt là $(5, 10)$, $(4, 40)$, $(6, 30)$, $(3, 50)$, bảng $f$ như sau.

| $i \backslash j$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 10 | 10 | 10 | 10 | 10 | 10 |
| 2 | 0 | 0 | 0 | 0 | 40 | 40 | 40 | 40 | 40 | 50 | 50 |
| 3 | 0 | 0 | 0 | 0 | 40 | 40 | 40 | 40 | 40 | 50 | 70 |
| 4 | 0 | 0 | 0 | 50 | 50 | 50 | 50 | 90 | 90 | 90 | **90** |

Đáp số là $f[4][10] = 90$, đạt được khi chọn đồ vật 2 và đồ vật 4, tổng trọng lượng $4 + 3 = 7 \le 10$ và tổng giá trị $40 + 50 = 90$. Chương trình vét cạn trên $2^4 = 16$ tập con cho cùng kết quả.

**Truy vết.** Đi từ ô $f[n][W]$ ngược lên: nếu $f[i][j] \neq f[i-1][j]$ thì đồ vật $i$ đã được lấy, chuyển sang ô $f[i-1][j-w_i]$; ngược lại chuyển sang $f[i-1][j]$.

**PYTHON**

```python
def cai_tui(w, v, W):
    n = len(w)
    f = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(W + 1):
            f[i][j] = f[i - 1][j]                       # không lấy đồ vật i
            if w[i - 1] <= j:                           # thử lấy đồ vật i
                f[i][j] = max(f[i][j], f[i - 1][j - w[i - 1]] + v[i - 1])
    chon = []                                           # truy vết
    j = W
    for i in range(n, 0, -1):
        if f[i][j] != f[i - 1][j]:
            chon.append(i)
            j -= w[i - 1]
    return f[n][W], sorted(chon)
```

**JULIA**

```julia
function cai_tui(w, v, W)
    n = length(w)
    f = zeros(Int, n + 1, W + 1)
    for i in 1:n, j in 0:W
        f[i+1, j+1] = f[i, j+1]
        if w[i] <= j
            f[i+1, j+1] = max(f[i+1, j+1], f[i, j-w[i]+1] + v[i])
        end
    end
    chon = Int[]
    j = W
    for i in n:-1:1
        if f[i+1, j+1] != f[i, j+1]
            push!(chon, i)
            j -= w[i]
        end
    end
    return f[n+1, W+1], sort(chon)
end
```

C++

```cpp
pair<int, vector<int>> cai_tui(const vector<int> &w, const vector<int> &v,
                               int W) {
    int n = w.size();
    vector<vector<int>> f(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 0; j <= W; j++) {
            f[i][j] = f[i - 1][j];
            if (w[i - 1] <= j)
                f[i][j] = max(f[i][j], f[i - 1][j - w[i - 1]] + v[i - 1]);
        }
    vector<int> chon;
    for (int i = n, j = W; i > 0; i--)
        if (f[i][j] != f[i - 1][j]) { chon.push_back(i); j -= w[i - 1]; }
    reverse(chon.begin(), chon.end());
    return {f[n][W], chon};
}
```

**Chỉ số của mảng $f$ trong Julia.** Bảng $f$ có hàng 0 và cột 0, nhưng mảng Julia đánh chỉ số từ 1. Vì vậy bản Julia phải viết `f[i+1, j+1]` cho ô $f[i][j]$ của công thức toán học. Cách khác là dùng `OffsetArrays` để giữ nguyên chỉ số 0, nhưng phải cài thêm gói. Trong bài thi, cách cộng thêm 1 an toàn hơn.

**Phân tích.** Bảng có $(n+1)(W+1)$ ô, mỗi ô tính trong $O(1)$, nên độ phức tạp là $O(nW)$ cả về thời gian lẫn bộ nhớ. Đây là độ phức tạp **giả đa thức**: nó là đa thức theo *giá trị* $W$ chứ không phải theo *số chữ số* của $W$. Với $W = 10^9$, bảng không thể lưu nổi dù $n$ chỉ bằng 10.

Có thể giảm bộ nhớ xuống $O(W)$ bằng cách chỉ giữ một hàng và duyệt $j$ giảm dần, vì $f[i][j]$ chỉ phụ thuộc hàng $i-1$.

C++

```cpp
// Bản tiết kiệm bộ nhớ: chỉ một mảng một chiều, duyệt j GIẢM dần
vector<int> f(W + 1, 0);
for (int i = 0; i < n; i++)
    for (int j = W; j >= w[i]; j--)          // duyệt ngược là bắt buộc
        f[j] = max(f[j], f[j - w[i]] + v[i]);
```

**Chú ý.** Duyệt $j$ giảm dần là bắt buộc. Nếu duyệt tăng dần, khi tính $f[j]$ thì $f[j-w_i]$ đã bị cập nhật ở chính vòng $i$ này, nghĩa là đồ vật $i$ có thể bị lấy nhiều lần. Chương trình khi đó giải bài toán *cái túi không giới hạn số lượng*, một bài toán khác.

### 4.3.3 Quy hoạch động trên ma trận

#### 4.3.3.1 Xâu con chung dài nhất

**Bài toán.** Cho hai xâu $X$ độ dài $m$ và $Y$ độ dài $n$. Tìm xâu con chung dài nhất, trong đó xâu con không cần gồm các ký tự liên tiếp.

**Trạng thái.** $f[i][j]$ là độ dài xâu con chung dài nhất của $i$ ký tự đầu của $X$ và $j$ ký tự đầu của $Y$.

**Truy hồi.**

$$f[i][j] = \begin{cases} f[i-1][j-1] + 1 & \text{nếu } X_i = Y_j, \\ \max(f[i-1][j],\ f[i][j-1]) & \text{nếu } X_i \neq Y_j. \end{cases}$$

**Cơ sở.** $f[0][j] = f[i][0] = 0$.

**Ví dụ 4.9.** Với $X =$ ABCBDAB và $Y =$ BDCABA, bảng $f$ như sau.

| | | | B | D | C | A | B | A |
|---|---|---|---|---|---|---|---|---|
| | | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| A | | 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| B | | 0 | 1 | 1 | 1 | 1 | 2 | 2 |
| C | | 0 | 1 | 1 | 2 | 2 | 2 | 2 |
| B | | 0 | 1 | 1 | 2 | 2 | 3 | 3 |
| D | | 0 | 1 | 2 | 2 | 2 | 3 | 3 |
| A | | 0 | 1 | 2 | 2 | 3 | 3 | **4** |
| B | | 0 | 1 | 2 | 2 | 3 | 4 | **4** |

Độ dài xâu con chung dài nhất là 4, và truy vết cho xâu BCBA.

PYTHON

```python
def xau_con_chung(x, y):
    m, n = len(x), len(y)
    f = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if x[i - 1] == y[j - 1]:
                f[i][j] = f[i - 1][j - 1] + 1
            else:
                f[i][j] = max(f[i - 1][j], f[i][j - 1])
    i, j, s = m, n, []                      # truy vết
    while i > 0 and j > 0:
        if x[i - 1] == y[j - 1]:
            s.append(x[i - 1]); i -= 1; j -= 1
        elif f[i - 1][j] >= f[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return f[m][n], "".join(reversed(s))
```

JULIA

```julia
function xau_con_chung(x, y)
    m, n = length(x), length(y)
    f = zeros(Int, m + 1, n + 1)
    for i in 1:m, j in 1:n
        f[i+1, j+1] = x[i] == y[j] ? f[i, j] + 1 : max(f[i, j+1], f[i+1, j])
    end
    i, j = m, n
    s = Char[]
    while i > 0 && j > 0
        if x[i] == y[j]
            push!(s, x[i]); i -= 1; j -= 1
        elseif f[i, j+1] >= f[i+1, j]
            i -= 1
        else
            j -= 1
        end
    end
```

```
    return f[m+1, n+1], String(reverse(s))
end
```

C++

```cpp
pair<int, string> xau_con_chung(const string &x, const string &y) {
    int m = x.size(), n = y.size();
    vector<vector<int>> f(m + 1, vector<int>(n + 1, 0));
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            f[i][j] = (x[i-1] == y[j-1]) ? f[i-1][j-1] + 1
                                         : max(f[i-1][j], f[i][j-1]);
    string s;
    for (int i = m, j = n; i > 0 && j > 0; ) {
        if (x[i-1] == y[j-1]) { s += x[i-1]; i--; j--; }
        else if (f[i-1][j] >= f[i][j-1]) i--;
        else j--;
    }
    reverse(s.begin(), s.end());
    return {f[m][n], s};
}
```

#### 4.3.3.2 Đường đi có tổng nhỏ nhất trên lưới

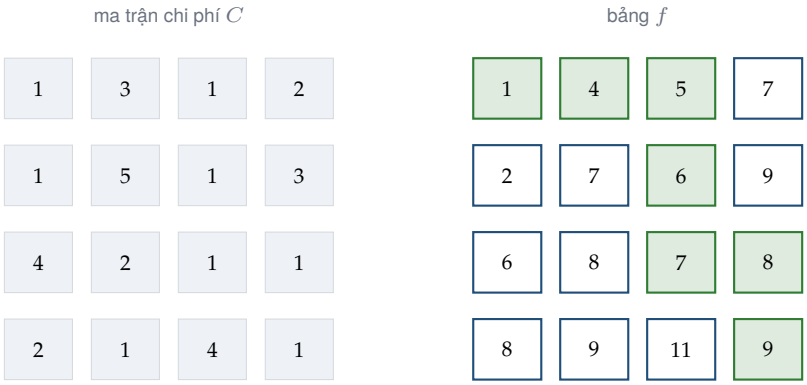
**Bài toán.** Cho lưới $m \times n$, ô $(i, j)$ có chi phí $C[i][j]$. Đi từ ô trên cùng bên trái đến ô dưới cùng bên phải, mỗi bước chỉ được sang phải hoặc xuống dưới. Tìm tổng chi phí nhỏ nhất.

**Trạng thái.** $f[i][j]$ là tổng chi phí nhỏ nhất để đi từ $(0, 0)$ tới $(i, j)$.

**Truy hồi.** Vì chỉ vào được ô $(i, j)$ từ ô bên trái hoặc ô bên trên:

$$f[i][j] = \min(f[i-1][j],\ f[i][j-1]) + C[i][j].$$

**Cơ sở.** $f[0][0] = C[0][0]$; hàng đầu và cột đầu chỉ có một hướng vào nên tính bằng tổng dồn.




**Hình 4.4.** Ma trận chi phí $C$ và bảng $f$ tương ứng. Các ô tô đậm tạo thành đường đi có tổng chi phí nhỏ nhất, bằng 9.

PYTHON

```python
def duong_di_luoi(C):
    m, n = len(C), len(C[0])
    f = [[0] * n for _ in range(m)]
    f[0][0] = C[0][0]
    for j in range(1, n):
```

```
        f[0][j] = f[0][j - 1] + C[0][j]     # hàng đầu: chỉ đi từ trái sang
    for i in range(1, m):
        f[i][0] = f[i - 1][0] + C[i][0]     # cột đầu: chỉ đi từ trên xuống
    for i in range(1, m):
        for j in range(1, n):
            f[i][j] = min(f[i - 1][j], f[i][j - 1]) + C[i][j]
    return f[m - 1][n - 1]
```

JULIA

```
function duong_di_luoi(C)
    m, n = size(C)
    f = zeros(Int, m, n)
    f[1, 1] = C[1, 1]
    for j in 2:n;  f[1, j] = f[1, j-1] + C[1, j];  end
    for i in 2:m;  f[i, 1] = f[i-1, 1] + C[i, 1];  end
    for i in 2:m, j in 2:n
        f[i, j] = min(f[i-1, j], f[i, j-1]) + C[i, j]
    end
    return f[m, n]
end
```

C++

```
int duong_di_luoi(const vector<vector<int>> &C) {
    int m = C.size(), n = C[0].size();
    vector<vector<int>> f(m, vector<int>(n, 0));
    f[0][0] = C[0][0];
    for (int j = 1; j < n; j++) f[0][j] = f[0][j-1] + C[0][j];
    for (int i = 1; i < m; i++) f[i][0] = f[i-1][0] + C[i][0];
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            f[i][j] = min(f[i-1][j], f[i][j-1]) + C[i][j];
    return f[m-1][n-1];
}
```

**Vì sao phải xử lý riêng hàng đầu và cột đầu.** Với $i = 0$, biểu thức $f[i-1][j]$ trở thành $f[-1][j]$. Trong C++ và Julia đó là truy cập ngoài mảng; trong Python đó là hàng cuối bảng, cho kết quả sai mà không báo lỗi. Cách viết gọn hơn là mở rộng bảng thêm một hàng và một cột với giá trị vô cùng lớn, khi đó công thức chung áp dụng được cho mọi ô.

**Bài 4.3.** Cài đặt cả ba bài toán quy hoạch động trong mục này bằng ba ngôn ngữ. Với mỗi bài, in ra toàn bộ bảng $f$ và đối chiếu từng ô với các bảng in trong tài liệu. Sau đó viết chương trình vét cạn cho từng bài và so sánh kết quả trên 1000 bộ dữ liệu ngẫu nhiên nhỏ. Cuối cùng, đo thời gian chạy của bài toán cái túi khi $W$ tăng từ $10^3$ lên $10^6$ với $n = 100$ cố định, để thấy rõ ý nghĩa của cụm từ "độ phức tạp giả đa thức".

## 4.4 Ứng dụng: thuật toán giải mã của mô hình ngôn ngữ

Mô hình ngôn ngữ dự đoán từ tiếp theo của một câu. Nhưng dự đoán từng từ chưa đủ: để sinh ra cả một câu, mô hình phải chọn một *dãy* từ, và số dãy có thể tăng theo hàm mũ theo độ dài

câu. Bài toán chọn dãy tốt nhất đó gọi là bài toán **giải mã** (*decoding*), và nó minh hoạ đúng ba mô hình thuật toán của chương này: tham lam, một biến thể hạn chế của duyệt, và quy hoạch động.

### 4.4.1 Mô hình ngôn ngữ hai từ

Để mọi con số trong mục này kiểm tra được bằng tay, ta dùng một mô hình rất nhỏ: xác suất của từ tiếp theo chỉ phụ thuộc từ liền trước.

**Bảng 4.2.** Mô hình ngôn ngữ dùng trong ví dụ. Mỗi hàng có tổng bằng 1.

| Từ hiện tại | Xác suất của từ tiếp theo |
|---|---|
| tôi | ăn: 0,55 đi: 0,45 |
| ăn | cơm: 0,42 sáng: 0,33 tối: 0,25 |
| đi | học: 0,56 làm: 0,24 chơi: 0,20 |

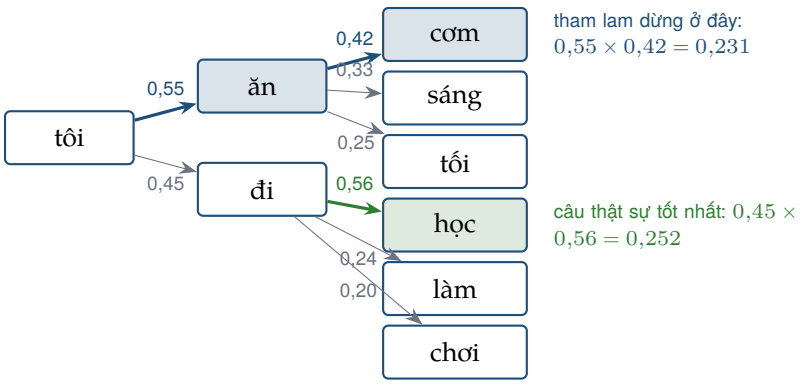
Điểm của một câu là tích các xác suất trên đường đi. Có sáu câu độ dài ba bắt đầu bằng "tôi", với điểm như sau.

**Bảng 4.3.** Toàn bộ sáu câu độ dài ba và điểm của chúng.

| Câu | Điểm | Cách tính |
|---|---|---|
| tôi đi học | **0,2520** | $0{,}45 \times 0{,}56$ |
| tôi ăn cơm | 0,2310 | $0{,}55 \times 0{,}42$ |
| tôi ăn sáng | 0,1815 | $0{,}55 \times 0{,}33$ |
| tôi ăn tối | 0,1375 | $0{,}55 \times 0{,}25$ |
| tôi đi làm | 0,1080 | $0{,}45 \times 0{,}24$ |
| tôi đi chơi | 0,0900 | $0{,}45 \times 0{,}20$ |

### 4.4.2 Giải mã tham lam

Cách đơn giản nhất là ở mỗi bước chọn từ có xác suất cao nhất.




**Hình 4.5.** Cây các câu có thể sinh ra. Tham lam đi theo nhánh đậm màu xanh đậm và bỏ lỡ nhánh tốt nhất vì bước đầu tiên của nhánh đó có xác suất thấp hơn.

Thuật toán tham lam chọn "ăn" ở bước một vì $0{,}55 > 0{,}45$, rồi chọn "cơm", cho câu "tôi ăn cơm" với điểm $0{,}231$. Nhưng câu tốt nhất là "tôi đi học" với điểm $0{,}252$. Tham lam sai vì nó không biết rằng nhánh "đi" tuy khởi đầu thấp hơn nhưng dẫn tới một từ tiếp theo có xác suất rất cao.

PYTHON

```python
def giai_ma_tham_lam(P, tu_dau, so_buoc):
    cau = [tu_dau]
    diem = 1.0
    for _ in range(so_buoc):
        tt = cau[-1]
        if tt not in P:
            break
        w = max(P[tt], key=lambda t: P[tt][t])   # từ có xác suất cao nhất
        diem *= P[tt][w]
        cau.append(w)
    return cau, diem
```

JULIA

```julia
function giai_ma_tham_lam(P, tu_dau, so_buoc)
    cau = [tu_dau]
    diem = 1.0
    for _ in 1:so_buoc
        tt = cau[end]
        haskey(P, tt) || break
        w = argmax(P[tt])
        diem *= P[tt][w]
        push!(cau, w)
    end
    return cau, diem
end
```

C++

```cpp
pair<vector<string>, double> giai_ma_tham_lam(
        const map<string, map<string,double>> &P,
        const string &tu_dau, int so_buoc) {
    vector<string> cau{tu_dau};
    double diem = 1.0;
    for (int b = 0; b < so_buoc; b++) {
        auto it = P.find(cau.back());
        if (it == P.end()) break;
        string tot; double cao = -1;
        for (auto &[w, p] : it->second)
            if (p > cao) { cao = p; tot = w; }
        diem *= cao;
        cau.push_back(tot);
    }
    return {cau, diem};
}
```

**Phân tích.** Với từ vựng $V$ từ và câu dài $L$, giải mã tham lam tốn $O(LV)$. Nhanh nhất có thể, nhưng không bảo đảm tối ưu.

### 4.4.3 Giải mã theo chùm

Duyệt toàn bộ $V^L$ câu là bất khả thi: với $V = 50\,000$ và $L = 20$, con số đó vượt xa mọi khả năng tính toán. Giải mã theo chùm (*beam search*) là bước trung gian: thay vì giữ một câu như tham lam hoặc giữ tất cả như vét cạn, nó giữ lại $k$ câu tốt nhất ở mỗi bước.

GIẢ MÃ

```
GIAI-MA-CHUM(k, L)
1  chùm ← { (câu rỗng, điểm 1) }
2  for bước ← 1 to L
3      mở_rộng ← rỗng
4      for mỗi (câu, điểm) trong chùm
5          for mỗi từ w có thể nối tiếp
6              thêm (câu + w, điểm × P(w | từ cuối)) vào mở_rộng
7      sắp xếp mở_rộng theo điểm giảm dần
8      chùm ← k phần tử đầu của mở_rộng
9  return phần tử đầu của chùm
```

PYTHON

```python
def giai_ma_chum(P, tu_dau, so_buoc, k):
    chum = [([tu_dau], 1.0)]
    for _ in range(so_buoc):
        mo_rong = []
        for cau, d in chum:
            tt = cau[-1]
            if tt not in P:
                mo_rong.append((cau, d))        # câu đã kết thúc
                continue
            for w, p in P[tt].items():
                mo_rong.append((cau + [w], d * p))
        mo_rong.sort(key=lambda t: -t[1])       # sắp xếp theo điểm giảm dần
        chum = mo_rong[:k]                      # giữ lại k câu tốt nhất
    return chum
```

JULIA

```julia
function giai_ma_chum(P, tu_dau, so_buoc, k)
    chum = [([tu_dau], 1.0)]
    for _ in 1:so_buoc
        mo_rong = Tuple{Vector{String}, Float64}[]
        for (cau, d) in chum
            tt = cau[end]
            if !haskey(P, tt)
                push!(mo_rong, (cau, d))
                continue
            end
            for (w, p) in P[tt]
                push!(mo_rong, ([cau; w], d * p))
            end
        end
        sort!(mo_rong, by = t -> -t[2])
        chum = mo_rong[1:min(k, length(mo_rong))]
    end
    return chum
end
```

C++

```cpp
using Cau = pair<vector<string>, double>;

vector<Cau> giai_ma_chum(const map<string, map<string,double>> &P,
                         const string &tu_dau, int so_buoc, int k) {
```

```
    vector<Cau> chum{{{tu_dau}, 1.0}};
    for (int b = 0; b < so_buoc; b++) {
        vector<Cau> mo_rong;
        for (auto &[cau, d] : chum) {
            auto it = P.find(cau.back());
            if (it == P.end()) { mo_rong.push_back({cau, d}); continue; }
            for (auto &[w, p] : it->second) {
                auto moi = cau;
                moi.push_back(w);
                mo_rong.push_back({moi, d * p});
            }
        }
        sort(mo_rong.begin(), mo_rong.end(),
             [](const Cau &a, const Cau &b) { return a.second > b.second; });
        if ((int)mo_rong.size() > k) mo_rong.resize(k);
        chum = mo_rong;
    }
    return chum;
}
```

**Ví dụ 4.10.** *Ảnh hưởng của $k$.* Với mô hình ở bảng 4.2, chạy hai bước sinh:

| $k$ | Câu trả về | Điểm |
|---|---|---|
| 1 | tôi ăn cơm | 0,2310 |
| 2 | tôi đi học | 0,2520 |
| 3 | tôi đi học | 0,2520 |

Với $k = 1$, chùm chỉ giữ một câu nên thuật toán trùng hoàn toàn với giải mã tham lam. Từ $k = 2$ trở đi, chùm giữ cả "tôi ăn" lẫn "tôi đi", nên nhánh tốt hơn không bị loại và thuật toán tìm ra câu tối ưu.

> **Chùm không bảo đảm tối ưu.** Giải mã theo chùm chỉ giảm rủi ro chứ không loại bỏ nó. Nếu câu tốt nhất có một tiền tố nằm ngoài $k$ tiền tố tốt nhất ở một bước nào đó, nó vẫn bị loại vĩnh viễn. Tăng $k$ làm giảm khả năng đó nhưng chi phí tăng tuyến tính theo $k$. Trong thực tế các hệ dịch máy thường dùng $k$ từ 4 đến 10.

**Phân tích.** Mỗi bước mở rộng $k$ câu thành $kV$ ứng viên rồi giữ lại $k$ tốt nhất, nên độ phức tạp là $O(L\,kV\log(kV))$ nếu sắp xếp toàn bộ. Dùng kỹ thuật chọn $k$ phần tử lớn nhất ở mục 2.7 thay cho sắp xếp thì giảm được xuống $O(L\,kV\log k)$.

### 4.4.4 Thuật toán Viterbi

Trong bài toán *gán nhãn từ loại*, đầu vào là một câu cho trước và ta cần gán cho mỗi từ một nhãn. Khác với bài toán sinh câu, ở đây số nhãn là nhỏ và cố định, nên có thể tìm dãy nhãn tốt nhất một cách *chính xác* bằng quy hoạch động. Thuật toán đó mang tên Viterbi.

**Mô hình.** Cho tập nhãn $T$, xác suất khởi đầu $\pi[t]$, xác suất chuyển $A[s][t]$ và xác suất phát $B[t][w]$. Xác suất của một dãy nhãn $t_1, \dots, t_L$ ứng với câu $w_1, \dots, w_L$ là

$$\pi[t_1]\,B[t_1][w_1]\prod_{i=2}^{L} A[t_{i-1}][t_i]\,B[t_i][w_i].$$

Số dãy nhãn là $|T|^L$, nhưng chỉ cần bốn bước quy hoạch động là đủ.

**Bước 1 — trạng thái.** $f[i][t]$ là xác suất lớn nhất của một dãy nhãn cho $i$ từ đầu tiên mà từ thứ $i$ được gán nhãn $t$.

**Bước 2 — truy hồi.**

$$f[i][t] = \Big(\max_{s \in T} f[i-1][s] \cdot A[s][t]\Big) \cdot B[t][w_i].$$

**Bước 3 — cơ sở.** $f[1][t] = \pi[t] \cdot B[t][w_1]$.

**Bước 4 — truy vết.** Lưu lại nhãn $s$ đạt cực đại ở mỗi ô, rồi đi ngược từ ô có $f[L][t]$ lớn nhất.

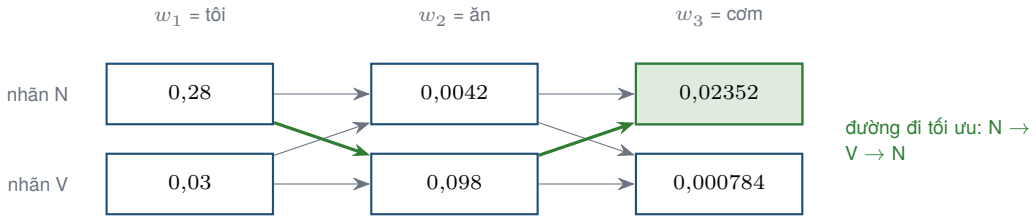
**Ví dụ 4.11.** *Gán nhãn câu "tôi ăn cơm".* Dùng hai nhãn: N (danh từ) và V (động từ). Tham số của mô hình:

| | | | | |
|---|---|---|---|---|
| Khởi đầu $\pi$ | N: 0,7 | V: 0,3 | | |
| Chuyển $A$ | N→N: 0,3 | N→V: 0,7 | V→N: 0,6 | V→V: 0,4 |
| Phát $B$ (nhãn N) | tôi: 0,4 | ăn: 0,05 | cơm: 0,4 | |
| Phát $B$ (nhãn V) | tôi: 0,1 | ăn: 0,5 | cơm: 0,02 | |

Bảng $f$ được điền như sau.

| | $i=1$ (tôi) | $i=2$ (ăn) | $i=3$ (cơm) |
|---|---|---|---|
| $f[i]$[N] | $0{,}7 \times 0{,}4 = 0{,}28$ | $0{,}084 \times 0{,}05 = 0{,}0042$ | $0{,}0588 \times 0{,}4 = \mathbf{0{,}02352}$ |
| $f[i]$[V] | $0{,}3 \times 0{,}1 = 0{,}03$ | $0{,}196 \times 0{,}5 = 0{,}098$ | $0{,}0392 \times 0{,}02 = 0{,}000784$ |
| nhãn trước | — | cả hai từ N | cả hai từ V |

Ô lớn nhất ở cột cuối là $f[3]$[N] $= 0{,}02352$. Truy vết ngược cho dãy nhãn N, V, N, tức "tôi/N ăn/V cơm/N". Chương trình vét cạn duyệt cả tám dãy nhãn cho cùng kết quả và cùng giá trị xác suất.




**Hình 4.6.** Lưới Viterbi cho câu "tôi ăn cơm". Mỗi ô giữ xác suất lớn nhất của một đường đi tới ô đó; đường đậm là đường được truy vết.

PYTHON

```python
def viterbi(cau, nhan, pi, A, B):
    T = len(cau)
    f  = [{} for _ in range(T)]
    tu = [{} for _ in range(T)]
    for t in nhan:                                  # cơ sở
        f[0][t]  = pi[t] * B[t][cau[0]]
        tu[0][t] = None
    for i in range(1, T):                           # truy hồi
        for t in nhan:
            tot, gt = None, -1.0
            for s in nhan:
                x = f[i - 1][s] * A[s][t]
```

```
                if x > gt:
                    gt, tot = x, s
            f[i][t]  = gt * B[t][cau[i]]
            tu[i][t] = tot
    cuoi = max(nhan, key=lambda t: f[T - 1][t])     # truy vết
    day = [cuoi]
    for i in range(T - 1, 0, -1):
        day.append(tu[i][day[-1]])
    return list(reversed(day)), f[T - 1][cuoi]
```

JULIA

```
function viterbi(cau, nhan, pi, A, B)
    T = length(cau)
    f  = [Dict{String,Float64}() for _ in 1:T]
    tu = [Dict{String,Union{String,Nothing}}() for _ in 1:T]
    for t in nhan
        f[1][t]  = pi[t] * B[t][cau[1]]
        tu[1][t] = nothing
    end
    for i in 2:T, t in nhan
        tot = nothing;  gt = -1.0
        for s in nhan
            x = f[i-1][s] * A[s][t]
            if x > gt
                gt = x;  tot = s
            end
        end
        f[i][t]  = gt * B[t][cau[i]]
        tu[i][t] = tot
    end
    cuoi = argmax(f[T])
    day = [cuoi]
    for i in T:-1:2
        push!(day, tu[i][day[end]])
    end
    return reverse(day), f[T][cuoi]
end
```

C++

```
pair<vector<string>, double> viterbi(
        const vector<string> &cau, const vector<string> &nhan,
        map<string,double> pi,
        map<string, map<string,double>> A,
        map<string, map<string,double>> B) {
    int T = cau.size();
    vector<map<string,double>> f(T);
    vector<map<string,string>> tu(T);
    for (auto &t : nhan) f[0][t] = pi[t] * B[t][cau[0]];
    for (int i = 1; i < T; i++)
        for (auto &t : nhan) {
            string tot; double gt = -1;
            for (auto &s : nhan) {
                double x = f[i-1][s] * A[s][t];
                if (x > gt) { gt = x; tot = s; }
            }
            f[i][t]  = gt * B[t][cau[i]];
```

```
            tu[i][t] = tot;
        }
    string cuoi; double cao = -1;
    for (auto &t : nhan)
        if (f[T-1][t] > cao) { cao = f[T-1][t]; cuoi = t; }
    vector<string> day{cuoi};
    for (int i = T - 1; i > 0; i--) day.push_back(tu[i][day.back()]);
    reverse(day.begin(), day.end());
    return {day, cao};
}
```

**Phân tích.** Bảng có $L \times |T|$ ô, mỗi ô cần duyệt $|T|$ nhãn trước đó, nên độ phức tạp là $O(L\,|T|^2)$. So với $|T|^L$ của phép vét cạn: với $L = 20$ và $|T| = 10$, đó là 2000 phép tính thay vì $10^{20}$.

**Nhân xác suất và hiện tượng tràn dưới.** Xác suất là số nhỏ hơn 1, nên tích của hàng trăm xác suất nhanh chóng nhỏ hơn số dương nhỏ nhất mà kiểu `double` biểu diễn được, và trở thành 0. Khi đó mọi đường đi đều có giá trị 0 và thuật toán mất khả năng phân biệt. Cách khắc phục chuẩn là làm việc với lô-ga-rit: thay $p_1 p_2 \cdots p_n$ bằng $\log p_1 + \log p_2 + \cdots + \log p_n$. Phép nhân thành phép cộng, thứ tự so sánh giữ nguyên vì log là hàm tăng, và giá trị không còn tràn dưới. Mọi thư viện xử lý ngôn ngữ đều làm như vậy.

**Bảng 4.4.** Ba thuật toán giải mã trong mục này.

| Thuật toán | Mô hình thuật toán | Độ phức tạp | Bảo đảm tối ưu |
|---|---|---|---|
| Giải mã tham lam | tham lam | $O(LV)$ | không |
| Giải mã theo chùm | duyệt có hạn chế | $O(LkV \log k)$ | không |
| Viterbi | quy hoạch động | $O(L\,\|T\|^2)$ | có |

Viterbi bảo đảm tối ưu còn hai thuật toán kia thì không. Lý do là bài toán gán nhãn có tập nhãn nhỏ và cố định, nên bảng quy hoạch động lưu được; còn bài toán sinh câu có tập từ vựng hàng chục nghìn từ, bảng $f$ sẽ có kích thước $L \times V$ với mỗi ô lại phụ thuộc toàn bộ lịch sử, nên quy hoạch động không áp dụng được và phải chấp nhận một thuật toán gần đúng.

**Bài 4.4.** Cài đặt cả ba thuật toán giải mã theo các bước sau.

1. Nạp mô hình ở bảng 4.2 vào một từ điển lồng nhau. Viết hàm kiểm tra tổng mỗi hàng bằng 1.
2. Cài giải mã tham lam, xác nhận kết quả “tôi ăn cơm” với điểm $0{,}231$.
3. Cài giải mã theo chùm, chạy với $k = 1, 2, 3$ và đối chiếu với bảng trong ví dụ.
4. Viết chương trình liệt kê cả sáu câu và điểm, xác nhận bảng 4.3.
5. Cài Viterbi với mô hình trong ví dụ, in ra toàn bộ bảng $f$ và bảng truy vết, xác nhận kết quả N V N với xác suất $0{,}02352$.
6. Viết chương trình vét cạn duyệt cả tám dãy nhãn và so sánh.
7. Sửa Viterbi sang dạng lô-ga-rit, chạy lại với một câu 200 từ sinh ngẫu nhiên và so sánh với bản dùng xác suất trực tiếp.

**Tóm tắt.** Chia để trị chia bài toán thành các phần rời nhau; sắp xếp trộn theo mô hình này đạt $O(n \log n)$ trong mọi trường hợp. Tham lam chọn khả năng tốt nhất tại mỗi bước và không xét lại; nó nhanh nhưng chỉ đúng khi chứng minh được, như bài toán chọn hoạt động, và sai trong những trường hợp như đổi tiền với bộ mệnh giá $\{1, 3, 4\}$. Quy hoạch động dùng khi các bài toán con chồng lấn: định nghĩa trạng thái, viết truy hồi, xác định cơ sở, rồi truy vết. Ba thuật toán giải mã của mô hình ngôn ngữ cho thấy ba mô hình cùng giải một bài toán với ba mức đánh đổi khác nhau giữa tốc độ và chất lượng lời giải.

## 4.5 Bài tập chương 4

### Câu hỏi ôn tập

1. Nêu ba bước của mô hình chia để trị và cho biết bước nào quyết định độ phức tạp trong trường hợp sắp xếp trộn.
2. Vì sao sắp xếp trộn đạt $O(n \log n)$ trong mọi trường hợp còn sắp xếp nhanh thì không?
3. Vì sao phép trộn hai dãy đã sắp xếp chỉ tốn $O(n)$?
4. Dấu <= trong phép trộn liên quan tới tính chất gì của thuật toán? Đổi thành < thì kết quả sắp xếp có sai không?
5. Nêu sơ đồ chung của thuật toán tham lam và cho biết đặc điểm nào khiến nó có thể sai.
6. Cho ví dụ bộ mệnh giá làm thuật toán tham lam đổi tiền cho kết quả không tối ưu, kèm số tiền cụ thể.
7. Phát biểu và chứng minh tính tối ưu của tiêu chí "kết thúc sớm nhất" trong bài toán chọn hoạt động.
8. Hai điều kiện để áp dụng quy hoạch động là gì? Điều kiện nào phân biệt nó với chia để trị?
9. Nêu bốn bước thiết kế một lời giải quy hoạch động.
10. Trong bài toán dãy con tăng dài nhất, vì sao phải định nghĩa $f[i]$ là độ dài dãy kết thúc *đúng tại* $a_i$?
11. Vì sao độ phức tạp $O(nW)$ của bài toán cái túi được gọi là giả đa thức?
12. Trong bản cái túi tiết kiệm bộ nhớ, vì sao phải duyệt $j$ giảm dần?
13. Vì sao giải mã tham lam bỏ lỡ câu tốt nhất trong ví dụ ở mục 4.4?
14. Giải mã theo chùm với $k = 1$ tương đương thuật toán nào?
15. Vì sao Viterbi bảo đảm tối ưu còn giải mã theo chùm thì không?
16. Hiện tượng tràn dưới khi nhân xác suất là gì và khắc phục thế nào?

### Bài tập tính toán

17. Lần vết sắp xếp trộn trên dãy $(38, 27, 43, 3, 9, 82, 10)$, vẽ cây đệ quy và ghi kết quả sau mỗi phép trộn.
18. Giải hệ thức $T(n) = 2T(n/2) + 2$ với $T(2) = 1$ và xác nhận công thức $T(n) = 3n/2 - 2$ của bài toán tìm đồng thời nhỏ nhất và lớn nhất.

19. Tính $7^{100} \bmod 13$ bằng luy thừa nhanh, ghi rõ từng lời gọi đệ quy.

20. Với bộ mệnh giá $\{1, 5, 6, 9\}$ và $S = 11$, so sánh kết quả của tham lam và của lời giải tối ưu.

21. Lập bảng $f$ đầy đủ cho bài toán dãy con tăng dài nhất trên dãy $(5, 2, 8, 6, 3, 6, 9, 7)$ và chỉ ra một dãy con đạt độ dài lớn nhất.

22. Lập bảng $f$ đầy đủ cho bài toán cái túi với $W = 8$ và ba đồ vật $(3, 4)$, $(4, 5)$, $(5, 6)$, rồi truy vết ra tập đồ vật được chọn.

23. Lập bảng $f$ cho xâu con chung dài nhất của `AGGTAB` và `GXTXAYB`, rồi truy vết.

24. Với mô hình ở bảng 4.2, tính điểm của tất cả các câu độ dài 3 và xác nhận bảng 4.3.

25. Với mô hình Viterbi trong ví dụ, tính xác suất của cả tám dãy nhãn và xác nhận N V N là dãy có xác suất lớn nhất.

### Bài tập lập trình mức cơ bản

26. Cài đặt sắp xếp trộn bằng ba ngôn ngữ, kiểm chứng với hàm sắp xếp thư viện trên 1000 mảng ngẫu nhiên.

27. Cài đặt hàm đếm số cặp nghịch thế của một mảng trong $O(n \log n)$ bằng cách sửa phép trộn.

28. Cài đặt luỹ thừa nhanh cho ma trận $2 \times 2$ và dùng nó để tính $F_n$ trong $O(\log n)$.

29. Cài đặt bài toán chọn hoạt động và kiểm chứng bằng vét cạn với $n \le 20$.

30. Cài đặt bài toán đổi tiền bằng cả tham lam lẫn quy hoạch động, chạy trên 100 bộ mệnh giá ngẫu nhiên và đếm số bộ mà tham lam cho kết quả không tối ưu.

31. Cài đặt dãy con tăng dài nhất $O(n^2)$ có truy vết.

32. Cài đặt bài toán cái túi cả hai bản: bảng hai chiều có truy vết và bản một chiều tiết kiệm bộ nhớ. So sánh bộ nhớ dùng khi $n = 100$, $W = 10^5$.

### Bài tập lập trình mức vận dụng

33. Cài đặt dãy con tăng dài nhất trong $O(n \log n)$: duy trì một mảng `duoi`, trong đó `duoi[k]` là phần tử cuối nhỏ nhất trong các dãy con tăng độ dài $k$, và dùng tìm kiếm nhị phân để cập nhật.

34. Bài toán nhân dãy ma trận: cho kích thước $n$ ma trận, tìm cách đặt dấu ngoặc sao cho tổng số phép nhân vô hướng nhỏ nhất. Đây là quy hoạch động trên đoạn với $O(n^3)$.

35. Bài toán khoảng cách chỉnh sửa: cho hai xâu, tìm số phép chèn, xoá, thay thế ít nhất để biến xâu này thành xâu kia. Cài đặt, in bảng và truy vết dãy phép biến đổi.

36. Bài toán đường đi trên lưới có vật cản, thêm ràng buộc không được đi qua ô bị đánh dấu, và cho phép đi cả bốn hướng. Giải thích vì sao lúc này quy hoạch động không còn áp dụng được và phải dùng thuật toán ở chương 6.

37. Bài toán chia mảng thành hai phần có tổng chênh lệch nhỏ nhất. Đưa về bài toán cái túi.

38. Cài đặt mã Huffman: dùng tham lam ghép hai ký tự có tần suất thấp nhất, lặp lại cho tới khi còn một cây. So sánh kích thước tệp nén với tệp gốc trên một văn bản tiếng Việt.

### Bài tập phần trí tuệ nhân tạo

39. Cài đặt đầy đủ ba thuật toán giải mã theo bài thực hành 4.4.

40. Mở rộng mô hình ngôn ngữ thành ba từ (từ hiện tại phụ thuộc hai từ liền trước) và sửa cả ba thuật toán giải mã cho phù hợp.

41. Thêm vào giải mã theo chùm cơ chế chuẩn hoá theo độ dài: chia log điểm cho độ dài câu trước khi so sánh. Giải thích vì sao không có cơ chế này thì thuật toán thiên vị câu ngắn.

42. Cài đặt Viterbi dạng lô-ga-rit và đo giá trị nhỏ nhất mà bản dùng xác suất trực tiếp còn biểu diễn được, bằng cách tăng dần độ dài câu cho tới khi kết quả bằng $0$.

43. Huấn luyện một mô hình gán nhãn từ loại đơn giản: đếm tần suất trên một tập văn bản đã gán nhãn để ước lượng $\pi$, $A$, $B$, rồi chạy Viterbi trên câu mới. Đánh giá độ chính xác trên tập kiểm tra.

44. So sánh số phép tính của Viterbi và của vét cạn khi $L$ tăng từ 5 đến 25 với $|T| = 10$, vẽ đồ thị hai đường trên thang lô-ga-rit.

CHƯƠNG 5

# Danh sách liên kết, Ngăn xếp, Hàng đợi

Bốn chương đầu tập trung vào thuật toán, và cấu trúc dữ liệu duy nhất được dùng là mảng. Từ chương này trở đi trọng tâm chuyển sang cấu trúc dữ liệu: cách tổ chức dữ liệu trong bộ nhớ để các thao tác cần thiết chạy nhanh.

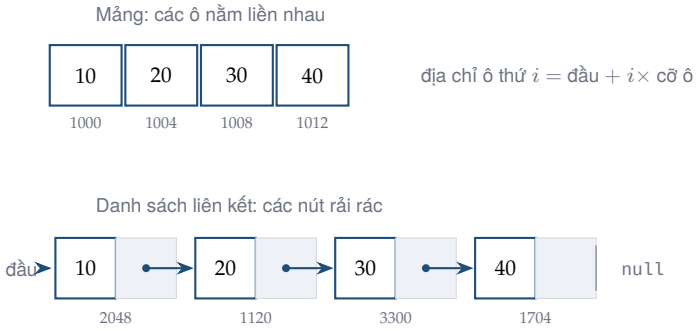
Xuất phát điểm là một nhận xét về mảng. Mảng cho phép truy cập phần tử thứ $i$ trong $O(1)$, vì địa chỉ của nó tính được trực tiếp từ địa chỉ đầu mảng. Đổi lại, việc chèn hoặc xoá một phần tử ở giữa buộc phải dịch chuyển toàn bộ phần đuôi, tốn $O(n)$. Ba cấu trúc trong chương này giải quyết đúng nhược điểm đó.

## 5.1 Danh sách và hai cách cài đặt

**Định nghĩa 5.1.** (*Danh sách*) **Danh sách** là một dãy hữu hạn các phần tử cùng kiểu, có thứ tự. Các thao tác cơ bản gồm: truy cập phần tử thứ $i$, tìm kiếm theo giá trị, chèn một phần tử vào vị trí bất kỳ, xoá một phần tử.

Có hai cách cài đặt danh sách.

- **Cài đặt bằng mảng**, còn gọi là danh sách kề: các phần tử nằm liên tiếp trong bộ nhớ.
- **Cài đặt bằng liên kết**: mỗi phần tử nằm ở một vùng nhớ riêng và lưu thêm địa chỉ của phần tử kế tiếp.




**Hình 5.1.** Hai cách cài đặt danh sách. Số nhỏ màu xám là địa chỉ ô nhớ.

### 5.1.1 So sánh hai cách

Bảng trên là lý thuyết. Số đo dưới đây cho thấy khác biệt trong thực tế, đo bằng C++ biên dịch với `-O2`.

Ở chiều ngược lại, với $n = 200\,000$ phần tử và 1000 lần truy cập ngẫu nhiên theo chỉ số, mảng mất 0,009 mi-li-giây còn danh sách liên kết mất 269,8 mi-li-giây, chậm hơn khoảng ba mươi nghìn lần. Hai bảng số liệu này là lý do không có cấu trúc nào "tốt hơn" cấu trúc nào: lựa chọn phụ thuộc vào thao tác nào được thực hiện nhiều nhất.

**Bảng 5.1.** Độ phức tạp các thao tác trên hai cách cài đặt danh sách.

| Thao tác | Mảng | Danh sách liên kết |
|---|---|---|
| Truy cập phần tử thứ $i$ | $O(1)$ | $O(n)$ |
| Tìm theo giá trị | $O(n)$ | $O(n)$ |
| Chèn vào đầu | $O(n)$ | $O(1)$ |
| Chèn sau một nút đã biết địa chỉ | $O(n)$ | $O(1)$ |
| Xoá một nút đã biết địa chỉ | $O(n)$ | $O(1)$ |
| Bộ nhớ cho mỗi phần tử | chỉ dữ liệu | dữ liệu + con trỏ |
| Cần biết trước số phần tử | có | không |

**Bảng 5.2.** Thời gian chèn $n$ phần tử vào *đầu* danh sách, đơn vị mi-li-giây.

| $n$ | `vector` | Danh sách liên kết | Tỉ lệ |
|---|---|---|---|
| 10 000 | 2,1 | 0,3 | 7 lần |
| 50 000 | 80,2 | 1,2 | 67 lần |
| 200 000 | 1 328,8 | 5,2 | 255 lần |

**Một yếu tố không xuất hiện trong ký hiệu $O$.** Chênh lệch 30 000 lần ở phép truy cập ngẫu nhiên lớn hơn nhiều so với dự đoán từ $O(1)$ và $O(n)$. Nguyên nhân là bộ nhớ đệm của bộ xử lý: các ô của mảng nằm liền nhau nên một lần đọc bộ nhớ nạp luôn nhiều ô kế tiếp, còn các nút của danh sách liên kết rải rác nên mỗi bước nhảy là một lần truy cập bộ nhớ chính. Ký hiệu $O$ bỏ qua yếu tố này, nhưng trong chương trình thực tế nó thường quyết định.

## 5.2 Danh sách liên kết đơn

### 5.2.1 Cấu trúc nút

Mỗi nút gồm hai phần: phần dữ liệu và phần liên kết trỏ tới nút kế tiếp. Nút cuối cùng có liên kết rỗng.

JULIA

```julia
mutable struct Nut
    gt::Int
    ke::Union{Nut, Nothing}      # nút kế tiếp, hoặc nothing nếu là nút cuối
end
Nut(gt) = Nut(gt, nothing)

mutable struct DSLK
    dau::Union{Nut, Nothing}
    n::Int
end
DSLK() = DSLK(nothing, 0)
```

PYTHON

```python
class Nut:
    __slots__ = ("gt", "ke")           # tiết kiệm bộ nhớ

```

```
    def __init__(self, gt, ke=None):
        self.gt = gt
        self.ke = ke                    # None nếu là nút cuối

class DSLK:
    def __init__(self):
        self.dau = None
        self.n = 0
```

C++

```
struct Nut {
    int gt;
    Nut *ke;
    Nut(int g, Nut *k = nullptr) : gt(g), ke(k) {}
};

struct DSLK {
    Nut *dau = nullptr;
    int n = 0;
};
```

**Ba cách diễn đạt cùng một ý.** C++ dùng *con trỏ* `Nut *`, tức biến chứa địa chỉ ô nhớ, và giá trị rỗng là `nullptr`. Python và Julia không có con trỏ tường minh: biến `ke` tham chiếu trực tiếp tới đối tượng nút, và giá trị rỗng là `None` hoặc `nothing`. Về mặt logic ba cách là một. Khác biệt thực tế là C++ yêu cầu người viết tự giải phóng bộ nhớ bằng `delete`, còn hai ngôn ngữ kia có bộ thu gom rác làm việc đó.
Trong Julia, phải so sánh bằng `===` và `!==` khi kiểm tra `nothing`, không dùng `==`.

### 5.2.2 Duyệt danh sách

Mọi thao tác đều bắt đầu bằng việc đi từ nút đầu theo các liên kết.

GIẢ MÃ

```
DUYET(dau)
1  p ← dau
2  while p ≠ null
3      xử lý p.gt
4      p ← p.ke
```

JULIA

```
function in_danh_sach(L::DSLK)
    p = L.dau
    while p !== nothing
        print(p.gt, " ")
        p = p.ke
    end
    println()
end
```

PYTHON

```python
def in_danh_sach(L):
    p = L.dau
    while p is not None:
        print(p.gt, end=" ")
        p = p.ke
    print()
```

C++

```cpp
void in_danh_sach(const DSLK &L) {
    for (Nut *p = L.dau; p != nullptr; p = p->ke)
        cout << p->gt << " ";
    cout << "\n";
}
```

### 5.2.3 Thêm phần tử

#### 5.2.3.1 Thêm vào đầu

Đây là thao tác rẻ nhất: tạo nút mới, cho nó trỏ vào nút đầu cũ, rồi cập nhật con trỏ đầu.

JULIA

```julia
function them_dau!(L::DSLK, gt)
    L.dau = Nut(gt, L.dau)
    L.n += 1
    return L
end
```

PYTHON

```python
def them_dau(L, gt):
    L.dau = Nut(gt, L.dau)
    L.n += 1
```

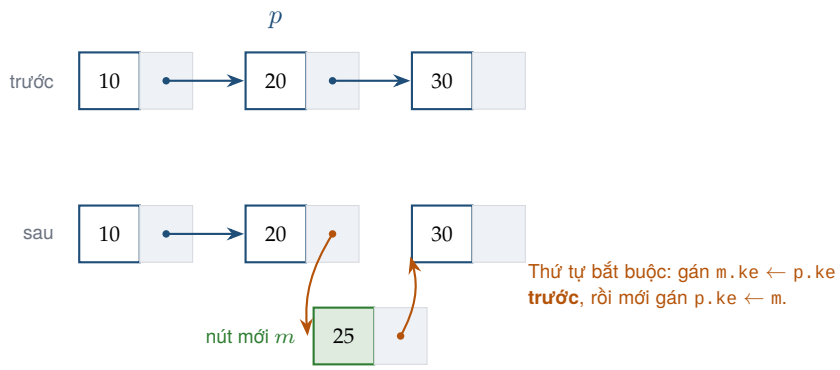
C++

```cpp
void them_dau(DSLK &L, int gt) {
    L.dau = new Nut(gt, L.dau);
    L.n++;
}
```

#### 5.2.3.2 Chèn sau một nút

GIẢ MÃ

```
CHEN-SAU(p, gt)
1  m ← nút mới với giá trị gt
2  m.ke ← p.ke          // phải làm bước này TRƯỚC
3  p.ke ← m
```




**Hình 5.2.** Chèn nút mới sau nút $p$. Đổi thứ tự hai phép gán sẽ làm mất phần đuôi của danh sách.

**JULIA**

```julia
function chen_sau!(L::DSLK, p::Nut, gt)
    m = Nut(gt, p.ke)       # m.ke = p.ke ngay khi tạo
    p.ke = m
    L.n += 1
    return m
end
```

**PYTHON**

```python
def chen_sau(L, p, gt):
    m = Nut(gt, p.ke)       # m.ke = p.ke ngay khi tạo
    p.ke = m
    L.n += 1
    return m
```

**C++**

```cpp
Nut *chen_sau(DSLK &L, Nut *p, int gt) {
    Nut *m = new Nut(gt, p->ke);    // m->ke = p->ke ngay khi tạo
    p->ke = m;
    L.n++;
    return m;
}
```

**Lỗi kinh điển.** Nếu viết `p.ke = m` trước rồi mới `m.ke = p.ke`, thì ở dòng thứ hai `p.ke` đã bằng `m`, nên `m.ke = m`: nút mới trỏ vào chính nó. Toàn bộ phần đuôi của danh sách bị mất, và mọi vòng lặp duyệt sau đó sẽ chạy vô hạn. Chương trình không báo lỗi, nó chỉ treo.

#### 5.2.3.3 Thêm vào cuối

Không có con trỏ tới nút cuối thì phải duyệt cả danh sách, tốn $O(n)$.

**PYTHON**

```python
def them_cuoi(L, gt):
    m = Nut(gt)
    if L.dau is None:
        L.dau = m
    else:
```

```
        p = L.dau
        while p.ke is not None:     # đi tới nút cuối
            p = p.ke
        p.ke = m
    L.n += 1
```

C++

```
void them_cuoi(DSLK &L, int gt) {
    Nut *m = new Nut(gt);
    if (L.dau == nullptr) L.dau = m;
    else {
        Nut *p = L.dau;
        while (p->ke != nullptr) p = p->ke;      // đi tới nút cuối
        p->ke = m;
    }
    L.n++;
}
```

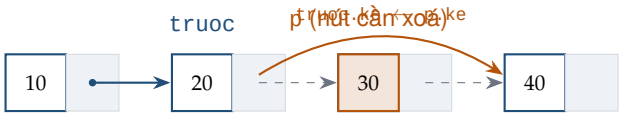
JULIA

```
function them_cuoi!(L::DSLK, gt)
    m = Nut(gt)
    if L.dau === nothing
        L.dau = m
    else
        p = L.dau
        while p.ke !== nothing
            p = p.ke
        end
        p.ke = m
    end
    L.n += 1
    return L
end
```

**Nhận xét 5.1.** Nếu chương trình thêm vào cuối thường xuyên, nên giữ thêm một con trỏ `cuoi` trỏ vào nút cuối cùng. Khi đó thao tác thêm vào cuối chỉ tốn $O(1)$, đổi lại mọi thao tác xoá phải nhớ cập nhật con trỏ đó.

### 5.2.4 Xoá phần tử

Muốn xoá một nút, phải biết nút *đứng trước* nó, vì liên kết chỉ đi một chiều. Vì vậy vòng lặp tìm kiếm phải giữ hai con trỏ.




**Hình 5.3.** Xoá nút $p$ khỏi danh sách liên kết đơn: nối nút đứng trước thẳng vào nút đứng sau.

GIẢ MÃ

```
XOA(L, gt)
1  truoc ← null;  p ← L.dau
2  while p ≠ null and p.gt ≠ gt
3      truoc ← p;  p ← p.ke
4  if p = null
5      return SAI                    // không có giá trị này
6  if truoc = null
7      L.dau ← p.ke                  // xoá nút đầu
8  else
9      truoc.ke ← p.ke
10 giải phóng p
11 return ĐÚNG
```

**PYTHON**

```python
def xoa(L, gt):
    truoc, p = None, L.dau
    while p is not None and p.gt != gt:
        truoc, p = p, p.ke
    if p is None:
        return False                  # không tìm thấy
    if truoc is None:
        L.dau = p.ke                  # xoá nút đầu
    else:
        truoc.ke = p.ke
    L.n -= 1
    return True
```

**C++**

```cpp
bool xoa(DSLK &L, int gt) {
    Nut *truoc = nullptr, *p = L.dau;
    while (p != nullptr && p->gt != gt) { truoc = p; p = p->ke; }
    if (p == nullptr) return false;          // không tìm thấy
    if (truoc == nullptr) L.dau = p->ke;     // xoá nút đầu
    else                  truoc->ke = p->ke;
    delete p;                                // bắt buộc, tránh rò bộ nhớ
    L.n--;
    return true;
}
```

**JULIA**

```julia
function xoa!(L::DSLK, gt)
    truoc = nothing
    p = L.dau
    while p !== nothing && p.gt != gt
        truoc = p
        p = p.ke
    end
    p === nothing && return false
    if truoc === nothing
        L.dau = p.ke
    else
        truoc.ke = p.ke
    end
    L.n -= 1
```

```
    return true
end
```

> **Hai chi tiết bắt buộc trong C++.** Thứ nhất, phải gọi `delete p` sau khi tách nút ra khỏi danh sách; quên là rò bộ nhớ. Thứ hai, phải gọi *sau* khi đã dùng xong `p->ke`; nếu viết `delete p` trước rồi mới đọc `p->ke`, chương trình đọc vùng nhớ đã giải phóng và cho kết quả không xác định.

### 5.2.5 Đảo ngược danh sách

Đây là bài tập kinh điển và cũng là câu hỏi phỏng vấn phổ biến. Thuật toán dùng ba con trỏ và đi đúng một lượt.

GIẢ MÃ

```
DAO(L)
1  truoc ← null;  p ← L.dau
2  while p ≠ null
3      sau ← p.ke        // giữ lại phần đuôi trước khi cắt
4      p.ke ← truoc      // đảo chiều liên kết
5      truoc ← p
6      p ← sau
7  L.dau ← truoc
```

PYTHON

```python
def dao(L):
    truoc, p = None, L.dau
    while p is not None:
        sau = p.ke          # giữ lại phần đuôi trước khi cắt
        p.ke = truoc        # đảo chiều liên kết
        truoc, p = p, sau
    L.dau = truoc
```

C++

```cpp
void dao(DSLK &L) {
    Nut *truoc = nullptr, *p = L.dau;
    while (p != nullptr) {
        Nut *sau = p->ke;   // giữ lại phần đuôi trước khi cắt
        p->ke = truoc;      // đảo chiều liên kết
        truoc = p;
        p = sau;
    }
    L.dau = truoc;
}
```

JULIA

```julia
function dao!(L::DSLK)
    truoc = nothing
    p = L.dau
    while p !== nothing
        sau = p.ke
```

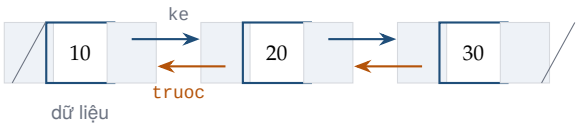
```
        p.ke = truoc
        truoc = p
        p = sau
    end
    L.dau = truoc
    return L
end
```

Biến `sau` ở dòng 3 là bắt buộc: dòng 4 ghi đè `p.ke`, nên nếu không lưu lại trước thì không còn cách nào đi tiếp.

**Ví dụ 5.1.** Danh sách $10 \to 20 \to 30$ sau khi đảo trở thành $30 \to 20 \to 10$. Chương trình kiểm thử chạy 3000 kịch bản ngẫu nhiên gồm các thao tác thêm đầu, thêm cuối, xoá, tìm và đảo, đối chiếu từng bước với danh sách chuẩn của ngôn ngữ; kết quả trùng khớp hoàn toàn ở cả ba bản cài đặt.

### 5.2.6 Danh sách liên kết kép

Danh sách liên kết kép cho mỗi nút hai liên kết: một trỏ tới nút sau, một trỏ tới nút trước. Nhờ đó có thể duyệt ngược, và có thể xoá một nút mà không cần biết nút đứng trước nó.




**Hình 5.4.** Danh sách liên kết kép: mỗi nút có hai liên kết.

PYTHON

```python
class NutKep:
    __slots__ = ("gt", "truoc", "ke")

    def __init__(self, gt):
        self.gt = gt
        self.truoc = None
        self.ke = None

class DSLKKep:
    def __init__(self):
        self.dau = None
        self.cuoi = None        # danh sách kép giữ thêm con trỏ nút cuối
        self.n = 0

def xoa_nut_kep(L, p):
    """Xoá nút p mà không cần duyệt tìm nút đứng trước."""
    if p.truoc is not None:
        p.truoc.ke = p.ke
    else:
        L.dau = p.ke                # p là nút đầu
    if p.ke is not None:
        p.ke.truoc = p.truoc
    else:
        L.cuoi = p.truoc            # p là nút cuối
    L.n -= 1
```

C++

```cpp
struct NutKep {
    int gt;
    NutKep *truoc = nullptr, *ke = nullptr;
    NutKep(int g) : gt(g) {}
};

struct DSLKKep {
    NutKep *dau = nullptr, *cuoi = nullptr;
    int n = 0;
};

void xoa_nut_kep(DSLKKep &L, NutKep *p) {
    if (p->truoc) p->truoc->ke = p->ke;
    else          L.dau = p->ke;        // p là nút đầu
    if (p->ke)    p->ke->truoc = p->truoc;
    else          L.cuoi = p->truoc;    // p là nút cuối
    delete p;
    L.n--;
}
```

JULIA

```julia
mutable struct NutKep
    gt::Int
    truoc::Union{NutKep, Nothing}
    ke::Union{NutKep, Nothing}
end
NutKep(gt) = NutKep(gt, nothing, nothing)

mutable struct DSLKKep
    dau::Union{NutKep, Nothing}
    cuoi::Union{NutKep, Nothing}
    n::Int
end
DSLKKep() = DSLKKep(nothing, nothing, 0)

function xoa_nut_kep!(L::DSLKKep, p::NutKep)
    if p.truoc !== nothing
        p.truoc.ke = p.ke
    else
        L.dau = p.ke
    end
    if p.ke !== nothing
        p.ke.truoc = p.truoc
    else
        L.cuoi = p.truoc
    end
    L.n -= 1
end
```

**Bài 5.1.** Cài đặt lớp danh sách liên kết đơn đầy đủ bằng ba ngôn ngữ, gồm các thao tác thêm đầu, thêm cuối, chèn sau, xoá theo giá trị, tìm kiếm, đảo ngược và in. Viết chương trình kiểm thử tự động: sinh ngẫu nhiên một dãy thao tác, thực hiện song song trên danh sách liên kết của mình và trên danh sách chuẩn của ngôn ngữ, so sánh trạng thái sau mỗi thao tác. Chạy 3000 kịch bản. Riêng bản C++, kiểm tra rò bộ nhớ bằng `valgrind` hoặc bằng cách

**Bảng 5.3.** So sánh danh sách liên kết đơn và kép.

| | Đơn | Kép |
|---|---|---|
| Bộ nhớ cho liên kết mỗi nút | 1 con trỏ | 2 con trỏ |
| Duyệt xuôi | $O(n)$ | $O(n)$ |
| Duyệt ngược | không được | $O(n)$ |
| Xoá nút khi đã biết địa chỉ | $O(n)$ | $O(1)$ |
| Số phép gán khi chèn | 2 | 4 |

đếm số lần `new` và `delete`.

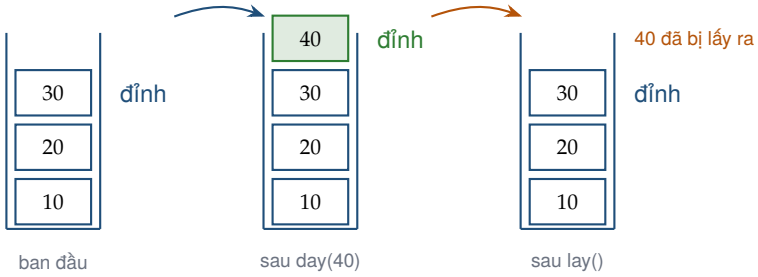
## 5.3 Ngăn xếp

### 5.3.1 Định nghĩa

**Định nghĩa 5.2.** (*Ngăn xếp*) **Ngăn xếp** (*stack*) là danh sách trong đó việc thêm và lấy phần tử chỉ được thực hiện ở *một đầu*, gọi là đỉnh. Phần tử vào sau cùng là phần tử ra trước tiên, nên ngăn xếp còn gọi là cấu trúc LIFO (*Last In First Out*).

Bốn thao tác cơ bản:

- `day(x)` — đưa $x$ vào đỉnh;
- `lay()` — lấy và trả về phần tử ở đỉnh;
- `dinh()` — xem phần tử ở đỉnh mà không lấy ra;
- `rong()` — kiểm tra ngăn xếp có rỗng không.




**Hình 5.5.** Ba trạng thái của ngăn xếp. Mọi thao tác chỉ xảy ra ở đỉnh.

### 5.3.2 Biểu diễn bằng mảng

Cách đơn giản và nhanh nhất: dùng một mảng và một biến `dinh` ghi chỉ số phần tử trên đỉnh.

JULIA

```julia
mutable struct NganXep
    a::Vector{Int}
end
NganXep() = NganXep(Int[])

day!(s::NganXep, x)  = (push!(s.a, x); s)
```

```
lay!(s::NganXep)     = isempty(s.a) ? nothing : pop!(s.a)
dinh(s::NganXep)     = isempty(s.a) ? nothing : s.a[end]
rong(s::NganXep)     = isempty(s.a)
```

PYTHON

```python
class NganXep:
    def __init__(self):
        self.a = []

    def day(self, x):
        self.a.append(x)

    def lay(self):
        return self.a.pop() if self.a else None

    def dinh(self):
        return self.a[-1] if self.a else None

    def rong(self):
        return len(self.a) == 0
```

C++

```cpp
struct NganXep {
    vector<int> a;
    void day(int x)  { a.push_back(x); }
    bool rong()      { return a.empty(); }
    int  dinh()      { return a.back(); }        // gọi khi biết chắc không rỗng
    int  lay()       { int x = a.back(); a.pop_back(); return x; }
};

// Thư viện chuẩn đã có sẵn: stack<int> s;  s.push(x);  s.top();  s.pop();
```

> **Chú ý.** Trong C++, `std::stack::pop()` *không* trả về giá trị; nó chỉ xoá phần tử trên đỉnh. Muốn lấy giá trị phải gọi `top()` trước rồi mới `pop()`. Ngoài ra gọi `top()` trên ngăn xếp rỗng là hành vi không xác định, không phải lỗi được báo. Luôn kiểm tra `empty()` trước.

Cả bốn thao tác đều là $O(1)$.

### 5.3.3 Ứng dụng 1: kiểm tra dấu ngoặc

**Bài toán.** Cho biểu thức chứa các loại ngoặc `()`, `[]`, `{}`. Kiểm tra các ngoặc có khớp nhau không.

**Ý tưởng.** Mỗi khi gặp ngoặc mở, đẩy vào ngăn xếp. Mỗi khi gặp ngoặc đóng, lấy phần tử ở đỉnh ra và kiểm tra có cùng loại không. Cuối cùng ngăn xếp phải rỗng.

Ngăn xếp phù hợp ở đây vì ngoặc đóng luôn khớp với ngoặc mở *gần nhất chưa được đóng*, đúng với quy tắc vào sau ra trước.

PYTHON

```python
def can_ngoac(s):
    cap = {")": "(", "]": "[", "}": "{"}
    st = []
    for c in s:
```

```
        if c in "([{":
            st.append(c)
        elif c in ")]}":
            if not st or st[-1] != cap[c]:
                return False
            st.pop()
    return len(st) == 0          # còn ngoặc mở chưa đóng thì sai
```

JULIA

```
function can_ngoac(s::AbstractString)
    cap = Dict(')' => '(', ']' => '[', '}' => '{')
    st = Char[]
    for c in s
        if c in "([{"
            push!(st, c)
        elseif c in ")]}"
            (isempty(st) || st[end] != cap[c]) && return false
            pop!(st)
        end
    end
    return isempty(st)
end
```

C++

```
bool can_ngoac(const string &s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else if (c == ')' || c == ']' || c == '}') {
            if (st.empty()) return false;
            char t = st.top(); st.pop();
            if ((c == ')' && t != '(') || (c == ']' && t != '[') ||
                (c == '}' && t != '{')) return false;
        }
    }
    return st.empty();
}
```

**Ví dụ 5.2.** *Kết quả kiểm thử.*

| Biểu thức | Kết quả |
|---|---|
| `(a+[b*c])-{d/e}` | đúng |
| `(a+[b)*c]` | sai (ngoặc lồng nhau chéo) |
| `(((())))` | đúng |
| `(()` | sai (còn một ngoặc mở) |
| xâu rỗng | đúng |
| `a+b` | đúng |

### 5.3.4 Ứng dụng 2: biểu thức hậu tố

Biểu thức viết theo cách quen thuộc gọi là **trung tố**, chẳng hạn $3 + 4 \times 2$. Cách viết này cần quy tắc ưu tiên phép toán và cần dấu ngoặc. Cách viết **hậu tố** đặt phép toán sau hai toán hạng: $3\ 4\ 2\ \times\ +$. Nó không cần dấu ngoặc, không cần quy tắc ưu tiên, và tính được bằng một ngăn xếp duy nhất. Vì vậy nhiều máy ảo và máy tính bỏ túi dùng dạng này.

#### 5.3.4.1 Chuyển trung tố sang hậu tố

Thuật toán mang tên Dijkstra, thường gọi là thuật toán sân ga.

GIẢ MÃ

```
TRUNG-TO-SANG-HAU-TO(bt)
1  ra ← rỗng;  st ← ngăn xếp rỗng
2  for mỗi ký hiệu c trong bt
3      if c là toán hạng:      thêm c vào ra
4      else if c = "(":        đẩy c vào st
5      else if c = ")":
6          while đỉnh st ≠ "(":  lấy đỉnh st, thêm vào ra
7          bỏ dấu "(" khỏi st
8      else                                    // c là phép toán
9          while st khác rỗng và đỉnh ≠ "(" và
               (ưu_tiên(đỉnh) > ưu_tiên(c) hoặc
                (ưu_tiên(đỉnh) = ưu_tiên(c) và c kết hợp từ trái sang phải))
10             lấy đỉnh st, thêm vào ra
11         đẩy c vào st
12 lấy hết st, thêm vào ra
13 return ra
```

PYTHON

```python
UU_TIEN = {"+": 1, "-": 1, "*": 2, "/": 2, "^": 3}
PHAI_SANG_TRAI = {"^"}              # luỹ thừa kết hợp từ phải sang trái

def trung_to_sang_hau_to(bt):
    ra, st = [], []
    for c in bt.split():
        if c.isalnum():
            ra.append(c)
        elif c == "(":
            st.append(c)
        elif c == ")":
            while st and st[-1] != "(":
                ra.append(st.pop())
            st.pop()                        # bỏ dấu "("
        else:
            while st and st[-1] != "(" and (
                    UU_TIEN[st[-1]] > UU_TIEN[c] or
                    (UU_TIEN[st[-1]] == UU_TIEN[c]
                     and c not in PHAI_SANG_TRAI)):
                ra.append(st.pop())
            st.append(c)
    while st:
        ra.append(st.pop())
    return " ".join(ra)
```

### 5.3.4.2 Tính giá trị biểu thức hậu tố

**GIẢ MÃ**

```
TINH-HAU-TO(bt)
1  st ← ngăn xếp rỗng
2  for mỗi ký hiệu c trong bt
3      if c là số:  đẩy c vào st
4      else
5          b ← lấy đỉnh st        // toán hạng PHẢI ra trước
6          a ← lấy đỉnh st        // toán hạng TRÁI ra sau
7          đẩy (a c b) vào st
8  return đỉnh st
```

**PYTHON**

```python
def tinh_hau_to(bt):
    st = []
    for c in bt.split():
        if c.isdigit():
            st.append(int(c))
        else:
            b = st.pop()                    # toán hạng PHẢI ra trước
            a = st.pop()                    # toán hạng TRÁI ra sau
            if   c == "+": st.append(a + b)
            elif c == "-": st.append(a - b)
            elif c == "*": st.append(a * b)
            elif c == "^": st.append(a ** b)
            else:          st.append(int(a / b))   # cắt về 0, giống C++
    return st[-1]
```

**C++**

```cpp
long long luy_thua_nguyen(long long a, long long n) {
    long long kq = 1;
    while (n-- > 0) kq *= a;
    return kq;
}

long long tinh_hau_to(const string &bt) {
    stringstream ss(bt);
    string tok;
    stack<long long> st;
    while (ss >> tok) {
        if (isdigit(tok[0])) st.push(stoll(tok));
        else {
            long long b = st.top(); st.pop();    // toán hạng PHẢI ra trước
            long long a = st.top(); st.pop();
            st.push(tok == "+" ? a + b : tok == "-" ? a - b
                  : tok == "*" ? a * b
                  : tok == "^" ? luy_thua_nguyen(a, b)
                  : a / b);                      // a / b cắt về 0
        }
    }
    return st.top();
}
```

JULIA

```julia
function tinh_hau_to(bt::AbstractString)
    st = Int[]
    for c in split(bt)
        if all(isdigit, c)
            push!(st, parse(Int, c))
        else
            b = pop!(st)                    # toán hạng PHẢI ra trước
            a = pop!(st)
            push!(st, c == "+" ? a + b : c == "-" ? a - b :
                      c == "*" ? a * b : c == "^" ? a^b :
                      Int(trunc(a / b)))           # cắt về 0, giống C++
        end
    end
    return st[end]
end
```

**Phép chia nguyên trong ba ngôn ngữ.** Toán tử `//` của Python làm tròn *xuống*, còn `/` của C++ và `÷` của Julia cắt phần thập phân về phía 0. Hai cách chỉ khác nhau khi số bị chia âm: $-7//2 = -4$ trong Python nhưng $-7/2 = -3$ trong C++. Vì vậy bản Python ở trên dùng `int(a / b)` để ba bản cài đặt cho cùng kết quả trên mọi dữ liệu vào.

**Thứ tự lấy hai toán hạng.** Toán hạng lấy ra *trước* là toán hạng *bên phải*, vì nó vào ngăn xếp sau. Viết ngược lại thì phép cộng và phép nhân vẫn đúng do có tính giao hoán, nhưng phép trừ và phép chia sai. Đây là lỗi rất khó phát hiện nếu chỉ kiểm thử bằng biểu thức chỉ có cộng và nhân.

**Ví dụ 5.3.**

| Trung tố | Hậu tố | Giá trị |
|---|---|---|
| `3 + 4 * 2 - ( 1 + 1 )` | `3 4 2 * + 1 1 + -` | 9 |
| `( 2 + 3 ) * ( 4 - 1 )` | `2 3 + 4 1 - *` | 15 |
| `5 * ( 6 + 2 ) - 12 / 4` | `5 6 2 + * 12 4 / -` | 37 |
| `2 ^ 3 ^ 2` | `2 3 2 ^ ^` | 512 |

Cả bốn kết quả trùng với giá trị tính trực tiếp bằng biểu thức của ngôn ngữ. Dòng cuối cho thấy phép luỹ thừa được xử lý đúng theo luật kết hợp từ phải sang trái: $2^{(3^2)} = 2^9 = 512$ chứ không phải $(2^3)^2 = 64$.

### 5.3.5 Ứng dụng 3: khử đệ quy

Mọi thuật toán đệ quy đều viết lại được bằng vòng lặp cộng một ngăn xếp, vì bản thân cơ chế đệ quy của ngôn ngữ cũng dùng ngăn xếp lời gọi. Kỹ thuật này cần đến khi độ sâu đệ quy vượt giới hạn của ngôn ngữ, như đã nêu ở mục 3.3.

PYTHON

```python
def duyet_sau_de_quy(cay, goc):
    if goc is not None:
        print(goc, end=" ")
```

```
        for con in cay[goc]:
            duyet_sau_de_quy(cay, con)

def duyet_khu_de_quy(cay, goc):
    st = [goc]
    while st:
        u = st.pop()
        print(u, end=" ")
        for con in reversed(cay[u]):    # đảo để thứ tự duyệt không đổi
            st.append(con)
```

Chú thích ở dòng cuối là chi tiết dễ bỏ sót: ngăn xếp lấy ra theo thứ tự ngược với thứ tự đưa vào, nên muốn giữ nguyên thứ tự duyệt của bản đệ quy, phải đẩy các con vào theo thứ tự ngược lại.

## 5.4 Hàng đợi

### 5.4.1 Định nghĩa

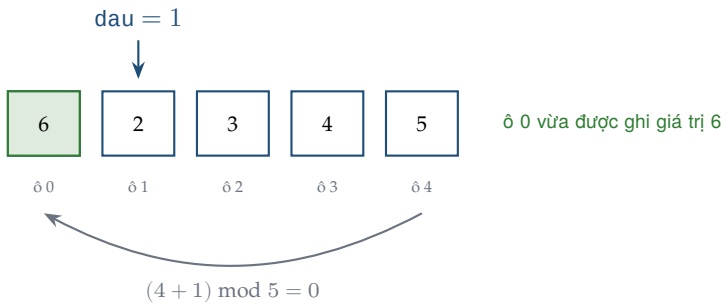
**Định nghĩa 5.3.** (*Hàng đợi*) **Hàng đợi** (*queue*) là danh sách trong đó phần tử được thêm ở một đầu và lấy ra ở đầu kia. Phần tử vào trước là phần tử ra trước, nên hàng đợi còn gọi là cấu trúc FIFO (*First In First Out*).

Hàng đợi mô hình hoá đúng hình ảnh xếp hàng: người đến trước được phục vụ trước. Nó xuất hiện ở mọi nơi cần xử lý theo thứ tự đến: hàng đợi in ấn, hàng đợi yêu cầu tới máy chủ, và quan trọng nhất với học phần này là thuật toán tìm kiếm theo chiều rộng ở chương 6.

### 5.4.2 Cách cài đặt sai và cách cài đặt đúng

Cách cài đặt trực tiếp bằng mảng có một vấn đề. Nếu lấy phần tử ra bằng cách xoá phần tử đầu mảng thì mọi phần tử còn lại phải dịch lên một ô, tốn $O(n)$. Nếu chỉ tăng chỉ số dau lên mà không dịch, thì phần đầu mảng bị bỏ phí và mảng sẽ đầy dù thực tế còn rất ít phần tử.

Cách khắc phục là **hàng đợi vòng**: coi mảng như một vòng tròn, ô cuối nối liền ô đầu. Phép toán lấy phần dư đảm nhiệm việc "quay vòng".




**Hình 5.6.** Hàng đợi vòng sức chứa 5 sau khi thêm $1..5$, lấy ra một phần tử, rồi thêm giá trị 6. Giá trị mới quay về ô 0.

PYTHON

```
class HangDoiVong:
    def __init__(self, n):
        self.a = [None] * n
        self.n = n
        self.dau = 0                # vị trí phần tử đầu hàng
```

```
        self.so = 0                 # số phần tử hiện có

    def them(self, x):
        if self.so == self.n:
            return False            # hàng đầy
        self.a[(self.dau + self.so) % self.n] = x
        self.so += 1
        return True

    def lay(self):
        if self.so == 0:
            return None             # hàng rỗng
        x = self.a[self.dau]
        self.dau = (self.dau + 1) % self.n
        self.so -= 1
        return x
```

**JULIA**

```
mutable struct HangDoiVong
    a::Vector{Union{Int, Nothing}}
    n::Int
    dau::Int        # đánh chỉ số từ 0 cho tiện phép chia dư
    so::Int
end
HangDoiVong(n) = HangDoiVong(fill(nothing, n), n, 0, 0)

function them!(q::HangDoiVong, x)
    q.so == q.n && return false
    q.a[(q.dau + q.so) % q.n + 1] = x       # +1 vì Julia đánh số từ 1
    q.so += 1
    return true
end

function lay!(q::HangDoiVong)
    q.so == 0 && return nothing
    x = q.a[q.dau + 1]
    q.dau = (q.dau + 1) % q.n
    q.so -= 1
    return x
end
```

**C++**

```
struct HangDoiVong {
    vector<int> a;
    int n, dau = 0, so = 0;
    HangDoiVong(int n) : a(n), n(n) {}

    bool them(int x) {
        if (so == n) return false;              // hàng đầy
        a[(dau + so) % n] = x;
        so++;
        return true;
    }
    bool lay(int &x) {
        if (so == 0) return false;              // hàng rỗng
        x = a[dau];
```

```
        dau = (dau + 1) % n;
        so--;
        return true;
    }
};

// Thư viện chuẩn: queue<int> q;  q.push(x);  q.front();  q.pop();
```

**Vì sao lưu `so` chứ không lưu `cuoi`.** Nhiều tài liệu cài hàng đợi vòng bằng hai chỉ số `dau` và `cuoi`. Cách đó gặp một vấn đề: khi hàng rỗng và khi hàng đầy, cả hai đều cho `dau == cuoi`, nên không phân biệt được. Người ta phải hy sinh một ô trống hoặc thêm một biến cờ. Cách lưu số phần tử hiện có tránh hẳn vấn đề đó và dễ đọc hơn.

**Ví dụ 5.4.** *Lần vết*. Hàng đợi vòng sức chứa 5:

| Thao tác | Nội dung mảng | Trạng thái |
|---|---|---|
| thêm 1..5 | `[1, 2, 3, 4, 5]` | đầy, `dau` $= 0$ |
| thêm 6 | `[1, 2, 3, 4, 5]` | bị từ chối, hàng đầy |
| lấy | `[1, 2, 3, 4, 5]` | trả về 1, `dau` $= 1$ |
| thêm 6 | `[6, 2, 3, 4, 5]` | ghi vào ô 0, quay vòng |

Sau bước cuối, hàng đợi chứa các giá trị $2, 3, 4, 5, 6$ theo đúng thứ tự vào. Giá trị 1 vẫn còn trong ô 0 cho tới khi bị ghi đè, nhưng không còn thuộc hàng đợi.

Cả hai thao tác đều là $O(1)$, và bộ nhớ cố định $O(n)$ không phụ thuộc số lần thêm.

**Bài 5.2.** Cài đặt ngăn xếp và hàng đợi vòng bằng ba ngôn ngữ, mỗi cấu trúc theo hai cách: bằng mảng và bằng danh sách liên kết. Viết chương trình kiểm thử so sánh với `stack` và queue của thư viện chuẩn trên 3000 dãy thao tác ngẫu nhiên. Sau đó cài chương trình chuyển trung tố sang hậu tố và tính giá trị, kiểm tra với hai biểu thức trong ví dụ và với mười biểu thức tự đặt có chứa phép trừ và phép chia.

## 5.5 Ứng dụng: bộ nhớ phát lại trong học tăng cường

### 5.5.1 Bài toán

Trong học tăng cường, một tác tử tương tác liên tục với môi trường. Mỗi bước sinh ra một **kinh nghiệm**: trạng thái hiện tại, hành động đã chọn, phần thưởng nhận được, trạng thái mới. Mạng nơ-ron của tác tử được huấn luyện từ các kinh nghiệm đó.

Nếu huấn luyện ngay trên kinh nghiệm vừa sinh ra, việc học gặp hai vấn đề.

1. **Các mẫu liên tiếp tương quan mạnh với nhau.** Trạng thái ở bước $t$ và bước $t + 1$ gần như giống hệt nhau, nên gradient tính từ chúng chỉ đi theo một hướng và việc học rất kém ổn định.
2. **Mỗi kinh nghiệm chỉ được dùng một lần.** Trong khi việc thu thập kinh nghiệm là phần tốn kém nhất của quá trình huấn luyện.

Giải pháp là **bộ nhớ phát lại** (*experience replay buffer*): lưu các kinh nghiệm gần đây vào một bộ đệm, và mỗi bước huấn luyện thì lấy ngẫu nhiên một lô mẫu từ bộ đệm đó. Việc lấy ngẫu nhiên phá vỡ tương quan; việc lưu lại cho phép mỗi kinh nghiệm được dùng nhiều lần.

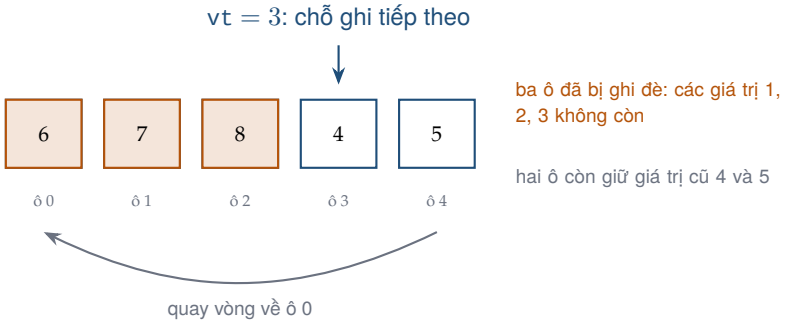
### 5.5.2 Yêu cầu đối với cấu trúc dữ liệu

Bộ đệm phải đáp ứng ba yêu cầu.

1. Thêm một kinh nghiệm mới trong $O(1)$, vì thao tác này chạy ở mỗi bước của môi trường.
2. Khi đầy, tự động loại kinh nghiệm cũ nhất, cũng trong $O(1)$. Đây đúng là hành vi FIFO của hàng đợi.
3. Lấy ngẫu nhiên phần tử thứ $i$ trong $O(1)$, để lấy một lô mẫu nhanh.

Yêu cầu thứ ba loại bỏ danh sách liên kết, vì truy cập theo chỉ số trên danh sách liên kết tốn $O(n)$; bảng số liệu ở mục 5.1 cho thấy khác biệt lên tới ba mươi nghìn lần. Yêu cầu thứ hai loại bỏ mảng thường, vì xoá phần tử đầu mảng tốn $O(n)$.

Cấu trúc thoả cả ba yêu cầu là *mảng vòng*, tức chính hàng đợi vòng ở mục 5.4, với một thay đổi nhỏ: khi đầy thì ghi đè lên phần tử cũ nhất thay vì từ chối.




**Hình 5.7.** Bộ nhớ phát lại sức chứa 5 sau khi thêm lần lượt các giá trị $1..8$. Ba giá trị cũ nhất đã bị ghi đè.

### 5.5.3 Cài đặt

GIẢ MÃ

```
THEM(x)
1  a[vt] ← x
2  vt ← (vt + 1) mod n              // quay vòng
3  so ← min(so + 1, n)              // số phần tử thực có

LAY-MAU(k)
1  chọn ngẫu nhiên k chỉ số phân biệt trong 0..so-1
2  return các phần tử tương ứng
```

PYTHON

```python
import random

class BoNhoPhatLai:
    def __init__(self, suc_chua):
        self.a = [None] * suc_chua
        self.n = suc_chua
```

```
        self.vt = 0                 # vị trí sẽ ghi tiếp theo
        self.so = 0                 # số phần tử thực có

    def them(self, kinh_nghiem):
        self.a[self.vt] = kinh_nghiem
        self.vt = (self.vt + 1) % self.n        # quay vòng, ghi đè cái cũ nhất
        self.so = min(self.so + 1, self.n)

    def lay_mau(self, k):
        return random.sample(self.a[:self.so], k)

    def du_de_hoc(self, k):
        return self.so >= k
```

JULIA

```
mutable struct BoNhoPhatLai
    a::Vector{Any}
    n::Int
    vt::Int         # đánh chỉ số từ 0 cho tiện phép chia dư
    so::Int
end
BoNhoPhatLai(suc_chua) = BoNhoPhatLai(Vector{Any}(undef, suc_chua),
                                      suc_chua, 0, 0)

using Random            # randperm nằm trong thư viện chuẩn Random

function them!(b::BoNhoPhatLai, kinh_nghiem)
    b.a[b.vt + 1] = kinh_nghiem
    b.vt = (b.vt + 1) % b.n
    b.so = min(b.so + 1, b.n)
    return b
end

lay_mau(b::BoNhoPhatLai, k) = b.a[randperm(b.so)[1:k]]
du_de_hoc(b::BoNhoPhatLai, k) = b.so >= k
```

C++

```
struct KinhNghiem {
    vector<double> trang_thai, trang_thai_moi;
    int hanh_dong;
    double phan_thuong;
    bool ket_thuc;
};

struct BoNhoPhatLai {
    vector<KinhNghiem> a;
    int n, vt = 0, so = 0;
    mt19937 rng{12345};

    BoNhoPhatLai(int suc_chua) : a(suc_chua), n(suc_chua) {}

    void them(const KinhNghiem &kn) {
        a[vt] = kn;
        vt = (vt + 1) % n;              // quay vòng, ghi đè cái cũ nhất
        so = min(so + 1, n);
    }
```

```
20
21      vector<KinhNghiem> lay_mau(int k) {
22          vector<int> chi_so(so);
23          iota(chi_so.begin(), chi_so.end(), 0);
24          shuffle(chi_so.begin(), chi_so.end(), rng);
25          vector<KinhNghiem> lo;
26          for (int i = 0; i < k; i++) lo.push_back(a[chi_so[i]]);
27          return lo;
28      }
29
30      bool du_de_hoc(int k) const { return so >= k; }
31  };
```

**Ví dụ 5.5.** *Kết quả đo.* Bộ nhớ sức chứa 5, thêm lần lượt các giá trị $1, 2, \ldots, 8$. Trạng thái cuối cùng của mảng là `[6, 7, 8, 4, 5]`, với `vt` $= 3$ và `so` $= 5$. Ba ô đầu đã bị ghi đè bởi các giá trị 6, 7, 8; hai ô cuối vẫn giữ 4 và 5. Nội dung logic của bộ nhớ là năm kinh nghiệm gần nhất $\{4, 5, 6, 7, 8\}$, đúng như yêu cầu.

**Thứ tự trong mảng không phải thứ tự thời gian.** Trong ví dụ trên, mảng vật lý là `[6, 7, 8, 4, 5]` nhưng thứ tự thời gian là $4, 5, 6, 7, 8$. Với bộ nhớ phát lại, điều này không quan trọng, vì mẫu được lấy *ngẫu nhiên* nên thứ tự vô nghĩa. Nhưng nếu dùng cùng cấu trúc để lấy "$k$ kinh nghiệm gần nhất theo thứ tự", phải khôi phục thứ tự bằng phép chia dư chứ không được đọc thẳng mảng.

**Bảng 5.4.** Vì sao mảng vòng là lựa chọn duy nhất thoả cả ba yêu cầu.

| Cấu trúc | Thêm mới | Loại cái cũ nhất | Truy cập ngẫu nhiên |
|---|---|---|---|
| Mảng động, xoá đầu | $O(1)$ | $O(n)$ | $O(1)$ |
| Danh sách liên kết đơn | $O(1)$ | $O(1)$ | $O(n)$ |
| Danh sách liên kết kép | $O(1)$ | $O(1)$ | $O(n)$ |
| **Mảng vòng** | $O(1)$ | $O(1)$ | $O(1)$ |

**Nhận xét 5.2.** Kích thước bộ nhớ phát lại là một tham số quan trọng của thuật toán học. Bộ nhớ quá nhỏ thì các mẫu vẫn còn tương quan; bộ nhớ quá lớn thì tác tử học từ những kinh nghiệm đã lỗi thời, sinh ra trong giai đoạn nó còn hành động rất kém. Các bản cài đặt phổ biến dùng sức chứa từ $10^5$ đến $10^6$ kinh nghiệm. Với $10^6$ kinh nghiệm, việc chọn cấu trúc dữ liệu không còn là chi tiết kỹ thuật: một thao tác $O(n)$ chạy ở mỗi bước sẽ khiến toàn bộ quá trình huấn luyện không thể hoàn thành.

**Bài 5.3.** Cài đặt bộ nhớ phát lại theo các bước sau.

1. Định nghĩa kiểu dữ liệu kinh nghiệm gồm bốn trường: trạng thái, hành động, phần thưởng, trạng thái mới.
2. Cài lớp `BoNhoPhatLai` với ba phương thức `them`, `lay_mau`, `du_de_hoc`.
3. Kiểm thử: tạo bộ nhớ sức chứa 5, thêm các số $1..8$, in nội dung mảng. Kết quả phải là `[6, 7, 8, 4, 5]`.
4. Viết một môi trường giả lập đơn giản, chẳng hạn tác tử đi trên đoạn thẳng từ 0 đến 10

và nhận phần thưởng khi tới đích. Chạy 10 000 bước, mỗi bước thêm một kinh nghiệm và lấy một lô 32 mẫu.

5. Đo thời gian chạy khi bộ nhớ được cài bằng mảng vòng và khi được cài bằng danh sách Python thường có `pop(0)`. Với sức chứa $10^5$, chênh lệch phải rất rõ.

**Tóm tắt.** Mảng và danh sách liên kết là hai cách cài đặt danh sách với ưu nhược điểm trái ngược nhau: mảng truy cập theo chỉ số trong $O(1)$ nhưng chèn đầu tốn $O(n)$; danh sách liên kết thì ngược lại. Số đo thực tế cho chênh lệch tới 255 lần theo chiều này và 30 000 lần theo chiều kia. Ngăn xếp là danh sách chỉ thao tác ở một đầu, phù hợp với mọi bài toán có cấu trúc lồng nhau như kiểm tra ngoặc, tính biểu thức hậu tố và khử đệ quy. Hàng đợi thao tác ở hai đầu và phải cài bằng mảng vòng để đạt $O(1)$. Bộ nhớ phát lại trong học tăng cường là một hàng đợi vòng có ghi đè, và ba yêu cầu của nó chỉ có mảng vòng đáp ứng được cả ba.

## 5.6 Bài tập chương 5

### Câu hỏi ôn tập

1. So sánh mảng và danh sách liên kết trên bốn thao tác: truy cập theo chỉ số, tìm theo giá trị, chèn đầu, xoá nút đã biết địa chỉ.
2. Vì sao chênh lệch thời gian truy cập ngẫu nhiên giữa mảng và danh sách liên kết lớn hơn nhiều so với dự đoán từ ký hiệu $O$?
3. Trong phép chèn sau một nút, vì sao thứ tự hai phép gán là bắt buộc? Điều gì xảy ra nếu đảo thứ tự?
4. Vì sao xoá một nút khỏi danh sách liên kết đơn cần biết nút đứng trước? Danh sách liên kết kép khắc phục điều này thế nào?
5. Trong thuật toán đảo danh sách, vai trò của biến `sau` là gì?
6. Trong C++, hai lỗi nào liên quan tới `delete` khi xoá nút?
7. Phát biểu định nghĩa ngăn xếp và giải thích tên gọi LIFO.
8. Vì sao bài toán kiểm tra dấu ngoặc phù hợp với ngăn xếp?
9. Trong thuật toán tính biểu thức hậu tố, vì sao toán hạng lấy ra trước lại là toán hạng bên phải? Với phép toán nào thì viết sai vẫn cho kết quả đúng?
10. Vì sao hàng đợi cài bằng mảng thường lại kém hiệu quả? Hàng đợi vòng khắc phục thế nào?
11. Vì sao nên lưu số phần tử hiện có thay vì lưu chỉ số cuối trong hàng đợi vòng?
12. Nêu ba yêu cầu của bộ nhớ phát lại và giải thích vì sao chỉ mảng vòng thoả cả ba.

### Bài tập tính toán

13. Vẽ sơ đồ danh sách liên kết đơn sau dãy thao tác: thêm cuối 5, thêm cuối 9, thêm đầu 2, xoá 9, thêm cuối 7, đảo.
14. Cho danh sách $1 \to 2 \to 3 \to 4 \to 5$. Vẽ trạng thái sau mỗi vòng lặp của thuật toán đảo.

15. Lần vết ngăn xếp khi kiểm tra biểu thức `{[(a+b)*c]-[d/(e-f)]}`.

16. Chuyển các biểu thức sau sang hậu tố và tính giá trị: `5 * ( 6 + 2 ) - 12 / 4` và `2 ^ 3 ^ 2` (phép luỹ thừa kết hợp từ phải sang trái).

17. Lần vết hàng đợi vòng sức chứa 4 với dãy thao tác: thêm 1, thêm 2, lấy, thêm 3, thêm 4, thêm 5, lấy, thêm 6. Ghi nội dung mảng, `dau` và `so` sau mỗi bước.

18. Bộ nhớ phát lại sức chứa 4, thêm các giá trị $1..11$. Xác định nội dung mảng, giá trị `vt` và tập kinh nghiệm còn giữ.

### Bài tập lập trình mức cơ bản

19. Cài đặt danh sách liên kết đơn đầy đủ bằng ba ngôn ngữ theo bài thực hành 5.1.

20. Viết hàm đếm số nút của danh sách liên kết bằng hai cách: dùng biến đếm cập nhật khi thêm hoặc xoá, và duyệt lại từ đầu. So sánh độ phức tạp.

21. Viết hàm tìm nút ở giữa danh sách liên kết chỉ bằng một lượt duyệt, dùng kỹ thuật hai con trỏ chạy với tốc độ khác nhau.

22. Viết hàm kiểm tra một danh sách liên kết có chứa chu trình không, dùng thuật toán hai con trỏ nhanh chậm của Floyd.

23. Viết hàm trộn hai danh sách liên kết đã sắp xếp thành một danh sách đã sắp xếp, không cấp phát nút mới.

24. Cài đặt ngăn xếp bằng danh sách liên kết và bằng mảng, so sánh thời gian với 10 triệu thao tác.

25. Cài đặt hàng đợi bằng hai ngăn xếp, mỗi thao tác đạt $O(1)$ khi tính trung bình.

26. Viết chương trình kiểm tra một xâu có phải xâu đối xứng không, dùng ngăn xếp.

### Bài tập lập trình mức vận dụng

27. Cài đặt danh sách liên kết vòng và viết chương trình giải bài toán Josephus: $n$ người đứng thành vòng tròn, đếm đến người thứ $k$ thì loại, tiếp tục cho tới khi còn một người.

28. Cài đặt một trình soạn thảo văn bản đơn giản với chức năng hoàn tác và làm lại, dùng hai ngăn xếp.

29. Cài đặt máy tính bỏ túi đọc biểu thức trung tố có ngoặc và bốn phép toán, chuyển sang hậu tố rồi tính. Xử lý cả trường hợp biểu thức sai cú pháp.

30. Cài đặt cấu trúc `MinStack`: ngăn xếp có thêm thao tác lấy giá trị nhỏ nhất trong $O(1)$. Gợi ý: dùng thêm một ngăn xếp phụ.

31. Cài đặt hàng đợi hai đầu (*deque*) bằng mảng vòng, cho phép thêm và lấy ở cả hai đầu trong $O(1)$.

32. Cho một dãy số và cửa sổ trượt độ dài $k$. In giá trị lớn nhất trong mỗi cửa sổ, tổng thời gian $O(n)$, dùng hàng đợi hai đầu.

33. Cài đặt bài toán tìm phần tử lớn hơn kế tiếp: với mỗi phần tử của mảng, tìm phần tử đầu tiên bên phải lớn hơn nó, tổng thời gian $O(n)$ bằng ngăn xếp.

### Bài tập phần trí tuệ nhân tạo

34. Cài đặt bộ nhớ phát lại theo bài thực hành 5.3 và xác nhận kết quả `[6, 7, 8, 4, 5]`.

35. Đo thời gian thêm $10^6$ kinh nghiệm vào bộ nhớ sức chứa $10^5$ với ba cách cài đặt: mảng vòng, danh sách Python với `pop(0)`, và `collections.deque` có tham số `maxlen`. Lập bảng kết quả.

36. Cài đặt *phát lại có ưu tiên*: mỗi kinh nghiệm có một trọng số, và xác suất được chọn tỉ lệ với trọng số đó. Cài bản đơn giản $O(n)$ mỗi lần lấy mẫu, rồi giải thích vì sao bản thực dụng cần cấu trúc cây tổng, một biến thể của đống ở chương 7.

37. Cài đặt hàng đợi dùng cho thuật toán tìm kiếm theo chiều rộng: đọc một lưới có tường, tìm đường ngắn nhất từ ô xuất phát tới ô đích. So sánh kết quả với chương 6 sau khi học xong chương đó.

38. Viết chương trình mô phỏng một hệ thống hàng đợi yêu cầu: các yêu cầu đến theo thời gian ngẫu nhiên, mỗi yêu cầu cần một thời gian xử lý ngẫu nhiên. Đo thời gian chờ trung bình khi số máy chủ thay đổi từ 1 đến 5.

CHƯƠNG 6

# Đồ thị và Đồ thị có trọng số

Đồ thị là mô hình toán học của quan hệ giữa các đối tượng. Bản đồ giao thông, mạng máy tính, mạng xã hội, sơ đồ phụ thuộc giữa các học phần, các trạng thái của một trò chơi — tất cả đều là đồ thị. Vì vậy các thuật toán trên đồ thị được dùng rộng rãi hơn hầu hết mọi thuật toán khác trong giáo trình này.

Chương này trình bày cách biểu diễn đồ thị trong máy tính, hai thuật toán duyệt cơ bản, bài toán đường đi ngắn nhất và bài toán cây bao trùm nhỏ nhất. Mục cuối áp dụng vào thuật toán tìm đường A sao, thuật toán tìm đường được dùng phổ biến nhất trong trò chơi điện tử và trong dẫn đường rô-bốt.

## 6.1 Khái niệm và định nghĩa

**Định nghĩa 6.1.** (*Đồ thị*) Một **đồ thị** là cặp $G = (V, E)$, trong đó $V$ là tập hữu hạn khác rỗng các **đỉnh**, còn $E$ là tập các **cạnh**, mỗi cạnh nối hai đỉnh của $V$.

Ta ký hiệu $n = |V|$ là số đỉnh và $m = |E|$ là số cạnh. Hai đại lượng này xuất hiện trong mọi công thức độ phức tạp của chương.

**Bảng 6.1.** Các loại đồ thị.

| Loại | Đặc điểm | Ví dụ |
|---|---|---|
| Vô hướng | cạnh không có chiều, $(u, v)$ và $(v, u)$ là một | đường hai chiều, quan hệ bạn bè |
| Có hướng | cạnh có chiều, $(u, v) \neq (v, u)$ | đường một chiều, quan hệ theo dõi |
| Có trọng số | mỗi cạnh gắn một số | khoảng cách, chi phí, thời gian |
| Đơn | không có khuyên, không có cạnh bội | phần lớn bài toán |

Một số khái niệm dùng thường xuyên:

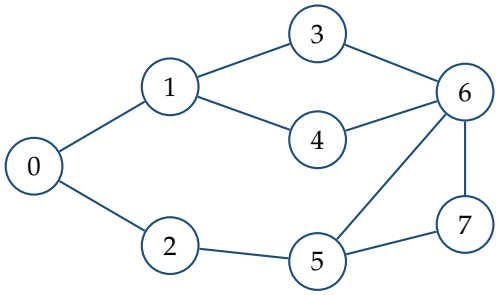
- **Bậc** của đỉnh $u$ là số cạnh nối với $u$. Với đồ thị có hướng, phân biệt bậc vào và bậc ra.
- **Đường đi** từ $u$ đến $v$ là dãy đỉnh $u = x_0, x_1, \dots, x_k = v$ trong đó mỗi cặp liên tiếp là một cạnh. **Độ dài** đường đi là số cạnh, hoặc tổng trọng số nếu đồ thị có trọng số.
- **Chu trình** là đường đi có đỉnh đầu trùng đỉnh cuối và không lặp cạnh.
- Đồ thị **liên thông** nếu giữa hai đỉnh bất kỳ luôn có đường đi.
- **Cây** là đồ thị liên thông không có chu trình. Cây $n$ đỉnh có đúng $n - 1$ cạnh.

**Định lý 6.1.** (*Bổ đề bắt tay*) *Trong đồ thị vô hướng, tổng bậc của mọi đỉnh bằng hai lần số cạnh, tức là*

$$\sum_{u \in V} \deg(u) = 2m.$$

*Chứng minh.* Mỗi cạnh $(u, v)$ đóng góp đúng 1 vào bậc của $u$ và đúng 1 vào bậc của $v$, tức đóng góp tổng cộng 2 vào tổng các bậc. Cộng qua tất cả $m$ cạnh được $2m$. ■

Đồ thị dùng trong toàn bộ chương này có 8 đỉnh, đánh số từ 0 đến 7, và 10 cạnh.




**Hình 6.1.** Đồ thị vô hướng dùng làm ví dụ trong chương: 8 đỉnh, 10 cạnh.

## 6.2 Biểu diễn đồ thị

### 6.2.1 Ma trận kề

Ma trận kề là mảng hai chiều $A$ kích thước $n \times n$, với $A[u][v] = 1$ nếu có cạnh nối $u$ và $v$, ngược lại bằng 0. Với đồ thị vô hướng, ma trận đối xứng qua đường chéo chính.

**Bảng 6.2.** Ma trận kề của đồ thị ở hình 6.1.

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| 2 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| 3 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 |
| 4 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 |
| 5 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
| 6 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 1 |
| 7 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 |

### 6.2.2 Danh sách kề

Danh sách kề lưu, với mỗi đỉnh, danh sách các đỉnh nối trực tiếp với nó.

JULIA

```julia
function doc_do_thi(n, canh)
    ke = [Int[] for _ in 1:n]          # KHÔNG viết fill([], n)
    for (u, v) in canh
        push!(ke[u], v)
        push!(ke[v], u)                # bỏ dòng này nếu đồ thị có hướng
    end
    return ke
end
```

| đỉnh | các đỉnh kề |
|---|---|
| 0 | 1, 2 |
| 1 | 0, 3, 4 |
| 2 | 0, 5 |
| 3 | 1, 6 |
| 4 | 1, 6 |
| 5 | 2, 6, 7 |
| 6 | 3, 4, 5, 7 |
| 7 | 5, 6 |

Tổng số phần tử trong tất cả các danh sách bằng $2m = 20$, đúng theo bổ đề bắt tay.

**Hình 6.2.** Danh sách kề của đồ thị ở hình 6.1.

PYTHON

```python
def doc_do_thi(n, canh):
    ke = [[] for _ in range(n)]          # KHÔNG viết [[]] * n
    for u, v in canh:
        ke[u].append(v)
        ke[v].append(u)                  # bỏ dòng này nếu đồ thị có hướng
    return ke
```

C++

```cpp
vector<vector<int>> doc_do_thi(int n, const vector<pair<int,int>> &canh) {
    vector<vector<int>> ke(n);
    for (auto &[u, v] : canh) {
        ke[u].push_back(v);
        ke[v].push_back(u);              // bỏ dòng này nếu đồ thị có hướng
    }
    return ke;
}
```

**Lỗi tạo danh sách rỗng.** Trong Python, `[[]] * n` tạo ra $n$ tham chiếu tới *cùng một* danh sách, nên thêm vào danh sách của đỉnh 0 sẽ làm mọi đỉnh khác cũng thay đổi. Phải viết `[[] for _ in range(n)]`. Julia có lỗi tương tự với `fill([], n)`. C++ không gặp vấn đề này vì `vector<vector<int>> ke(n)` tạo $n$ véc-tơ độc lập.

### 6.2.3 Chọn cách biểu diễn nào

**Bảng 6.3.** So sánh hai cách biểu diễn đồ thị.

| | Ma trận kề | Danh sách kề |
|---|---|---|
| Bộ nhớ | $O(n^2)$ | $O(n+m)$ |
| Kiểm tra có cạnh $(u, v)$ không | $O(1)$ | $O(\deg u)$ |
| Duyệt mọi đỉnh kề với $u$ | $O(n)$ | $O(\deg u)$ |
| Duyệt toàn bộ đồ thị | $O(n^2)$ | $O(n+m)$ |

Số cạnh của đồ thị đơn nhiều nhất là $n(n-1)/2$. Đồ thị được gọi là **dày** nếu $m$ gần cận trên đó, và **thưa** nếu $m$ nhỏ hơn nhiều, chẳng hạn $m = O(n)$.

Phần lớn đồ thị thực tế đều thưa. Bản đồ giao thông là ví dụ điển hình: mỗi ngã tư nối với nhiều nhất bốn hoặc năm ngã tư khác, bất kể thành phố lớn đến đâu. Với $n = 10^6$ ngã tư, ma trận kề cần $10^{12}$ ô, tức khoảng một nghìn tỉ giá trị — vượt xa mọi bộ nhớ máy tính thông thường, trong khi danh sách kề chỉ cần vài triệu ô.

**Nhận xét 6.1.** Vì lý do trên, trừ khi đề bài nói rõ đồ thị dày hoặc yêu cầu kiểm tra cạnh nhiều lần, danh sách kề là lựa chọn mặc định.

## 6.3 Các thuật toán tìm kiếm trên đồ thị

### 6.3.1 Tìm kiếm theo chiều sâu

**Ý tưởng.** Từ đỉnh hiện tại, đi sâu theo một cạnh chưa dùng cho tới khi không đi tiếp được nữa, rồi lùi lại và thử cạnh khác. Đây chính là kỹ thuật quay lui của chương 3 áp dụng cho đồ thị.

Mảng `tham` đánh dấu các đỉnh đã đi qua là bắt buộc. Không có nó, chương trình sẽ đi vòng quanh một chu trình mãi mãi.

GIẢ MÃ

```
DFS(u)
1  tham[u] ← true
2  xử lý u
3  for mỗi v kề u
4      if tham[v] = false
5          DFS(v)
```

JULIA

```julia
function dfs(ke, s)
    n = length(ke)
    tham = falses(n)
    thu_tu = Int[]
    function di(u)
        tham[u] = true
        push!(thu_tu, u)
        for v in ke[u]
            tham[v] || di(v)
        end
    end
    di(s)
    return thu_tu
end
```

PYTHON

```python
def dfs(ke, s):
    tham = [False] * len(ke)
    thu_tu = []

    def di(u):
        tham[u] = True
        thu_tu.append(u)
        for v in ke[u]:
            if not tham[v]:
                di(v)
```

```
    di(s)
    return thu_tu
```

C++

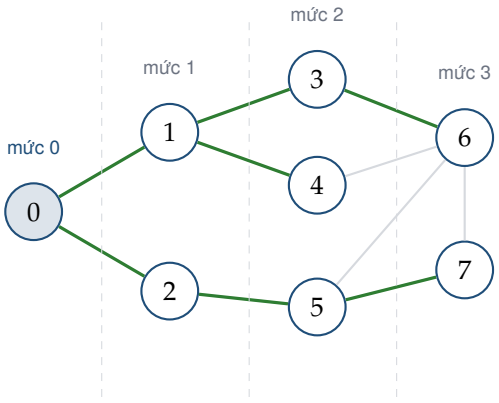
```cpp
vector<bool> tham;
vector<int> thu_tu;

void dfs(const vector<vector<int>> &ke, int u) {
    tham[u] = true;
    thu_tu.push_back(u);
    for (int v : ke[u])
        if (!tham[v]) dfs(ke, v);
}
```

**Chú ý.** Trong Julia, không được khai báo kiểu tham số là `Vector{Bool}` rồi truyền vào `falses(n)`: hàm `falses` trả về một `BitVector`, không phải `Vector{Bool}`, và chương trình sẽ dừng với lỗi `MethodError`. Hoặc bỏ khai báo kiểu, hoặc dùng `fill(false, n)`.

### 6.3.2 Tìm kiếm theo chiều rộng

**Ý tưởng.** Thăm toàn bộ các đỉnh cách đỉnh xuất phát 1 cạnh, rồi toàn bộ các đỉnh cách 2 cạnh, và cứ thế. Việc này thực hiện được nhờ một hàng đợi: đỉnh nào vào hàng trước thì được xử lý trước, đúng tính chất FIFO ở mục 5.4.




Các cạnh tô đậm tạo thành cây tìm kiếm theo chiều rộng.
Mức của đỉnh chính là số cạnh ít nhất từ đỉnh 0 tới nó.

**Hình 6.3.** Tìm kiếm theo chiều rộng chia đồ thị thành các mức theo khoảng cách tới đỉnh xuất phát.

GIẢ MÃ

```
BFS(s)
1  tham[s] ← true;  d[s] ← 0
2  đưa s vào hàng đợi Q
3  while Q khác rỗng
4      u ← lấy đầu Q
5      xử lý u
6      for mỗi v kề u
7          if tham[v] = false
8              tham[v] ← true
```

```
9               d[v] ← d[u] + 1
10              đưa v vào Q
```

**PYTHON**

```python
from collections import deque

def bfs(ke, s):
    n = len(ke)
    tham = [False] * n
    d = [-1] * n
    q = deque([s])
    tham[s] = True
    d[s] = 0
    thu_tu = []
    while q:
        u = q.popleft()
        thu_tu.append(u)
        for v in ke[u]:
            if not tham[v]:
                tham[v] = True          # đánh dấu NGAY khi đưa vào hàng
                d[v] = d[u] + 1
                q.append(v)
    return thu_tu, d
```

**JULIA**

```julia
function bfs(ke, s)
    n = length(ke)
    tham = falses(n)
    d = fill(-1, n)
    q = Int[s]
    dau = 1                 # chỉ số phần tử đầu hàng, thay cho popfirst!
    tham[s] = true
    d[s] = 0
    thu_tu = Int[]
    while dau <= length(q)
        u = q[dau]; dau += 1
        push!(thu_tu, u)
        for v in ke[u]
            if !tham[v]
                tham[v] = true
                d[v] = d[u] + 1
                push!(q, v)
            end
        end
    end
    return thu_tu, d
end
```

**C++**

```cpp
pair<vector<int>, vector<int>> bfs(const vector<vector<int>> &ke, int s) {
    int n = ke.size();
    vector<bool> tham(n, false);
    vector<int> d(n, -1), thu_tu;
    queue<int> q;
```

```
    q.push(s);  tham[s] = true;  d[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        thu_tu.push_back(u);
        for (int v : ke[u])
            if (!tham[v]) {
                tham[v] = true;         // đánh dấu NGAY khi đưa vào hàng
                d[v] = d[u] + 1;
                q.push(v);
            }
    }
    return {thu_tu, d};
}
```

**Đánh dấu lúc nào.** Phải đặt `tham[v] = true` *ngay khi đưa $v$ vào hàng đợi*, không phải khi lấy $v$ ra. Nếu đánh dấu lúc lấy ra, một đỉnh có nhiều đỉnh kề đã thăm sẽ bị đưa vào hàng nhiều lần; chương trình vẫn cho khoảng cách đúng nhưng độ phức tạp tăng và với đồ thị lớn thì hàng đợi phình ra rất nhanh.

**Ví dụ 6.1.** *Kết quả trên đồ thị ví dụ.* Xuất phát từ đỉnh 0, với danh sách kề đã sắp xếp tăng dần:

| Thuật toán | Thứ tự thăm |
|---|---|
| Tìm kiếm theo chiều sâu | 0, 1, 3, 6, 4, 5, 2, 7 |
| Tìm kiếm theo chiều rộng | 0, 1, 2, 3, 4, 5, 6, 7 |

Khoảng cách tính bằng số cạnh từ đỉnh 0 mà tìm kiếm theo chiều rộng cho ra:

| đỉnh | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| $d$ | 0 | 1 | 1 | 2 | 2 | 2 | 3 | 3 |

**Định lý 6.2.** (*Tính đúng đắn của tìm kiếm theo chiều rộng*) *Trên đồ thị không trọng số, tìm kiếm theo chiều rộng cho $d[v]$ bằng số cạnh ít nhất của một đường đi từ $s$ tới $v$.*

*Chứng minh.* Chứng minh bằng quy nạp theo giá trị của $d$. Đúng với $d = 0$ vì chỉ có đỉnh $s$. Giả sử mọi đỉnh có khoảng cách thật bằng $k$ đều được đưa vào hàng đợi với nhãn $d = k$, và chúng đứng liền nhau trong hàng. Xét đỉnh $v$ có khoảng cách thật bằng $k + 1$. Theo định nghĩa, tồn tại đỉnh $u$ kề $v$ có khoảng cách thật bằng $k$. Khi $u$ được lấy ra, nếu $v$ chưa được thăm thì nó nhận nhãn $d[u] + 1 = k + 1$. Nếu $v$ đã được thăm trước đó thì nhãn của nó nhận từ một đỉnh lấy ra sớm hơn, tức có nhãn không lớn hơn $k$, cho $d[v] \le k + 1$; kết hợp với việc $d[v]$ không thể nhỏ hơn khoảng cách thật, ta được $d[v] = k + 1$. ∎

**Bài 6.1.** Cài đặt cả hai thuật toán duyệt bằng ba ngôn ngữ trên đồ thị ở hình 6.1, xác nhận thứ tự thăm trùng với ví dụ trên. Sau đó dùng chúng để giải ba bài toán: đếm số thành phần liên thông, kiểm tra đồ thị có chu trình không, và kiểm tra đồ thị có hai phía không (tô hai màu sao cho hai đỉnh kề khác màu).

**Bảng 6.4.** So sánh hai thuật toán duyệt.

| | Chiều sâu | Chiều rộng | Độ phức tạp |
|---|---|---|---|
| Cấu trúc phụ | ngăn xếp (thường là ngăn xếp lời gọi) | hàng đợi | — |
| Cho đường ngắn nhất | không | có, khi không trọng số | — |
| Bộ nhớ | $O(h)$ với $h$ là độ sâu | $O(b)$ với $b$ là bề rộng lớn nhất | — |
| Thời gian | $O(n+m)$ | $O(n+m)$ | danh sách kề |

## 6.4 Đồ thị có trọng số và bài toán đường đi ngắn nhất

### 6.4.1 Biểu diễn đồ thị có trọng số

Danh sách kề của đồ thị có trọng số lưu, với mỗi đỉnh, danh sách các cặp (đỉnh kề, trọng số cạnh).

PYTHON

```python
def doc_do_thi_trong_so(n, canh):
    ke = [[] for _ in range(n)]
    for u, v, w in canh:
        ke[u].append((v, w))
        ke[v].append((u, w))
    return ke
```

C++

```cpp
vector<vector<pair<int,int>>> doc_do_thi_trong_so(
        int n, const vector<array<int,3>> &canh) {
    vector<vector<pair<int,int>>> ke(n);
    for (auto &e : canh) {
        ke[e[0]].push_back({e[1], e[2]});
        ke[e[1]].push_back({e[0], e[2]});
    }
    return ke;
}
```

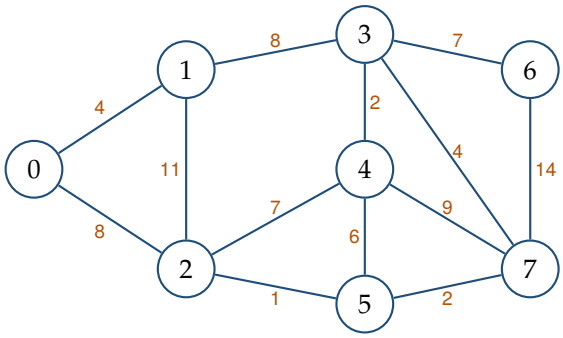
JULIA

```julia
function doc_do_thi_trong_so(n, canh)
    ke = [Tuple{Int,Int}[] for _ in 1:n]
    for (u, v, w) in canh
        push!(ke[u], (v, w))
        push!(ke[v], (u, w))
    end
    return ke
end
```

Đồ thị có trọng số dùng trong mục này cũng có 8 đỉnh và 13 cạnh.

### 6.4.2 Vì sao tìm kiếm theo chiều rộng không đủ

Tìm kiếm theo chiều rộng đếm số cạnh, nên nó chỉ đúng khi mọi cạnh có trọng số bằng nhau. Trên đồ thị ở hình 6.4, đường $0 \to 1 \to 2$ chỉ dùng 2 cạnh nhưng có tổng trọng số $4 + 11 = 15$,




**Hình 6.4.** Đồ thị có trọng số dùng trong mục này. Số trên mỗi cạnh là trọng số.

trong khi đường $0 \to 2$ dùng 1 cạnh và có trọng số 8. Đường ít cạnh hơn ở đây lại là đường ngắn hơn, nhưng nói chung hai tiêu chí không trùng nhau.

### 6.4.3 Thuật toán Dijkstra

**Điều kiện áp dụng.** Mọi trọng số phải không âm.

**Ý tưởng.** Giữ một mảng $d$ chứa khoảng cách ngắn nhất *tạm thời* từ đỉnh nguồn tới mỗi đỉnh. Lặp lại: lấy đỉnh chưa xử lý có $d$ nhỏ nhất, coi giá trị $d$ của nó là chính thức, rồi dùng nó để cập nhật các đỉnh kề. Bước cập nhật gọi là **nới lỏng** cạnh:

$$\text{nếu } d[u] + w(u,v) < d[v] \text{ thì đặt } d[v] \leftarrow d[u] + w(u,v).$$

Việc lấy đỉnh có $d$ nhỏ nhất được thực hiện bằng **hàng đợi ưu tiên**, một cấu trúc trả về phần tử nhỏ nhất trong $O(\log n)$. Cấu trúc này được cài đặt bằng đống, trình bày ở mục 7.6.

GIẢ MÃ

```
DIJKSTRA(ke, s)
1  d[v] ← ∞ với mọi v;  d[s] ← 0
2  Q ← hàng đợi ưu tiên chứa (0, s)
3  while Q khác rỗng
4      (du, u) ← lấy phần tử nhỏ nhất của Q
5      if u đã xong: bỏ qua
6      đánh dấu u đã xong
7      for mỗi (v, w) kề u
8          if d[u] + w < d[v]
9              d[v] ← d[u] + w
10             tu[v] ← u
11             đưa (d[v], v) vào Q
```

PYTHON

```python
import heapq

def dijkstra(ke, s):
    n = len(ke)
    INF = float("inf")
    d = [INF] * n
    tu = [-1] * n
    xong = [False] * n
    d[s] = 0
    pq = [(0, s)]
    while pq:
```

```
        du, u = heapq.heappop(pq)
        if xong[u]:
            continue                    # bản cũ đã lỗi thời, bỏ qua
        xong[u] = True
        for v, w in ke[u]:
            if d[u] + w < d[v]:
                d[v] = d[u] + w
                tu[v] = u
                heapq.heappush(pq, (d[v], v))
    return d, tu

def truy_vet(tu, t):
    duong = []
    while t != -1:
        duong.append(t)
        t = tu[t]
    return list(reversed(duong))
```

C++

```
const int INF = 1e9;

pair<vector<int>, vector<int>> dijkstra(
        const vector<vector<pair<int,int>>> &ke, int s) {
    int n = ke.size();
    vector<int> d(n, INF), tu(n, -1);
    vector<bool> xong(n, false);
    priority_queue<pair<int,int>, vector<pair<int,int>>,
                   greater<pair<int,int>>> pq;   // greater = lấy nhỏ nhất
    d[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (xong[u]) continue;          // bản cũ đã lỗi thời, bỏ qua
        xong[u] = true;
        for (auto &[v, w] : ke[u])
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                tu[v] = u;
                pq.push({d[v], v});
            }
    }
    return {d, tu};
}
```

JULIA

```
using DataStructures       # cung cấp PriorityQueue

function dijkstra(ke, s)
    n = length(ke)
    d = fill(typemax(Int), n)
    tu = fill(-1, n)
    xong = falses(n)
    d[s] = 0
    pq = PriorityQueue{Int, Int}()
    pq[s] = 0
    while !isempty(pq)
```

```
        u = dequeue!(pq)
        xong[u] && continue
        xong[u] = true
        for (v, w) in ke[u]
            if d[u] + w < d[v]
                d[v] = d[u] + w
                tu[v] = u
                pq[v] = d[v]
            end
        end
    end
    return d, tu
end
```

**Ba chi tiết bắt buộc.**

1. Trong C++, `priority_queue` mặc định trả về phần tử *lớn nhất*. Phải thêm `greater<...>` để lấy nhỏ nhất, hoặc đẩy vào giá trị âm.
2. Cặp đưa vào hàng phải theo thứ tự `(khoảng cách, đỉnh)`, không phải `(đỉnh, khoảng cách)`, vì phép so sánh cặp lấy thành phần đầu trước.
3. Dòng `if xong[u]: continue` là bắt buộc. Một đỉnh có thể được đẩy vào hàng nhiều lần với các giá trị khác nhau; chỉ lần lấy ra đầu tiên là giá trị đúng, các lần sau đã lỗi thời. Bỏ dòng này thì kết quả vẫn đúng nhưng mỗi đỉnh bị xử lý nhiều lần.

**Ví dụ 6.2.** *Lần vết.* Chạy Dijkstra từ đỉnh 0 trên đồ thị ở hình 6.4. Thứ tự các đỉnh được lấy ra khỏi hàng đợi ưu tiên và giá trị $d$ chính thức của chúng:

| Thứ tự lấy ra | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| Đỉnh | 0 | 1 | 2 | 5 | 7 | 3 | 4 | 6 |
| $d$ chính thức | 0 | 4 | 8 | 9 | 11 | 12 | 14 | 19 |

Dãy $d$ tăng dần, đúng như lý thuyết dự đoán. Đường đi ngắn nhất tới từng đỉnh:

| Đỉnh đích | $d$ | Đường đi |
|---|---|---|
| 1 | 4 | $0 \to 1$ |
| 2 | 8 | $0 \to 2$ |
| 3 | 12 | $0 \to 1 \to 3$ |
| 4 | 14 | $0 \to 1 \to 3 \to 4$ |
| 5 | 9 | $0 \to 2 \to 5$ |
| 6 | 19 | $0 \to 1 \to 3 \to 6$ |
| 7 | 11 | $0 \to 2 \to 5 \to 7$ |

Chương trình Bellman-Ford chạy độc lập trên cùng dữ liệu cho đúng dãy $d$ này.

**Định lý 6.3.** (*Tính đúng đắn của Dijkstra*) *Nếu mọi trọng số không âm thì khi một đỉnh $u$ được lấy ra khỏi hàng đợi ưu tiên lần đầu tiên, giá trị $d[u]$ chính là độ dài đường đi ngắn nhất từ $s$ tới $u$.*

*Chứng minh.* Giả sử ngược lại, gọi $u$ là đỉnh đầu tiên bị lấy ra với $d[u]$ lớn hơn độ dài $\delta(u)$ của đường đi ngắn nhất thật. Xét một đường đi ngắn nhất thật từ $s$ tới $u$, và gọi $y$ là đỉnh đầu tiên trên đường

đó chưa được lấy ra, còn $x$ là đỉnh ngay trước $y$. Vì $x$ đã được lấy ra trước $u$ và $u$ là đỉnh sai đầu tiên nên $d[x] = \delta(x)$. Khi $x$ được lấy ra, cạnh $(x, y)$ đã được nới lỏng, cho $d[y] \le d[x] + w(x, y) = \delta(y)$. Mặt khác $y$ nằm trên đường đi ngắn nhất tới $u$ và mọi trọng số không âm, nên $\delta(y) \le \delta(u)$. Kết hợp lại: $d[y] \le \delta(u) < d[u]$. Nhưng khi đó thuật toán phải lấy $y$ ra trước $u$, mâu thuẫn với cách chọn $y$. ∎

> **Trọng số âm.** Bước "coi $d[u]$ là chính thức" dựa vào giả thiết trọng số không âm: đi thêm cạnh chỉ làm đường dài ra. Nếu có cạnh trọng số âm, một đường vòng qua cạnh âm có thể ngắn hơn, và Dijkstra cho kết quả sai. Khi có trọng số âm phải dùng thuật toán Bellman-Ford, độ phức tạp $O(nm)$; thuật toán này còn phát hiện được chu trình âm.

**Độ phức tạp.** Mỗi cạnh gây ra nhiều nhất một lần đẩy vào hàng, nên hàng có nhiều nhất $m$ phần tử. Mỗi thao tác trên hàng tốn $O(\log m)$, và $\log m \le \log n^2 = 2 \log n$. Tổng cộng $O((n+m) \log n)$.

## 6.5 Bài toán cây bao trùm nhỏ nhất

**Định nghĩa 6.2.** (*Cây bao trùm nhỏ nhất*) Cho đồ thị vô hướng liên thông có trọng số $G = (V, E)$. **Cây bao trùm** là tập con của $E$ tạo thành cây nối tất cả các đỉnh. **Cây bao trùm nhỏ nhất** là cây bao trùm có tổng trọng số nhỏ nhất.

Bài toán này mô hình hoá tình huống: cần nối tất cả các điểm với chi phí thấp nhất, chẳng hạn kéo cáp mạng giữa các toà nhà hay xây đường nối các xã. Cây bao trùm của đồ thị $n$ đỉnh luôn có đúng $n - 1$ cạnh.

Cả hai thuật toán dưới đây đều theo mô hình tham lam của mục 4.2, và cả hai đều chứng minh được là tối ưu.

### 6.5.1 Thuật toán Prim

**Ý tưởng.** Bắt đầu từ một đỉnh. Ở mỗi bước, thêm cạnh nhẹ nhất nối một đỉnh đã có trong cây với một đỉnh chưa có. Lặp $n - 1$ lần.

PYTHON

```python
import heapq

def prim(ke, s=0):
    n = len(ke)
    trong_cay = [False] * n
    tong = 0
    canh_chon = []
    pq = [(0, s, -1)]                  # (trọng số, đỉnh, đỉnh cha)
    while pq:
        w, u, cha = heapq.heappop(pq)
        if trong_cay[u]:
            continue
        trong_cay[u] = True
        tong += w
        if cha != -1:
            canh_chon.append((cha, u, w))
        for v, ww in ke[u]:
            if not trong_cay[v]:
                heapq.heappush(pq, (ww, v, u))
    return tong, canh_chon
```

C++

```cpp
pair<int, vector<array<int,3>>> prim(
        const vector<vector<pair<int,int>>> &ke, int s = 0) {
    int n = ke.size(), tong = 0;
    vector<bool> trong_cay(n, false);
    vector<array<int,3>> canh_chon;
    priority_queue<array<int,3>, vector<array<int,3>>,
                   greater<array<int,3>>> pq;
    pq.push({0, s, -1});                // {trọng số, đỉnh, đỉnh cha}
    while (!pq.empty()) {
        auto [w, u, cha] = pq.top(); pq.pop();
        if (trong_cay[u]) continue;
        trong_cay[u] = true;
        tong += w;
        if (cha != -1) canh_chon.push_back({cha, u, w});
        for (auto &[v, ww] : ke[u])
            if (!trong_cay[v]) pq.push({ww, v, u});
    }
    return {tong, canh_chon};
}
```

### 6.5.2 Thuật toán Kruskal

**Ý tưởng.** Sắp xếp mọi cạnh theo trọng số tăng dần. Duyệt lần lượt, nhận cạnh nếu nó nối hai thành phần khác nhau, bỏ qua nếu nó tạo chu trình.

Câu hỏi "hai đỉnh có cùng thành phần không" được trả lời bằng cấu trúc **tập cha con** (*disjoint set union*), gồm hai thao tác: `tim` tìm đại diện của một thành phần, và `hop` gộp hai thành phần.

PYTHON

```python
def kruskal(n, canh):
    cha = list(range(n))

    def tim(x):
        while cha[x] != x:
            cha[x] = cha[cha[x]]        # nén đường đi
            x = cha[x]
        return x

    tong = 0
    chon = []
    for w, u, v in sorted(canh):        # sắp theo trọng số tăng dần
        a, b = tim(u), tim(v)
        if a != b:                      # không tạo chu trình
            cha[a] = b
            tong += w
            chon.append((u, v, w))
    return tong, chon
```

C++

```cpp
struct TapChaCon {
    vector<int> cha;
    TapChaCon(int n) : cha(n) { iota(cha.begin(), cha.end(), 0); }
    int tim(int x) {
        while (cha[x] != x) { cha[x] = cha[cha[x]]; x = cha[x]; }
        return x;                       // nén đường đi
```

```
    }
    bool hop(int u, int v) {
        int a = tim(u), b = tim(v);
        if (a == b) return false;
        cha[a] = b;
        return true;
    }
};

int kruskal(int n, vector<array<int,3>> canh) {   // {w, u, v}
    sort(canh.begin(), canh.end());
    TapChaCon t(n);
    int tong = 0;
    for (auto &[w, u, v] : canh)
        if (t.hop(u, v)) tong += w;
    return tong;
}
```

JULIA

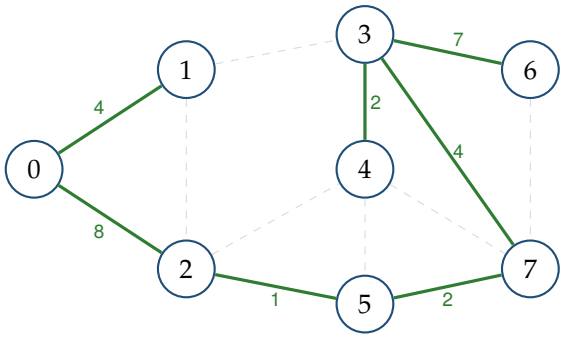
```
function kruskal(n, canh)              # canh: vector các bộ (w, u, v)
    cha = collect(1:n)
    function tim(x)
        while cha[x] != x
            cha[x] = cha[cha[x]]
            x = cha[x]
        end
        return x
    end
    tong = 0
    chon = Tuple{Int,Int,Int}[]
    for (w, u, v) in sort(canh)
        a, b = tim(u), tim(v)
        if a != b
            cha[a] = b
            tong += w
            push!(chon, (u, v, w))
        end
    end
    return tong, chon
end
```

**Ví dụ 6.3.** *Kết quả trên đồ thị ví dụ.* Trên đồ thị ở hình 6.4, cả hai thuật toán đều cho tổng trọng số **28**, nhưng chọn các cạnh theo thứ tự khác nhau:

| | |
|---|---|
| Prim (bắt đầu từ 0) | $(0,1,4), (0,2,8), (2,5,1), (5,7,2), (7,3,4), (3,4,2), (3,6,7)$ |
| Kruskal | $(2,5,1), (3,4,2), (5,7,2), (0,1,4), (3,7,4), (3,6,7), (0,2,8)$ |

Cả hai đều chọn đúng 7 cạnh, bằng $n-1$. Tập cạnh của hai cây trùng nhau hoàn toàn; nói chung khi các trọng số đôi một khác nhau thì cây bao trùm nhỏ nhất là duy nhất, còn khi có trọng số bằng nhau thì có thể có nhiều cây khác nhau nhưng tổng trọng số luôn như nhau.

**Bài 6.2.** Cài đặt Dijkstra, Prim và Kruskal bằng ba ngôn ngữ trên đồ thị ở hình 6.4. Với Dijkstra, in ra thứ tự các đỉnh được lấy khỏi hàng đợi ưu tiên và đối chiếu với bảng trong ví




Bảy cạnh tô đậm tạo thành cây bao trùm nhỏ nhất,
tổng trọng số 28. Các cạnh nét đứt không được chọn.

**Hình 6.5.** Cây bao trùm nhỏ nhất của đồ thị ở hình 6.4.

**Bảng 6.5.** So sánh Prim và Kruskal.

| | Prim | Kruskal |
|---|---|---|
| Cách xây | mở rộng một cây duy nhất | ghép dần nhiều cây rời nhau |
| Cấu trúc phụ | hàng đợi ưu tiên | sắp xếp + tập cha con |
| Độ phức tạp | $O(m \log n)$ | $O(m \log m)$ |
| Phù hợp với | đồ thị dày | đồ thị thưa |

dụ. Viết thêm chương trình Bellman-Ford để kiểm chứng kết quả Dijkstra, và một chương trình vét cạn duyệt mọi cây bao trùm với $n \le 7$ để kiểm chứng kết quả 28.

## 6.6 Ứng dụng: thuật toán tìm đường A sao

### 6.6.1 Bài toán

Trong trò chơi điện tử và trong dẫn đường rô-bốt, bản đồ thường được chia thành lưới ô vuông; mỗi ô là một đỉnh, hai ô kề nhau nối bởi một cạnh trọng số 1, và các ô là tường thì không đi qua được. Cần tìm đường ngắn nhất từ ô xuất phát tới ô đích.

Dijkstra giải được bài toán này. Nhưng nó có một điểm yếu: nó lan đều ra mọi hướng, kể cả hướng ngược hẳn với đích. Trên bản đồ trò chơi có hàng trăm nghìn ô và hàng chục nhân vật cùng tìm đường mỗi khung hình, chi phí đó là không chấp nhận được.

### 6.6.2 Ý tưởng của A sao

Dijkstra luôn chọn đỉnh có $g(u)$ nhỏ nhất, trong đó $g(u)$ là chi phí *đã đi* từ điểm xuất phát. Thuật toán A sao chọn đỉnh có

$$f(u) = g(u) + h(u)$$

nhỏ nhất, trong đó $h(u)$ là **hàm ước lượng** chi phí *còn lại* từ $u$ tới đích. Nói cách khác, A sao không chỉ nhìn vào quãng đường đã đi mà còn nhìn vào quãng đường có vẻ còn phải đi.

**Định nghĩa 6.3.** (*Hàm ước lượng chấp nhận được*) Hàm $h$ được gọi là **chấp nhận được** (*admissible*) nếu $h(u)$ không bao giờ lớn hơn chi phí thật của đường đi ngắn nhất từ $u$ tới đích.

**Định lý 6.4.** (*Tính đúng đắn của A sao*) *Nếu hàm ước lượng $h$ chấp nhận được thì A sao trả về đường đi ngắn nhất.*

*Chứng minh.* Giả sử thuật toán lấy đích $G$ ra khỏi hàng đợi với giá trị $f(G) = g(G)$ lớn hơn độ dài $\delta$ của đường đi ngắn nhất thật. Xét một đường đi ngắn nhất thật và gọi $y$ là đỉnh đầu tiên trên đường đó còn nằm trong hàng đợi. Khi đó $g(y)$ bằng chi phí thật từ đầu tới $y$, nên

$$f(y) = g(y) + h(y) \le g(y) + (\delta - g(y)) = \delta < g(G) = f(G),$$

trong đó bất đẳng thức giữa dùng đúng giả thiết $h$ chấp nhận được. Vậy $y$ có $f$ nhỏ hơn $G$ và phải được lấy ra trước $G$, mâu thuẫn. ∎

**Nhận xét 6.2.** Hai trường hợp đặc biệt đáng chú ý. Nếu $h \equiv 0$ thì $f = g$ và A sao trở thành đúng Dijkstra — Dijkstra là trường hợp riêng của A sao với hàm ước lượng không mang thông tin gì. Nếu $h$ bằng đúng chi phí thật còn lại thì thuật toán đi thẳng tới đích không xét thừa ô nào.

**Hàm ước lượng trên lưới.** Khi chỉ được đi bốn hướng, khoảng cách Manhattan

$$h(u) = |u_x - G_x| + |u_y - G_y|$$

là hàm chấp nhận được: bất kỳ đường đi nào cũng phải bù đủ chênh lệch theo cả hai trục, và mỗi bước chỉ bù được 1 đơn vị theo đúng một trục.

> **Chọn sai hàm ước lượng.** Nếu cho phép đi chéo mà vẫn dùng khoảng cách Manhattan, hàm ước lượng có thể lớn hơn chi phí thật và A sao cho đường không tối ưu. Với lưới đi tám hướng phải dùng khoảng cách Chebyshev $h(u) = \max(|u_x - G_x|, |u_y - G_y|)$. Nhân hàm ước lượng với một hệ số lớn hơn 1 cũng phá tính chấp nhận được; đổi lại thuật toán chạy nhanh hơn nhiều, và đây là một đánh đổi được dùng có chủ đích trong nhiều trò chơi.

### 6.6.3 Cài đặt

GIẢ MÃ

```
A-SAO(S, G)
1  g[S] ← 0
2  Q ← hàng đợi ưu tiên chứa (h(S), S)
3  while Q khác rỗng
4      (f, u) ← lấy phần tử nhỏ nhất của Q
5      if u đã xét: bỏ qua
6      đánh dấu u đã xét
7      if u = G: return g[u]
8      for mỗi ô v kề u và đi được
9          if g[u] + 1 < g[v]
10             g[v] ← g[u] + 1
11             tu[v] ← u
12             đưa (g[v] + h(v), v) vào Q
13 return KHÔNG-CÓ-ĐƯỜNG
```

PYTHON

```python
import heapq

def a_sao(ban_do, S, G):
```

```python
    H, W = len(ban_do), len(ban_do[0])
    DI = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    def di_duoc(i, j):
        return 0 <= i < H and 0 <= j < W and ban_do[i][j] != "#"

    def h(u):                                   # khoảng cách Manhattan
        return abs(u[0] - G[0]) + abs(u[1] - G[1])

    g = {S: 0}
    tu = {}
    da_xet = set()
    pq = [(h(S), 0, S)]
    while pq:
        f, gu, u = heapq.heappop(pq)
        if u in da_xet:
            continue
        da_xet.add(u)
        if u == G:
            return gu, tu, len(da_xet)
        for di, dj in DI:
            v = (u[0] + di, u[1] + dj)
            if di_duoc(*v) and gu + 1 < g.get(v, float("inf")):
                g[v] = gu + 1
                tu[v] = u
                heapq.heappush(pq, (g[v] + h(v), g[v], v))
    return None, tu, len(da_xet)
```

C++

```cpp
using O = pair<int,int>;

pair<int,int> a_sao(const vector<string> &ban_do, O S, O G, bool dung_h) {
    int H = ban_do.size(), W = ban_do[0].size();
    int di[] = {-1, 1, 0, 0}, dj[] = {0, 0, -1, 1};
    auto h = [&](O u) {
        return dung_h ? abs(u.first - G.first) + abs(u.second - G.second) : 0;
    };
    map<O,int> g;  g[S] = 0;
    set<O> da_xet;
    priority_queue<tuple<int,int,O>, vector<tuple<int,int,O>>,
                   greater<tuple<int,int,O>>> pq;
    pq.push({h(S), 0, S});
    while (!pq.empty()) {
        auto [f, gu, u] = pq.top(); pq.pop();
        if (da_xet.count(u)) continue;
        da_xet.insert(u);
        if (u == G) return {gu, (int)da_xet.size()};
        for (int k = 0; k < 4; k++) {
            O v = {u.first + di[k], u.second + dj[k]};
            if (v.first < 0 || v.first >= H || v.second < 0 || v.second >= W)
                continue;
            if (ban_do[v.first][v.second] == '#') continue;
            if (!g.count(v) || gu + 1 < g[v]) {
                g[v] = gu + 1;
                pq.push({gu + 1 + h(v), gu + 1, v});
            }
        }
    }
```

```
    return {-1, (int)da_xet.size()};
}
// Gọi với dung_h = false thì hàm này chính là Dijkstra.
```

JULIA

```julia
using DataStructures

function a_sao(ban_do, S, G; dung_h = true)
    H, W = length(ban_do), length(ban_do[1])
    DI = [(-1,0), (1,0), (0,-1), (0,1)]
    di_duoc(i, j) = 1 <= i <= H && 1 <= j <= W && ban_do[i][j] != '#'
    h(u) = dung_h ? abs(u[1]-G[1]) + abs(u[2]-G[2]) : 0

    g = Dict(S => 0)
    tu = Dict{Tuple{Int,Int}, Tuple{Int,Int}}()
    da_xet = Set{Tuple{Int,Int}}()
    pq = PriorityQueue{Tuple{Int,Int}, Int}()
    pq[S] = h(S)
    while !isempty(pq)
        u = dequeue!(pq)
        u in da_xet && continue
        push!(da_xet, u)
        u == G && return g[u], tu, length(da_xet)
        for (di, dj) in DI
            v = (u[1]+di, u[2]+dj)
            if di_duoc(v...) && g[u] + 1 < get(g, v, typemax(Int))
                g[v] = g[u] + 1
                tu[v] = u
                pq[v] = g[v] + h(v)
            end
        end
    end
    return nothing, tu, length(da_xet)
end
```

### 6.6.4 Đo hiệu quả

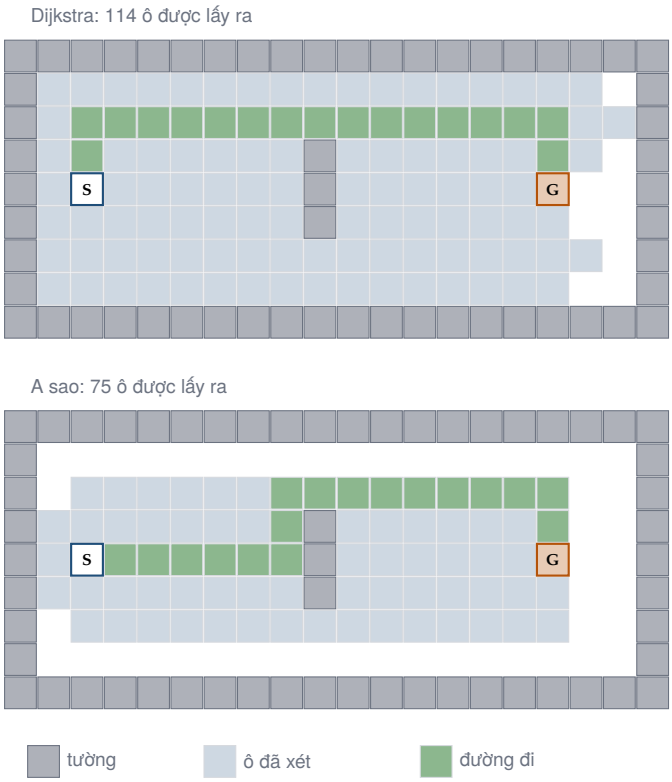
Chương trình chạy trên bản đồ $9 \times 20$ dưới đây, có một bức tường thẳng đứng chắn giữa. Cả hai thuật toán đều tìm ra đường dài 18 bước; khác biệt nằm ở số ô phải lấy ra khỏi hàng đợi.

**Bảng 6.6.** Số ô được lấy ra khỏi hàng đợi trên bản đồ $9 \times 20$.

| Bản đồ | Dijkstra | A sao | Tỉ lệ |
|---|---|---|---|
| Có tường chắn (đường dài 18) | 114 | 75 | 1,5 lần |
| Không có tường (đường dài 14) | 97 | 15 | 6,5 lần |

Bảng số liệu cho thấy một điều quan trọng: mức tiết kiệm của A sao *phụ thuộc vào chất lượng hàm ước lượng*. Trên bản đồ trống, khoảng cách Manhattan đúng bằng chi phí thật, và A sao đi thẳng tới đích, chỉ xét 15 ô. Khi có tường chắn, hàm ước lượng đánh giá thấp hơn chi phí thật vì nó không biết về bức tường, nên A sao vẫn phải xét khá nhiều ô. Trong trường hợp xấu nhất, chẳng hạn một mê cung mà đường đi thật đi vòng hẳn ra xa đích, A sao xét đúng bằng số ô Dijkstra xét.

**Nhận xét 6.3.** Nhìn hình 6.6 có thể thấy rõ khác biệt về hình dạng vùng tìm kiếm: Dijkstra loang đều thành hình tròn quanh điểm xuất phát, còn A sao loang thành hình bầu dục kéo dài về phía




**Hình 6.6.** Dijkstra và A sao trên cùng bản đồ. Cả hai cho đường dài 18 bước, nhưng A sao chỉ xét 75 ô so với 114 ô.

đích, chừa hẳn hai hàng trên và hai hàng dưới. Đó chính là hình ảnh của cụm từ "tìm kiếm có định hướng".

**Bài 6.3.** Cài đặt A sao theo các bước sau.

1. Đọc bản đồ từ tệp văn bản, ký tự `#` là tường, `S` là điểm xuất phát, `G` là đích.
2. Viết một hàm chung nhận tham số `dung_h`; khi `dung_h` bằng sai thì hàm ước lượng trả về 0 và chương trình trở thành Dijkstra. Nhờ vậy hai thuật toán dùng chung một đoạn mã và so sánh được công bằng.
3. Đếm số ô lấy ra khỏi hàng đợi trong mỗi trường hợp.
4. In bản đồ kết quả với ba ký hiệu: `+` cho ô đã xét, `o` cho ô trên đường đi, dấu chấm cho ô chưa xét.
5. Chạy trên bản đồ ở hình 6.6; kết quả phải là 114 và 75 ô, cùng độ dài 18. Sau đó xoá bức tường và chạy lại; kết quả phải là 97 và 15 ô, cùng độ dài 14.
6. Thay hàm ước lượng bằng $h(u) \times 2$ và ghi lại số ô cùng độ dài đường đi. Đường đi có thể dài hơn 18; hãy giải thích bằng định lý về tính chấp nhận được.

**Tóm tắt.** Đồ thị biểu diễn được bằng ma trận kề, tốn $O(n^2)$ bộ nhớ, hoặc bằng danh sách kề, tốn $O(n+m)$; với đồ thị thưa — tức phần lớn đồ thị thực tế — danh sách kề là lựa chọn bắt buộc. Tìm kiếm theo chiều sâu dùng ngăn xếp, tìm kiếm theo chiều rộng dùng hàng đợi và cho đường ngắn nhất khi mọi cạnh có trọng số bằng nhau. Khi trọng số khác nhau và không âm, Dijkstra cho đường ngắn nhất trong $O((n+m)\log n)$. Prim và Kruskal đều là

thuật toán tham lam cho cây bao trùm nhỏ nhất, và trên đồ thị ví dụ cả hai cho tổng trọng số 28. Cuối cùng, A sao là Dijkstra cộng thêm hàm ước lượng chi phí còn lại; nó vẫn cho đường tối ưu miễn là hàm ước lượng không bao giờ đánh giá cao hơn chi phí thật, và tiết kiệm được từ 1,5 lần tới 6,5 lần số ô phải xét tuỳ theo hàm ước lượng bám sát thực tế đến đâu.

## 6.7 Bài tập chương 6

### Câu hỏi ôn tập

1. Định nghĩa đồ thị, bậc của đỉnh, đường đi, chu trình, tính liên thông.
2. Phát biểu và chứng minh bổ đề bắt tay.
3. So sánh ma trận kề và danh sách kề về bộ nhớ và về bốn thao tác cơ bản. Khi nào nên dùng cách nào?
4. Vì sao bản đồ giao thông là đồ thị thưa? Điều đó dẫn tới lựa chọn biểu diễn nào?
5. Vì sao thuật toán duyệt bắt buộc phải có mảng đánh dấu?
6. Trong tìm kiếm theo chiều rộng, vì sao phải đánh dấu đỉnh ngay khi đưa vào hàng đợi chứ không phải khi lấy ra?
7. Chứng minh tìm kiếm theo chiều rộng cho đường đi ít cạnh nhất.
8. Vì sao tìm kiếm theo chiều rộng không dùng được cho đồ thị có trọng số?
9. Phát biểu quy tắc nới lỏng cạnh trong thuật toán Dijkstra.
10. Ba chi tiết dễ sai khi cài Dijkstra bằng hàng đợi ưu tiên là gì?
11. Vì sao Dijkstra sai khi có cạnh trọng số âm? Nêu thuật toán thay thế.
12. So sánh Prim và Kruskal về cách xây, cấu trúc phụ và độ phức tạp.
13. Thế nào là hàm ước lượng chấp nhận được? Vì sao tính chất này cần cho tính đúng đắn của A sao?
14. Vì sao Dijkstra là trường hợp riêng của A sao?
15. Vì sao mức tiết kiệm của A sao thay đổi từ 1,5 lần lên 6,5 lần giữa hai bản đồ trong bảng 6.6?

### Bài tập tính toán

16. Cho đồ thị ở hình 6.1. Viết ma trận kề và danh sách kề, tính bậc của mọi đỉnh và kiểm chứng bổ đề bắt tay.
17. Lần vết tìm kiếm theo chiều sâu và theo chiều rộng từ đỉnh 3 của đồ thị ở hình 6.1, ghi rõ nội dung ngăn xếp hoặc hàng đợi ở mỗi bước.
18. Lần vết Dijkstra từ đỉnh 2 trên đồ thị ở hình 6.4, lập bảng giá trị $d$ sau mỗi lần lấy đỉnh ra khỏi hàng.
19. Áp dụng Kruskal cho đồ thị ở hình 6.4, liệt kê các cạnh theo thứ tự xét và ghi rõ cạnh nào bị bỏ vì tạo chu trình.

20. Chứng minh cây $n$ đỉnh có đúng $n - 1$ cạnh.

21. Cho lưới $5 \times 5$ không có tường, điểm xuất phát ở góc trên bên trái và đích ở góc dưới bên phải. Tính $h$ theo khoảng cách Manhattan cho mọi ô và xác định thứ tự A sao lấy các ô ra khỏi hàng.

22. Chứng minh rằng nếu nhân hàm ước lượng chấp nhận được với hệ số $\alpha > 1$ thì đường đi thu được dài không quá $\alpha$ lần đường đi ngắn nhất.

### Bài tập lập trình mức cơ bản

23. Cài đặt cả hai cách biểu diễn đồ thị và hai hàm chuyển đổi giữa chúng.

24. Cài đặt cả hai thuật toán duyệt bằng ba ngôn ngữ, in thứ tự thăm.

25. Đếm số thành phần liên thông của một đồ thị vô hướng.

26. Kiểm tra một đồ thị vô hướng có chu trình không.

27. Kiểm tra một đồ thị có hai phía không, dùng tô hai màu bằng tìm kiếm theo chiều rộng.

28. Tìm đường đi ít cạnh nhất giữa hai đỉnh và in ra đường đi đó.

29. Cài đặt Dijkstra có truy vết đường đi, in cả độ dài lẫn dãy đỉnh.

30. Cài đặt Prim và Kruskal, so sánh tập cạnh của hai cây thu được.

### Bài tập lập trình mức vận dụng

31. Cài đặt thuật toán sắp xếp tô-pô cho đồ thị có hướng không chu trình, bằng cả hai cách: dựa trên tìm kiếm theo chiều sâu và dựa trên bậc vào.

32. Tìm thành phần liên thông mạnh của đồ thị có hướng bằng thuật toán Kosaraju hoặc Tarjan.

33. Cài đặt Bellman-Ford, phát hiện chu trình âm, và so sánh kết quả với Dijkstra trên đồ thị chỉ có trọng số dương.

34. Cài đặt Floyd-Warshall tìm đường đi ngắn nhất giữa mọi cặp đỉnh trong $O(n^3)$, so sánh với việc chạy Dijkstra $n$ lần.

35. Bài toán luồng cực đại: cài đặt thuật toán Ford-Fulkerson với tìm đường tăng luồng bằng tìm kiếm theo chiều rộng.

36. Cho một mê cung đọc từ tệp, tìm đường ngắn nhất bằng tìm kiếm theo chiều rộng và in đường đi lên màn hình.

37. Tìm chu trình Euler của một đồ thị vô hướng liên thông có mọi đỉnh bậc chẵn.

### Bài tập phần trí tuệ nhân tạo

38. Cài đặt A sao theo bài thực hành 6.3 và xác nhận các số liệu 114, 75, 97, 15.

39. Mở rộng A sao cho lưới đi tám hướng, dùng khoảng cách Chebyshev, và kiểm tra bằng thực nghiệm rằng dùng khoảng cách Manhattan trong trường hợp này cho đường không tối ưu.

40. Thêm chi phí địa hình: mỗi ô có chi phí đi qua từ 1 đến 9 thay vì luôn bằng 1. Sửa cả Dijkstra lẫn A sao cho phù hợp và kiểm tra hàm ước lượng vẫn chấp nhận được khi chi phí nhỏ nhất bằng 1.

41. Cài đặt thuật toán *tham lam theo hàm ước lượng*, tức chỉ dùng $f(u) = h(u)$ mà bỏ $g(u)$. Đo số ô phải xét và độ dài đường đi trên cùng bản đồ; thuật toán sẽ nhanh hơn A sao nhưng không cho đường tối ưu.

42. So sánh ba thuật toán tìm đường — tìm kiếm theo chiều rộng, Dijkstra, A sao — trên một bản đồ $200 \times 200$ sinh ngẫu nhiên với 30 phần trăm ô là tường. Lập bảng số ô đã xét và thời gian chạy.

43. Cài đặt thuật toán A sao có trọng số với $\alpha = 1$, $1{,}5$, $2$, $3$. Với mỗi giá trị, ghi lại số ô đã xét và độ dài đường đi, rồi vẽ đồ thị biểu diễn sự đánh đổi giữa hai đại lượng đó.

CHƯƠNG 7

# Cây nhị phân

Chương 5 trình bày các cấu trúc tuyến tính: mọi phần tử xếp thành một dãy. Chương 6 mở rộng sang đồ thị, cấu trúc tổng quát nhất. Chương này quay lại một lớp đồ thị đặc biệt nhưng quan trọng bậc nhất: cây, và cụ thể hơn là cây nhị phân.

Cây nhị phân giải quyết một bài toán mà cả mảng lẫn danh sách liên kết đều không giải được trọn vẹn. Mảng đã sắp xếp cho tìm kiếm $O(\log n)$ nhưng chèn và xoá tốn $O(n)$; danh sách liên kết cho chèn và xoá $O(1)$ nhưng tìm kiếm tốn $O(n)$. Cây nhị phân tìm kiếm cân bằng cho *cả ba* thao tác trong $O(\log n)$.

## 7.1 Định nghĩa và phân loại

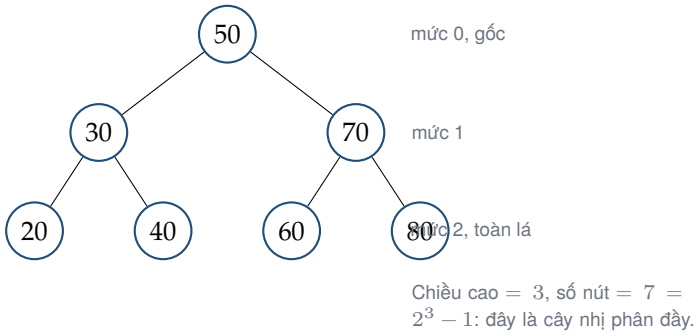
### 7.1.1 Định nghĩa

**Định nghĩa 7.1.** (*Cây nhị phân*) Một **cây nhị phân** là tập hữu hạn các nút, hoặc rỗng, hoặc gồm một nút gọi là **gốc** cùng hai cây nhị phân rời nhau gọi là **cây con trái** và **cây con phải** của gốc.

Định nghĩa trên là định nghĩa đệ quy, và đó là lý do gần như mọi thuật toán trên cây đều viết bằng đệ quy một cách rất tự nhiên.

Các thuật ngữ dùng trong chương:

- **Nút cha** của $v$ là nút có $v$ làm con; **nút lá** là nút không có con nào.
- **Mức** của nút là số cạnh từ gốc tới nó; gốc có mức 0.
- **Chiều cao** của cây là mức lớn nhất cộng 1, tức số nút trên đường dài nhất từ gốc xuống lá. Cây rỗng có chiều cao 0.
- **Bậc** của nút là số con của nó, nhận giá trị 0, 1 hoặc 2.




**Hình 7.1.** Cây nhị phân dùng làm ví dụ trong chương.

### 7.1.2 Phân loại

**Bảng 7.1.** Các loại cây nhị phân.

| Loại | Định nghĩa | Tính chất |
|---|---|---|
| Cây suy biến | mỗi nút có nhiều nhất một con | chiều cao $= n$, thoái hoá thành danh sách |
| Cây nhị phân đủ | mọi nút hoặc có 0 con hoặc có 2 con | số lá $=$ số nút trong $+1$ |
| Cây nhị phân đầy | mọi mức đều đầy | $n = 2^h - 1$ |
| Cây gần đầy | đầy ở mọi mức trừ mức cuối, mức cuối dồn về trái | lưu được bằng mảng, không có ô trống |
| Cây cân bằng | chênh lệch chiều cao hai cây con của mọi nút không quá 1 | chiều cao $= O(\log n)$ |

**Định lý 7.1.** (*Quan hệ giữa số nút và chiều cao*) *Cây nhị phân chiều cao $h$ có nhiều nhất $2^h - 1$ nút. Ngược lại, cây $n$ nút có chiều cao ít nhất $\lceil \log_2(n+1) \rceil$.*

*Chứng minh.* Mức $k$ có nhiều nhất $2^k$ nút, chứng minh bằng quy nạp: mức 0 có 1 nút, và mỗi nút ở mức $k$ sinh ra nhiều nhất 2 nút ở mức $k+1$. Cộng qua các mức 0 đến $h-1$ được $1+2+4+\cdots+2^{h-1} = 2^h - 1$. Mệnh đề thứ hai suy ra từ mệnh đề thứ nhất: từ $n \le 2^h - 1$ ta được $h \ge \log_2(n+1)$. ■

Định lý này là nền tảng của toàn chương: nó nói rằng $O(\log n)$ là giới hạn tốt nhất có thể cho một thao tác đi từ gốc xuống lá, và mọi kỹ thuật cân bằng đều nhằm đạt được giới hạn đó.

## 7.2 Biểu diễn cây nhị phân

### 7.2.1 Biểu diễn bằng danh sách liên kết

Cách tổng quát: mỗi nút chứa dữ liệu và hai liên kết tới hai con.

JULIA

```julia
mutable struct Nut
    gt::Int
    trai::Union{Nut, Nothing}
    phai::Union{Nut, Nothing}
end
Nut(gt) = Nut(gt, nothing, nothing)
```

PYTHON

```python
class Nut:
    __slots__ = ("gt", "trai", "phai")

    def __init__(self, gt):
        self.gt = gt
        self.trai = None
        self.phai = None
```

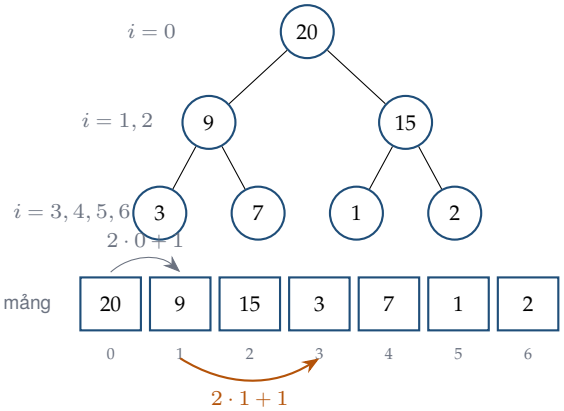
C++

```cpp
struct Nut {
    int gt;
    Nut *trai = nullptr, *phai = nullptr;
    Nut(int g) : gt(g) {}
};
```

### 7.2.2 Biểu diễn bằng mảng

Với cây *gần đầy*, có một cách biểu diễn không cần con trỏ nào. Đánh số các nút theo mức từ trên xuống, trong mỗi mức từ trái sang phải, bắt đầu từ 0. Khi đó với nút ở chỉ số $i$:

- con trái ở chỉ số $2i + 1$;
- con phải ở chỉ số $2i + 2$;
- nút cha ở chỉ số $\lfloor (i-1)/2 \rfloor$.




**Hình 7.2.** Cây gần đầy và mảng biểu diễn nó. Không có ô trống nào, và quan hệ cha con tính được bằng phép số học trên chỉ số.

> **Chỉ số trong Julia.** Ba công thức trên viết cho cách đánh số từ 0. Với Julia đánh số từ 1, công thức trở thành: con trái ở $2i$, con phải ở $2i + 1$, cha ở $\lfloor i/2 \rfloor$. Cách đánh số từ 1 thực ra gọn hơn, và đó là lý do nhiều giáo trình cố tình bỏ trống ô 0 khi cài đống bằng mảng trong C++.

**Nhận xét 7.1.** Biểu diễn bằng mảng chỉ hiệu quả với cây gần đầy. Với cây suy biến chiều cao $n$, chỉ số của nút sâu nhất là $2^n - 1$, nên mảng phải có kích thước $2^n$ dù chỉ chứa $n$ giá trị. Vì vậy quy tắc là: cây gần đầy dùng mảng, cây có hình dạng bất kỳ dùng liên kết.

## 7.3 Thao tác trên cây nhị phân tổng quát

### 7.3.1 Ba cách duyệt

Duyệt cây là thăm mọi nút đúng một lần. Vì mỗi nút có hai cây con, thứ tự giữa ba việc — thăm gốc, duyệt cây con trái, duyệt cây con phải — cho ba cách duyệt khác nhau.

Dãy duyệt giữa cho kết quả tăng dần. Đó không phải ngẫu nhiên mà là tính chất đặc trưng của cây nhị phân tìm kiếm ở mục sau.

**Bảng 7.2.** Ba cách duyệt cây nhị phân và kết quả trên cây ở hình 7.1.

| Cách duyệt | Thứ tự | Kết quả |
|---|---|---|
| Trước (*preorder*) | gốc, trái, phải | 50, 30, 20, 40, 70, 60, 80 |
| Giữa (*inorder*) | trái, gốc, phải | 20, 30, 40, 50, 60, 70, 80 |
| Sau (*postorder*) | trái, phải, gốc | 20, 40, 30, 60, 80, 70, 50 |

PYTHON

```python
def duyet_truoc(g, kq=None):
    kq = [] if kq is None else kq
    if g is not None:
        kq.append(g.gt)                # thăm gốc TRƯỚC
        duyet_truoc(g.trai, kq)
        duyet_truoc(g.phai, kq)
    return kq

def duyet_giua(g, kq=None):
    kq = [] if kq is None else kq
    if g is not None:
        duyet_giua(g.trai, kq)
        kq.append(g.gt)                # thăm gốc GIỮA
        duyet_giua(g.phai, kq)
    return kq

def duyet_sau(g, kq=None):
    kq = [] if kq is None else kq
    if g is not None:
        duyet_sau(g.trai, kq)
        duyet_sau(g.phai, kq)
        kq.append(g.gt)                # thăm gốc SAU
    return kq
```

C++

```cpp
void duyet_truoc(Nut *g, vector<int> &kq) {
    if (!g) return;
    kq.push_back(g->gt);
    duyet_truoc(g->trai, kq);
    duyet_truoc(g->phai, kq);
}

void duyet_giua(Nut *g, vector<int> &kq) {
    if (!g) return;
    duyet_giua(g->trai, kq);
    kq.push_back(g->gt);
    duyet_giua(g->phai, kq);
}

void duyet_sau(Nut *g, vector<int> &kq) {
    if (!g) return;
    duyet_sau(g->trai, kq);
    duyet_sau(g->phai, kq);
    kq.push_back(g->gt);
}
```

JULIA

```julia
function duyet_giua(g, kq = Int[])
    if g !== nothing
        duyet_giua(g.trai, kq)
        push!(kq, g.gt)
        duyet_giua(g.phai, kq)
    end
    return kq
end
```

**Nhận xét 7.2.** [Khi nào dùng cách duyệt nào] Duyệt trước dùng khi cần xử lý nút cha trước con, chẳng hạn khi sao chép cây hoặc khi ghi cây ra tệp. Duyệt giữa dùng khi cần lấy dữ liệu theo thứ tự tăng dần. Duyệt sau dùng khi cần xử lý con trước cha, chẳng hạn khi giải phóng bộ nhớ của cả cây: phải xoá hai cây con trước rồi mới xoá gốc, ngược lại sẽ mất đường đi tới chúng.

### 7.3.2 Duyệt theo mức

Cách duyệt thứ tư dùng hàng đợi thay vì đệ quy, thăm các nút theo từng mức từ trên xuống. Đây chính là tìm kiếm theo chiều rộng của mục 6.3 áp dụng cho cây.

PYTHON

```python
from collections import deque

def duyet_theo_muc(g):
    if g is None:
        return []
    kq = []
    q = deque([g])
    while q:
        u = q.popleft()
        kq.append(u.gt)
        if u.trai:
            q.append(u.trai)
        if u.phai:
            q.append(u.phai)
    return kq
```

### 7.3.3 Các thao tác đệ quy khác

PYTHON

```python
def dem_nut(g):
    return 0 if g is None else 1 + dem_nut(g.trai) + dem_nut(g.phai)

def chieu_cao(g):
    return 0 if g is None else 1 + max(chieu_cao(g.trai), chieu_cao(g.phai))

def dem_la(g):
    if g is None:
        return 0
    if g.trai is None and g.phai is None:
        return 1
    return dem_la(g.trai) + dem_la(g.phai)
```

C++

```cpp
int dem_nut(Nut *g)    { return g ? 1 + dem_nut(g->trai) + dem_nut(g->phai) : 0; }
int chieu_cao(Nut *g)  { return g ? 1 + max(chieu_cao(g->trai),
                                           chieu_cao(g->phai)) : 0; }
int dem_la(Nut *g) {
    if (!g) return 0;
    if (!g->trai && !g->phai) return 1;
    return dem_la(g->trai) + dem_la(g->phai);
}
```

Cả ba hàm đều theo cùng một khuôn: trường hợp cơ sở là cây rỗng, bước đệ quy gọi trên hai cây con rồi ghép kết quả. Khuôn này áp dụng được cho hầu hết bài toán trên cây, và độ phức tạp luôn là $O(n)$ vì mỗi nút được thăm đúng một lần.

## 7.4 Cây nhị phân tìm kiếm

**Định nghĩa 7.2.** (*Cây nhị phân tìm kiếm*) Cây nhị phân được gọi là **cây nhị phân tìm kiếm** nếu với mọi nút $v$: mọi giá trị trong cây con trái của $v$ đều nhỏ hơn giá trị của $v$, và mọi giá trị trong cây con phải đều lớn hơn.

Chú ý điều kiện áp dụng cho *toàn bộ cây con*, không chỉ hai nút con trực tiếp. Đây là chỗ hay bị hiểu nhầm khi làm bài tập kiểm tra tính hợp lệ của cây.

**Định lý 7.2.** *Duyệt giữa một cây nhị phân tìm kiếm cho dãy giá trị tăng dần.*

*Chứng minh.* Quy nạp theo số nút. Cây rỗng đúng hiển nhiên. Với cây gốc $v$, duyệt giữa cho dãy gồm ba phần nối tiếp: dãy của cây con trái, giá trị của $v$, dãy của cây con phải. Theo giả thiết quy nạp hai dãy con đều tăng dần. Theo định nghĩa, mọi giá trị trong cây con trái nhỏ hơn $v$ và mọi giá trị trong cây con phải lớn hơn $v$, nên dãy ghép lại vẫn tăng dần. ∎

### 7.4.1 Tìm kiếm

Từ gốc, so sánh với giá trị cần tìm: bằng thì xong, nhỏ hơn thì sang trái, lớn hơn thì sang phải. Mỗi bước đi xuống một mức, nên số phép so sánh không vượt quá chiều cao của cây.

PYTHON

```python
def tim(g, gt):
    while g is not None:
        if gt == g.gt:
            return True
        g = g.trai if gt < g.gt else g.phai
    return False
```

C++

```cpp
bool tim(Nut *g, int gt) {
    while (g) {
        if (gt == g->gt) return true;
        g = (gt < g->gt) ? g->trai : g->phai;
    }
    return false;
}
```

JULIA

```julia
function tim(g, gt)
    while g !== nothing
        gt == g.gt && return true
        g = gt < g.gt ? g.trai : g.phai
    end
    return false
end
```

### 7.4.2 Chèn

Đi xuống đúng như khi tìm kiếm, tới khi gặp vị trí rỗng thì tạo nút mới ở đó.

PYTHON

```python
def chen(g, gt):
    if g is None:
        return Nut(gt)                  # tạo nút mới ở chỗ rỗng
    if gt < g.gt:
        g.trai = chen(g.trai, gt)
    elif gt > g.gt:
        g.phai = chen(g.phai, gt)
    return g                            # gt == g.gt: đã có, không chèn lại
```

C++

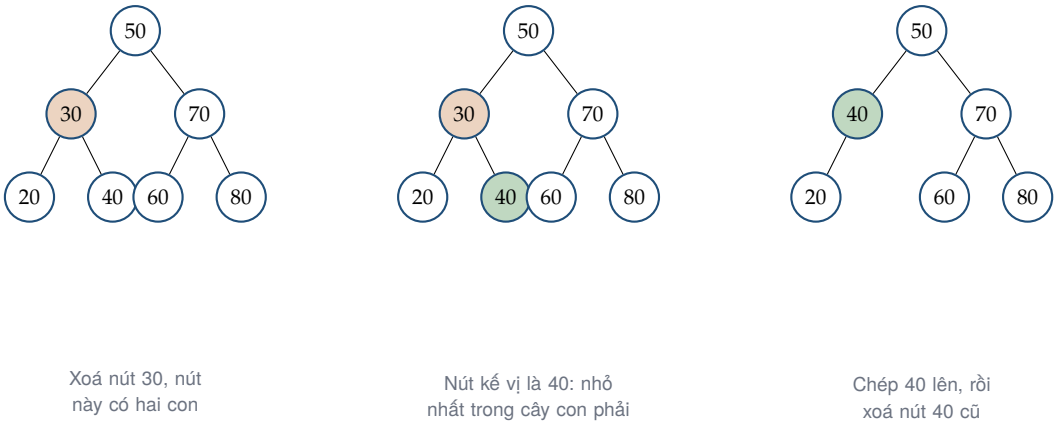
```cpp
Nut *chen(Nut *g, int gt) {
    if (!g) return new Nut(gt);
    if (gt < g->gt)      g->trai = chen(g->trai, gt);
    else if (gt > g->gt) g->phai = chen(g->phai, gt);
    return g;
}
```

> **Vì sao hàm chèn phải trả về gốc.** Hàm nhận vào gốc của một cây con và trả về gốc của cây con đó *sau khi chèn*. Khi cây con rỗng, gốc mới khác gốc cũ, nên giá trị trả về phải được gán lại vào `g.trai` hoặc `g.phai` của nút cha. Viết `chen(g.trai, gt)` mà không gán lại thì nút mới bị mất và cây không thay đổi. Cách khác là truyền tham chiếu tới con trỏ; trong C++ viết `void chen(Nut *&g, int gt)`.

### 7.4.3 Xoá

Đây là thao tác phức tạp nhất, với ba trường hợp.

1. **Nút lá.** Xoá thẳng, thay nó bằng cây rỗng.
2. **Nút có một con.** Nối nút cha thẳng vào cây con duy nhất đó.
3. **Nút có hai con.** Không xoá được ngay vì cả hai cây con đều cần chỗ. Thay vào đó, tìm nút **kế vị**: nút nhỏ nhất trong cây con phải, tức nút đi sang phải một bước rồi đi hết sang trái. Chép giá trị của nó lên nút đang xoá, rồi xoá nó khỏi cây con phải — và nút kế vị chắc chắn không có con trái nên rơi vào trường hợp 1 hoặc 2.




**Hình 7.3.** Ba bước xoá một nút có hai con khỏi cây nhị phân tìm kiếm.

**GIẢ MÃ**

```
XOA(g, gt)
1  if g = null:  return null
2  if gt < g.gt:      g.trai ← XOA(g.trai, gt)
3  else if gt > g.gt: g.phai ← XOA(g.phai, gt)
4  else                              // tìm thấy nút cần xoá
5      if g.trai = null:  return g.phai
6      if g.phai = null:  return g.trai
7      kv ← nút nhỏ nhất của g.phai
8      g.gt ← kv.gt
9      g.phai ← XOA(g.phai, kv.gt)
10 return g
```

**PYTHON**

```python
def nho_nhat(g):
    while g.trai is not None:
        g = g.trai
    return g

def xoa(g, gt):
    if g is None:
        return None
    if gt < g.gt:
        g.trai = xoa(g.trai, gt)
    elif gt > g.gt:
        g.phai = xoa(g.phai, gt)
    else:
        if g.trai is None:              # 0 hoặc 1 con
            return g.phai
        if g.phai is None:
            return g.trai
        kv = nho_nhat(g.phai)           # 2 con: dùng nút kế vị
        g.gt = kv.gt
        g.phai = xoa(g.phai, kv.gt)
    return g
```

C++

```cpp
Nut *nho_nhat(Nut *g) {
    while (g->trai) g = g->trai;
    return g;
}

Nut *xoa(Nut *g, int gt) {
    if (!g) return nullptr;
    if (gt < g->gt)      g->trai = xoa(g->trai, gt);
    else if (gt > g->gt) g->phai = xoa(g->phai, gt);
    else {
        if (!g->trai) { Nut *t = g->phai; delete g; return t; }
        if (!g->phai) { Nut *t = g->trai; delete g; return t; }
        Nut *kv = nho_nhat(g->phai);
        g->gt = kv->gt;
        g->phai = xoa(g->phai, kv->gt);
    }
    return g;
}
```

**Ví dụ 7.1.** *Ba trường hợp trên cây ở hình 7.1.*

| Thao tác | Trường hợp | Duyệt giữa sau khi xoá |
|---|---|---|
| xoá 20 | nút lá | 30, 40, 50, 60, 70, 80 |
| xoá 30 | hai con | 20, 40, 50, 60, 70, 80 |
| xoá 70 | hai con | 20, 30, 40, 50, 60, 80 |

Sau khi xoá 30, con trái của gốc trở thành nút 40. Chương trình kiểm thử chạy 2000 kịch bản chèn và xoá ngẫu nhiên, mỗi lần so sánh dãy duyệt giữa với dãy đã sắp xếp của tập giá trị tương ứng; kết quả trùng khớp ở cả ba ngôn ngữ.

### 7.4.4 Vấn đề chiều cao

Mọi thao tác trên cây nhị phân tìm kiếm đều tốn $O(h)$ với $h$ là chiều cao. Câu hỏi là $h$ bằng bao nhiêu.

**Bảng 7.3.** Chiều cao đo được khi chèn 100 giá trị khác nhau.

| Thứ tự chèn | Chiều cao |
|---|---|
| Tăng dần $1, 2, \ldots, 100$ | 100 |
| Ngẫu nhiên | 14 |
| Cây AVL, chèn tăng dần $1, 2, \ldots, 100$ | 7 |
| Giới hạn lý thuyết $\lceil \log_2 101 \rceil$ | 7 |

Dòng đầu là trường hợp xấu nhất và cũng là trường hợp rất hay gặp: dữ liệu vào đã được sắp xếp sẵn. Khi đó mỗi giá trị mới đều lớn hơn mọi giá trị đã có, nên luôn đi sang phải, và cây thoái hoá thành một danh sách liên kết chiều cao $n$. Mọi ưu điểm của cây biến mất.

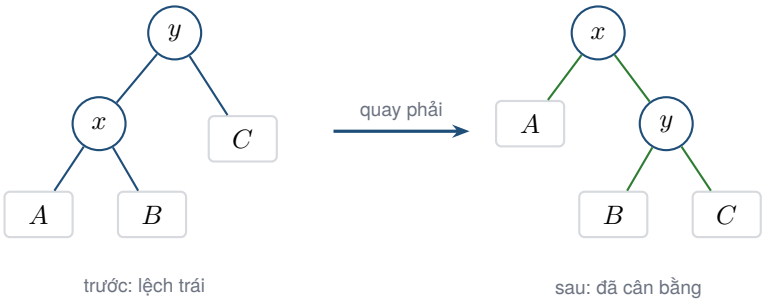
## 7.5 Cây nhị phân tìm kiếm cân bằng

Cây AVL, mang tên hai tác giả Adelson-Velsky và Landis, là cây nhị phân tìm kiếm tự giữ cân bằng bằng cách quay lại sau mỗi lần chèn.

**Định nghĩa 7.3.** (*Hệ số cân bằng*) **Hệ số cân bằng** của nút $v$ là hiệu chiều cao cây con trái trừ chiều cao cây con phải. Cây là cây AVL nếu mọi nút có hệ số cân bằng thuộc $\{-1, 0, 1\}$.

### 7.5.1 Phép quay

Khi chèn làm một nút mất cân bằng, ta khôi phục bằng phép quay: đổi lại quan hệ cha con giữa hai nút mà vẫn giữ nguyên thứ tự duyệt giữa.




Cả hai cây đều có thứ tự duyệt giữa là $A, x, B, y, C$,
nên tính chất cây nhị phân tìm kiếm được giữ nguyên.

**Hình 7.4.** Phép quay phải. Chỉ ba liên kết thay đổi, nên phép quay tốn $O(1)$.

Có bốn tình huống mất cân bằng, xác định bởi vị trí nút vừa chèn:

**Bảng 7.4.** Bốn tình huống mất cân bằng và cách xử lý.

| Tình huống | Điều kiện | Cách xử lý |
|---|---|---|
| Trái–Trái | hệ số $> 1$ và nút mới nhỏ hơn con trái | quay phải |
| Phải–Phải | hệ số $< -1$ và nút mới lớn hơn con phải | quay trái |
| Trái–Phải | hệ số $> 1$ và nút mới lớn hơn con trái | quay trái ở con trái, rồi quay phải |
| Phải–Trái | hệ số $< -1$ và nút mới nhỏ hơn con phải | quay phải ở con phải, rồi quay trái |

PYTHON

```python
class NutA:
    __slots__ = ("gt", "trai", "phai", "h")

    def __init__(self, gt):
        self.gt = gt
        self.trai = None
        self.phai = None
        self.h = 1                      # chiều cao, lá bằng 1

def h(x):
    return 0 if x is None else x.h
```

```python

def cap_nhat(x):
    x.h = 1 + max(h(x.trai), h(x.phai))

def can(x):
    return 0 if x is None else h(x.trai) - h(x.phai)

def quay_phai(y):
    x = y.trai
    t = x.phai
    x.phai = y                          # đúng thứ tự: nối x lên trước
    y.trai = t
    cap_nhat(y)                         # cập nhật y TRƯỚC vì y giờ là con
    cap_nhat(x)
    return x

def quay_trai(x):
    y = x.phai
    t = y.trai
    y.trai = x
    x.phai = t
    cap_nhat(x)
    cap_nhat(y)
    return y

def chen_avl(g, gt):
    if g is None:
        return NutA(gt)
    if gt < g.gt:
        g.trai = chen_avl(g.trai, gt)
    elif gt > g.gt:
        g.phai = chen_avl(g.phai, gt)
    else:
        return g
    cap_nhat(g)
    c = can(g)
    if c > 1 and gt < g.trai.gt:                    # Trái - Trái
        return quay_phai(g)
    if c < -1 and gt > g.phai.gt:                   # Phải - Phải
        return quay_trai(g)
    if c > 1 and gt > g.trai.gt:                    # Trái - Phải
        g.trai = quay_trai(g.trai)
        return quay_phai(g)
    if c < -1 and gt < g.phai.gt:                   # Phải - Trái
        g.phai = quay_phai(g.phai)
        return quay_trai(g)
    return g
```

C++

```cpp
struct NutA {
    int gt, h = 1;
    NutA *trai = nullptr, *phai = nullptr;
    NutA(int g) : gt(g) {}
};

int  H(NutA *x)        { return x ? x->h : 0; }
void cap_nhat(NutA *x) { x->h = 1 + max(H(x->trai), H(x->phai)); }
int  can(NutA *x)      { return x ? H(x->trai) - H(x->phai) : 0; }
```

```

NutA *quay_phai(NutA *y) {
    NutA *x = y->trai, *t = x->phai;
    x->phai = y;  y->trai = t;
    cap_nhat(y);  cap_nhat(x);
    return x;
}

NutA *quay_trai(NutA *x) {
    NutA *y = x->phai, *t = y->trai;
    y->trai = x;  x->phai = t;
    cap_nhat(x);  cap_nhat(y);
    return y;
}

NutA *chen_avl(NutA *g, int gt) {
    if (!g) return new NutA(gt);
    if (gt < g->gt)      g->trai = chen_avl(g->trai, gt);
    else if (gt > g->gt) g->phai = chen_avl(g->phai, gt);
    else return g;
    cap_nhat(g);
    int c = can(g);
    if (c >  1 && gt < g->trai->gt) return quay_phai(g);
    if (c < -1 && gt > g->phai->gt) return quay_trai(g);
    if (c >  1 && gt > g->trai->gt) { g->trai = quay_trai(g->trai);
                                      return quay_phai(g); }
    if (c < -1 && gt < g->phai->gt) { g->phai = quay_phai(g->phai);
                                      return quay_trai(g); }
    return g;
}
```

JULIA

```
mutable struct NutA
    gt::Int
    trai::Union{NutA, Nothing}
    phai::Union{NutA, Nothing}
    h::Int
end
NutA(gt) = NutA(gt, nothing, nothing, 1)

H(x) = x === nothing ? 0 : x.h
cap_nhat!(x) = (x.h = 1 + max(H(x.trai), H(x.phai)))
can(x) = x === nothing ? 0 : H(x.trai) - H(x.phai)

function quay_phai!(y)
    x = y.trai;  t = x.phai
    x.phai = y;  y.trai = t
    cap_nhat!(y);  cap_nhat!(x)
    return x
end

function quay_trai!(x)
    y = x.phai;  t = y.trai
    y.trai = x;  x.phai = t
    cap_nhat!(x);  cap_nhat!(y)
    return y
end

```

```
function chen_avl!(g, gt)
    g === nothing && return NutA(gt)
    if gt < g.gt
        g.trai = chen_avl!(g.trai, gt)
    elseif gt > g.gt
        g.phai = chen_avl!(g.phai, gt)
    else
        return g
    end
    cap_nhat!(g)
    c = can(g)
    c >  1 && gt < g.trai.gt && return quay_phai!(g)
    c < -1 && gt > g.phai.gt && return quay_trai!(g)
    if c > 1 && gt > g.trai.gt
        g.trai = quay_trai!(g.trai);  return quay_phai!(g)
    end
    if c < -1 && gt < g.phai.gt
        g.phai = quay_phai!(g.phai);  return quay_trai!(g)
    end
    return g
end
```

**Thứ tự cập nhật chiều cao.** Trong hàm quay, phải cập nhật chiều cao của nút *xuống dưới* trước rồi mới tới nút *lên trên*. Ở `quay_phai`, sau khi quay thì $y$ là con của $x$, nên phải gọi `cap_nhat(y)` trước `cap_nhat(x)`. Đảo thứ tự thì chiều cao của $x$ tính từ giá trị cũ của $y$ và sai; cây vẫn hoạt động nhưng dần mất cân bằng mà không có dấu hiệu gì.

**Ví dụ 7.2.** *Chiều cao đo được.*

| $n$ | Chiều cao AVL | $\log_2 n$ |
|---:|---:|---:|
| 10 | 4 | 3,32 |
| 100 | 7 | 6,64 |
| 1 000 | 10 | 9,97 |
| 10 000 | 14 | 13,29 |
| 100 000 | 17 | 16,61 |

Mọi phép chèn ở đây đều theo thứ tự tăng dần, tức trường hợp xấu nhất của cây nhị phân tìm kiếm thường. Chiều cao AVL luôn nhỏ hơn $1{,}44 \log_2 n$, đúng với chặn lý thuyết.

**Bảng 7.5.** So sánh ba cấu trúc lưu tập giá trị có thứ tự.

| | Mảng đã sắp xếp | Cây tìm kiếm thường | Cây AVL |
|---|---|---|---|
| Tìm kiếm | $O(\log n)$ | $O(h)$ | $O(\log n)$ |
| Chèn | $O(n)$ | $O(h)$ | $O(\log n)$ |
| Xoá | $O(n)$ | $O(h)$ | $O(\log n)$ |
| Chiều cao xấu nhất | — | $n$ | $1{,}44 \log_2 n$ |

**Nhận xét 7.3.** Trong thư viện chuẩn, `std::map` và `std::set` của C++ được cài bằng cây đỏ đen, một loại cây cân bằng khác có cùng độ phức tạp $O(\log n)$ nhưng ít phép quay hơn khi chèn và xoá. Sinh viên trong học phần này chỉ cần hiểu AVL; cây đỏ đen là nội dung của học phần nâng cao.

**Bài 7.1.** Cài đặt cây nhị phân tìm kiếm đầy đủ bằng ba ngôn ngữ: chèn, tìm kiếm, xoá cả ba trường hợp, ba cách duyệt, đếm nút, đếm lá, tính chiều cao. Viết chương trình kiểm thử tự động: sinh ngẫu nhiên một tập giá trị, chèn vào cây theo thứ tự ngẫu nhiên, so sánh dãy duyệt giữa với tập đã sắp xếp; rồi xoá ngẫu nhiên một giá trị và so sánh lại. Chạy 2000 kịch bản.
Sau đó cài AVL, chèn $1, 2, \ldots, n$ với $n = 10, 10^2, 10^3, 10^4, 10^5$ và lập bảng chiều cao đối chiếu với bảng trong ví dụ trên.

## 7.6 Cây nhị phân gần đầy và đống

**Định nghĩa 7.4.** (*Đống*) **Đống** (*heap*) là cây nhị phân gần đầy thoả tính chất đống: giá trị của mỗi nút không nhỏ hơn giá trị của hai con (đống cực đại), hoặc không lớn hơn (đống cực tiểu).

Chú ý khác biệt so với cây nhị phân tìm kiếm: đống chỉ ràng buộc quan hệ giữa cha và con, không ràng buộc gì giữa hai anh em. Vì vậy duyệt giữa một đống không cho dãy có thứ tự.

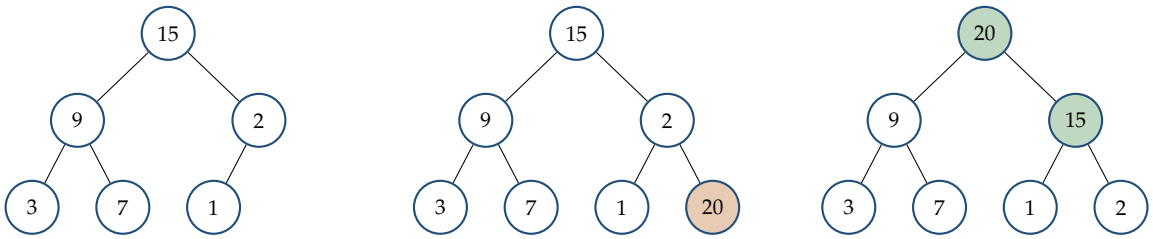
Vì đống luôn gần đầy, nó được lưu bằng mảng theo cách ở mục 7.2, không cần con trỏ.

### 7.6.1 Hai thao tác cơ bản

**Thêm phần tử.** Đặt giá trị mới vào cuối mảng, rồi *đẩy lên*: so sánh với cha, nếu lớn hơn thì đổi chỗ, lặp lại cho tới khi đúng chỗ hoặc tới gốc.

**Lấy phần tử lớn nhất.** Giá trị lớn nhất luôn ở gốc. Lấy nó ra, chuyển phần tử cuối mảng lên gốc, rồi *đẩy xuống*: so sánh với hai con, nếu nhỏ hơn con lớn hơn thì đổi chỗ, lặp lại.

Cả hai thao tác đi dọc một đường từ gốc xuống lá hoặc ngược lại, nên đều tốn $O(\log n)$.




Trước khi thêm 20

Đặt 20 vào cuối, vi phạm tính chất đống

Đẩy lên hai lần, đống đã đúng

**Hình 7.5.** Thêm giá trị 20 vào đống cực đại. Phần tử mới đi lên dọc theo đường từ lá về gốc.

PYTHON

```python
class Dong:
    def __init__(self):
        self.a = []

    def them(self, x):
```

```
        self.a.append(x)
        i = len(self.a) - 1
        while i > 0 and self.a[(i - 1) // 2] < self.a[i]:    # đẩy lên
            cha = (i - 1) // 2
            self.a[cha], self.a[i] = self.a[i], self.a[cha]
            i = cha

    def lay(self):
        if not self.a:
            return None
        goc = self.a[0]
        self.a[0] = self.a[-1]
        self.a.pop()
        i, n = 0, len(self.a)
        while True:                                          # đẩy xuống
            trai, phai, lon = 2 * i + 1, 2 * i + 2, i
            if trai < n and self.a[trai] > self.a[lon]:
                lon = trai
            if phai < n and self.a[phai] > self.a[lon]:
                lon = phai
            if lon == i:
                break
            self.a[i], self.a[lon] = self.a[lon], self.a[i]
            i = lon
        return goc
```

C++

```
struct Dong {
    vector<int> a;

    void them(int x) {
        a.push_back(x);
        int i = a.size() - 1;
        while (i > 0 && a[(i - 1) / 2] < a[i]) {              // đẩy lên
            swap(a[(i - 1) / 2], a[i]);
            i = (i - 1) / 2;
        }
    }

    int lay() {
        int goc = a[0];
        a[0] = a.back();
        a.pop_back();
        int i = 0, n = a.size();
        while (true) {                                       // đẩy xuống
            int trai = 2*i + 1, phai = 2*i + 2, lon = i;
            if (trai < n && a[trai] > a[lon]) lon = trai;
            if (phai < n && a[phai] > a[lon]) lon = phai;
            if (lon == i) break;
            swap(a[i], a[lon]);
            i = lon;
        }
        return goc;
    }
};
// Thư viện chuẩn: priority_queue<int> pq;  pq.push(x);  pq.top();  pq.pop();
```

JULIA

```julia
mutable struct Dong
    a::Vector{Int}
end
Dong() = Dong(Int[])

function them!(d::Dong, x)
    push!(d.a, x)
    i = length(d.a)
    while i > 1 && d.a[i ÷ 2] < d.a[i]          # Julia: cha của i là i ÷ 2
        d.a[i ÷ 2], d.a[i] = d.a[i], d.a[i ÷ 2]
        i = i ÷ 2
    end
    return d
end

function lay!(d::Dong)
    isempty(d.a) && return nothing
    goc = d.a[1]
    d.a[1] = d.a[end]
    pop!(d.a)
    i, n = 1, length(d.a)
    while true
        trai, phai, lon = 2i, 2i + 1, i
        trai <= n && d.a[trai] > d.a[lon] && (lon = trai)
        phai <= n && d.a[phai] > d.a[lon] && (lon = phai)
        lon == i && break
        d.a[i], d.a[lon] = d.a[lon], d.a[i]
        i = lon
    end
    return goc
end
```

**Ví dụ 7.3.** *Lần vết*. Thêm lần lượt các giá trị $3, 9, 2, 15, 7, 1, 20$ vào đống cực đại:

| Thêm | Nội dung mảng |
|---|---|
| 3 | `[3]` |
| 9 | `[9, 3]` |
| 2 | `[9, 3, 2]` |
| 15 | `[15, 9, 2, 3]` |
| 7 | `[15, 9, 2, 3, 7]` |
| 1 | `[15, 9, 2, 3, 7, 1]` |
| 20 | `[20, 9, 15, 3, 7, 1, 2]` |

Lấy ra liên tiếp bảy lần cho dãy $20, 15, 9, 7, 3, 2, 1$, tức đúng thứ tự giảm dần. Chương trình kiểm thử 2000 dãy ngẫu nhiên xác nhận điều này ở cả ba ngôn ngữ.

**Nhận xét 7.4.** [Sắp xếp vun đống] Ví dụ trên gợi ra một thuật toán sắp xếp: đưa toàn bộ $n$ phần tử vào đống rồi lấy ra liên tiếp. Mỗi thao tác tốn $O(\log n)$, tổng cộng $O(n \log n)$, không cần bộ nhớ phụ nếu dùng chính mảng đầu vào làm đống. Đây là thuật toán **sắp xếp vun đống**, thuật toán $O(n \log n)$ duy nhất vừa chạy tại chỗ vừa không có trường hợp xấu nhất $O(n^2)$.

Đống cũng chính là cấu trúc dùng để cài hàng đợi ưu tiên trong thuật toán Dijkstra và A sao ở chương 6, và trong hàm `partial_sort` đã nhắc tới ở mục 2.7.

## 7.7 Ứng dụng: cây quyết định và thuật toán chơi cờ

Mục này trình bày hai ứng dụng của cây trong trí tuệ nhân tạo. Cây quyết định là mô hình học máy dễ giải thích nhất; minimax là thuật toán nền của mọi chương trình chơi cờ.

### 7.7.1 Cây quyết định

**Ý tưởng.** Mỗi nút trong của cây hỏi một câu về một thuộc tính của dữ liệu, mỗi nhánh ứng với một câu trả lời, mỗi lá cho một kết luận. Phân loại một mẫu mới nghĩa là đi từ gốc xuống lá theo các câu trả lời.

Khác với mạng nơ-ron, cây quyết định giải thích được: đường đi từ gốc tới lá chính là chuỗi lý do dẫn tới kết luận. Đó là lý do cây quyết định vẫn được dùng trong y tế và trong tài chính, những lĩnh vực đòi hỏi giải trình.

**Câu hỏi trung tâm.** Ở mỗi nút, chọn thuộc tính nào để hỏi? Tiêu chuẩn phổ biến là chỉ số Gini.

**Định nghĩa 7.5.** (*Chỉ số Gini*) Cho một nhóm mẫu, gọi $p_i$ là tỉ lệ mẫu thuộc lớp $i$. **Chỉ số Gini** của nhóm là

$$\text{Gini} = 1 - \sum_i p_i^2.$$

Chỉ số Gini bằng $0$ khi nhóm thuần nhất, tức mọi mẫu cùng một lớp, và đạt lớn nhất khi các lớp chia đều. Với hai lớp, giá trị lớn nhất là $0{,}5$.

Khi chia một nhóm theo một thuộc tính, chỉ số Gini của phép chia là trung bình có trọng số của các nhóm con:

$$\text{Gini}_{\text{chia}} = \sum_k \frac{n_k}{n}\,\text{Gini}_k.$$

Thuộc tính cho $\text{Gini}_{\text{chia}}$ nhỏ nhất được chọn.

**Bảng 7.6.** Tập dữ liệu huấn luyện: có đi chơi hay không.

| Mẫu | Thời tiết | Gió | Đi chơi |
|---|---|---|---|
| 1 | nắng | nhẹ | có |
| 2 | nắng | mạnh | không |
| 3 | mây | nhẹ | có |
| 4 | mây | mạnh | có |
| 5 | mưa | nhẹ | không |
| 6 | mưa | mạnh | không |
| 7 | nắng | nhẹ | có |
| 8 | mưa | mạnh | không |

**Ví dụ 7.4.** *Tính chỉ số Gini.* Tập dữ liệu có 4 mẫu "có" và 4 mẫu "không", nên

$$\text{Gini}_{\text{ban đầu}} = 1 - \left(\tfrac{4}{8}\right)^2 - \left(\tfrac{4}{8}\right)^2 = \tfrac{1}{2} = 0{,}5.$$

**Chia theo thời tiết:**

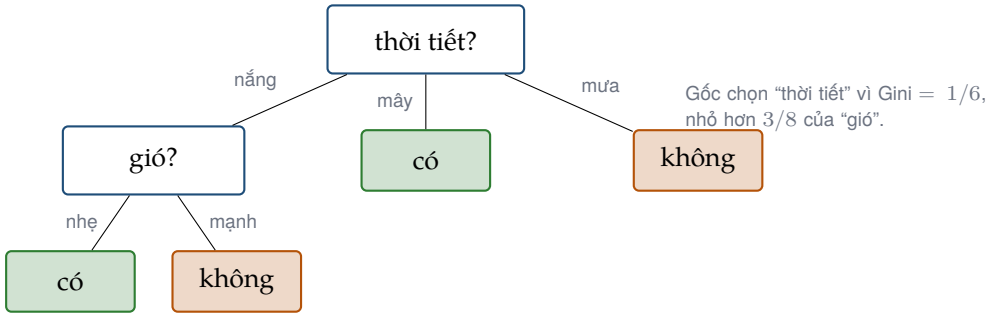
| Giá trị | Số mẫu | Nhãn | Gini |
|---|---|---|---|
| nắng | 3 | có, không, có | $4/9 \approx 0{,}4444$ |
| mây | 2 | có, có | 0 |
| mưa | 3 | không, không, không | 0 |
| Trung bình có trọng số | | | $\frac{3}{8}\cdot\frac{4}{9} = \frac{1}{6} \approx 0{,}1667$ |

**Chia theo gió:**

| Giá trị | Số mẫu | Nhãn | Gini |
|---|---|---|---|
| nhẹ | 4 | có, có, không, có | $3/8 = 0{,}375$ |
| mạnh | 4 | không, có, không, không | $3/8 = 0{,}375$ |
| Trung bình có trọng số | | | $3/8 = 0{,}375$ |

Vì $0{,}1667 < 0{,}375$, thuộc tính "thời tiết" được chọn làm gốc.

Sau khi chia theo thời tiết, hai nhánh "mây" và "mưa" đã thuần nhất nên trở thành lá. Nhánh "nắng" còn ba mẫu với Gini $= 4/9$, phải chia tiếp: chia theo gió cho hai nhóm thuần nhất, Gini bằng 0.




**Hình 7.6.** Cây quyết định học được từ tập dữ liệu ở bảng 7.6.

PYTHON

```python
def gini(nhan):
    n = len(nhan)
    if n == 0:
        return 0.0
    dem = {}
    for x in nhan:
        dem[x] = dem.get(x, 0) + 1
    return 1 - sum((c / n) ** 2 for c in dem.values())

def gini_chia(dl, cot):
    n = len(dl)
    nhom = {}
    for hang in dl:
        nhom.setdefault(hang[cot], []).append(hang[-1])
    return sum(len(nh) / n * gini(nh) for nh in nhom.values())

def xay_cay(dl, cac_cot):
    nhan = [h[-1] for h in dl]
    if len(set(nhan)) == 1:                    # đã thuần nhất
        return ("la", nhan[0])
```

```
    if not cac_cot:                              # hết thuộc tính để hỏi
        return ("la", max(set(nhan), key=nhan.count))
    tot = min(cac_cot, key=lambda c: gini_chia(dl, c))
    con = {}
    for gt in {h[tot] for h in dl}:
        con_dl = [h for h in dl if h[tot] == gt]
        con[gt] = xay_cay(con_dl, [c for c in cac_cot if c != tot])
    return ("hoi", tot, con)

def phan_loai(cay, mau):
    while cay[0] == "hoi":
        _, cot, con = cay
        cay = con[mau[cot]]
    return cay[1]
```

**JULIA**

```
function gini(nhan)
    n = length(nhan)
    n == 0 && return 0.0
    dem = Dict{String,Int}()
    for x in nhan
        dem[x] = get(dem, x, 0) + 1
    end
    return 1 - sum((c / n)^2 for c in values(dem))
end

function gini_chia(dl, cot)
    n = length(dl)
    nhom = Dict{String, Vector{String}}()
    for hang in dl
        push!(get!(nhom, hang[cot], String[]), hang[end])
    end
    return sum(length(nh) / n * gini(nh) for nh in values(nhom))
end
```

**C++**

```
double gini(const vector<string> &nhan) {
    int n = nhan.size();
    if (n == 0) return 0.0;
    map<string,int> dem;
    for (auto &x : nhan) dem[x]++;
    double s = 0;
    for (auto &[k, c] : dem) s += (double)c / n * c / n;
    return 1 - s;
}

double gini_chia(const vector<vector<string>> &dl, int cot) {
    int n = dl.size();
    map<string, vector<string>> nhom;
    for (auto &hang : dl) nhom[hang[cot]].push_back(hang.back());
    double tong = 0;
    for (auto &[gt, nh] : nhom) tong += (double)nh.size() / n * gini(nh);
    return tong;
}
```

**Hiện tượng quá khớp.** Nếu để cây mọc tới khi mọi lá đều thuần nhất, cây sẽ nhớ thuộc lòng tập huấn luyện và hoạt động kém trên dữ liệu mới. Hiện tượng đó gọi là **quá khớp** (*overfitting*). Hai cách hạn chế là giới hạn chiều cao cây, và cắt tỉa các nhánh không cải thiện độ chính xác trên một tập kiểm tra riêng.

### 7.7.2 Thuật toán minimax

**Bài toán.** Trong trò chơi hai người luân phiên đi, thông tin đầy đủ, tổng bằng không — cờ ca-rô, cờ vua, cờ vây — hãy chọn nước đi tốt nhất.

**Mô hình.** Mọi thế cờ có thể xảy ra tạo thành một cây: gốc là thế hiện tại, mỗi nhánh là một nước đi, mỗi lá là một thế kết thúc có giá trị xác định.

**Ý tưởng.** Hai người chơi có mục tiêu trái ngược. Người thứ nhất muốn giá trị lớn nhất, người thứ hai muốn nhỏ nhất. Giá trị của một nút tính từ giá trị các nút con: lấy giá trị lớn nhất nếu tới lượt người thứ nhất, lấy nhỏ nhất nếu tới lượt người thứ hai.

GIẢ MÃ

```
MINIMAX(thế cờ, lượt_của_người_lớn)
1  if thế cờ đã kết thúc
2      return giá trị của thế cờ
3  if lượt_của_người_lớn
4      tot ← -∞
5      for mỗi nước đi hợp lệ
6          đi nước đó
7          tot ← max(tot, MINIMAX(thế mới, false))
8          rút nước đi lại
9      return tot
10 else
11     tot ← +∞
12     for mỗi nước đi hợp lệ
13         đi nước đó
14         tot ← min(tot, MINIMAX(thế mới, true))
15         rút nước đi lại
16     return tot
```

### 7.7.3 Cắt tỉa alpha-beta

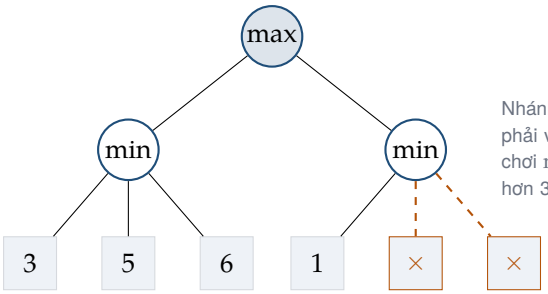
Số nút của cây trò chơi tăng theo hàm mũ. Cờ ca-rô $3 \times 3$ đã có gần 550 000 nút; cờ vua có cỡ $10^{120}$ nút, nhiều hơn số nguyên tử trong vũ trụ. Vì vậy phải cắt bỏ.

**Ý tưởng.** Giữ hai giá trị dọc đường đi: $\alpha$ là giá trị tốt nhất mà người chơi lớn đã bảo đảm được, $\beta$ là giá trị tốt nhất mà người chơi nhỏ đã bảo đảm được. Nếu tại một nút ta phát hiện $\beta \le \alpha$, nghĩa là nhánh này không thể tốt hơn nhánh đã xét, thì dừng ngay, không xét nốt các nước đi còn lại.

PYTHON

```python
def thang(b, p):
    HANG = [(0,1,2), (3,4,5), (6,7,8), (0,3,6), (1,4,7), (2,5,8),
            (0,4,8), (2,4,6)]
    return any(b[i] == b[j] == b[k] == p for i, j, k in HANG)

def minimax(b, la_X):
    if thang(b, "X"): return 1
    if thang(b, "O"): return -1
```




Nhánh trái cho giá trị 3, nên $\alpha = 3$. Nhánh phải vừa gặp giá trị 1, nên $\beta = 1 \leq \alpha$: người chơi $\min$ chắc chắn không cho quá 1, kém hơn 3. Hai nút còn lại không cần xét.

**Hình 7.7.** Một lần cắt tỉa alpha-beta. Hai nút đánh dấu không bao giờ được sinh ra.

```
    if all(c != " " for c in b): return 0
    if la_X:
        tot = -2
        for i in range(9):
            if b[i] == " ":
                b[i] = "X"
                tot = max(tot, minimax(b, False))
                b[i] = " "
        return tot
    else:
        tot = 2
        for i in range(9):
            if b[i] == " ":
                b[i] = "O"
                tot = min(tot, minimax(b, True))
                b[i] = " "
        return tot

def alpha_beta(b, la_X, alpha, beta):
    if thang(b, "X"): return 1
    if thang(b, "O"): return -1
    if all(c != " " for c in b): return 0
    if la_X:
        tot = -2
        for i in range(9):
            if b[i] == " ":
                b[i] = "X"
                tot = max(tot, alpha_beta(b, False, alpha, beta))
                b[i] = " "
                alpha = max(alpha, tot)
                if beta <= alpha:
                    break                       # cắt nhánh
        return tot
    else:
        tot = 2
        for i in range(9):
            if b[i] == " ":
                b[i] = "O"
                tot = min(tot, alpha_beta(b, True, alpha, beta))
                b[i] = " "
                beta = min(beta, tot)
                if beta <= alpha:
                    break                       # cắt nhánh
        return tot
```

C++

```cpp
int alpha_beta(string &b, bool la_X, int alpha, int beta) {
    if (thang(b, 'X')) return  1;
    if (thang(b, 'O')) return -1;
    if (b.find(' ') == string::npos) return 0;
    if (la_X) {
        int tot = -2;
        for (int i = 0; i < 9; i++)
            if (b[i] == ' ') {
                b[i] = 'X';
                tot = max(tot, alpha_beta(b, false, alpha, beta));
                b[i] = ' ';
                alpha = max(alpha, tot);
                if (beta <= alpha) break;       // cắt nhánh
            }
        return tot;
    } else {
        int tot = 2;
        for (int i = 0; i < 9; i++)
            if (b[i] == ' ') {
                b[i] = 'O';
                tot = min(tot, alpha_beta(b, true, alpha, beta));
                b[i] = ' ';
                beta = min(beta, tot);
                if (beta <= alpha) break;       // cắt nhánh
            }
        return tot;
    }
}
// Gọi lần đầu: alpha_beta(b, true, -2, 2);
```

JULIA

```julia
function alpha_beta(b, la_X, alpha, beta)
    thang(b, 'X') && return  1
    thang(b, 'O') && return -1
    all(c -> c != ' ', b) && return 0
    if la_X
        tot = -2
        for i in 1:9
            if b[i] == ' '
                b[i] = 'X'
                tot = max(tot, alpha_beta(b, false, alpha, beta))
                b[i] = ' '
                alpha = max(alpha, tot)
                beta <= alpha && break          # cắt nhánh
            end
        end
        return tot
    else
        tot = 2
        for i in 1:9
            if b[i] == ' '
                b[i] = 'O'
                tot = min(tot, alpha_beta(b, true, alpha, beta))
                b[i] = ' '
                beta = min(beta, tot)
                beta <= alpha && break          # cắt nhánh
            end
```

```
            end
            return tot
        end
    end
```

### 7.7.4 Đo hiệu quả cắt tỉa

Cả hai chương trình chạy trên bàn cờ ca-rô $3 \times 3$ trống, người chơi X đi trước.

**Bảng 7.7.** Số nút phải sinh ra khi giải cờ ca-rô $3 \times 3$ từ bàn cờ trống.

| Thuật toán | Số nút | Giá trị trả về |
|---|---|---|
| Minimax | 549 946 | 0 |
| Minimax có cắt tỉa alpha-beta | 18 297 | 0 |

Cắt tỉa loại bỏ 96,7 phần trăm số nút, tương ứng nhanh hơn 30,1 lần, mà *kết quả không đổi*: cả hai đều trả về 0, nghĩa là nếu hai bên chơi đúng thì ván cờ hoà. Đây là điểm quan trọng nhất của kỹ thuật này: nó không phải một phép xấp xỉ mà là một phép cắt bỏ chính xác những nhánh chắc chắn không ảnh hưởng tới kết quả.

**Nhận xét 7.5.** [Thứ tự nước đi] Hiệu quả cắt tỉa phụ thuộc mạnh vào thứ tự xét các nước đi. Trong trường hợp lý tưởng, khi nước đi tốt nhất luôn được xét trước, alpha-beta xét khoảng $O(b^{d/2})$ nút thay vì $O(b^d)$ — nghĩa là với cùng thời gian, chương trình nhìn xa được *gấp đôi* số nước. Vì vậy mọi chương trình cờ đều đầu tư nhiều vào việc sắp thứ tự nước đi trước khi tìm kiếm.

Với các trò chơi lớn hơn, không thể duyệt tới lá. Chương trình dừng ở một độ sâu cố định và dùng **hàm lượng giá** để ước lượng giá trị thế cờ, ví dụ tổng giá trị quân cờ cộng các yếu tố về vị trí. Toàn bộ khác biệt giữa các chương trình cờ vua nằm ở chất lượng hàm lượng giá và ở kỹ thuật sắp thứ tự nước đi, chứ không nằm ở thuật toán minimax vốn giống nhau.

**Bài 7.2.** Cài đặt hai phần sau.
**Phần 1 — cây quyết định.**

1. Nạp tập dữ liệu ở bảng 7.6.
2. Viết hàm `gini` và kiểm tra: chỉ số Gini của toàn tập phải bằng $0{,}5$.
3. Viết hàm `gini_chia` và kiểm tra: chia theo thời tiết cho $1/6 \approx 0{,}1667$, chia theo gió cho $3/8 = 0{,}375$.
4. Xây cây bằng đệ quy và in nó ra màn hình theo dạng thụt đầu dòng. Kết quả phải trùng hình 7.6.
5. Phân loại mẫu mới (nắng, mạnh) và (mây, nhẹ).

**Phần 2 — cờ ca-rô.**

1. Cài minimax và alpha-beta dùng chung hàm kiểm tra thắng thua.
2. Thêm biến đếm số nút, chạy từ bàn cờ trống, xác nhận 549 946 và 18 297.
3. Kiểm tra hai thuật toán luôn trả về cùng giá trị trên 100 thế cờ ngẫu nhiên sinh bằng cách đi ngẫu nhiên từ 1 đến 5 nước.
4. Viết giao diện dòng lệnh cho người chơi với máy và xác nhận không bao giờ thắng được

máy.

**Tóm tắt.** Cây nhị phân chiều cao $h$ có nhiều nhất $2^h - 1$ nút, nên mọi thao tác đi từ gốc xuống lá tốn ít nhất $\Omega(\log n)$. Cây nhị phân tìm kiếm đạt được $O(h)$ cho cả ba thao tác tìm, chèn, xoá, nhưng $h$ có thể bằng $n$ khi dữ liệu vào đã sắp xếp — số đo cho chiều cao 100 khi chèn $1..100$ theo thứ tự tăng. Cây AVL giữ $h = O(\log n)$ bằng các phép quay $O(1)$, cho chiều cao 7 trên cùng dữ liệu đó. Đống là cây gần đầy lưu bằng mảng, cho hàng đợi ưu tiên $O(\log n)$ dùng trong Dijkstra và A sao. Trong trí tuệ nhân tạo, cây quyết định chọn thuộc tính theo chỉ số Gini, còn minimax với cắt tỉa alpha-beta giảm số nút của cờ ca-rô từ 549 946 xuống 18 297 mà không đổi kết quả.

## 7.8 Bài tập chương 7

### Câu hỏi ôn tập

1. Định nghĩa cây nhị phân. Vì sao định nghĩa này dẫn tới việc mọi thuật toán trên cây đều tự nhiên viết bằng đệ quy?
2. Phân biệt cây nhị phân đủ, cây nhị phân đầy và cây gần đầy.
3. Chứng minh cây nhị phân chiều cao $h$ có nhiều nhất $2^h - 1$ nút.
4. Nêu ba công thức tính chỉ số con trái, con phải và cha khi lưu cây bằng mảng. Chúng đổi thế nào khi đánh chỉ số từ 1?
5. Vì sao chỉ nên lưu cây gần đầy bằng mảng?
6. Nêu ba cách duyệt cây và cho biết cách nào phù hợp với việc giải phóng bộ nhớ của cả cây, vì sao.
7. Phát biểu định nghĩa cây nhị phân tìm kiếm. Vì sao điều kiện phải áp dụng cho toàn bộ cây con chứ không chỉ hai nút con?
8. Chứng minh duyệt giữa cây nhị phân tìm kiếm cho dãy tăng dần.
9. Nêu ba trường hợp khi xoá một nút. Vì sao trường hợp hai con lại quy về được hai trường hợp còn lại?
10. Vì sao hàm chèn phải trả về gốc của cây con?
11. Trường hợp nào làm cây nhị phân tìm kiếm thoái hoá thành danh sách? Đây có phải trường hợp hiếm không?
12. Định nghĩa hệ số cân bằng và cây AVL.
13. Vì sao phép quay giữ nguyên tính chất cây nhị phân tìm kiếm?
14. Nêu bốn tình huống mất cân bằng và cách xử lý từng tình huống.
15. Trong hàm quay, vì sao thứ tự cập nhật chiều cao lại quan trọng?
16. Phân biệt tính chất đống và tính chất cây nhị phân tìm kiếm.
17. Vì sao cả hai thao tác trên đống đều là $O(\log n)$?
18. Chỉ số Gini bằng bao nhiêu khi nhóm thuần nhất, và bằng bao nhiêu khi hai lớp chia đều?

19. Vì sao cắt tỉa alpha-beta không làm thay đổi kết quả của minimax?

20. Vì sao thứ tự xét các nước đi lại ảnh hưởng mạnh tới hiệu quả cắt tỉa?

### Bài tập tính toán

21. Vẽ cây nhị phân tìm kiếm khi chèn lần lượt $45, 15, 79, 90, 12, 38, 60, 25$. Ghi ba dãy duyệt và chiều cao.

22. Với cây ở bài 21, vẽ cây sau khi xoá lần lượt 12, 15, 45, ghi rõ trường hợp nào.

23. Cho dãy duyệt trước $50, 30, 20, 40, 70, 60, 80$ và dãy duyệt giữa $20, 30, 40, 50, 60, 70, 80$. Khôi phục cây.

24. Chèn lần lượt $10, 20, 30, 40, 50, 25$ vào cây AVL, vẽ cây sau mỗi phép chèn và ghi rõ phép quay được dùng. Kết quả cuối phải có gốc bằng 30 và chiều cao 3.

25. Cho mảng đống $[20, 9, 15, 3, 7, 1, 2]$. Vẽ cây tương ứng, kiểm tra tính chất đống, rồi vẽ mảng sau khi thêm 18 và sau khi lấy ra phần tử lớn nhất.

26. Tính chỉ số Gini của các nhóm sau: $(5$ có, $0$ không$)$, $(3$ có, $3$ không$)$, $(4$ có, $1$ không$)$.

27. Với tập dữ liệu ở bảng 7.6, tính lại toàn bộ chỉ số Gini bằng tay và xác nhận các giá trị $1/2$, $1/6$, $3/8$.

28. Cho cây trò chơi ba mức với các lá lần lượt là $3, 12, 8, 2, 4, 6, 14, 5, 2$. Tính giá trị minimax của gốc và đánh dấu các nhánh bị alpha-beta cắt bỏ khi duyệt từ trái sang phải.

### Bài tập lập trình mức cơ bản

29. Cài đặt cây nhị phân tìm kiếm đầy đủ theo bài thực hành 7.1.

30. Viết hàm kiểm tra một cây nhị phân có phải cây tìm kiếm không, độ phức tạp $O(n)$. Chú ý điều kiện áp dụng cho toàn bộ cây con.

31. Viết hàm tìm giá trị nhỏ nhất và lớn nhất trong cây nhị phân tìm kiếm.

32. Viết hàm đếm số nút có đúng một con.

33. Viết hàm tính tổng giá trị mọi nút và tổng giá trị mọi lá.

34. Viết hàm in cây ra màn hình theo dạng thụt đầu dòng, mỗi mức thụt thêm bốn dấu cách.

35. Cài đặt ba cách duyệt bằng vòng lặp và ngăn xếp, không dùng đệ quy.

36. Cài đặt đống cực tiểu và dùng nó để cài lại thuật toán Dijkstra của chương 6.

### Bài tập lập trình mức vận dụng

37. Khôi phục cây nhị phân từ hai dãy duyệt: duyệt trước và duyệt giữa.

38. Cài đặt AVL đầy đủ, gồm cả thao tác xoá có cân bằng lại.

39. Đo và so sánh thời gian của $10^6$ phép chèn rồi $10^6$ phép tìm kiếm trên ba cấu trúc: cây nhị phân tìm kiếm thường, cây AVL, và `std::set`. Chạy với hai loại dữ liệu: tăng dần và ngẫu nhiên.

40. Cài đặt sắp xếp vun đống tại chỗ và so sánh thời gian với sắp xếp nhanh ở chương 2 trên $10^6$ phần tử.

41. Cài đặt cây Fenwick hoặc cây phân đoạn để trả lời truy vấn tổng đoạn và cập nhật một phần tử, mỗi thao tác $O(\log n)$.

42. Cài đặt mã Huffman bằng đống cực tiểu và nén một tệp văn bản tiếng Việt, báo cáo tỉ lệ nén.

43. Tìm tổ tiên chung gần nhất của hai nút trong cây nhị phân tìm kiếm và trong cây nhị phân tổng quát.

44. Kiểm tra hai cây nhị phân có giống nhau về hình dạng và giá trị không; sau đó kiểm tra một cây có phải cây con của cây kia không.

### Bài tập phần trí tuệ nhân tạo

45. Cài đặt cây quyết định theo phần 1 của bài thực hành 7.2.

46. Thay chỉ số Gini bằng entropy $-\sum_i p_i \log_2 p_i$ và độ lợi thông tin. So sánh cây thu được với cây dùng Gini trên cùng dữ liệu.

47. Bổ sung xử lý thuộc tính liên tục: với mỗi thuộc tính số, thử mọi ngưỡng chia và chọn ngưỡng cho Gini nhỏ nhất.

48. Thêm cơ chế giới hạn chiều cao cây và đo độ chính xác trên tập kiểm tra khi chiều cao tối đa thay đổi từ 1 đến 10, để thấy rõ hiện tượng quá khớp.

49. Cài đặt rừng ngẫu nhiên đơn giản: xây 100 cây quyết định, mỗi cây học trên một mẫu con ngẫu nhiên của dữ liệu, rồi cho chúng bỏ phiếu. So sánh độ chính xác với một cây đơn lẻ.

50. Cài đặt minimax và alpha-beta theo phần 2 của bài thực hành 7.2, xác nhận các số 549 946 và 18 297.

51. Thêm cơ chế sắp thứ tự nước đi: ưu tiên ô giữa, rồi bốn góc, rồi bốn cạnh. Đo lại số nút của alpha-beta và so sánh với 18 297.

52. Mở rộng chương trình sang cờ ca-rô $4 \times 4$ thắng 4 quân, giới hạn độ sâu tìm kiếm và viết một hàm lượng giá đơn giản. Đo số nút ở các độ sâu 3, 5, 7 với và không có cắt tỉa.

53. Cài đặt bảng ghi nhớ thế cờ đã tính, một dạng quy hoạch động trên cây trò chơi. Đo số nút giảm thêm bao nhiêu so với alpha-beta thuần.

# Tài liệu tham khảo


**Tài liệu chính**

[1] Nguyễn Duy Phương, Nguyễn Mạnh Sơn. *Bài giảng Cấu trúc dữ liệu và Giải thuật*. Học viện Công nghệ Bưu chính Viễn thông, 2020.

**Tài liệu tham khảo**

[2] Donald E. Knuth. *The Art of Computer Programming*, Vol. 1–4. Addison-Wesley.[1]

[3] Đinh Mạnh Tường. *Cấu trúc dữ liệu và giải thuật*. Nhà xuất bản Khoa học và Kỹ thuật, Hà Nội, 2008.

[4] Steven Halim, Felix Halim. *Competitive Programming 3*, 2013.

**Tài liệu bổ sung dùng khi biên soạn**

[5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*. MIT Press.

[6] Stuart Russell, Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson.


**Về môi trường chạy các chương trình trong tài liệu**

- C++: biên dịch bằng `g++` chuẩn C++17, tuỳ chọn `-O2`. Lệnh biên dịch: `g++ -std=c++17 -O2 -o bai bai.cpp`
- Python: bản 3.11 trở lên, không dùng thư viện ngoài trừ các mô-đun chuẩn `heapq`, `collections`, `bisect`, `random`, `math`.
- Julia: bản 1.9 trở lên. Riêng các đoạn dùng `PriorityQueue` cần gói `DataStructures`, cài bằng lệnh `using Pkg; Pkg.add("DataStructures")`.

[1] Đề cương học phần ghi tên tác giả là "J. Knuth" và tên sách là *The Art of Programming*, nhà xuất bản McGraw-Hill. Đây là lỗi in ấn; bộ sách này của Donald E. Knuth và do Addison-Wesley xuất bản.